Reference textbook: Operating System Concepts, 9th Edition
Abraham Silberschatz, Peter B. Galvin, Greg Gagne

Operating Systems

Full course handout

A4 · Two slides per page · Lin Cing

Contents

1. Kernels, interfaces, and system structure
Chapter 1 · Introduction 2
Chapter 2 · Operating system structures 17
2. Processes, synchronization, and scheduling
Chapter 3 · Processes 38
Chapter 4 · Threads 52
Chapter 5 · Process synchronization 64
Chapter 6 · CPU scheduling 80
Chapter 7 · Deadlocks 106
3. Address translation and virtual memory
Chapter 8 · Main memory 123
Chapter 9 · Virtual memory 138
4. Storage, files, and I/O
Chapter 10 · Mass storage 164
Chapter 11 · File-system interface 178
Chapter 12 · File-system implementation 194
Chapter 13 · I/O systems 212
5. Protection and security
Chapter 14 · Protection 226
Chapter 15 · Security 237
6. Virtualization and distributed systems
Chapter 16 · Virtual machines 254
Chapter 17 · Distributed systems 267
7. System case studies and design context
Chapter 18 · The Linux system 282
Chapter 19 · Windows 7 301
Chapter 20 · Influential operating systems 323

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Managed abstractions

Resource	Program abstraction	Required enforcement
CPU and timer	Process or thread	Who runs, when control returns, who changes privilege
RAM and MMU	Address space	Ordinary programs cannot change others' mappings or permissions
Disk and controller	File and block service	Names, permissions, ordering, and error reporting
NIC	Socket and message	Endpoints, buffer ownership, packet and connection state

Convenience, utilization, response time, fairness, isolation, and reliability are separate goals. A long batch job can keep the CPU busy while interactive requests wait. Specify the workload and objective before claiming efficiency. The kernel is the trusted privileged core. Shells, GUIs, libraries, and daemons usually run in user space. A library call need not enter the kernel, and changing a shell need not change the kernel.

Policy and mechanism

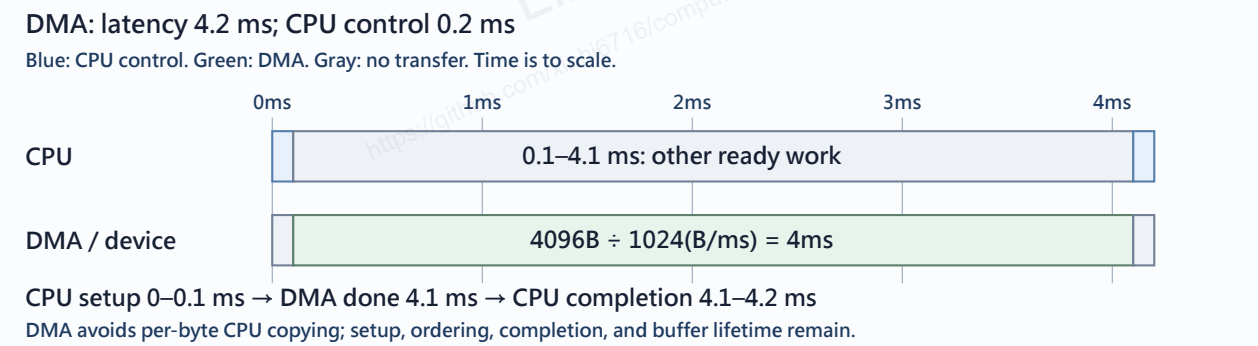
Alice and Bob share a machine. Bob's program is untrusted. Policy denies Bob access to Alice's private pages and files and prevents permanent CPU monopolization. Mechanisms include protected page tables and memory-access checks, kernel file-authority checks, and a protected timer and scheduler entry.

An application's check is insufficient because Bob can bypass it. User and kernel modes are insufficient if the kernel reads sensitive files for Bob without validating authority. Enforcement must mediate every relevant operation; untrusted programs must not modify its policy or metadata. The boundary includes CPU control, memory, object rights, and mutable shared state. A hidden GUI button enforces nothing. Revoking an already authorized reference has a separate contract in Chapters 11 and 14.

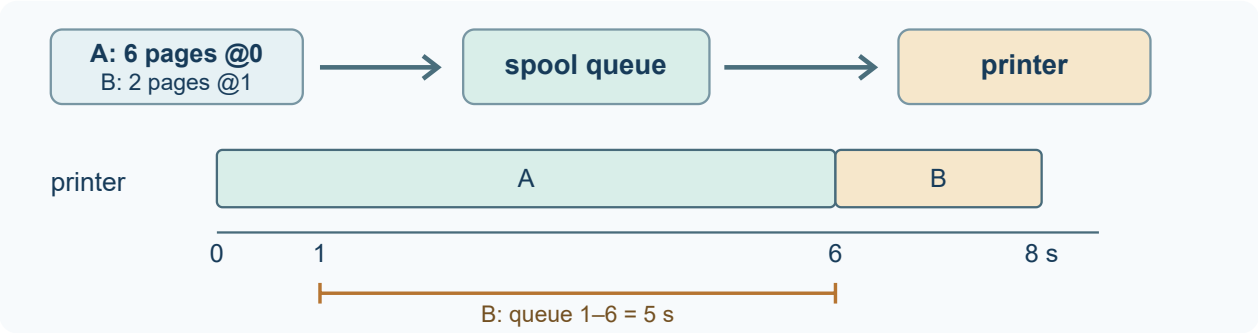
Controller, driver, and DMA

The controller implements device control in hardware or firmware. The driver is OS software that understands its registers, queues, and completion status. CPU submission, device transfer, and CPU completion handling are distinct events. Polling or an interrupt can discover completion.

The fixed model gives setup 0–0.1 ms, DMA 0.1–4.1 ms, and completion handling 4.1–4.2 ms. During the middle 4 ms, the CPU can run another ready job. DMA reduces per-byte CPU copying but does not eliminate setup, cache synchronization, bandwidth, or completion costs. A 4096 B block at 1024 B/ms takes 4 ms. CPU control costs 0.2 ms, whereas request latency is 4.2 ms. CPU service time is not application waiting time. Required data visibility and DMA completion precede buffer return.



Spooling: queued jobs and device completion



At one page per second with non-interleaved FCFS jobs, A's six pages arrive at t=0 and B's two at t=1. A prints during 0–6; B waits five seconds and prints during 6–8.

Submission confirms acceptance, not printing or survival after power failure. Jobs may reside on disk, in memory, or elsewhere; durability depends on the contract.

A spool and a buffer solve different problems

Mechanism	Managed object	Main purpose	Does not by itself imply
Spool / job queue	Complete jobs, order and state	Schedule a shared device; decouple submission from service	Durability, FCFS, or unlimited capacity
Buffer	Data temporarily held in transit	Absorb producer/consumer rate differences and batch transfers	RAM-only storage, completed work, or durability

They can be combined: the spool holds print jobs while the driver transfers each job in buffered chunks. When space runs out an API may block, reject, or report an error. Priority insertion also invalidates the FCFS timeline.

Interrupt, exception, and system call

Event	Relation to current instruction	Typical continuation
Device or timer interrupt	External, asynchronous	Save state, handle event, possibly schedule
Page fault or divide exception	Synchronous consequence	Repair and restart, report error, or terminate
System call	Deliberate synchronous request	Controlled entry, validation, service, return

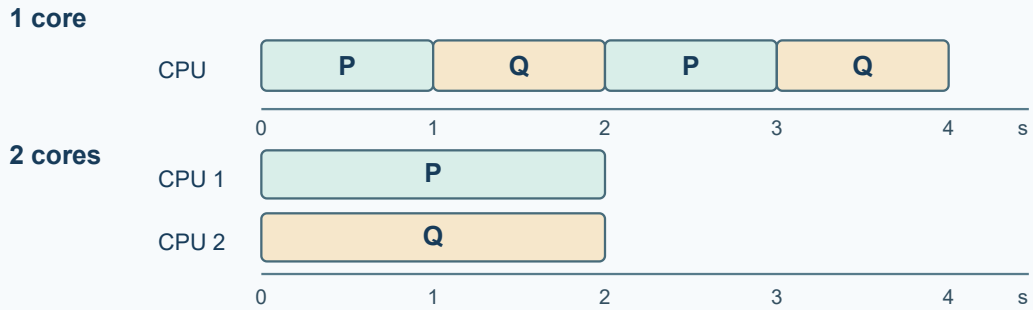
ISA terminology and return points differ; not every exception resumes at the next instruction. An interrupt in kernel mode may cause no user-to-kernel transition, but still needs state preservation and reentry synchronization. An interrupt can return to the same thread, and a system call need not block. Neither implies a process switch. DMA can use polling or batched notification rather than one interrupt per block. State the completion rule.

Architecture follows sharing

Architecture	Typical sharing	Additional issue
One general-purpose CPU	One general execution resource	Concurrency with DMA and interrupts remains
SMP	Peer CPUs and common RAM	Synchronization and cache coherence
NUMA shared memory	Common addressable RAM, unequal locality	Location changes latency and bandwidth
Cluster	Independent networked nodes	Messages, partial failure, retry and consensus assumptions

Core, socket/package, and hardware thread count different things. SMP peer control and NUMA access asymmetry are separate dimensions; shared memory need not be UMA. Multiple CPUs alone do not establish fault tolerance. Power, RAM, kernel, or storage may fail jointly. Failover also needs detection, recovery, and exclusion of two simultaneous owners of an exclusive resource.

Concurrency and parallelism: read the timeline



Concurrency means tasks make progress in an overlapping period. On one core P and Q take turns. **Parallelism** means work runs simultaneously on distinct resources, such as the two cores shown.

Shared data still needs synchronization. A single-core program can also run in parallel with DMA; state which resources are being compared.

NUMA mean and parallel speedup

Assume sequential nonoverlapping requests without cache effects. Local memory costs 100 ns, remote 180 ns. At 80% local access, mean cost is 0.8×100+0.2×180=116 ns. At 20% local it is 164 ns. Their ratio is not automatically whole-program speedup.

A 100 s baseline has 20 s serial and 80 s perfectly parallel work with no overhead. Four processors give T=20+80/4=40 s and speedup 2.5. Generally, $T(P) = 100[s + (1 - s)/P]$, so speedup is at most $1/s$, before synchronization and data-movement costs. Operation-count fractions cannot replace baseline-time fractions. Hardware does not automatically discover the parallel work.

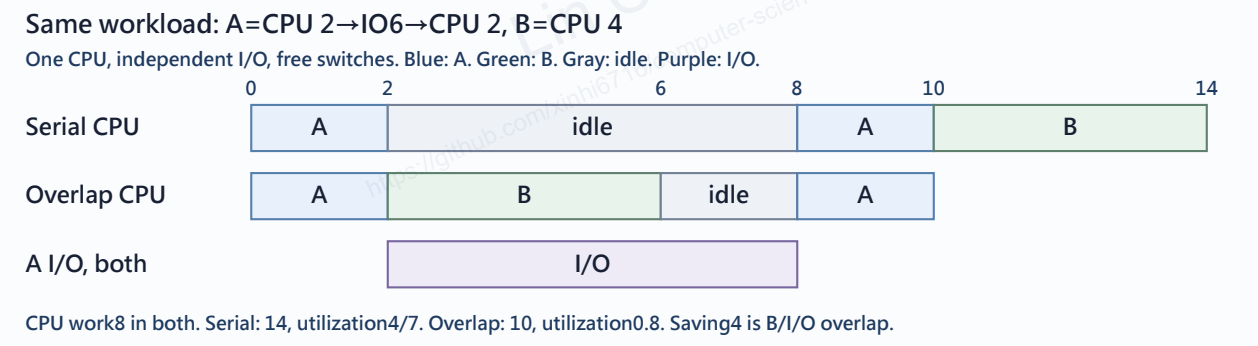
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multiprogramming overlaps CPU and I/O

Assume one CPU, one I/O device, free switching, A=CPU 2→I/O6→CPU 2, B=CPU 4, both ready at t=0.

Policy	CPU intervals	I/O	All complete	CPU utilization
Finish A before B	A0–2, idle2–8, A8–10, B10–14	A2–8	14	8/14=4/7
Run B while A blocks	A0–2, B2–6, idle6–8, A8–10	A2–8	10	8/10=0.8

CPU work remains 8. One CPU does not execute both CPU bursts simultaneously. The saving of 4 is exactly the overlap of B with A's I/O. Without other ready work, or when all I/O waits on one bottleneck, this saving need not exist.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Time sharing requires recoverable control

Switching only on blocking cannot protect other jobs from an infinite CPU loop. Time sharing normally adds timer preemption and a ready-queue rule. For n continuously ready jobs, quantum q, switching cost c, and fixed RR, a job that just missed its turn waits at most $(n-1)q+nc$, conservatively including switch costs. The bound fails with unbounded n, unbounded nonpreemptible kernel execution, or indefinitely inserted higher-priority work.

Smaller q reduces rotation delays but raises $c/(q+c)$. With q=4 ms and c=0.1 ms, this CPU-only model gives 0.1/4.1≈2.44%. This is a specified model, not measured universal overhead.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Privilege and execution context

Protected privilege state prevents ordinary code changing the timer, page-table root, or interrupt entry. A system call saves ISA-required return state and enters controlled kernel code. Validation of arguments and authority precedes service.

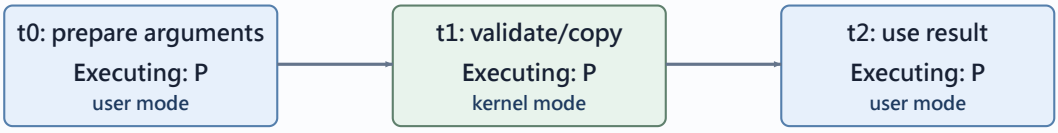
t	One thread's execution	Mode	Process switch required?
0	Prepare read arguments	User	No
1	Entry and validation	Kernel	No
2a	Cache hit, copy, return	Kernel→user	No
2b	Cache miss, wait for I/O	Blocked in kernel	Another ready thread may run

A mode switch changes privilege. A context switch saves and loads an execution context. Another thread in the same address space need not require an address-space switch.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Privilege and execution context (continued)

Cache-hit syscall: 2 mode transitions,0 thread switches



Blue: user. Green: kernel. Entry/return change mode; P and its AS stay the same.

No other event here. Blocking/preemption may schedule Q. Count each switch type separately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A timer does not prove fairness

The OS sets a protected timer before handing control to untrusted user code. Allowing that code to disable or modify the timer defeats control recovery. Progress also requires serviceable interrupts, bounded nonpreemptible kernel sections, terminating handlers and scheduler actions, and selection without permanent starvation.

Interrupts can be delayed, batched, or prioritized. Setting q does not imply a precise switch at each q boundary. Kernel faults and malicious arguments remain possible. Mode enforcement is a foundation, not a proof of all kernel behavior.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Programs and process state

A program is passive bytes, perhaps in a file. A process has an address space, credentials, open references, and at least one thread. A thread has a PC, registers, stack, and execution context. Threads share some process resources, not one PC.

Event	Transition	Actor
Wait for I/O	Running→waiting	Current thread and kernel register waiting
I/O completion	Waiting→ready	Completion path makes it schedulable
Selection	Ready→running	Scheduler and dispatcher
Quantum expiration	Running→ready	Controlled preemption

Completion does not immediately make a thread running; it must obtain the CPU. Blocked time is not ready-queue waiting. Process management also includes creation, termination, synchronization, IPC, and cleanup.

A non-atomic increment loses updates

Initially counter=0. Each thread loads, increments, and stores.

Step	T1	T2	Counter
1	Load r1=0		0
2		Load r2=0	0
3	Store r1+1=1		1
4		Store r2+1=1	1

The result is 1 rather than 2. A single CPU can switch between these steps; multiple CPUs add interleavings. A suitable mutex or atomic RMW changes the contract to an indivisible update. `volatile` alone does not provide mutual exclusion. Chapter 5 supplies the ordering conditions.

Equal virtual addresses, different frames

Pages are 4096 B. P maps VPN 1→PFN 7 read/write; Q maps VPN 1→PFN 9 read-only. VA0x1234 has VPN 1 and offset0x234=564. P's PA is 0x7234, Q's 0x9234.

Operation	P	Q
Read VA0x1234	Frame 7+564	Frame 9+564
Write VA0x1234	Permission allows	Permission fault
Change page table to frame 7	Ordinary user cannot modify protected tables	Same restriction

Mapping existence, residency, and authorization are distinct. The OS may repair a not-present mapping; adding a page need not repair forbidden writing. Sharing requires explicit mappings to the same frame and defined permissions. Equal VAs do not create sharing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Frame allocation and reclamation

Allocated and free sets cannot both claim exclusive ownership of one frame. Legitimate sharing needs consistent references and permissions. Reclamation requires old mappings to be unusable, including TLB entries and outstanding I/O holders.

Eight RAM frames with kernel 2, P3, Q2 leave free 1. Releasing one unshared Q frame gives free 2. If P still references it, it cannot enter the free pool. Remaining reachability matters more than one process declaring release. Virtual memory need not keep all virtual pages resident, but RAM remains finite. Working sets, backing I/O, and commit limits constrain performance and allocation guarantees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Define miss penalty before averaging

L1 lookup costs 1 cycle, hit probability is 0.9, and a miss adds 50 cycles. Without overlap or prefetch, mean cost is 1+0.1×50=6, equivalently 0.9×1+0.1×51. If 50 denotes total miss latency, the mean is 0.9×1+0.1×50=5.9. The inputs differ. A mean of 6 does not make every access 6 or specify tail latency.

Caching introduces copy coherence and invalidation. CPU coherence does not establish filesystem durability; data may remain volatile. Write completion, flush completion, and power-loss survival need separate definitions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Allocation and storage failures

Applications use file names and offsets. Filesystems map ranges to blocks, and block layers and drivers submit device requests. Allocation prevents duplicate exclusive owners; free-space indexes must agree with object pointers. A memory lock handles concurrency but cannot repair a crash that persisted half an update.

For 2^{20} blocks at one bit per block, bitmap capacity is 2^{20} bits=2^{17} B=128 KiB. This is logical capacity. An uncached query still reads its containing storage block, not one physical bit. Corruption, hardware failure, accidental deletion, and hostile access need different responses. RAID, backup, and protection do not replace one another.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Protection and security

Protection restricts known subjects' use of objects. Security also concerns authentication, intrusion, malware, and threats to the system. A simple internal/external split cannot classify every attack.

Question	Mechanism	Still insufficient
Who requests?	Authentication and trusted credentials	Identity does not grant authority
What object and operation?	ACL or capability and access check	UI visibility cannot enforce an API
Can messages be intercepted?	Threat-appropriate encryption and key management	Cannot repair compromised endpoints
Can mappings exceed authority?	MMU rights and protected tables	Kernel bugs may bypass policy

Confidentiality, integrity, and availability are separate. Denying every request may prevent disclosure while destroying availability.

Least privilege and authorization time

An image converter needs input-file read and specified-output write rights, not write authority over a whole home directory. Bounded authorized references limit damage better than unrestricted paths or root identity. An ordinary user still has authority through inherited descriptors, shared memory, and sockets.

Removing a name's permission need not invalidate existing handles. Distinguish reference-creation checks from later-operation checks. A complete decision identifies credentials, object identity, requested rights, policy, and check time. File names, displayed identities, and privilege icons alone are insufficient.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Kernel structures and their premises

Structure	Operations	Conditions
Linked list	O(1) insertion/deletion with required node or predecessor	Worst-case key search O(n), pointer/cache costs
Stack or FIFO queue	O(1) endpoints with suitable representation	FIFO is not priority order
Balanced search tree	Ordered lookup/ranges, O(log n)	Maintain balancing invariant
Hash table	Average O(1) in specified model	Worst-case collision O(n), resizing costs
Bitmap	Membership in finite integer domain	Space follows domain size; claims need atomicity

With distinct BST keys, all left-subtree keys are below the node and all right-subtree keys above. Immediate-child comparisons are insufficient. Balanced height is O(log n), not an exact log2 n level count for any tree.

Collisions, subtree bounds, and lifetime

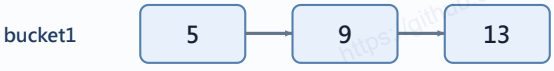
For h(k)=k mod4, tail-insert 5,9,13 into bucket1 as [5,9,13]. Searching 13 takes three comparisons, and absent17 also takes three. n congruent keys give linear worst-case search. Constant average cost requires distribution/load premises or a stronger design.

A tree with root 10, left5, right20, and 12 as 5's right child satisfies 12>5 but violates root 10's left-subtree bound. Searching 12 follows right20 and then an empty left link. Full subtree ranges must hold.

A reader may retain an old node after unlinking. Immediate free/reuse permits use-after-free. Locks, reference counting, or an RCU-style grace period have different reclamation contracts. Stale values do not authorize invalid memory.

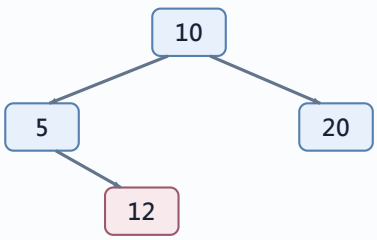
Hash collision cost and complete BST subtree bounds

h(k)=k mod4; chaining



Find 13: 3 comparisons. Reject 17: 3 comparisons.

n congruent keys: worst case O(n).



Red 12 is left of root 10 but12>10: subtree-bound violation.

Computing environments

Environment	Boundary	Unsupported inference
Client/server	Request and service roles	A server cannot also be a client
Peer-to-peer	Nodes request and provide	No central discovery/authentication exists
Cluster/distributed	Independent nodes and network	Timeout proves remote failure
VM	Guest virtual hardware through VMM	Guest privilege equals host privilege
Conventional container	Views normally share host kernel	Each has an independent kernel
Embedded/real-time	Device or timing constraints	Every embedded system is hard real-time

IaaS, PaaS, and SaaS divide provision and management responsibilities. They are not CPU architectures. Virtualization can support them, but every cloud service is not one VM.

Deadlines are part of correctness

Assume one CPU, no blocking/overhead, A(C=1, T=D=4), B(C=2, T=D=6). Demand is 1/4+2/6=7/12. Utilization below one alone does not prove deadlines without a scheduler.

A feasible 12-unit EDF schedule is A0–1, B1–3, idle3–4, A4–5, idle5–6, B6–8, A8–9, idle9–12. A completes at1,5,9 before4,8,12; B at3,8 before6,12. It repeats because lcm=12 and no work remains at the boundary. Nonpreemptible sections, jitter, or I/O change the premises and require fresh blocking/response analysis. Low average latency does not replace a worst-case deadline check. Hard/soft real-time differ in consequences and permitted violations.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Source visibility and open-source rights

The Open Source Definition requires more than readable source, including specified redistribution and derived-work rights. Source-available, freeware, public domain, and open source are different. Commercial charging need not conflict with open source.

A distribution combines a Linux kernel with tools, libraries, services, and packages. BSD is another family and licensing tradition. One open component does not give all files one license. Comparing implementations requires fixed versions, configuration, and dependencies. Linux 3.2 and Windows 7 retain historical contracts here; modern supplementary interfaces are identified separately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Match evidence to the question

Question	Evidence	Insufficient substitute
Which API outcomes are allowed?	Versioned interface/standard	One successful run
How does this version implement it?	Matching source/configuration/architecture	Same function name in another version
What is observed latency?	Workload time/resource breakdown	Operation proportions treated as time
Is an update durable?	Storage/flush contract and failure order	Write returned success

Source access permits inspection, not a proof of no races, optimal speed, or security. Preserve premises, error outcomes, and general invariants. One example does not establish all inputs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One read connects the course

P reads4096 B through an authorized fd. Controlled entry validates the descriptor/OFD and buffer. A cache miss creates I/O and blocks P; the scheduler selects Q. DMA completion makes P ready. P later obtains the CPU, observes valid data, and copies/returns actual bytes.

Part	State	Chapters
Call/protection	Mode, credentials, requested right	2,14,15
Process/thread	Ready/waiting/running, saved registers	3–7
Mapping/data	VA/PTE/frame, cache, buffer lifetime	8,9,13
Name/storage	OFD offset, inode, block, completion/durability	10–12

These states do not complete simultaneously. Waiting→ready is not immediately running. Completing one request does not flush unrelated files.

Quantities that must remain distinct

Mode transitions, context switches, and address-space switches are separate counts. One thread's cache-hit system call may require only the first. CPU busy time, request latency, throughput, and deadlines use different denominators; less CPU work need not shorten a request. Names, object identities, and open references differ; renaming a path need not alter an fd. Average cost, worst-case cost, and general correctness differ, as collisions, timer delay, and races demonstrate.

Fix the version and abstraction, enumerate observable states/results, then use invariants and counterexamples to assess the mechanism.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 1. Open Source Initiative: Open Source Definition.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Services and completion contracts

User interfaces, execution, I/O, file operations, communication, and error detection support applications. Allocation, accounting, protection, and security support coexistence. A service is not merely a syscall name. Kernels, libraries, daemons, and tools can cooperate to provide one interface. Actual privileges and object access determine protection boundaries.

Copy service	Provider in this model	Observable completion
Interpret source/destination	Shell and copy tool	Two pathnames, not kernel interpretation of shell syntax
Find file and check rights	VFS/filesystem	Successful open descriptor or explicit error
Transfer	Library API→kernel I/O	Actual completed bytes, possibly below request
Allocate and protect	Kernel/filesystem	Valid descriptors, cache/storage capacity, rights
Store result	Filesystem/device	File synchronization distinct from directory-entry persistence

User and system programs normally obey the same protection rules. The name system tool grants no kernel privilege. A privileged user-space daemon still runs in user mode; credentials and CPU mode are separate.

A 6000-byte copy with short writes

Source is a fixed6000 B, destination a new empty regular file, buffer 4096 B, blocking I/O, no competing writer.

Event	Returned bytes	Source offset	Destination offset/length	Buffer pending
Open source/create destination		0	0	0
read 4096	4096	4096	0	4096
write 4096	1000	4096	1000	3096
write buffer[1000:]	3096	4096	4096	0
read 4096	1904	6000	4096	1904
write 1904	1024	6000	5120	880
write buffer[1024:]	880	6000	6000	0
read 4096	0, EOF	6000	6000	0

Excluding directory synchronization, the path has2 opens,3 reads,4 writes,1 fsync,2 closes, totaling12 API operations. The essential rule is progress from actual returned bytes. One write after each read would lose3976 B.

Copy-loop invariant and failure boundary

Let R be bytes read, W bytes written, and [j, n) the pending buffer interval. Maintain $W \le R$ and buffer[j: n] equal to the next contiguous source fragment not yet written. A positive return $k \le n-j$ increments both W and j by k without changing sequence. Only j=n permits overwriting the buffer with another read. EOF with no pending bytes proves W=R=6000.

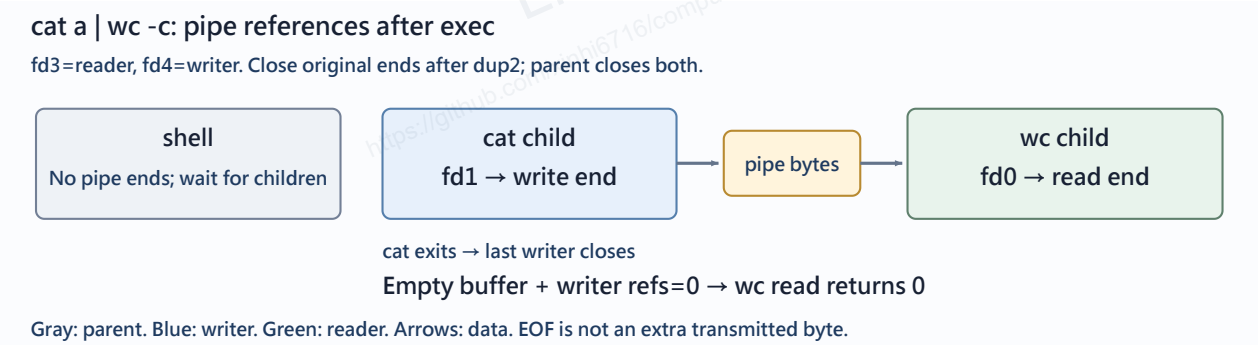
Handle zero-progress EINTR according to the interface and retry the same remaining range. A positive count advances progress first; rewriting the whole chunk duplicates data. Unexpected write=0 must report no progress or otherwise avoid an infinite loop. ENOSPC/EIO leaves partial output and cannot return success.

A successful write is not crash durability. Under a compliant local filesystem/device flush contract, successful destination fsync confirms its data and required metadata. A new name also requires synchronization of its containing directory. This copy still exposes a partial destination on failure. Atomic replacement needs a same-directory temporary file, data synchronization, rename, and directory synchronization protocol; close alone does not supply it.

Shell, GUI, and kernel

A command interpreter translates text/scripts into processes and I/O connections; a GUI translates input events into application actions. Both can use the same kernel services. A child-only `cd` cannot change the parent shell's cwd, which explains its usual builtin implementation. External `cat` can execute in a child. A builtin is not necessarily kernel code. Privileged frontends also expose attack surfaces such as command injection and accidental overwriting.

For POSIX-style `cat a | wc -c`, assume no alias, job control, or extra subshell. The shell creates a pipe, forks two children, connects cat's stdout to its write end and wc's stdin to its read end, then waits. Two execs replace child images without creating two more processes. The parent retains its image.



Endpoint references determine EOF

pipe returns fd3 read and fd4 write. Initially all three processes inherit both. Cat uses dup2(4,1), wc uses dup2(3,0), then closes redundant descriptors.

Process	Remaining pipe reference	Close	Reason
Shell	None	3,4	Do not keep a writer alive
cat	fd1→write end	3 and original4	Do not hide loss of readers with own read reference
wc	fd0→read end	Original3 and4	Do not retain a writer that prevents EOF

Cat writes6000 B and exits. Wc can read arbitrary chunks and accumulate6000. Only an empty pipe with every write-end reference closed makes the next read return 0. If the shell retains fd4, wc may block after cat exits while the shell waits for wc. Process exit alone does not establish the reference condition.

Output, status, and permissions

If cat cannot open a and returns ENOENT, it can exit nonzero without writing bytes. Wc can correctly count the empty stream, output0, and exit0. Collect both wait statuses. Pipeline-wide failure follows the shell's specified rule, independently of data correctness.

A GUI cannot claim copy success merely because a progress bar ends. It must await the I/O/error and synchronization contract it promises. CLI versus GUI does not change access checks, capacity, or crash consistency. `cat 'a b'` supplies one pathname; `cat a b` supplies two. The shell parses quotes and pipes. The kernel receives an exec argument array and does not reapply shell syntax.

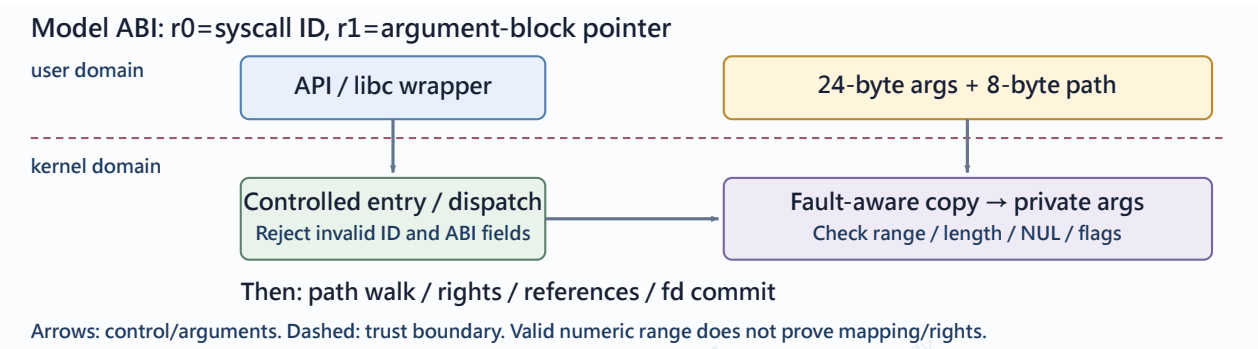
API, ABI, and implementation

An API defines function names, types, and observable behavior. An ABI defines machine calling conventions, syscall numbers, registers/stack layout, and returns. Kernel implementation operates on protected objects. Source compatibility does not make a binary portable across ISAs/ABIs.

`strlen` can run entirely in user space. Buffered `printf` may retain output and combine several calls into one write. A copy API can combine many opens/reads/writes. API-call counts therefore differ from trap counts. If libc converts kernel errors into−1/errno, distinguish the two return formats.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

API, ABI, and implementation (continued)



The teaching ABI puts syscall ID in r0 and a pointer to a24 B argument block in r1. Its three64-bit fields are pathname pointer, flags, and mode. This is a specified model, not a claim that Linux open always uses such a block; real platforms may use registers and vary by version.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Copy arguments before validating their meaning

`/data/x` plus NUL occupies8 B. A fault-aware user copy obtains the24 B fixed block, then bounded string copying obtains at most PATH_LIMIT bytes,8 here. Total copy is32 B. Parse and use the same kernel-owned copy so later user modifications cannot substitute another pathname.

Step	Check	Failure
Decode ID/ABI	Known ID, valid fields/flags	Reject, never jump to arbitrary handler
Copy block	User range and actual fault-safe access	EFAULT or model equivalent, no partial arguments used
Copy pathname	Accessible, NUL-terminated, bounded	EFAULT/ENAMETOOLONG, no published fd
Path walk	Directory search and final-object rights	EACCES/ENOENT or other error
Allocate/publish fd	Stable reference, rollback incomplete allocation	Valid fd or no leaked reference

Copy arguments before validating their meaning (continued)

A readable-looking VMA is insufficient: another thread can unmap it and the actual access can fault. Use fault-aware copying, check that the copy completed, and validate and consume the same kernel-owned bytes. **Bytes stable after copying are not an atomic snapshot of the source during copying.**

For example, user fields begin at (length=4, capacity=4). The kernel reads length=4; user code writes length=8 and then capacity=8; the kernel reads capacity=8. The private result is (4,8), although the source visited only (4,4), (8,4), and (8,8). Treat the result as arbitrary untrusted input and revalidate cross-field relations, lengths, and authority. One copy does not guarantee coherent source fields. An API requiring a coherent snapshot needs an additional enforceable synchronization or version protocol; an adversarial caller cannot be trusted to honor a user-space lock. Filesystem namespace stability during path walk separately requires kernel object references and synchronization.

Prove arithmetic before accessing memory

For32-bit unsigned words, nonempty ranges in user addresses[0, U), U=0xC0000000, use `p < U && n <= U-p`. Do not first calculate wrapping p+n. Arithmetic success still needs mapping, permission, and fault-safe copy checks.

p	n	Mathematical exclusive end	Result
0xBFFFFF00	0x100	0xC0000000	Valid, final byte0xBFFFFFFF
0xBFFFFF00	0x101	0xC0000001	Crosses boundary
0xFFFFFFF0	0x40	0x100000030	Invalid,32-bit addition disguises it as0x30

For count elements of4 B, first require count≤SIZE_MAX/4. count 0x40000001 has product0x100000004, truncated to4 in32 bits. Allocating4 B and iterating the original count overruns memory. Zero-length pointer rules belong to each API; the present condition explicitly concerns nonempty ranges.

Syscall categories connect objects to state

Category	Requests	Protected state	Success does not imply
Process control	fork, exec, wait	Task, image, exit status	exec creates another process
Files	open/read/write/lseek/close	fd, OFD, inode	Write durability
Devices	read/write/control	Driver queue, device state	Queueing is completion
Information	Attributes, time	Credentials, metadata	Object ID grants rights
Communication	Pipe, messages, shared memory	Endpoint, message, mapping	Stream read is one record
Protection	Permissions, credentials	Policy, principal	Authentication is every authorization

Categories overlap. Socket reads involve I/O, communication, and protection. An fd need not name a regular file. Files, pipes, and terminals share read-like interfaces with different blocking, EOF, and seek rules. Parameters may use registers, blocks, or a stack. Persistent channels and bounded message units have different correctness obligations.

Returns are state-machine outputs

For a regular-file read without concurrent modification, count 4096 from offset0 may return 3000 and advance to3000, then1096 and advance to4096, then0 at that moment's EOF. A short read is not permanent EOF; on a pipe it may merely reflect available bytes.

Model kill(pid100, sig9) as two32-bit arguments,8 B of representation, not necessarily8 B copied. The kernel resolves the PID namespace and validates authority/signal. Missing target gives ESRCH; insufficient authority EPERM. Success means delivery/queueing under signal rules, not observed complete termination and reclamation. Wait supplies lifecycle synchronization. Multiple successful syscalls are not automatically one atomic transaction. Identify the commit point; protection, blocking, visibility, durability, and termination remain separate.

Source to running process

For hello.c with main and libc output, the path is preprocess→compile→assemble→link→image. Dynamic execution additionally needs a compatible interpreter/loader and libraries. Compiler/linker are user programs using kernel file, memory, and process services; generating machine code does not give kernel mode.

Stage	Input/output	Independent failure
Preprocess	Source/headers→tokens	Missing header, wrong macro branch
Compile/assemble	Tokens→assembly→object	Syntax/type error, target ISA mismatch
Link	Objects/libraries→image	Unresolved symbol, ABI mismatch
Exec/load	Image/interpreter→process image	Permission, format, missing loader
Run	Entry→APIs/syscalls	Fault, I/O error

File tools, status tools, programming support, loading, communication, and background services have different purposes. Daemon lifetime changes neither CPU-mode enforcement nor kernel-interface rules.

Build errors must propagate

For parser, compiler, linker, success is0 and failure nonzero. `parser && compiler && linker` runs a stage only after success. Semicolons continue regardless, and the last status may reflect only the linker. Prevent stale outputs through explicit versions and dependencies, not mere existence.

Case	Parser	Compiler	Linker	Conclusion
All succeed	0	0	0	Image exists, execution still unproved
Parser fails with &&	2	Not run	Not run	Build failed, do not publish stale image
Compiler fails with ;	0	1	0 using stale object	Last0 hides earlier failure

Source API compatibility, binary ABI compatibility, and complete runtime dependencies are separate. Link success or a matching digest alone proves neither a bootable target nor error-free execution.

Mechanism and policy

Timer interrupts, context save/restore, and controlled ready-queue operations are mechanisms. Quantum and selection order are policy. Timer resolution, nonpreemptible code, and switch cost constrain achievable policy latency.

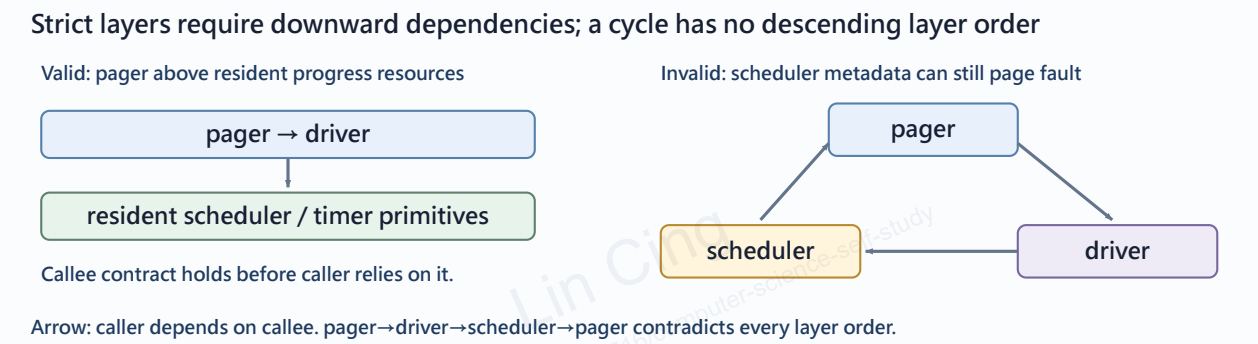
One CPU has A/B/C bursts5/3/1 ms, all ready at0, initial RR order A, B, C, no I/O, zero switch cost.

q	CPU timeline	Completion A, B, C	First response A, B, C
1	A0–1, B1–2, C2–3, A3–4, B4–5, A5–6, B6–7, A7–8, A8–9	9,7,3	0,1,2
4	A0–4, B4–7, C7–8, A8–9	9,7,8	0,4,7

Both do9 ms useful work. Mean response is1 versus11/3 ms. The q 1 trace has7 actual task switches; q 4 has3. Final A→A is no task switch. Nonzero switch cost changes wall times and requires recalculation.

Layered proof requires acyclic dependencies

Strict layers use only lower-layer contracts. If layer0 satisfies C0 and each layer i establishes Ci assuming C0…C(i−1), induction establishes upper properties. Tests do not make lower layers infallible; false contracts, premises, or coverage defeat the inference.



Pager→disk driver→scheduler→pager cannot have strictly descending layer numbers. Keep necessary scheduler metadata and fault/I/O-progress resources resident, and separate lower scheduling primitives that do not depend on paging. Hiding an arrow does not remove the cycle.

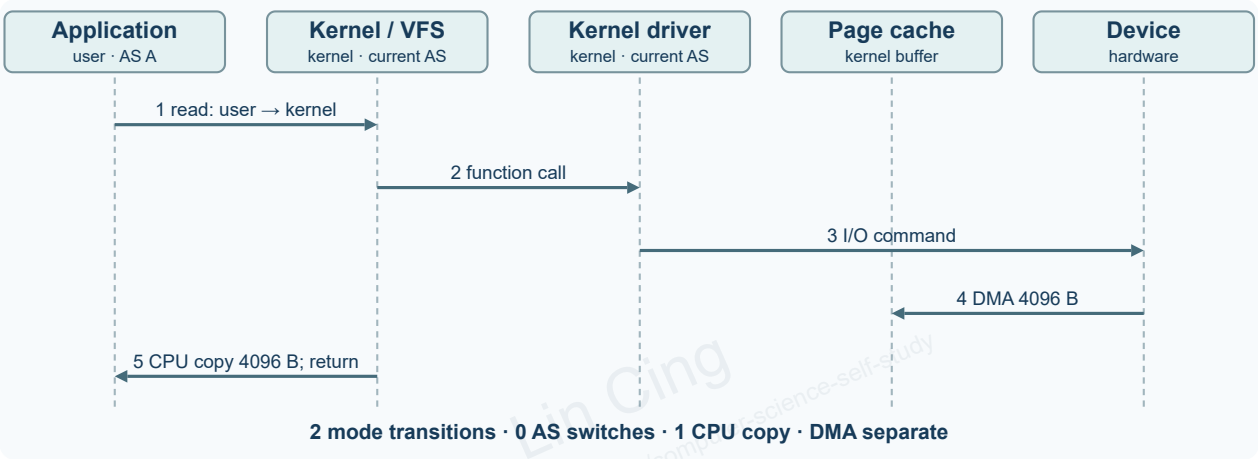
Implementation language and measurable goals

Higher-level languages can improve readability, portability, and optimization. They do not automatically provide memory safety, atomicity, or hardware ordering; C pointer errors can corrupt a kernel. Assembly remains at machine-specific entry, context, and privileged-instruction boundaries. One library API does not erase ABI differences.

With queue capacity1024, identical pregenerated operations, and identical machine resources, candidate A loses no elements. Candidate B loses an enqueued element in one recorded interleaving. B violates correctness even if faster. This is logical exclusion, independent of a throughput threshold.

CPU time per request, end-to-end latency, throughput, and p99 have different denominators. Define workload and quantile rules. Zero-cost schedule tables describe only that model. Real comparison needs switching, cache, and wait measurements, without inventing throughput or p99.

Monolithic read: sequence and boundaries



A fixed 4096 B cold read; follow events downward. A cache hit omits device work but retains the cache-to-user copy.

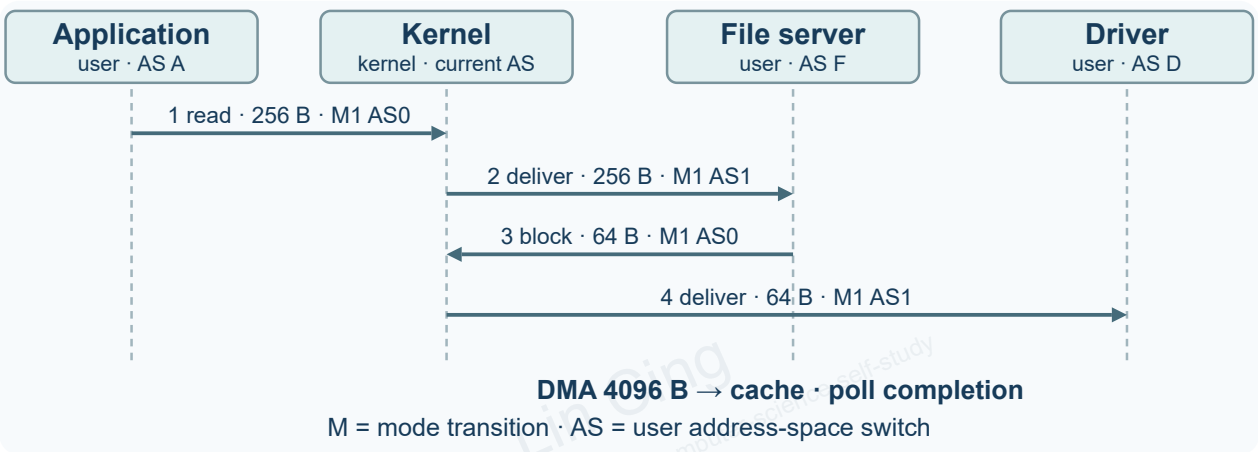
A fixed cold read in two structures

Use read(fd, user_buf,4096) from `/srv/data.bin`, offset0, cold page cache. ABI arguments use registers, with no additional parameter copy. The monolithic VFS, filesystem, and block driver share kernel execution. In the microkernel model, file server and driver have separate user address spaces, kernel mapping is in the current user AS, and each IPC uses sender→kernel→receiver two-copy delivery.

The application uses libc, enters kernel mode, validates fd/rights/buffer/offset, calls VFS→filesystem→driver, transfers4096 B by DMA into page cache, copies4096 B to user, and returns.

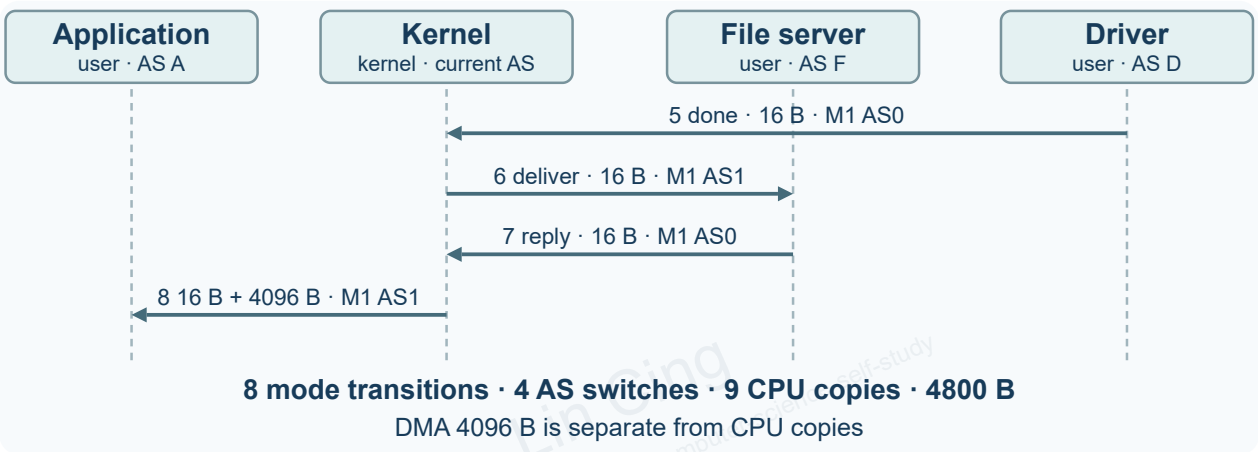
Event	Boundary	Cost
libc read	ABI call	0
sys_read	User→kernel, register args	Mode+1
VFS/filesystem/driver	Same-AS function calls	AS+0
Disk→page cache	4096 B DMA	CPU copies+0
Cache→user	4096 B CPU copy	CPU copies+1
Return4096	Kernel→user	Mode+1

Microkernel: request and device work



Fixed two-copy IPC; DMA buffer and device authorization are preconfigured.

Microkernel: completion and reply



Fixed two-copy IPC; DMA buffer and device authorization are preconfigured.

Eight microkernel boundary events

App, FS, Driver are three user processes. Entering the kernel retains the current user AS; handing control to another user process changes AS.

Assume the kernel has already allocated and pinned the DMA buffer and authorized device access and mappings for the driver. Setup and teardown are outside the counts; an IOMMU, if used, must enforce the device mapping.

Step	Boundary	IPC action	Control copy	Mode	AS	CPU copies
1	App→Kernel	SYS_IPC_SEND(read_req), register args	256 sender→kernel	1	0	1
2	Kernel→FS	Deliver request	256 kernel→receiver	1	1	1
3	FS→Kernel	SYS_IPC_SEND(block_req)	64 sender→kernel	1	0	1
4	Kernel→Driver	Deliver block request	64 kernel→receiver	1	1	1
5	Driver→Kernel	Completion send	16 sender→kernel	1	0	1
6	Kernel→FS	Deliver completion	16 kernel→receiver	1	1	1
7	FS→Kernel	Reply send	16 sender→kernel	1	0	1
8	Kernel→App	Deliver reply and copy cache	16 kernel→receiver plus4096data	1	1	2

Eight microkernel boundary events (continued)

Total8 modes,4 user-AS switches,9 CPU copies. Four messages256/64/16/16 B take8 copies and704 B. Add one4096 B data copy for4800 B. One 4096 B DMA remains separate. AS handoffs are App→FS, FS→Driver, Driver→FS, FS→App.

A function call is not automatically zero-copy, nor does all microkernel IPC require these eight transitions. Merging servers, private kernel AS, shared memory, or one-copy delivery requires a new ABI/deployment/count table.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

DMA boundary and model sensitivity

Baseline counts cover only the listed request-chain handoffs. DMA completion uses polling, no other runnable work exists, and no timer preemption or completion interrupt occurs. Interrupt entry/exit, wakeup, and scheduling need separate rows if introduced.

Change	Mode	AS	CPU copies	Bytes copied
Monolithic, current-AS kernel	2	0	1	4096
Microkernel,256/64/16/16	8	4	9	4800
Microkernel request shrinks to128	8	4	9	4544
FS and driver in one process	4	2	5	4640
Monolithic, private kernel AS	2	2	1	4096
Microkernel, private kernel AS	8	8	9	4800

The128 B request gives2×128+2×64+4×16=448 control bytes, plus4096=4544. Merged FS/driver retains256 B request and16 B reply,544 control bytes plus4096=4640. Private-kernel-AS changes AS at every mode transition in both architectures. These are model sensitivity results, not universal syscall costs.

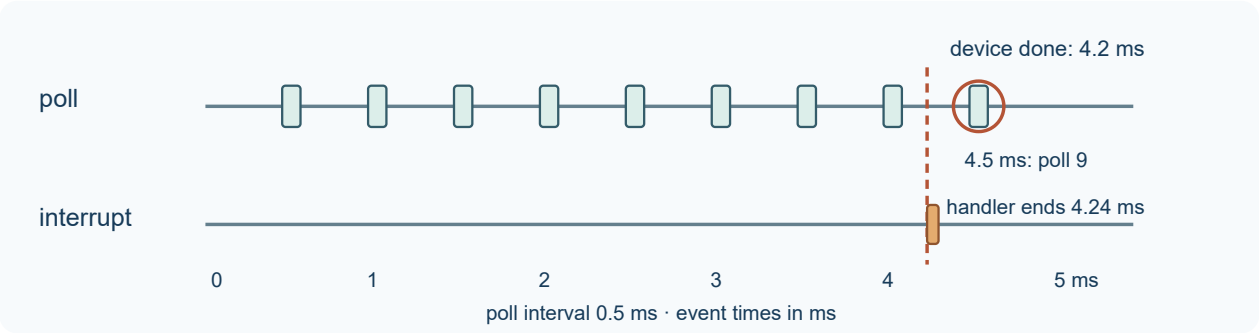
How is I/O completion discovered?

Polling has the CPU read device status; an **interrupt** notifies the CPU to run a handler; **DMA** moves data between a device and memory. DMA completion can be discovered by polling or by interrupt: DMA is not a third mutually exclusive notification method.

Method	CPU while waiting	Detection and cost
Busy polling	Repeated checks occupy the core	May detect quickly but consumes CPU
Periodic polling	Checks on a timer; other work can run between checks	Detection can wait until the next check; each check and timer costs work
Interrupt	Device notifies CPU on completion	No continuous checks, but handling and synchronization still cost work

Real devices may batch notifications, combine methods, or poll under high load. Workload, hardware, and scheduling determine which approach performs better.

A fixed polling and interrupt timeline



Suppose a device completes at **4.2 ms**. Periodic polls start at 0.5, 1.0, …, 4.5 ms and each consumes **0.01 ms** of CPU. Assume reliable status at the check instant and no scheduling delay. The ninth poll detects completion at 4.5 ms: detection delay is 0.3 ms and nine checks use 0.09 ms of CPU.

In a separate model, interrupt delivery is immediate and the handler takes 0.04 ms, ending at 4.24 ms. Submission, scheduling, and return-to-application costs are excluded. Busy waiting from 0 to 4.2 ms instead consumes 4.2 core-ms; the core is not idle.

A wakeup does not immediately run the thread

After completion, the poller or interrupt handler must establish required data visibility and update completion state. A waiting thread may transition **waiting → ready**. It becomes **ready → running** only after selection by the scheduler and restoration of its context.

In the preceding interrupt model the handler ends at 4.24 ms and makes P ready. If another thread yields and P runs at 4.34 ms, P has waited 0.10 ms in the ready queue. Handler completion at 4.24 ms and P's execution at 4.34 ms are different milestones; return to the application and data copying can require more work.

Safe DMA-buffer reuse also needs device completion, required synchronization, and ownership transfer. Early reuse can let the device and a new owner modify the same memory concurrently.

Four independent structural dimensions

Structure	Calls/dependencies	Privilege and AS	Extension	Failure contract
Monolithic	Mostly kernel function calls	Shared kernel privilege	May load modules	Bad pointer can corrupt kernel
Strict layers	Only lower interfaces	Layer need not be protection domain	Static or dynamic	Contracts support induction, not automatic isolation
Microkernel	Small kernel plus server IPC	Isolated user domains	Replace servers separately	Parsers, authority, recovery still need correctness
Modular kernel	Defined module interfaces	May share kernel privilege	Runtime load/unload	Maintainability is not fault isolation
Hybrid	Deployment varies by subsystem	Label each actual domain	Mixed module/server	Name determines neither cost nor security

Layers describe dependencies, modules organization/linking, and protection domains access control. Separate files/modules create no automatic user/kernel boundary. Stacked boxes do not prove acyclic dependencies.

Isolation and recovery are separate

A file server crashes after receiving append R but before replying. It may have crashed before commit or committed and lost the reply. Retrying repairs the first history but duplicates the second. Memory isolation alone cannot distinguish them.

Recoverable request IDs and dedup/result records can return the same result on retry. Records must persist atomically with the append or crashes can still break consistency. Limit device, memory, and handle authority. A server with unrestricted disk writes can corrupt disk data even when unable to overwrite kernel memory.

Module unloading separately must stop new entry and wait for references, DMA, callbacks. Removing a symbol-table entry does not stop old function pointers. This differs from restarting a user-space server.

Historical hybrid systems

The textbook's Mac OS X has Aqua/Cocoa above a kernel environment containing Mach, BSD, I/O Kit/extensions. Mach's presence does not imply all BSD services are separate user processes. The historical iOS stack has Cocoa Touch, Media Services, Core Services, Core OS.

The historical Android stack is applications→framework→libraries/Dalvik→Linux kernel. Dalvik is the textbook runtime, not a timeless claim about all Android. Linux supplies process, memory, device services without implementing every Java API's high-level meaning. Compare the contract, domain, boundary of each component, not stack names or assumptions about later versions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Evidence at failure time

A user core preserves some process state; a kernel crash dump preserves kernel failure state. Logs contain selected events; traces expose recorded calls, scheduling, I/O ordering. None alone reconstructs every past event.

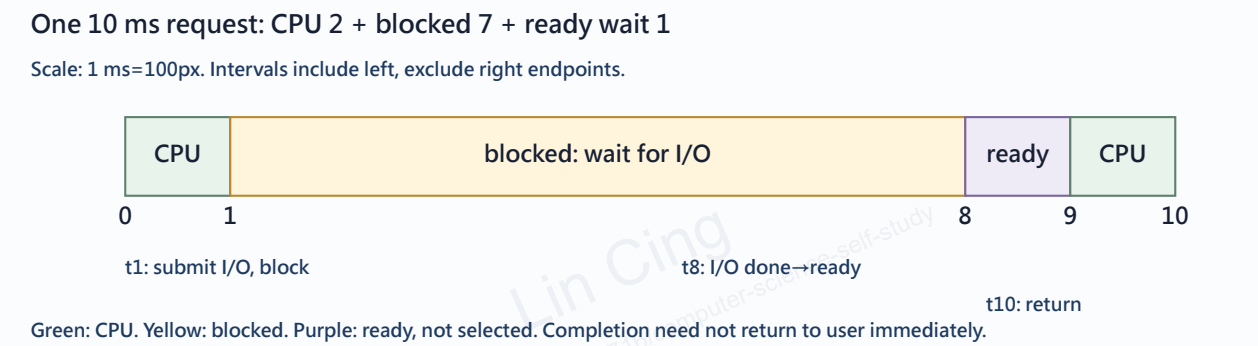
Evidence	Can answer	Cannot alone establish
PC, registers, stack	Failing instruction/call chain	First corruption happened there
Object/heap snapshot	Values at crash	Who changed them and when
Timestamped trace	Recorded order and timing	Unprobed events never occurred
CPU sampling	CPU-active stacks	Every long I/O wait was sampled

A broken filesystem may also fail when asked to store its kernel's dump. Reserve a reliable dump path/space and retrieval procedure. Collection failure does not establish absence of failure.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Decompose a 10 ms request

Assume comparable monotonic time and no lost events. Handler starts0, blocks for I/O at1, becomes ready at8, runs at9, returns 10. CPU[0,1)+[9,10)=2 ms; blocked[1,8)=7 ms; ready wait[8,9)=1 ms; total10 ms.



Seven blocked milliseconds are not disk CPU time. Device completion at8 is not return at8. Probes can change scheduling, so compare observable behavior with uninstrumented execution. For1,1,1,1,1,1,100 ms, mean107/8=13.375 ms. Nearest-rank p99 uses ceil(.99×8)=8 and yields100 ms. This small sample's order statistic is not a stable population-tail estimate.

Instrumentation can conceal a race

From x=0, T1: r0→T2: r0→T2: w1→T1: w1 produces1; serial increments produce2. printf can serialize the execution without repairing synchronization. Correctness requires indivisibility for all allowed interleavings, through a mutex covering the whole RMW or a suitable atomic RMW. Locking writes alone leaves both initial reads0 possible.

Dynamic probes must retain event IDs, time domain, drop counts, and necessary object identities. State assumptions about ordering per-CPU timestamps.

System generation and dependency closure

SYSGEN can select source at compile time, precompiled components at link time, or runtime functions through tables/discovery. These trade image size, portability, hardware-change cost, and early availability. Recompilation alone does not classify the design.

Configuration	Requirement	Failure if missing
CPU/ABI	Compatible kernel/user ISA/ABI	Unsupported instruction or exec format
RAM/reservations	Avoid firmware/device-reserved ranges	Corruption or invalid mapping
Boot storage	Controller driver reads root device	Root device unavailable
Filesystem	Root-format driver available early	Device exists but mount fails
Root/init	Correct root, executable init, loader	Mount works but userspace cannot start

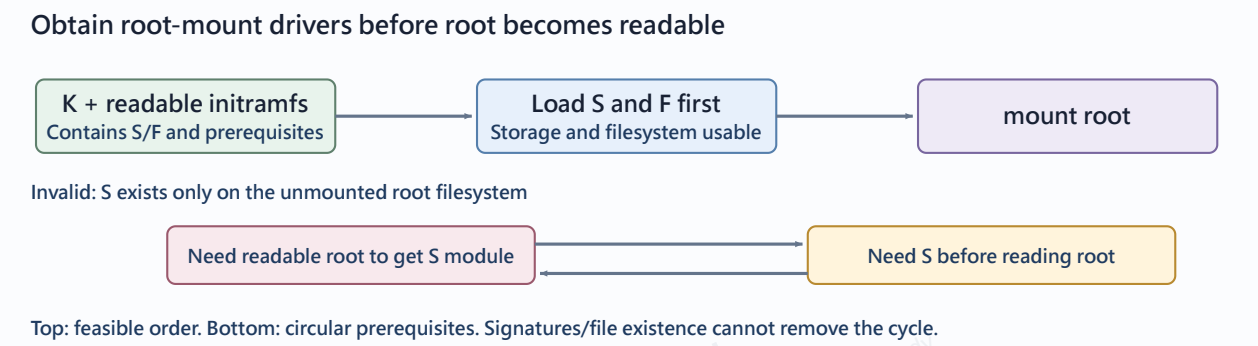
Discovery still needs a usable driver. A module's existence establishes neither version/ABI compatibility nor its availability before use.

Root drivers cannot exist only on inaccessible root

Root resides on storage S with filesystem F. Other dependencies are built in. B means built-in, I an accessible compatible initramfs module, R root-only.

S	F	Initramfs	Root mount reachable?
B	B	Neither needed	Yes
I	I	S, F, and all prerequisites	Yes, load first
R	B	S absent	No, need root to obtain root driver
B	R	F absent	No, device exists but format unavailable
I	I	Only F	No, closure incomplete

Root drivers cannot exist only on inaccessible root (continued)



Start from built-ins and readable initramfs components. Repeatedly add components whose prerequisites are all available. If mount-root never enters the set, a dependency is absent or cyclic. Closure only excludes this failure class; hardware, driver logic, and root contents still require correctness.

Reset to init

Assume firmware finds a loader, which loads kernel/initramfs and provides memory map/command line. These are roles; BIOS single-sector and UEFI implementations are not one identical flow.

Stage	Input	Next-stage premise
Reset/firmware	Reset entry, hardware	Usable CPU/RAM, executable loader
Loader	Configuration, images	Correct placement and parameters
Early kernel	Boot/hardware information	Tables, interrupts, scheduler, memory
Root setup	Storage/filesystem drivers	Mounted root, readable user files
Userspace init	Image, interpreter, libraries	Services/login under their policies

Linuxv3.2 start_kernel initializes early infrastructure; rest_init creates the task running kernel_init. It does not directly launch an arbitrary shell. Kernel_init completes initcalls/root preparation and tries the appropriate userspace init. Executable initramfs `/init` can run first; otherwise root namespace preparation is required. Failure points leave different diagnostics.

Locate the failed handoff

Use role gates F→L→K→R→I for firmware, loader, kernel/root access, readable init, and successful exec.

Fault	Last successful stage	First failing action	Incorrect diagnosis
Loader cannot find kernel	L	Load K	Init services failed
Missing S driver	K	Read root device	Only a misspelled root path
Missing F driver	K	Mount existing device	Disk absent
Missing init interpreter	R	Exec init	Kernel never started

This role table does not claim every Linux failure returns one exit code to firmware. Panic, loader error, and userspace nonzero exit are different failure paths.

Verification, measurement, and compatibility

Verified boot has a trusted predecessor authenticate/authorize the next stage and refuse execution on failure. Computing or recording a digest is measurement without automatic enforcement. Policy must name covered images/configuration/initramfs. A kernel filename alone is insufficient.

A validly signed old/revoked version may still be unacceptable, requiring rollback/revocation policy. Signatures do not remove authorized code's vulnerabilities. ABI compatibility and dependency closure support compatibility, not provenance. This abstract trust chain is separate from Linuxv3.2's control flow; it does not claim mainline3.2 implements every policy or that init immediately drops all capabilities. Service authority depends on actual work and security policy.

One read crosses several contracts

The4096 B cold-read API supplies fd, buffer, count; ABI selects registers; controlled entry establishes kernel context; validation and VFS/filesystem resolve objects; driver/DMA fills cache; the kernel copies4096 B and returns. Function calls, mode transitions, AS handoffs, CPU copies, and DMA are distinct events.

A microkernel can retain the API while adding internal IPC. GUI replaces the frontend without removing read protection. Modules change loading without automatically reducing privilege. Returned bytes mean this read's data is available, not that later writes are complete or durable. Pipe EOF requires empty buffers and no writers; missing server replies do not prove nonexecution.

Follow each object’ s contract and state across handoffs.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapters 1–2. write(2); fsync(2); pipe(7); Linux 3.2 init/main.c.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Program, process, and PCB

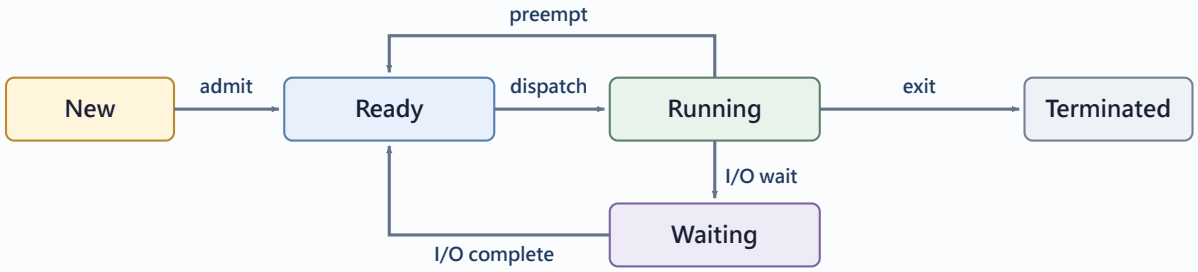
A program describes executable instructions/data. A process is one identified execution with resources and state. Two launches of a counter image, P100 and P101, can share read-only physical text while retaining private globals, heap, stack, registers, and descriptor tables.

State	P100	P101	Sharing condition
PID, PC, r1	100,0x40,7	101,0x120,3	Execution context is independent
Private counter at VA 0x5000	0→1	0→1	No shared mapping, no combined increment
fd3	OA→file A	OB→file B	Same number, different object
Code	Read-only image text	Same text	Frames can be shared without sharing execution

The PCB tracks identity, state, PC/registers, scheduling, and memory/I/O references. It can reference page tables and open descriptions without embedding the entire heap. Multithreaded execution separates additional thread context in Chapter 4.

Five states and their causes

Event completion makes Ready; dispatch supplies the CPU

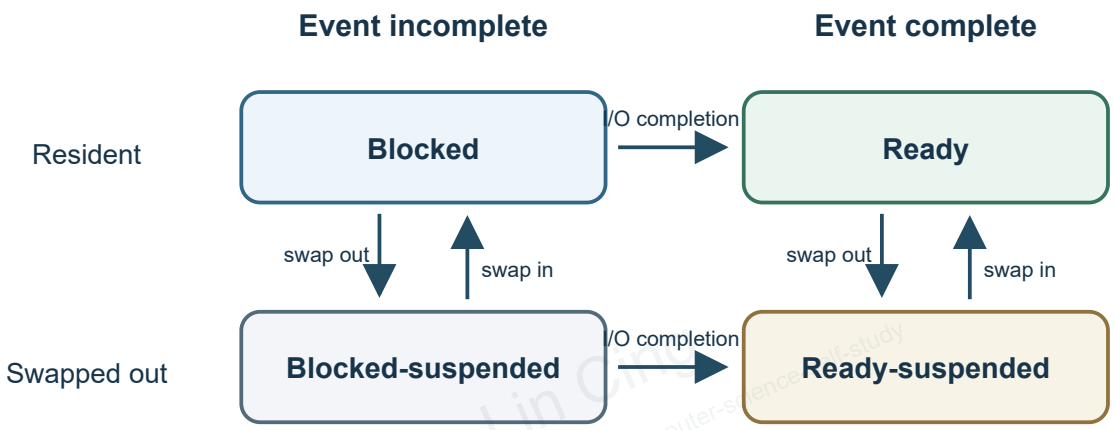


Arrows: legal state transitions. One CPU runs at most one process. Mode changes need not leave Running.

Assume one CPU and one execution stream per process. At most one process is running. Ready means executable but awaiting CPU; waiting means an event is missing. This abstraction does not enumerate every OS substate or imply that terminated metadata has disappeared.

Admission is new→ready; dispatch ready→running; preemption running→ready; blocking running→waiting; event completion waiting→ready; exit running→terminated. I/O completion normally establishes readiness; policy determines immediate execution. An interrupt can wake a waiter without first running the waiter itself.

Waiting outside memory: a seven-state model



Ready → dispatch → Running; ready-suspended must first be activated

This is a teaching model of whole-process swapping. **Blocked** means the event is incomplete; **suspended** means the process is temporarily ineligible to run in memory. These conditions are independent. Modern virtual memory can reclaim individual pages rather than swapping an entire process.

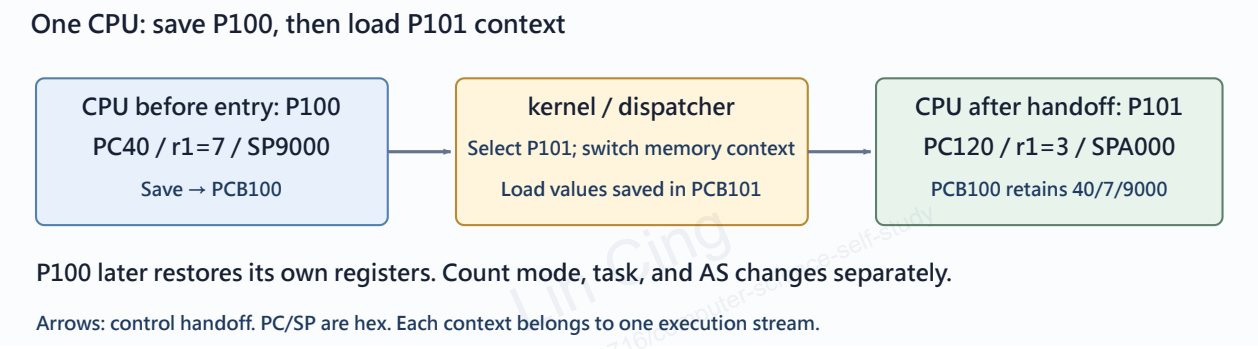
Completion does not mean immediate execution

Time	Event	State of A	Can A receive the CPU?
0	A starts I/O	blocked	No: the event is incomplete
1	Scheduler swaps A out	blocked-suspended	No: incomplete and swapped out
2	I/O completes	ready-suspended	No: it still needs activation
3	A is swapped in	ready	Not yet; awaits selection
4	Dispatcher selects A	running	Yes

Another process may run during times 1–3. If A is swapped in before I/O completes, its path is blocked-suspended → blocked → ready: the same eventual conditions, in the opposite order.

What a context switch preserves

P100 runs PC 0x40, r1=7, SP 0x9000. P101 has saved PC 0x120, r1=3, SP 0xA000. Entry first preserves ISA-required state. The kernel associates the resumable context with P100, then loads P101 and its memory context. Execution resumes at P101's PC, not P100's.



Moment	CPU ownership	Saved/loaded state
Before entry	P100	40/7/9000
Save P100	Kernel	PCB100 retains resumable 40/7/9000
Select/load P101	Kernel→P101	120/3/A000 and correct memory context
Later select P100	P100	Its own context, not P101's registers

What a context switch preserves (continued)

A syscall can change mode without changing process. Switching threads in one process can change registers/stack without user-AS change. Saved PC/restart semantics depend on ISA and event; not every event repeats its instruction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Queues, scheduler, and dispatcher

Ready queues contain runnable work; device/event queues contain waiters for predicates. Admission controls the resource-constrained job population. The short-term scheduler selects ready execution, and the dispatcher performs context/mode handoff. Long-term admission adjusts multiprogramming; medium-term suspension can remove work temporarily. An OS need not have separate components with all these names.

P1 runs, P2 is ready, and disk R1 is incomplete. P1 submits R1 and waits; P2 runs. Completion makes P1 ready, but a nonpreemptive policy can let P2 continue. Duplicate completion must not duplicate queue membership. A running task must not retain a dispatchable copy that another CPU could run simultaneously.

With 10 ms useful work and 1 ms scheduling/dispatch per cycle, the overhead fraction is 1/(10+1)=1/11. The ratio 1/10 instead measures added cost relative to useful work. This model does not measure cache/TLB disruption.

Lost wakeup: checking again is not enough

Even if a waiter registers, unlocks, and observes WAITING, completion can occur after that observation but before the actual sleep.

Time	Waiter	Completion handler
1	Reads WAITING after unlock	—
2	Has not slept	Sets `done=true` and wakes it
3	Acts on the old decision and sleeps	No further completion

Without coordination with the scheduler, the waiter may sleep forever. The protocol must make release-and-park atomic relative to wakeup, or use an equivalent scheduler-state protocol that preserves an early wakeup.

Connect the predicate to an atomic wait

Protect the predicate and wait queue with the same lock `L`. The waiter acquires `L` and, while `done=false`, registers and calls `park_and_release(L)`; on return it reacquires `L` and checks the predicate again. The completion handler acquires `L`, sets `done=true`, wakes waiters, and releases `L`.

`park_and_release` is an abstract primitive: relative to the handler holding `L`, releasing the lock and committing to wait form an atomic handoff; a wakeup during the handoff must be preserved. It is not ordinary `unlock(); sleep()`. A condition variable with this contract or a kernel wait-queue/scheduler protocol can implement it.

Here the completion handler executes in a context allowed to acquire `L`. A hardware interrupt handler cannot use a sleeping mutex or condition variable; use interrupt-safe locking and wakeup, or defer the work to an appropriate context.

If completion acquires the lock first, the waiter sees `done=true` and does not sleep. Otherwise the handler runs only after registration and the atomic handoff, so it can wake the waiter. Wakeup grants eligibility, not immediate execution. The while loop also handles spurious wakeups or another thread changing the predicate.

CPU-bound and I/O-bound are demand patterns

A alternates CPU 1 ms/I/O 9 ms; B needs CPU 10 ms. One CPU can run B during A's independent I/O. Admitting more work whenever CPU utilization is low can overload I/O queues or memory instead. Admission must consider complete demand and queueing.

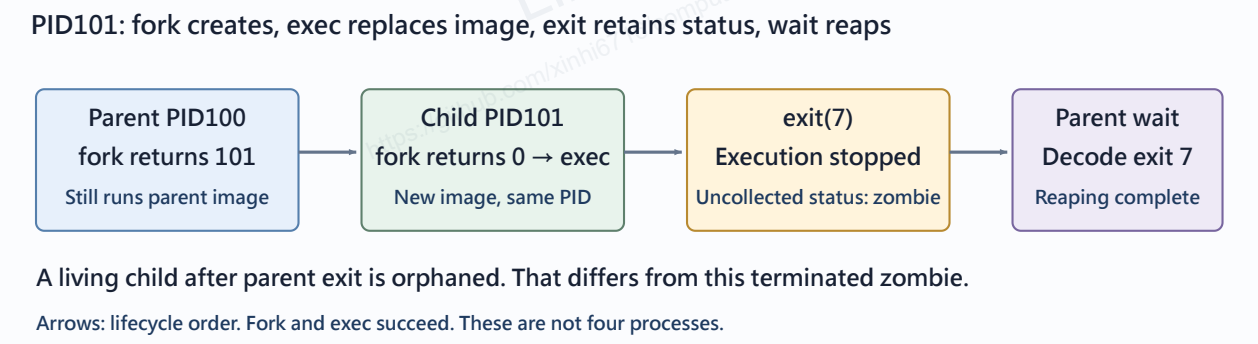
A larger ready population need not improve throughput. Working sets exceeding frames can cause thrashing and more faults. Suspension/resumption is medium-term resource control, not each CPU selection. Chapter 6 defines waiting, response, and turnaround on common workloads.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Fork, exec, exit, and wait

Fork creates a child with its own task/address-space metadata and required resource references. Private data pages can defer copying with COW. Exec replaces the caller's image; success does not return to its old call site and creates no additional process. Exit ends execution and releases most resources. Wait retrieves child status and completes reaping.

Parent 100 forks 101: parent returns 101, child 0. Child execs worker but remains 101. Worker exits 7; collectible status remains until wait. The parent checks normal exit and decodes 7. Zombie means terminated but unreaped; orphan means the parent relationship has ended, and the child may still run. Reparenting depends on OS, namespace, and subreaper policy. Parent termination can trigger cascading termination or reparenting under the actual system contract.



Two unconditional forks produce four processes

Assume successful forks, no early exit, and every current process executes both unconditional lines. P0 first creates P1. On the second line, P0 creates P2 and P1 creates P3.

Completed point	Executing processes	New children	Alive
Initial	P0	None	1
Fork1	P0	P1	2
Fork2	P0, P1	P2, P3	4

For k such forks, N(k+1)=2N(k), N0=1; induction gives 2^k. Parent-only continuation, child exec/exit, or failure breaks the premises. Only the original parent's second fork gives three total. The tree does not fix PID allocation or print order.

Inherited descriptors can share an open description

Parent fd3 points to O1 at offset 0. Fork copies the fd table entries but can preserve their shared O1 reference. Parent read 100 advances O1 to 100; child read 50 starts at 100 and advances to 150. A separate open of the same inode creates O2 with an independent offset, possibly 0. This differs from private-memory COW.

Fork failure is an error, not the child branch. Exec failure usually returns an error, but after an implementation's point of no return the old image cannot universally be guaranteed intact. Repeated unwaited child exits accumulate status records. General memory/handle release differs from retaining minimal reap metadata.

A multithreaded fork preserves only the calling thread in the child. Copied mutex state can have no surviving owner. Chapter 4 restricts operations before exec; the single-thread flow cannot simply be reused.

Shared memory and message passing

Shared memory establishes common backing/mappings and then exchanges data through loads/stores. It still needs authority and synchronization. Message-passing contracts determine copying, waiting, capacity, and delivery. They do not imply cross-machine execution, zero-copy, or exactly two copies.

Contract	Options to state	Risk if confused
Naming	Process/endpoint or mailbox	PID reuse, rights, cleanup
Synchronization	Each send/receive blocking or nonblocking	Send may await queueing, not processing
Capacity	Rendezvous, bounded, ideal unbounded	Backpressure or finite physical-resource limits
Data boundary	Messages or byte stream	One read need not be one record
Lifetime/failure	Close, unlink, remove, peer crash	EOF, error, and queue removal differ

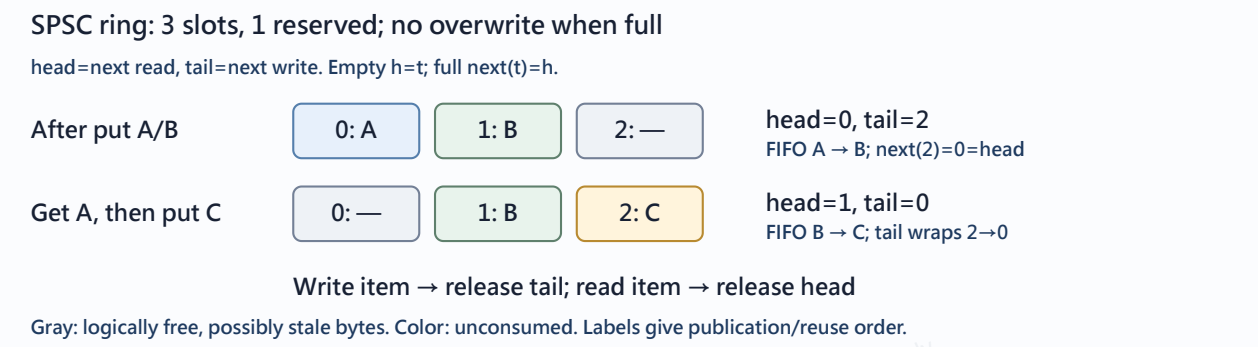
A capacity 2 channel holds complete J1/J2; J3 waits while full. Receiving J1 permits J3. Blocking APIs do not imply zero-capacity rendezvous if send completion means queue insertion.

Three physical ring slots hold two items

One producer and one consumer share slots[3] and atomic head/tail, initially 0. Head is next to consume, tail next to fill. Reserve one slot. Empty means head=tail; full means next(tail)=head; next(x)=(x+1) mod 3.

Event	Head	Tail	Pending FIFO	Result
Initial	0	0	[]	Empty
Put A at 0	0	1	[A]	Publish tail 1
Put B at 1	0	2	[A, B]	Full, next(2)=0=head
Attempt C	0	2	[A, B]	Must not publish overwrite
Get A at 0	1	2	[B]	Release head 1
Put C at 2	1	0	[B, C]	Wrapped, still full
Get B then C	0	0	[]	Empty

Three physical ring slots hold two items (continued)



Publication and reuse need two ordering arguments

Producer acquire-loads head, checks not full, writes slots[tail], then release-stores new tail. Consumer acquire-loads tail, checks not empty, reads slots[head], then release-stores new head. Producer alone writes tail; consumer alone writes head. Local indexes and cross-party loads follow the stated atomic rules. Process-shared atomics require platform support; mapping an arbitrary C++ atomic object across processes is not itself a portable guarantee.

First, slot write precedes release tail, and the consumer's corresponding acquire makes the completed item visible. Second, slot read precedes release head, and the producer's acquire permits reuse only after consumption. Successful put/get increases/decreases logical size by one. Empty/full checks preserve $0 \le size \le 2$ and FIFO.

Plain shared integers introduce language data races and invalid ordering assumptions. Publishing tail before data permits stale reads. Multiple producers compete for the same tail/slot and violate SPSC. Finite index traces do not replace the general release/acquire argument.

Frame records over a byte stream

Each record has a 2 B big-endian length and payload, max 8 B. Payloads ABC and DE form 00 03 41 42 43 00 02 44 45,9 B. Reads can return [00],[03 41 42],[43 00 02 44 45]. These are not three messages.

Accumulate two header bytes to decode 3, then collect ABC. Decode the next length 2 and collect DE. EOF inside a header or payload is truncation, not an empty completed message. Validate length ≤ 8 and allocation arithmetic before receiving untrusted-sized data. Pipe/TCP ordered streams do not provide application records or atomic remote procedures. Datagram/message truncation and boundaries follow their own APIs.

Shared names, descriptors, and mappings

Creator makes `/course-ring`, sets length 4096 B, and maps MAP_SHARED. Consumer opens/maps the name. P1's base0x100000 and P2's0x700000 can reference the same backing offset 16. Store offsets/indexes, not P1 absolute pointers intended for P2.

Closing a descriptor does not unmap existing mappings. shm_unlink removes the name, preventing new opens through it, while existing references/mappings can survive. MAP_SHARED does not publish completed initialization. A consumer accessing beyond valid object length before sizing is not merely reading an ordinary zero-filled buffer.

Put magic, version, length, and offsets in the region; complete initialization before a synchronized ready notification. Sleeping one second is no substitute for publication.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Removal and drain differ

Object	Identifier	Removal	Existing users
POSIX message queue	Name/descriptors	mq_unlink removes name now	Open references survive until final close
System V queue	msqid/key	IPC_RMID removes queue	Blocked send/receive wake with EIDRM
Custom drain channel	Application handle	Close sends, drain, destroy	Only its specified state machine applies

Removal does not universally await consumers draining. Unlink does not automatically send every receiver EOF. Termination and errors require application protocols.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sentinel termination

Use capacity 4 FIFO, one producer/consumer, positive jobs, and 0 reserved as STOP. Send 1,2,3, stop accepting new jobs, then send 0. Queue states are[1]→[1,2]→[1,2,3]→[1,2,3,0]. Consumer completes the jobs before STOP, acknowledges termination, and manager cleans up after acknowledgment. This is not kernel-provided drain.

If 0 is a legal job, use tagged messages. With two consumers, one STOP can leave the other waiting forever; use two sentinels or a closed/broadcast protocol. A crashed consumer can prevent acknowledgment forever, requiring recovery/reassignment policy. One normal trace does not establish exactly-once effects.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mach port and reply rights

In the textbook's historical port/mailbox model, the server has receive right and clients need send rights. Headers specify destination/reply port; typed bodies carry data or controlled rights. Knowing a local port name grants neither global pointer identity nor receive authority.

A sends A1 then A2, and B sends B1. A1, B1, A2 or B1, A1, A2 are allowed, but A2 must not overtake A1 from the same sender under this model. Different senders lack automatic global order. Replying to A1 leaves other requests pending; send success does not equal reply success.

A full queue can wait, time out, or fail under its selected contract. Receiving requires rights. Out-of-line VM/COW can reduce large-data copies without making all inline headers or cross-host messages zero-copy. Rights and shared-memory lifetimes remain protocol obligations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Historical Windows ALPC connection

A server publishes a connection port. An authorized client handle sends a connection request; acceptance creates private communication endpoints for both directions. Connection setup differs from repeatedly treating each payload as a new connection.

The textbook distinguishes copied small messages from shared-section large data using a historical 256 B threshold. This is not a fixed modern Windows limit. A 128 B request fits the small-message model; a 4096 B payload can use a prepared section with offset/length control. Validate mapping bounds, rights, and writers; a client pointer is untrusted. Applications commonly reach this local mechanism through public RPC interfaces. ALPC, Mach, and POSIX queues share questions of naming, rights, waiting, data, and replies but not interchangeable lifetime contracts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sockets and RPC stubs

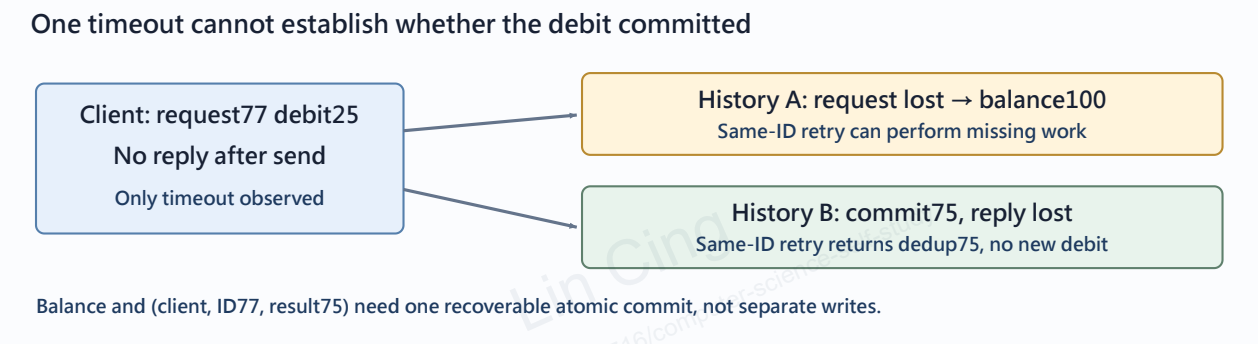
Sockets identify endpoints through address family, local/remote addresses, and ports. A stream server bind/listen/accepts; accept returns a connected endpoint while the listener accepts more. A port is not a PID. Connect success does not establish application-operation completion.

A client stub encodes procedure/request IDs and parameters. A server stub validates/decodes, invokes service, and encodes results. Native endianness, alignment, widths, and pointers are not a wire format. Use id 32/opcode 16/amount 32, all big-endian. ID 77, opcode 1, amount 25 produce 00 00 00 4D | 00 01 | 00 00 00 19,10 B. Validate length/opcode before decoding amount. A transmitted client pointer does not grant memory access.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One timeout, two histories

Balance starts at 100; request 77 debits 25. In history A, the request never arrives and balance stays 100. In B, the server commits 75 but loses the reply. The client sees timeout in both. Retrying with a new ID makes B become 50; not retrying leaves A unperformed.



Deduplicate by authenticated(client identity, request ID). Commit balance and recorded result atomically in one recoverable transaction. The first request 77 changes 100→75 and records 75; duplicate 77 returns 75 without another debit. Concurrent duplicates need a transaction/unique constraint, not independent absent checks followed by two debits.

At-most-once is separate from availability

Dedup records can bound an ID's effects to at most one during retention. Debit then separately record ID leaves a crash window. RAM-only records disappear on restart. Expiring a record while retries remain possible needs an epoch/retention agreement; identities and ID spaces cannot be arbitrarily reused.

At-most-once permits zero execution after permanent loss. At-least-once needs retries and eventual reachability but can repeat non-idempotent effects. Exactly-once requires explicit transaction, durable record, ID scope, failure, and communication assumptions. Reads or setting a value may be idempotent; debit 25 is not.

Anonymous pipes/named FIFOs support local stream inheritance/naming; sockets generalize endpoints across hosts. All still need framing, backpressure, and peer-failure handling. IPC costs also include waking, synchronization, references, and retries.

Execution and data lifetimes

Capacity 2 holds J1/J2. P waits trying J3. C receives J1, frees a slot, and wakes P. P becomes ready and sends J3 after selection. FIFO/capacity remain valid; waking is not immediate execution. Receiving an item is not durable job completion; process exit is not parental reaping.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Identities and safety conditions

PID/PCB identifies execution, fd entries reference open descriptions, shared offsets identify data, and request IDs identify retry effects. Equal numbers or paths do not replace references/lifetime proofs.

Correctness requires matching saved/restored execution contexts; one synchronized wait predicate; write-before-publication and read-before-reuse; only complete valid framed records; and joint RPC effect/result commit. Numerical traces expose intermediate states. General invariants explain additional allowed events. Finite testing does not prove every interleaving/crash. New APIs, participant counts, memory orders, or persistence strategies require rechecking their premises.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 3. read(2); pipe(7); msgctl(2); mq_unlink(3).

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Private context and shared resources

A thread has its own PC, registers, stack, scheduling state, and identity. Threads usually share process code, globals, heap, address space, and open resources. One bad pointer can corrupt another thread's data. Separate stacks describe logical ownership, not hardware isolation within one address space. Another thread can access a passed stack pointer while its object remains alive.

Worker data	W1	W2	Coordination
Register/local index	Own index 0	Own index 0	Independent context
Scratch	B1 holds 110	B2 holds 120	Disjoint ownership
Shared queue	Takes job 11	Takes job 12	One protocol prevents duplicate removal
Global total	Adds 110	Adds 120	Atomic/mutex, or one reducer after joins

Responsiveness can come from a UI thread continuing while workers await I/O. Sharing can reduce some process-creation costs. Scalability needs cores and separable work. These are conditional benefits, not fixed speed factors. More threads do not remove locks, bandwidth limits, or serial dependencies.

Globals, TLS, and lifetime

Unsynchronized increments of one non-atomic C/C++ counter are a data race and undefined behavior. Read 0/read 0/write 1/write 1 is an abstract lost-update witness, not the language's unique output.

With per-thread TLS counters, each independently goes 0→1. Each worker copies its value into a distinct result slot that stays alive before it exits. Main joins both workers successfully, then reads the slots and sums 2; it does not access a destroyed TLS object. TLS persists across functions for the thread's lifetime, unlike one function's automatic local. A pool worker does not reset TLS per job automatically; initialize and clean it at job boundaries. A mutex cannot make a pointer to an ended thread's stack-local result valid.

Concurrency and decomposition

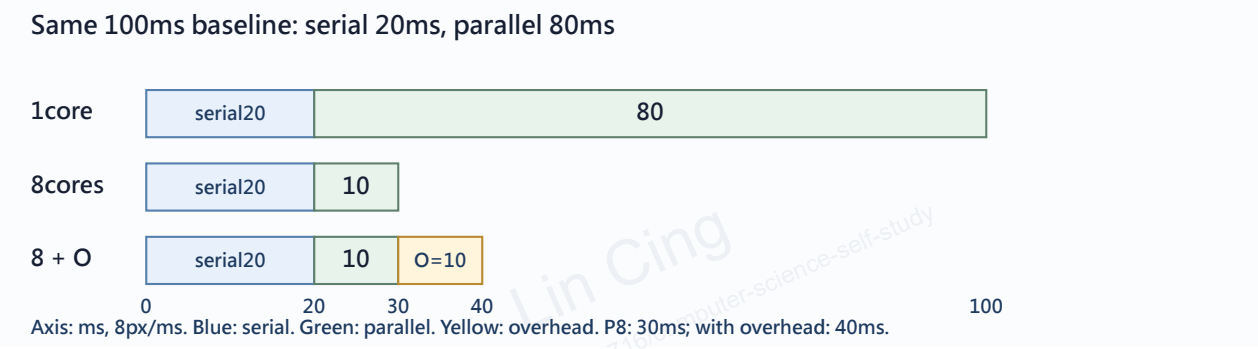
Concurrency permits overlapping or interleaved work, even on one CPU. Parallelism requires simultaneous hardware execution resources. SMT contexts share core resources; logical CPUs are not necessarily independent full cores.

Data parallelism applies one operation to partitions. Task parallelism overlaps functions or stages. Decomposition must address partitioning, balance, dependencies, sharing, and testing. A 64 MiB array of 32-bit integers contains 16,777,216 elements. Eight partitions each contain 8 MiB or 2,097,152 elements. Each worker uses a sufficiently wide 64-bit accumulator. Seven additions combine eight partials. Every element must still be read.

Per-element global atomic addition instead performs 16,777,216 shared RMWs. Publishing eight partials changes operation counts, not a measured speedup. Bandwidth, caches, and NUMA placement still matter.

Amdahl uses baseline time

For the same fixed problem, T1=100 ms contains 20 ms serial and 80 ms ideally divisible among P cores, without overhead. With s=0.2, $T_P = 100[s + (1 - s)/P]$, and speedup is $1/[s + (1 - s)/P]$. At P=8, T8=30 ms, speedup 10/3, efficiency 5/12. As P grows, speedup$\leq$5.

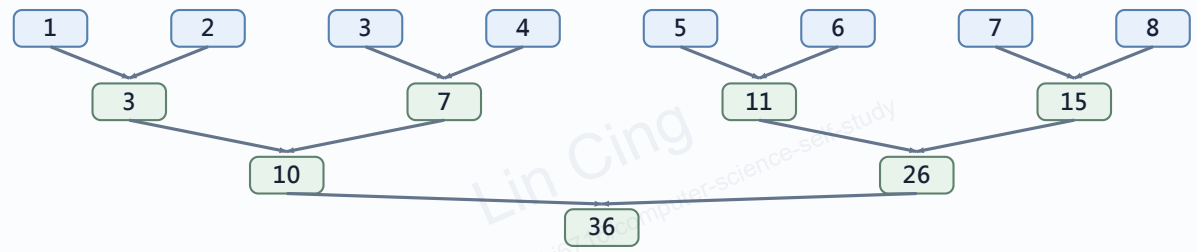


Adding 10 ms launch/synchronization overhead makes T8=40 ms and speedup 2.5. Eighty percent of operations need not mean 80% of baseline time. Eighty 1-cycle and twenty 10-cycle operations cost 280 cycles. Accelerating only the first group eightfold gives 10+200=210 cycles, speedup 4/3. Retain one denominator.

Work, span, and reduction

Eight leaves read values 1…8, and seven binary merges add them. Each leaf/merge costs one unit. Work=15. The longest path leaf→pair→quartet→root has span 4. Every P-processor schedule has $T_P \geq \max(15/P, 4)$, so speedup≤15/4. Work/span is average parallelism, not a theorem that processors beyond 3.75 cannot help.

Sum 1…8: 8 leaves +7 merges; work 15/span 4, unit node cost



Arrows: data dependencies. Each merge waits for both children. Actual partial sums; TP≥max(15/P, 4).

Pair sums are 3,7,11,15; quartets 10,26; root 36. Exactly seven merges occur. Root must await both quartets. Unequal partitions impose a lower bound equal to the largest partition time, not the average.

False sharing differs from a data race

With 64 B lines, 4 B integers, and a 64 B-aligned base, W1 writes a[0] and W2 a[1]. Distinct objects may avoid a language data race while repeatedly transferring one coherence line's ownership. Separately aligned regions at least 64 B apart can reduce interference; measurement is required to claim improvement.

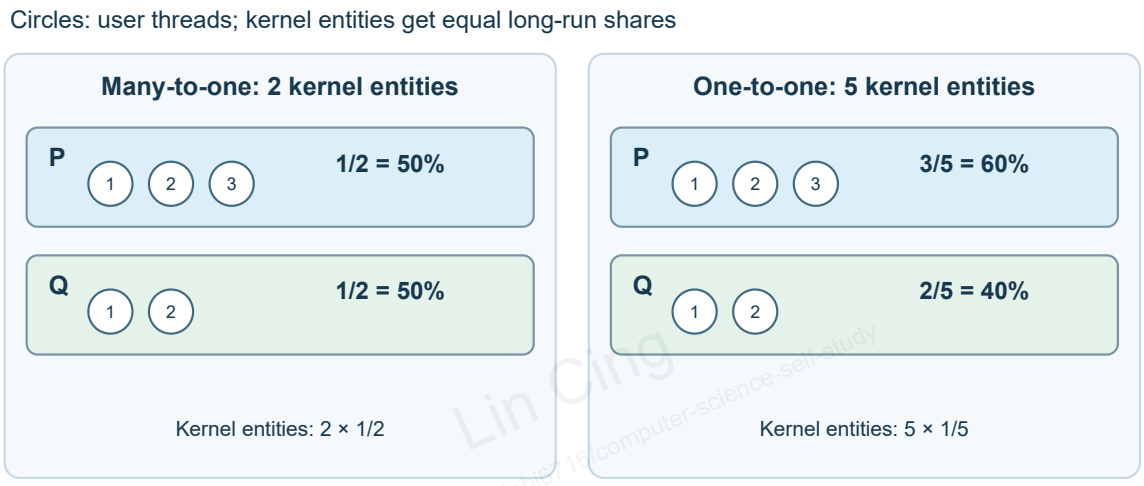
A mutex can add serialization. Replacing a shared counter with TLS changes abstract data and still needs reduction. Correctness, line-sharing cost, and result combination are separate obligations.

User-to-kernel mappings

Model	Mapping	Real blocking syscall	Parallelism
Many-to-one	Many user threads, one carrier	Without async substitution, blocks all execution there	One carrier
One-to-one	One kernel entity per thread	Other entities remain schedulable	Ready entities and cores
Many-to-many	Runtime multiplexes carriers	Depends on carriers and compensation	Runnable carriers and cores
Two-level	M:N with selected bound threads	Bound/unbound rules differ	Binding restricts runtime choices

A Pthreads API does not choose one universal mapping. Many user threads do not imply equal kernel-object counts. Nonblocking I/O and an event loop can leave many tasks logically waiting without blocking carriers.

Scheduling entities determine the modeled CPU shares



Assume one CPU, continuously runnable threads, equal priority, and long-run equal CPU shares among runnable kernel scheduling entities. Process P has three user threads; Q has two.

In this many-to-one example, each process has one carrier. The two kernel entities each receive 1/2, so P and Q get 1/2 each. With one-to-one mapping, five kernel entities each receive 1/5: P receives 3/5 in total and Q receives 2/5.

A different fairness rule changes the answer

Keep the single CPU and five continuously runnable threads. Now divide CPU equally between process groups P and Q first, then equally among threads in each group:

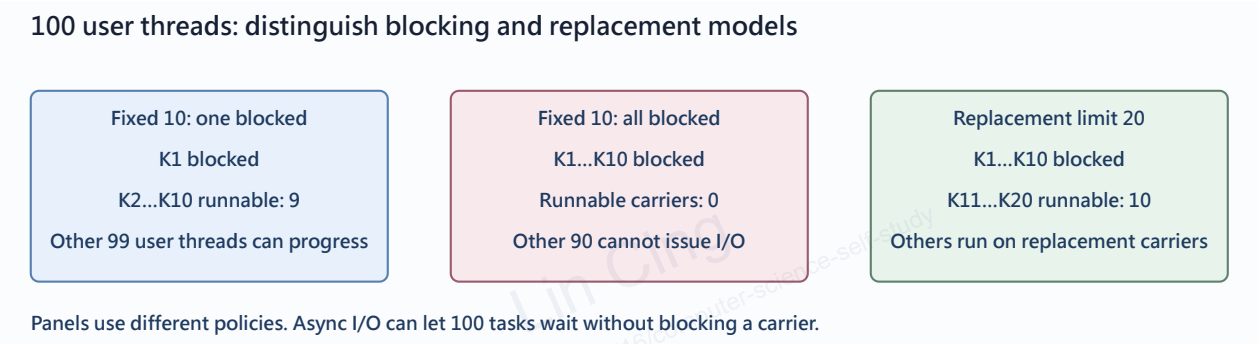
Policy	Total P	Each P thread	Total Q	Each Q thread
Five kernel entities get 1/5 each	3/5	1/5	2/5	1/5
Two process groups get 1/2 each	1/2	1/6	1/2	1/4

Over a modeled 60 ms, group fairness assigns 30 ms to each process; each P thread gets 10 ms and each Q thread gets 15 ms. This calculation excludes switching overhead.

Different weights, blocking, or multiple CPUs can also change the outcome. In many-to-one the runtime determines how to divide P's 1/2 among its user threads; the kernel share alone does not determine that division.

One hundred user threads, ten carriers

Fix K1…K10, no compensation or async I/O, and 100 threads each wanting blocking I/O. After U1 blocks K1, nine carriers remain. After ten block, none runs. The other 90 threads stay ready inside the runtime and cannot issue their syscalls.



A separate policy provides replacement carriers up to 20. K11…K20 can run after K1…K10 block and can issue additional blocking calls. Async I/O can let all 100 tasks wait while no carrier blocks. A ten-waiter bound is conditional, not universal M:N behavior.

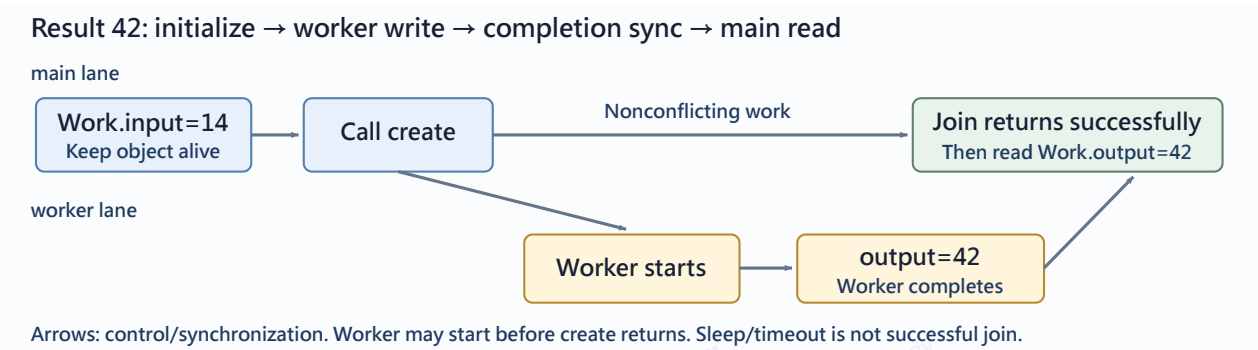
Historical scheduler activations/upcalls coordinate kernel resource notifications with runtime selection. They need rules for runtime locks, notification context, and allocation. They create no CPU capacity. Binding does not let one thread run on two carriers simultaneously.

Creation, completion, and join

Main owns Work{input14,output}; worker writes output=3×input=42. Initialize input before create, keep Work alive until successful join, and do not read output before completion synchronization. After create, the worker may already have started or finished.

Stage	Main	Worker	Result rule
Before create	Initialize 14	Absent	Valid object and arguments
After success	Nonconflicting work	May start, finish, write 42	No fixed parent/worker order
Worker complete	May await join	No longer uses Work	Publication follows library contract
Join success	Read 42, then end lifetime	Finished	No overlapping write

Creation, completion, and join (continued)



Join is not a fixed sleep. Sleeping 10 ms proves no completion. Detached threads need another lifetime/notification protocol. Detach does not authorize immediate parameter free.

Pthreads, Windows, and Java

Pthreads uses pthread_create with a routine and void* argument. Successful pthread_join waits for a joinable target and obtains its result. Handle its returned error number, not arbitrarily errno. Define join ownership; multiple joiners are not a generally safe broadcast. Never return a pointer to the worker's dead stack local.

Windows uses CreateThread and a handle. Read 42 after successful signaled WaitForSingleObject, not timeout or WAIT_FAILED. CloseHandle remains necessary and does not terminate the thread. Synchronization supplies ordering; volatile is insufficient.

Java 8 Thread.start creates execution; directly calling run is an ordinary method call on the current thread. Pre-start actions happen-before the worker. Worker actions happen-before successful join return. InterruptedException does not establish completion. API/language contracts and host mapping are separate layers.

Pools and fork-join

Eight 1-unit jobs on four FIFO workers with no overhead run J1–J4 in [0,1), J5–J8 in [1,2), makespan 2. Worker reuse avoids per-job creation, but bounded queues, backpressure, cancellation, and errors remain. Worker count and queue capacity limit different resources.

Fork-join splits independent subtasks and combines after completion. The sum tree gives 3/7/11/15→10/26→36. Too-fine granularity can make task, queue, and synchronization costs exceed useful work.

In a two-worker pool, A and B each submit A1/B1, then block waiting. Children are ready but no worker is free. This starvation deadlock can occur with an acyclic task DAG. Help-join/work stealing may execute dependencies while waiting, but arbitrary pools and Futures do not promise it.

OpenMP and GCD

OpenMP parallel-for partitions iterations. Independent c[i]=a[i]+b[i] can use distinct i; shared sum+=a[i] needs reduction/synchronization. Assume four workers actually receive [1,2], [3,4], [5,6], and [7,8]; their partial sums are 3, 7, 11, and 15. One possible merge is 10+26=36; OpenMP does not guarantee this partition or reduction order. Reduction is not arbitrary concurrent plain-sum writes. Floating-point reassociation can change final bits.

Historical GCD queues submit blocks to managed workers. A serial queue completes one at a time. Concurrent queues allow overlap; FIFO enqueue does not imply FIFO completion. A block on serial Q that synchronously submits another to Q and waits creates self-wait. Different queues do not automatically exclude shared-object access.

Runtime scheduling leaves dependencies, shared/private data, lifetime, and unsafe blocking to the application. A directive cannot erase loop-carried dependencies.

Multithread fork and signals

T2 holds M when T1 forks in a POSIX-style process. Child retains only T1, but copied M can appear owned by absent T2. A pre-exec library call needing M can stall. Before exec, use the required async-signal-safe operations. Atfork requires coordinated library/lock design and cannot universally repair unknown third-party locks.

Disposition is generally process-shared; mask is per thread. Synchronous faults target their causing thread. Process-directed signals go to an eligible thread under mask/implementation rules, not once per thread. Dedicated sigwait needs coordinated masks elsewhere. Signals and condition variables are different objects.

Successful exec replaces the image; other threads do not continue the old image. Ordinary thread creation shares it. These choices determine lock/resource inheritance.

Deferred cancellation

Successful pthread_cancel queues a request. The target may disable cancellation or not reach a point. Deferred cancellation acts at specified points. Asynchronous cancellation can interrupt broken invariants or lock ownership.

Worker owns M and buffer B and waits on a stable predicate in pthread_cond_wait. With cancellation disabled, establish resources and push unlock-M then free-B cleanup handlers. Enable deferred cancellation. Cancellation during wait first reacquires M, then runs LIFO cleanup, freeing B and unlocking M. Main successfully joins before reclaiming joinable state. Do not enable arbitrary cancellation before registration.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Cleanup invariants

State	M	B	Free count	Unlock count
Resources/handlers ready	Owned	Live	0	0
Cond wait	Atomically released	Live	0	0
Cancellation handled	Reacquired	Live	0	0
First cleanup	Owned	Freed	1	0
Second cleanup	Unlocked	Freed	1	1

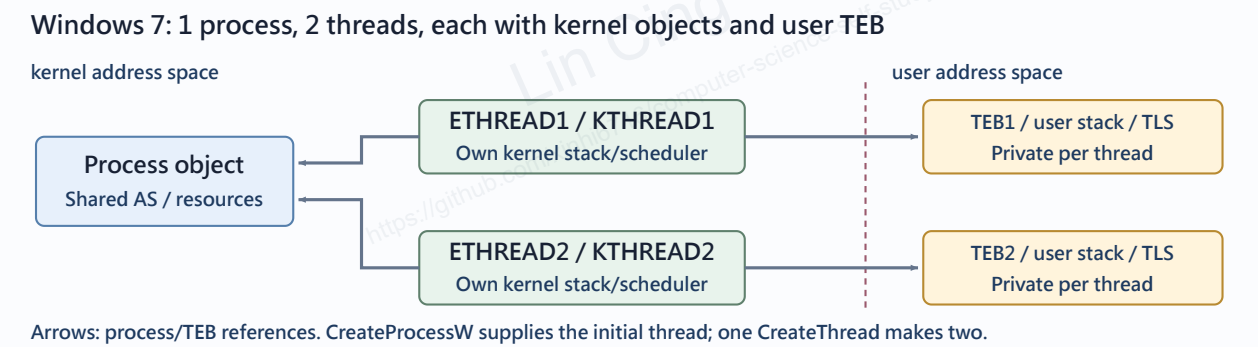
Normal execution also pops or cleans exactly once. Normal free followed by cancellation free is wrong. Releasing locks cannot repair arbitrary partial updates. Disable cancellation or avoid points while invariants are broken, or provide rollback. Include normal, error, and timeout ownership paths.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Windows 7 and Linux 3.2 objects

Fix the textbook snapshot. Windows 7 processes own address spaces/resources; threads have ETHREAD, KTHREAD, and user TEB. CreateProcessW creates an initial thread; one additional CreateThread gives two one-to-one threads. Session 0 system threads are outside this ordinary interactive-process example.

Linux 3.2 task_struct represents every schedulable task. Fork's copy_process→dup_task_struct creates a new task/kernel stack. Without CLONE_VM, copy_mm→dup_mm creates independent mm/page tables. Clone flags choose sharing of fs_struct, files_struct, sighand_struct, and mm.



Windows 7 and Linux 3.2 objects (continued)

Aspect	Windows 7	Linux 3.2
Process creation	CreateProcessW	fork
Thread-like creation	CreateThread	clone with VM,FS,FILES,SIGHAND,THREAD
Execution objects	Process and thread	One task_struct per stream
Private state	Registers, stacks, TEB, TLS	Kernel stack, scheduling, registers/stack
Shared state	AS and process resources	mm, files, fs, handlers per flags
Scheduling unit	Kernel thread	Task

One process/two streams gives Windows 1 process, 2 ETHREAD, 2 KTHREAD, 2 TEB, 1 AS, 2 scheduling entities. Ordinary Linux fork gives 2 tasks, 2 mm, 2 page tables. Specified thread-like clone gives 2 tasks, 1 mm, 1 file table, 1 handler table, 2 kernel stacks.

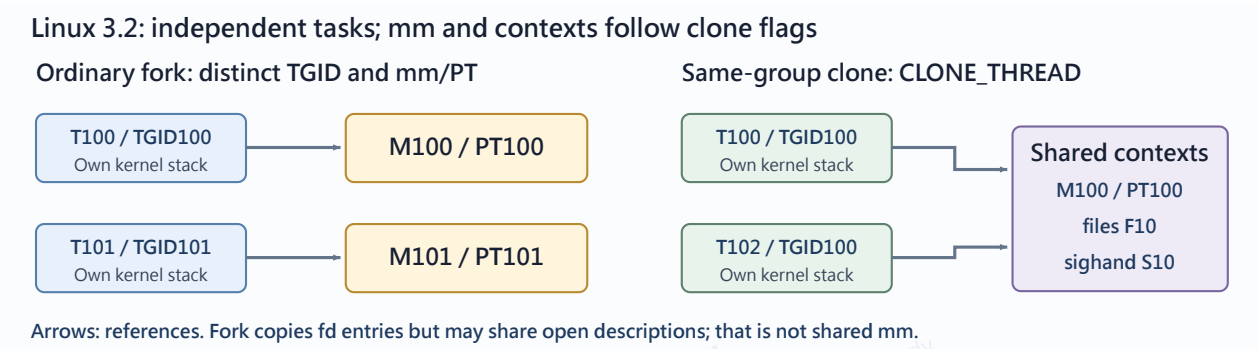
References and version counterexamples

ETHREAD references its process; KTHREAD tracks scheduling/synchronization; TEB is user space. The second thread owns independent stacks, registers, and TEB while referring to the same process/AS. Identity remains independent.

Fork creates child PID, state, stack, mm, and tables before private-page copying. COW sharing concerns physical frames, not a shared task object. Exec retains identity and changes image. Failure before destructive transition usually returns an error; some v3.2 failures after flush_old_exec terminate execution, so the old image is not unconditionally preserved.

Thread-group clone includes CLONE_THREAD, different TIDs, and one TGID. THREAD requires SIGHAND, which requires VM. VM/FS/FILES/SIGHAND with SIGCHLD may share resources without one thread group and is not a complete pthread implementation. Shared heap access still needs synchronization; for a still-shared COW page after fork, a write fault may require a private physical copy, but the kernel can sometimes reuse an exclusive frame.

References and version counterexamples (continued)



These counts describe fixed historical models, not today's kernel ABI guarantees.

Results, completion, and dependencies

Sum 1…8=36 can use one thread, four partition workers, or fork-join. Scheduling, objects, and synchronization costs differ. Read partials after joins, combine with the stated operation, and preserve lifetime.

Amdahl uses baseline-time fractions; work/span bounds a DAG; carriers bound kernel-schedulable streams. These are different bottlenecks, not proof that threads=cores is optimal or processors above average parallelism never help.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

When implementation assumptions change

Changing 1:1 to M:N requires blocking/compensation analysis. Changing scratch to global requires conflict analysis. Detach replaces join's lifetime/completion proof. Multithread fork adds inherited-lock risk. Cancellation adds resource and abstract-state recovery.

For measurements, fix inputs/results, cores/carriers, memory placement, and time denominator. Retain failures and wait locations; mean throughput can hide lock contention, pool exhaustion, and cleanup tails.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 4. Linux 3.2 fork.c; pthread_join(3); pthread_cond_wait(3p); C++ thread storage; OpenMP reduction.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Learning objectives

- Reason about mutual exclusion, progress, and bounded waiting
- Use semaphores for ordering and backpressure
- Identify deadlock, starvation, priority inversion, and lost wake-ups

Central question. How can arbitrary interleavings preserve shared state while allowing progress and bounded waiting?

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Race invariants and the complete counter trace

Producer executes R1=counter; R1=R1+1; counter=R1. Consumer executes R2=counter; R2=R2−1; counter=R2. The net serial effect is zero, but interleavings can leave 4,5,6 from initial 5. Treating counter++ as one step hides the conflict. This is an abstract interleaving model; unsynchronized non-atomic conflicting C/C++ accesses can be a data race with undefined behavior, not a guarantee of only these results.

Event	Producer	Consumer	Counter	Observation
T0	R1=5		5	Old value loaded
T1	R1=6		5	Only register changed
T2		R2=5	5	Same old value
T3		R2=4	5	Only register changed
T4	counter=6		6	Producer stores
T5		counter=4	4	Consumer overwrites

Swapping T4/T5 leaves 6; the intended result is 5. Mutual exclusion covers only agreed regions. Visibility, reordering, non-atomic widths, and failure retries need the same synchronization contract.

Race invariants and the complete counter trace (continued)

Identify all shared mutable locations, state cross-field invariants such as $0 \le size \le capacity$ or readers=0 or writer=0, identify operations temporarily breaking them, and define which observers must be excluded plus acquire/release visibility. Atomic stores to a list's next and size do not make their combined update indivisible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Safety, progress, and bounded waiting

Each of n processes P0…P(n−1) has entry, critical, exit, and remainder sections.

Requirement	Definition	Failure
Mutual exclusion	While Pi is in its critical section, no other participant is in its critical section	Concurrent writers break one invariant
Progress	If the section is empty and contenders exist, only non-remainder contenders participate in selection, and selection cannot be postponed indefinitely	No contender is selected
Bounded waiting	Every requester has a finite bound on other entries allowed before its admission	Global progress while one requester starves

An unfair exclusive lock can repeatedly admit high-priority arrivals without bounding a lower-priority wait. Progress with exclusion still permits indefinite overtaking. Conversely, admitting everyone can be fast yet violate exclusion. Safety must cover every reachable interleaving; progress and fairness need separate arguments.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Safety, progress, and bounded waiting (continued)

For enter_i/leave_i, prove exclusion at arbitrary relative speeds; finite-step admission progress when free with contenders; a finite overtaking bound per requester; and the atomicity, ordering, and failure recovery of entry/exit themselves. An overtaking bound counts other admissions and is not a wall-clock bound. Eventual entry also needs scheduling and termination assumptions. A finite sample of schedules can find counterexamples, not prove arbitrary n and speeds.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Peterson's complete protocol

There are exactly two processes, j=1−i, atomic flags initially false, and shared turn.

```
P_i:
  flag[i] = true
  turn = j
  while (flag[j] && turn == j) { wait }
  critical_section_i
  flag[i] = false
  remainder_i
```

Flag means competing. Turn resolves simultaneous contention, not permanent lock ownership. The slogan that the last turn writer always loses omits required contention premises.

Assume atomic sequentially consistent reads/writes. Suppose the two current critical-section attempts overlap. Let W0 be P0's turn=1 and W1 P1's turn=0. If W0<W1, P0's flag0=true preceded W0 and W1. Before P0 leaves, flag0 remains true; neither participant writes turn again inside this attempt. P1 therefore faces flag0=true and turn=0 when it would need to pass, making its while condition true, a contradiction. If W1<W0, symmetry blocks P0. Thus overlapping critical sections are impossible. This compares the current overlapping attempts' final turn writes, not arbitrary past iterations.

Peterson's complete protocol (continued)

For progress, one contender sees the other's false flag and enters. With both competing, turn selects one side whose while is not blocked by turn. A participant already inside eventually clears its flag on exit. For bounded waiting, blocked Pi has flag[j]=true and turn=j. Pj does not change turn in its critical section. After leaving it clears flag; any new attempt first writes turn=i, giving Pi priority. From the point Pi has completed flag[i]=true and turn=j, Pj can enter at most once before Pi, assuming relevant threads continue taking steps and critical sections terminate. Counting from the earlier flag write alone does not give this one-entry bound.

The proof needs shared order/visibility. Use corresponding atomics with memory_order_seq_cst, or a proved equivalent fencing protocol. Renaming ordinary variables flag/turn supplies no such proof.

Bakery algorithm: equal numbers still have an order

There are n fixed participants, each with at most one outstanding request. `choosing[i]` starts false and `number[i]` starts 0, meaning no request. For this teaching proof, all shared accesses are sequentially consistent atomic operations and ticket numbers do not overflow. Lamport's original paper considers a weaker read/write model.

```
choosing[i] = true
number[i] = 1 + max(number[0..n-1])
choosing[i] = false
for each j != i:
  while choosing[j]: wait
  while number[j] != 0 and (number[j],j) < (number[i],i): wait
critical section
number[i] = 0
```

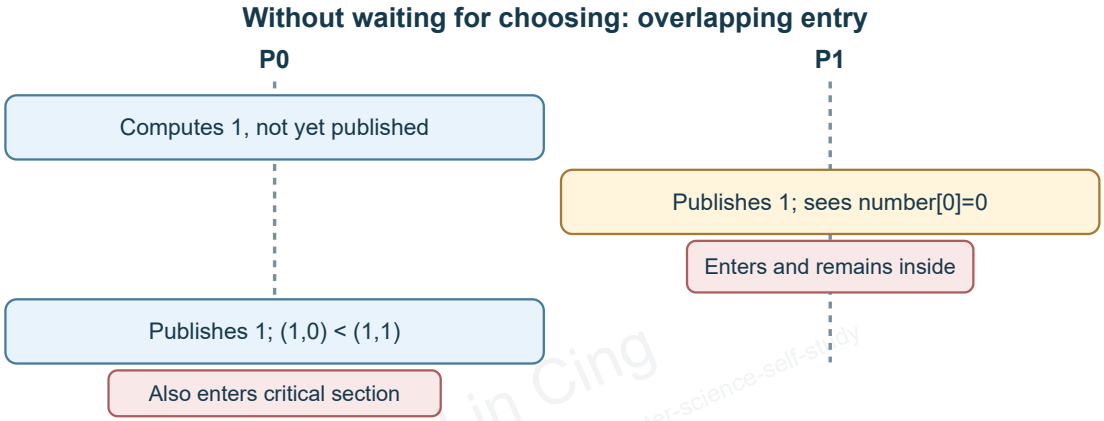
Pairs use lexicographic order: compare the number first, then the participant ID. The max scan is not an indivisible snapshot: two participants can both observe maximum 0 and choose 1. If P0 and P1 finish choosing with pairs (1,0) and (1,1), P0 precedes P1; P1 waits until P0 clears its number.

Bakery: why the choosing flag matters

`choosing[j]` means Pj is selecting a number. An observer waits for that selection to finish before comparing the published number. Without that step, initially all zero:

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bakery: why the choosing flag matters (continued)



Fix: P1 waits for choosing[0] to finish before comparing

The correct protocol holds P1 at `choosing[0]=true` until P0 publishes its choice, after which (1,0) has priority. The proof uses completed selection and pair order; it cannot assume tickets are always different.

After Pi completes choosing, at most n−1 other requests are already present or choosing. A participant that requests again while Pi's number remains published must choose after it. To conclude eventual entry, also assume selection and critical sections finish and relevant threads continue to be scheduled; this is not a wall-clock bound.

Bakery: mutual exclusion for general n

Suppose the current requests of Pi and Pj overlap in their critical sections. Pi has already published its number before checking Pj and keeps it unchanged until Pi exits.

If Pj had published its current nonzero number when Pi checked, Pi waits for `choosing[j]=false` and can pass only if `(number[i],i) < (number[j],j)`. If Pj has not yet checked Pi, the strict total order forces Pj to wait. If Pj checked and passed before Pi began its current request, Pj's number was already published and remains so until exit; Pi's later max scan sees it and selects a larger number, so Pi cannot pass. If selections overlap, `choosing` makes the check wait until the other selection is published, after which the pair order applies.

Another interleaving is that Pi reads `choosing[j]=false`, Pj then starts choosing, and Pi subsequently reads `number[j]=0`. These two reads are not an atomic snapshot. But Pj started after Pi had published its number; if the critical sections overlap, Pi's nonzero number remains valid while Pj scans, so Pj chooses a strictly larger number and waits for Pi to clear it. If Pj was already choosing when Pi read `choosing[j]`, Pi waits for that selection to complete. The flag does not stop a new selection from starting after Pi's check.

Both cases contradict overlap. Under the stated SC, nonoverflowing-number, and one-outstanding-request assumptions, mutual exclusion holds for arbitrary n.

Atomic hardware and spinning

Test-and-set atomically returns old L and sets true. CAS(&L,0,1) sets 1 only when L was 0 and returns the old value whether it succeeds or not.

```
acquire(L):
  repeat old = test_and_set(L)
  until old == false
release(L):
  L = false
```

One acquire sees false and owns the lock; others see true and retry. Owner-only release with proper publication preserves exclusion. Basic test-and-set does not bound waiting. Queueing, such as a circular waiting array with explicit successor handoff, adds fairness.

Spinning avoids sleep/wakeup and context costs for short sections. Long sections, one core, or a spinner competing with its owner can waste CPU. An interrupt handler spinning on a lock held by the interrupted task can prevent that owner resuming, producing livelock/deadlock. Interrupt contexts need controlled non-sleeping primitives and suitable interrupt/spin rules.

Atomic hardware and spinning (continued)

Instruction indivisibility does not alone provide language-level acquire/release. Later owners must see prior protected writes; writes before release must remain before publication; compilers must not move critical accesses outside the lock. Use atomic acquire/release semantics or corresponding fences. CAS alone does not solve ABA, reclamation, or lock-free progress.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mutex ownership and ordering proof

Every participating critical section uses acquire(m), protected work, release(m). Acquisition/release transitions are atomic. A mutex usually requires its owner to release it; a semaphore need not enforce ownership. Error, cancellation, and exception paths depend on that distinction. Blocking mutex contenders wait; spinners continue consuming CPU and require short sections with a runnable owner.

Assign a strict global lock order, ord(A)=1, ord(B)=2. Acquire a new lock only above every held lock. Any resource-wait cycle would require ord(A)<…<ord(A), a contradiction. In particular holding B while acquiring A violates ord(B)<ord(A). Callbacks, condition waits, upgrades, exceptions, and cancellation must obey the same rule. Sort dynamic locks by a stable address or unique ID, not transaction argument order such as from/to accounts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Permit abstraction and signed implementation

A resource semaphore counts permits; a binary abstraction has 0 or 1 free permits. A signed blocking implementation also encodes waiters. Counter and queue actions must be atomic with blocking/wakeup.

```
busy_wait(S):
  loop: old = atomic_load(S)
        if old > 0 and CAS(S, old, old-1) succeeds: return
busy_signal(S): atomic_fetch_add(S,1)

wait(S):
  S--
  if S < 0:
    enqueue current on S.queue
    block current
signal(S):
  S++
  if S <= 0:
    dequeue one waiter
    wake it
```


Permit abstraction and signed implementation (continued)

The busy-wait notation requires indivisible successful test/decrement, not two racing ordinary operations. Negative signed state represents waiters, not negative resources. At quiescent boundaries, free=max(S,0), queued=max(−S,0). A handed-off permit is allocated even if its recipient has not received CPU; newcomers cannot take it. Resource semaphores preserve free+allocated=capacity. Event semaphores need not have fixed capacity.

For capacity 1: T0 wait 1→0 obtains the permit; T1 wait 0→−1 queues/sleeps. T0 signal−1→0 wakes T1 because the incremented value≤0 and hands it the permit. T1 resumes after its original wait without decrementing again. T1 signal0→1 leaves a free permit because nobody waits. Waking only on0→1 strands T1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Permit abstraction and signed implementation (continued)

For capacity 3, empty=3, full=0, mutex=1:

Completed operation	Empty	Full	Buffer
Initial	3	0	[]
Put A	2	1	[A]
Put B	1	2	[A, B]
Put C	0	3	[A, B, C]
Get A	1	2	[B, C]
Put D	0	3	[B, C, D]

Producer uses wait(empty), wait(mutex), update, signal(mutex), signal(full). Consumer first waits full. Holding mutex while awaiting capacity can prevent the consumer releasing capacity.

If P0 takes S then Q and P1 Q then S, each initially1, P0 can hold S and P1 Q before waiting forever for the other. Starvation instead leaves some work progressing while a waiter is indefinitely skipped; LIFO arrivals can do that. FIFO removes one starvation pattern but not incorrect pairing.

Permit abstraction and signed implementation (continued)

For priority inversion, low-priority L owns H's needed resource while medium M preempts L. In the single-lock example, inheritance temporarily raises L; after release its effective priority is recomputed. With multiple locks or chained donations it need not revert to the base priority. The historical Mars Pathfinder VxWorks incident illustrates this mechanism.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Classical coordination protocols

Empty prevents producing into a full buffer, full prevents consuming an empty one, and mutex protects compound slot/head/tail/size updates. Capacity waits occur outside mutex. Otherwise producer waits for consumer capacity while consumer waits for producer's mutex. The occupancy safety condition is $0 \le \text{items} \le n$. General capacity accounting is $E+F+P+C=n$. E and F are the nonnegative, unreserved free permits of empty and full. P counts producers already granted an empty permit that have not yet signaled full; C counts consumers already granted a full permit that have not yet signaled empty. A permit handed directly to an awakened waiter counts as its P or C reservation even before it runs.

Every atomic protocol transition preserves the sum. Successful producer wait moves E to P; producer signal(full) moves P to F, or directly to a consumer's C on handoff. Consumer wait moves F to C; signal(empty) moves C to E or directly to another producer's P. Writing or removing the item does not change this capacity classification. With capacity 3 and an empty buffer, $(E,F,P,C)=(3,0,0,0)$. After only one successful wait(empty), it is $(2,0,1,0)$, so $E+F=2$, even before insertion. $E+F=n$ holds at quiescent boundaries with no reservations, $P=C=0$.

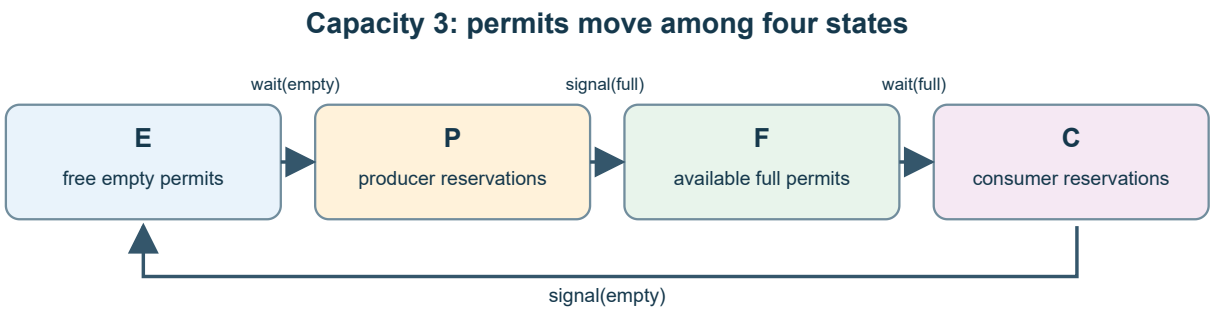
Classical coordination protocols (continued)

E/F are not signed semaphore raw counters. A negative raw value also encodes queued waiters and cannot be substituted into permit conservation. Raw zero can coexist with an outstanding handoff grant. Modify the structure under mutex, publish full only after insertion, and publish empty only after removal. These rules preserve occupancy safety; F need not equal the number of stored items during an operation.

Reader-preference allows new readers while writers wait, but not while a writer owns access; writers may starve. Writer-preference stops new readers after a writer starts waiting; readers may starve. Both allow concurrent readers when permitted. In the first protocol, read_count is protected by an owner-bound mutex; the first reader waits on rw_sem and the last reader signals it. Since they can be different threads, rw_sem is an ownerless binary semaphore, not a POSIX owner-bound mutex. A lock protects its actual state, not everything its name suggests.

Five philosophers each take chopstick[i], then chopstick[(i+1)%5], eat, release right then left. All can hold left and wait right, producing a five-edge cycle. Alternatives limit simultaneous contenders to four, acquire both atomically when available, or use opposite acquisition orders for odd/even philosophers. They exclude this deadlock pattern without automatically proving starvation freedom; waiter selection still matters.

Capacity conservation with in-flight reservations



Direct handoff: P can transfer to C; C can transfer to P

Event	E	F	P	C	Meaning
Empty buffer	3	0	0	0	Three unreserved empty slots
Producer succeeds in wait(empty)	2	0	1	0	Slot reserved, item not yet published
Producer writes and signals full	2	1	0	0	One item available
Consumer succeeds in wait(full)	2	0	0	1	Item reserved; slot not yet released
Consumer removes and signals empty	3	0	0	0	Buffer empty again

Every row has E+F+P+C=3. Cancellation must restore or complete any outstanding reservation.

Wait for a predicate, not a notification ticket

Use Mesa semantics, one mutex `m`, condition variable `not_empty`, and an initially empty shared queue `q`. P inserts A once. C1 and C2 each call `get()` once. Every queue mutation and `q.empty()` check holds the same mutex.

```
get():                          put_A():
  lock(m)                         lock(m)
  while q.empty():                q.push_back(A)
    wait(not_empty, m)            signal(not_empty)
  x = q.pop_front()               unlock(m)
  unlock(m)
  return x
```

Wait combines lock release and wait registration atomically relative to another thread acquiring m and notifying; successful return reacquires m. Signal does not transfer the mutex or reserve A for the waiter. With no waiter, no permit is accumulated.

Notification does not determine the next lock owner

Time flows down; P inserts one A; no spurious wakeups in this trace.

	Consumer C1	Producer P	Consumer C2	mutex / queue
T0	wait: unlock + enqueue			m=—, q=[]
T1		lock; enqueue A		m=P, q=[A]
T2	notified; no lock yet	signal C1; keep m		m=P, q=[A]
T3		unlock →	lock; take A; unlock	m=—, q=[]
T4	relock; q is empty			m=C1, q=[]
T5	while → wait again			m=—, q=[]

T3 orders P's unlock, C2's lock and removal of A, then C2's unlock. The mutex is not assumed FIFO. C1's T4 wait return owns m, but the predicate is false again. No spurious wakeup occurs in this counterexample: `if` would allow an empty pop; `while` sends C1 back to waiting at T5.

Track three distinct forms of state

CV holds condition waiters. E contains notified waiters awaiting m, **not a kernel ready queue**. Notification changes eligibility, not ownership or predicate truth. States follow each event.

Event	m owner	q	CV	E
T0 C1 waits	None	`[]`	`{C1}`	`{}`
T1 P inserts A	P	`[A]`	`{C1}`	`{}`
T2 P signals C1	P	`[A]`	`{}`	`{C1}`
T3 C2 removes A and unlocks	None	`[]`	`{}`	`{C1}`
T4 C1 reacquires	C1	`[]`	`{}`	`{}`
T5 C1 waits again	None	`[]`	`{C1}`	`{}`

Lost wakeup comes from a gap before waiting

Broken: separate unlock and wait

1. C1: sees empty, unlocks; not yet queued
2. P: inserts A, signals; nobody waiting
3. C1: queues and sleeps too late; q=[A]

Data exists, but C1 sleeps; signals do not accumulate.

Correct: atomic wait contract

1. C1: sees empty; wait unlocks + registers
2. P: locks, inserts A, signals, unlocks
3. C1: relocks, rechecks, then takes A

If P arrives first, C1 sees A and skips wait.

The left side is a broken abstract wait implementation: unlock first, register and block later. It is **not legal `pthread_cond_wait` usage**. P inserts and signals in the gap while CV is empty; C1 then sleeps despite queued data. Atomic wait excludes that gap. If P completes first and no other consumer removes A, C1 sees a nonempty queue under m and skips waiting. If A has already been consumed, C1 correctly waits again.

Prove safety and liveness separately

Safety. Every pop checks `!q.empty()` while holding m. No other thread can acquire m and change q between this check and removal. Wait return guarantees reacquisition, not predicate truth.

Liveness. The while loop prevents an invalid removal, but does not promise data for every consumer. There is only one A; once C2 consumes it, C1 may legitimately continue waiting. Scheduler fairness, mutex fairness, and future arrivals require separate assumptions.

If broadcast wakes two consumers for one A, the first owner takes A; the other returns to the while loop. Waking more threads creates neither data nor a bounded-waiting guarantee.

Handoff contracts: Hoare, Mesa, and signal-and-exit

The trace uses Mesa semantics: the signaler continues, and the waiter reacquires later. Hoare's handoff-time condition guarantee must not be imported into that model.

Handoff	Lock/execution after signal	Waiter reasoning
Hoare signal-and-wait	Immediate handoff; signaler waits	May rely on a predicate established before signal only under the immediate-handoff model without spurious wakeups or timeouts
Mesa signal-and-continue	Signaler retains the lock; waiter reacquires later	Recheck the predicate
Concurrent Pascal signal-and-exit	Signaler leaves, then hands off	Language-defined

Historical synchronization environments

Historical case	Mechanism	Boundary
Windows 7 kernel	SMP spinlocks for short global-resource sections	At the applicable IRQL ordinary thread scheduling does not preempt the holder; higher-IRQL interrupts may still occur
Windows 7 user services	Dispatcher mutex, semaphore, event, timer	Signaled/nonsignaled states; mutex grants one owner
Linux 2.6 onward textbook snapshot	Preemptible kernel, atomic_t, mutex, spinlock, semaphore	Short SMP spin regions; preempt_count>0 prevents safe preemption
Solaris textbook snapshot	Adaptive mutex, RW lock, turnstile	Spin while holder runs, otherwise sleep; turnstile supports inheritance
Pthreads/POSIX	Mutex, CV, RW lock; POSIX SEM semaphores	API determines waits, wakeups, errors

The textbook Windows critical-section object first spins in user mode and uses kernel assistance under prolonged contention. The cited uniprocessor Linux case replaces ordinary SMP-style spinning with preemption control. Solaris selects spin/sleep according to holder execution. These are versioned cases, not all-current-system claims.

Historical synchronization environments (continued)

Before substitution, identify user/kernel/interrupt context, scheduling entity, ownership and process-sharing support, fairness/inheritance/cancellation/wakeup order, and versioned memory ordering. Similar names do not establish equivalence.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Alternatives and their remaining boundaries

A memory transaction makes a sequence of shared accesses all-or-nothing. Commit publishes it; abort rolls it back, excluding partial observation. For an example with (x,y)= (100,40), a successful atomic transaction x=x−25; y=y+25 commits (75,65), while an aborted attempt publishes neither change. Conflicting read/write sets can force a retry; isolation applies to observers following the same transaction contract. Aborting can avoid lock-order deadlock but still permits starvation and must handle I/O, irreversible effects, and retries. STM instruments accesses through compiler/runtime; HTM uses coherent-cache mechanisms. Capacity and failure paths differ.

Within the same contention group, OpenMP critical regions of the same name exclude one another; unnamed critical regions share one unspecified name, while different names do not exclude one another. The programmer still identifies shared conflicts. Immutable functional values avoid in-place writes and permit sharing old values, but files, networks, mutable databases, and mailboxes remain shared resources. Message copying/ownership transfer reduces shared mutation without eliminating capacity, ordering, backpressure, or retry coordination.

Choosing a synchronization primitive

Need	Required model	Check
One critical section	Length, sleepability, ownership	Paired acquire/release
Fixed resources	Capacity, initial permits, authorized signalers	No excess signal
Predicate wait	Variables, notification, recheck	One protecting mutex
Multiple locks	Global order, dynamic key, cancel paths	Every acquisition follows order
Readers/writers	Preference/fairness rules	Reader/writer starvation

Trace modes, owned resources, waiters, and predicates. A successful finite test can reveal a bug but does not prove correctness for all schedules.

Chapter conclusions

- Critical-section requirements separate safety, progress, and fairness
- Semaphore and monitor errors often arise from predicates and wait order
- Classical proofs require their stated memory-model assumptions

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 5. Lamport, A New Solution of Dijkstra's Concurrent Programming Problem, 1974. pthread_cond_wait(3p); Windows spin locks; Windows critical sections; Linux lock types; OpenMP critical.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

One timeline, several different questions

Scheduling chooses who runs next. Fairness, throughput, response and deadlines then ask different questions about that choice. **A smaller average waiting time can coexist with a later completion for one job.**

Progression	Question to reconstruct
States and events	Waiting for CPU or I/O? Does a new arrival actually preempt?
General policies	What do FCFS, SJF, SRTF, RR and feedback queues select now?
Multicore and deadlines	How do migration costs and timing constraints change the result?
System cases	Which version, timer and weight rules produce this schedule?

Prerequisites: process/thread states, context switching and basic algebra. Every numerical example fixes its units and assumptions.

Ready, running and blocked

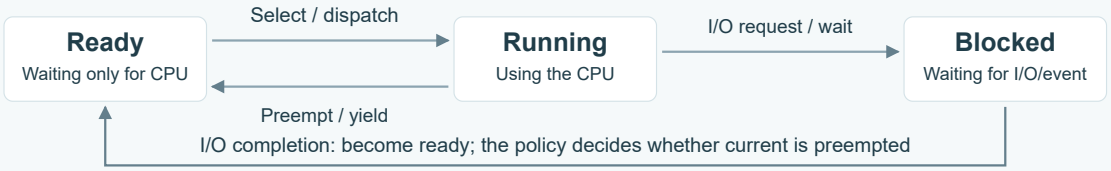
Ready means able to proceed except for CPU access; **running** means currently executing; **blocked** means awaiting I/O or an event that CPU time alone cannot supply.

A CPU burst is the CPU demand before the next blocking event or completion. Preemption pauses that demand; it neither finishes nor restarts it. The short-term scheduler selects an eligible execution context; the dispatcher saves/restores context and transfers control.

The ready queue is a candidate set, not necessarily a FIFO implementation: it may use priority queues or a tree. One modeled CPU runs at most one task, and a task cannot be both ready and running.

Which events require a new task, and which only change the candidates?

A state change and a dispatch decision are separate steps



New arrivals enter ready; completed tasks leave the system (exit omitted).

Blocked tasks are ineligible; preempted tasks retain their remaining CPU demand.

Blocking or completion requires selecting another ready job or idling. Arrival or wakeup only adds a candidate; **the policy decides whether to preempt**.

Nonpreemptive policies wait for completion, blocking or yield; preemptive policies may withdraw CPU access sooner.

Busy time need not be useful application work

One CPU executes P1 for 5 ms before it blocks, switches for 1 ms, then executes P2 for 2 ms.

Interval (ms)	CPU activity	Application progress
[0,5)	P1	5 ms
[5,6)	Context save/restore	None
[6,8)	P2	2 ms

There is no idle time: busy-time utilization is 100%, but useful-work fraction is 7/8=87.5%. Their numerators differ. The trace does not specify when P1's subsequent I/O or whole job completes.

Long-term admission selects jobs entering the system; short-term scheduling selects the next execution context; classical medium-term suspension/swapping changes memory pressure. These are different decision scales.

Fix the metric before comparing policies

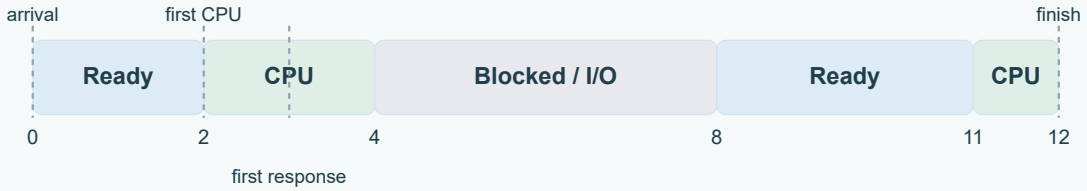
Let a be arrival, s first CPU dispatch, o first response production, and f completion.

Metric	Calculation	Distinction
Waiting W	Sum of ready intervals	Excludes CPU service and blocked time
Turnaround T	f−a	Includes intervening waits and I/O
First-CPU delay R_CPU	s−a	Does not imply application output
First-response delay R_out	o−a	Not complete output or job completion

Numerical scheduling exercises use R_CPU when they say response unless specified otherwise. The reference textbook § 6.2 describes interactive response production; that need not coincide with first dispatch. This is an explicit exercise convention, not an asserted identity between the events.

Why four milliseconds of I/O do not count as ready waiting

One task A: account for ready, execution and I/O intervals separately



Ready waiting	CPU service	Blocked I/O	Turnaround
2+3 = 5 ms	**2+1 = 3 ms**	**4 ms**	**12 ms**

First-CPU delay =2; first-response delay =3; 5+3+4 =12.

A arrives at 0: ready[0,2), CPU[2,4), blocked[4,8), ready[8,11), CPU[11,12). It first responds at 3. Thus W=2+3=5, service=3, blocked=4 and T=12: **5+3+4=12**.

R_CPU=2 and R_out=3. First execution, first response and completion differ. Counting I/O would incorrectly give W=9; T minus CPU service equals waiting only when other interval types are absent.

Define the selection rule, not just its name

General-policy examples use one CPU, one known CPU burst per job and zero switching cost. Unless overridden, ties follow arrival order then identifier. Intervals are half-open [start, end).

Policy	Selection key	Reconsideration
FCFS	Ready-enqueue order	Current releases CPU
Nonpreemptive SJF	Next CPU-burst length	Current releases CPU
SRTF	Remaining CPU demand	Arrivals/completions; shorter jobs may preempt
RR	FIFO and remaining quantum	Completion, blocking or quantum expiry
Priority	Explicitly ordered priority	Preemption depends on policy

Exact future burst lengths are a teaching assumption. Estimated lengths do not inherit guarantees that require exact lengths.

FCFS: short jobs can queue behind a long one

P1/P2/P3 all arrive at 0, require 8/4/2 ms and enqueue in that order.

Job	Execution	Completion f	Waiting W	Turnaround T
P1	[0,8)	8	0	8
P2	[8,12)	12	8	12
P3	[12,14)	14	12	14

Mean waiting is $20/3 \approx 6.67$ ms; mean turnaround is $34/3 \approx 11.33$ ms. Short jobs waiting together behind a long burst illustrate the convoy effect; interleaved I/O can further affect CPU/device coordination.

This does not make FCFS universally worst. Finite predecessors with finite service give a finite FIFO wait, but no uniform numerical bound follows without workload bounds.

SJF: reorder the same available batch

The 8/4/2 ms batch executes P3→P2→P1. **Rows remain in P1/P2/P3 order**, distinct from execution order.

Job	Execution	Completion f	Waiting W	Turnaround T
P1	[6,14)	14	6	14
P2	[2,6)	6	2	6
P3	[0,2)	2	0	2

Mean waiting is 8/3≈2.67 ms,4 ms lower than FCFS; P1 nevertheless finishes at 14 instead of 8. An improved average is not an improvement for every job.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The exchange argument and its assumptions

All jobs are available together, with one known burst each, nonpreemptive execution and zero switching cost. Adjacent jobs have lengths x>y and a shared preceding delay h.

Order	First wait	Second wait	Combined wait
x then y	h	h+x	2h+x
y then x	h	h+y	2h+y

Exchanging them reduces the sum by x−y>0. Their total execution is unchanged, so later start times are unchanged. Removing every inversion yields ascending lengths and minimum total/mean waiting. Fixed total service also gives minimum mean turnaround.

Different arrivals, deliberate idling, weights, deadlines or prediction error require their own argument.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Predicting the next burst

Exponential averaging uses $\hat{b}_{n+1} = \alpha b_n + (1-\alpha)\hat{b}_n$, where b_n is the latest measured burst, the hat denotes prediction and $0 \le \alpha \le 1$.

α=1/2; initial prediction 10 ms	Measured b_n	Next prediction
First	6	(6+10)/2=8
Second	4	(4+8)/2=6
Third	8	(8+6)/2=7

Expansion assigns older observations weights $\alpha(1-\alpha)^j$. For $0<\alpha<1$ these decay; $\alpha=0$ ignores new observations and $\alpha=1$ uses only the latest burst.

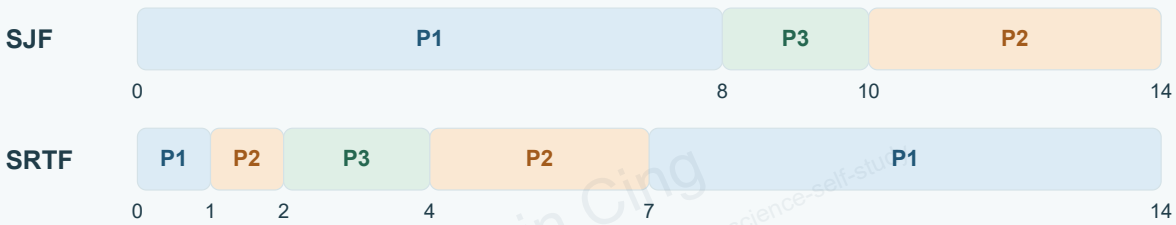
A phase change can defeat this predictor. Prediction does not remove the need for fairness or turn an estimated-shortest choice into an exact-shortest guarantee.

SRTF compares an arrival with remaining demand

Let P1=(arrival0, demand 8), P2=(1,4), P3=(2,2), in ms. Equal remaining demand retains current; otherwise choose the shortest.

Identical arrivals and demand; only preemption differs

P1: a=0, b=8 P2: a=1, b=4 P3: a=2, b=2 (ms)



t=1: 4 < 7 t=2: 2 < 3 t=4: P2 resumes with 3 ms left

P1/P2/P3 waits: SJF =0/9/6; SRTF =6/2/0 ms

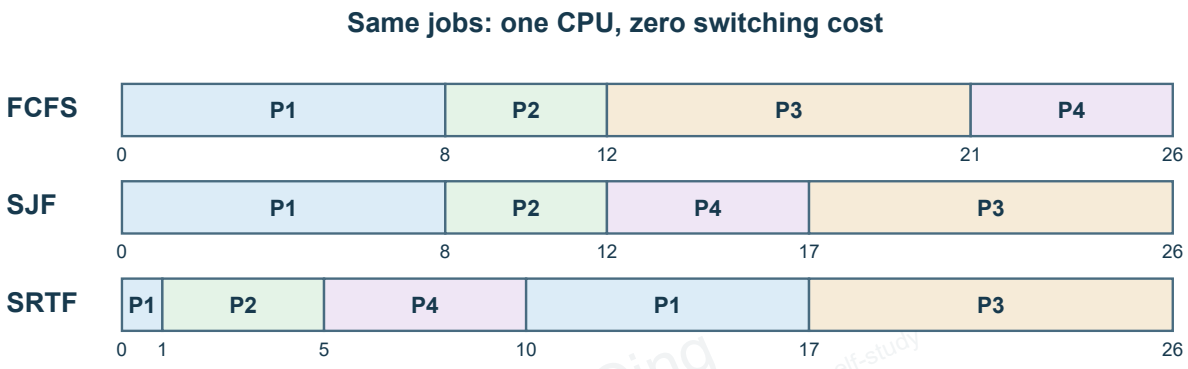
At 1, P1 has 7 left versus P2's4, so preempt. At 2, P2 has 3 left versus P3's2, so preempt again. At 4 choose P2's remaining 3 before P1's7. **P2 does not restart its original 4 ms.**

Reconstruct every SRTF event and metric

Event	Candidate remaining demand	Next segment	Reason
0	P1=8	P1[0,1)	Only candidate
1	P1=7, P2=4	P2[1,2)	4<7
2	P1=7, P2=3, P3=2	P3[2,4)	2 is smallest
4	P1=7, P2=3	P2[4,7)	3<7
7	P1=7	P1[7,14)	Only candidate

In P1/P2/P3 order: completion 14/7/4, turnaround 14/6/2, waiting 6/2/0 and mean waiting 8/3 ms. First-CPU delays are all 0. Nonpreemptive SJF instead completes 8/14/10 with waits 0/9/6 and mean 5 ms. Identical arrivals/demands make the policy comparison meaningful.

Three policies on the same arrivals and work



Assume one CPU, no switching cost, and one CPU burst per job. Inputs are P1=(arrival 0, demand 8), P2=(1,4), P3=(2,9), and P4=(3,5), in milliseconds. FCFS uses arrival order; nonpreemptive SJF picks the shortest available job whenever the CPU becomes free; SRTF picks the smallest remaining demand.

At t=1, SRTF preempts P1 because P2 needs 4 while P1 has 7 left. When P2 completes at t=5, P4's 5 is less than P1's 7 and P3's 9, so P4 runs next. SJF does not preempt on arrival, so P1 continues to t=8.

Derive waiting time from completion

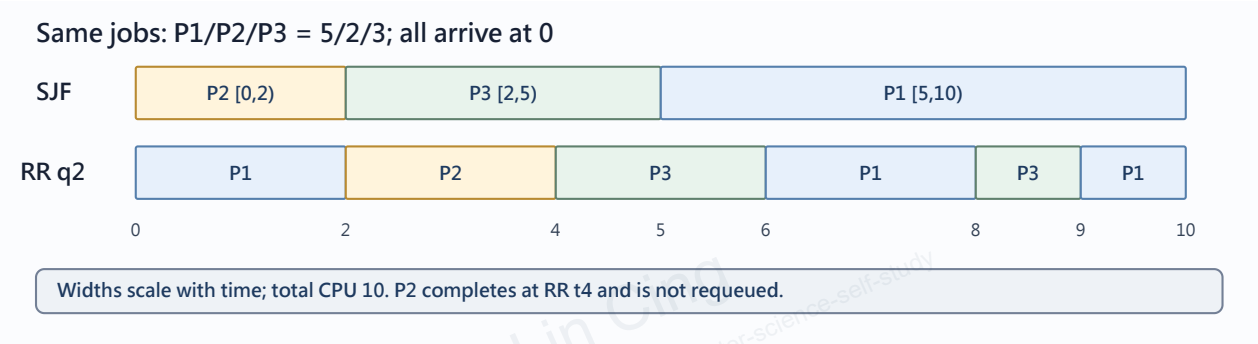
There is no I/O in this example, so each waiting time W is completion minus arrival minus CPU demand. Rows follow job IDs rather than execution order.

Job	FCFS completion / wait	SJF completion / wait	SRTF completion / wait
P1	8 / 0	8 / 0	17 / 9
P2	12 / 7	12 / 7	5 / 0
P3	21 / 10	26 / 15	26 / 15
P4	26 / 18	17 / 9	10 / 2
Mean wait	35/4 = 8.75	31/4 = 7.75	26/4 = 6.5

For example, under SRTF P1 runs for 1 ms in [0,1), leaving 7 ms. It resumes at t=10 and completes at t=17, so its waiting time is 17−0−8=9 ms. These results describe the stated workload; switching costs or a different objective can change the comparison.

RR: requeue only unfinished jobs

All arrive at 0; P1/P2/P3 demand 5/2/3 ms. Initial order P1→P2→P3, q=2, cost 0. Completed jobs leave; unfinished jobs rejoin the tail after a full slice.



In P1/P2/P3 order, RR completion is 10/4/9, waiting 5/2/6 and first-CPU delay0/2/4. P2 completes at 4 and cannot requeue. Mean waiting is 7/3 for SJF versus 13/3 ms for RR; both execute10 ms of work.

RR: separate a rotation bound from switching cost

For n continuously ready jobs, quantum q and handoff cost c, a job just requeued can face at most n−1 other quanta. Including the final handoff back to it gives bound **(n−1)q+nc**. This assumes a fixed eligible set, no higher scheduling class and no blocking.

Other jobs remain ready; c=0.1 ms	q	Useful fraction q/(q+c)
Larger quantum	4 ms	4/4.1≈97.56%
Smaller quantum	0.1 ms	0.1/0.2=50%

If q covers every entire burst, RR reduces to FCFS for that batch. A smaller q shortens rotations but raises overhead. With only one runnable job, expiry need not cause an actual cross-task switch.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Priority, aging and long finite waits

Specify whether higher numbers mean higher priority. With strict priority, a low-priority job can starve if higher-priority jobs remain ready forever.

A finite trace shows only a finite wait. An infinite counterexample has L ready at 0 and high-priority H_k arriving at every integer k, each demanding 1 ms. Higher-priority service fills the CPU, so L never advances.

Aging increases priority with waiting. A finite-wait proof also needs conditions such as capped levels, fair equal-priority service and finite predecessors/service demands. An unspecified increase alone gives no bound. Changing real-time priorities requires renewed deadline analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fixed multilevel queues versus feedback queues

Multilevel queues keep jobs in fixed classes, with both intra-queue and inter-queue policies. Multilevel feedback queues (MLFQ) let jobs move between levels.

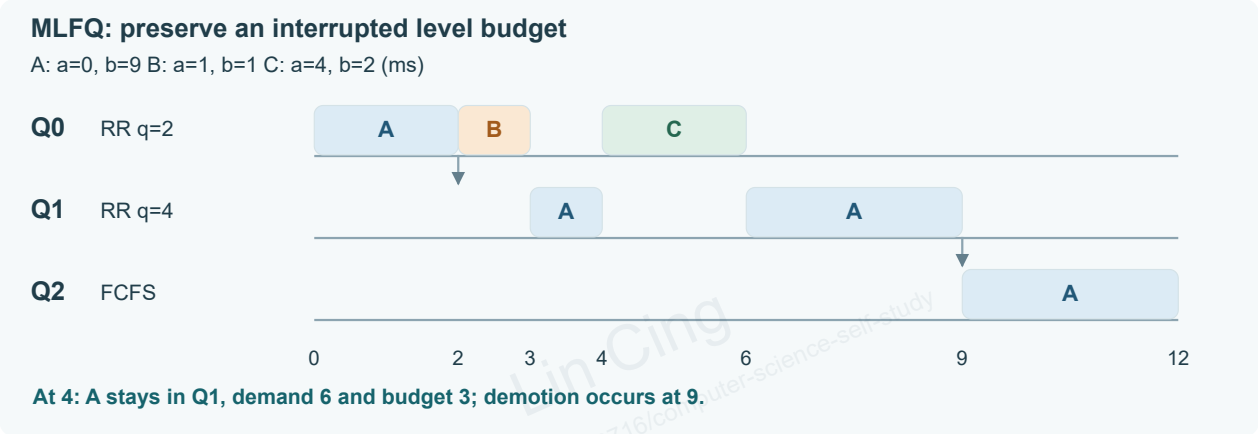
The following is one explicit teaching variant, **not a universal MLFQ definition**.

Rule	Chosen policy
Highest to lowest	Q0:RR q=2; Q1:RR q=4; Q2:FCFS
New arrival	Enter Q0; a nonempty higher queue preempts lower levels
Budget exhausted	Unfinished job drops one level; Q2 remains Q2
Higher-level preemption	Retain original-level head position and remaining budget
Promotion/blocking	No blocking or promotion in this finite example

Different requeue, boost or budget-reset rules can produce different schedules.

MLFQ: preemption does not replenish the budget

A=(arrival0, demand 9), B=(1,1), C=(4,2), all in ms, with the preceding rules.



At 4, A retains **Q1 budget 3 but CPU demand 6**. It resumes over 6–9, exhausts that budget, then completes in Q2 over 9–12. The preceding rules and following event table account for every segment.

Check every MLFQ queue and budget transition

States follow event processing and precede dispatch. Parentheses show remaining CPU/remaining level budget; Q2 has no quantum.

t	Q0	Q1	Q2	Next execution
0	A(9/2)	Empty	Empty	A0–2
2	B(1/2)	A(7/4)	Empty	B2–3
3	Empty	A(7/4)	Empty	A3–4
4	C(2/2)	A(6/3)	Empty	C4–6
6	Empty	A(6/3)	Empty	A6–9
9	Empty	Empty	A(3/—)	A9–12

Resetting A to a full 4 ms after preemption defines another policy and delays its demotion. Without aging/boosts, strict inter-level priority does not prevent starvation under infinite higher-level arrivals.

User-thread scheduling and CPU scheduling

A library manages user threads; the kernel selects kernel-visible execution entities. Process-contention scope (PCS) distributes available entities within a process; system-contention scope (SCS) competes for CPUs system-wide.

Mapping	Can T2 proceed while T1 blocks?	Can both execute on separate CPUs?
Many-to-one	Not if T1 blocks the sole kernel entity	No, only one kernel entity
One-to-one	If T2's own entity remains ready	Only with at least two available CPUs

If T2 needs 4 ms, one-to-one alone does not prove completion at 4: it must also run continuously from 0. Mapping makes execution possible; policy and load determine its timing.

Multicore balancing pays for locality changes

Processor affinity preserves useful cache/TLB state; load balancing moves work toward idle CPUs. Queue length alone cannot resolve the tradeoff.

One job must not be claimed by two CPUs. Synchronized migration removes its old queue entry, transfers unique ownership and enqueues it at the destination. A running job and an unstarted queued job are not interchangeable forms of stealing.

The following eight-core example fixes all jobs ready at 0, indivisible execution, no job on two CPUs at once and 1 ms per transfer plus warmup. These are model costs, not measurements.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

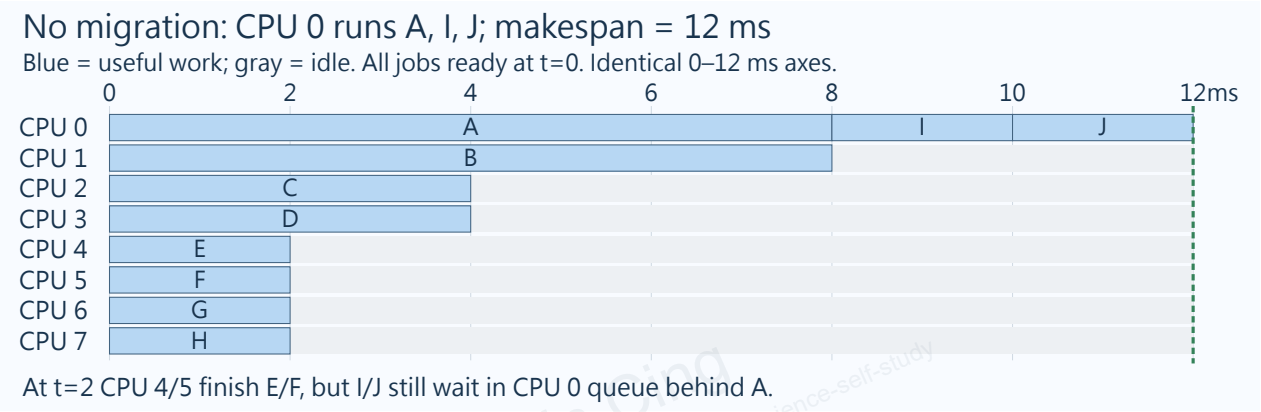
Eight cores: account for transfer and warmup

Fix eight cores, all jobs ready at t=0, indivisible jobs, and no job running on two cores at once. A/B each need 8 ms, C/D each need 4 ms, and E/F/G/H/I/J each need 2 ms: 36 ms of useful work. CPU 0 initially queues [A, I, J]; CPU 1 through CPU 7 initially hold B, C, D, E, F, G, H respectively.

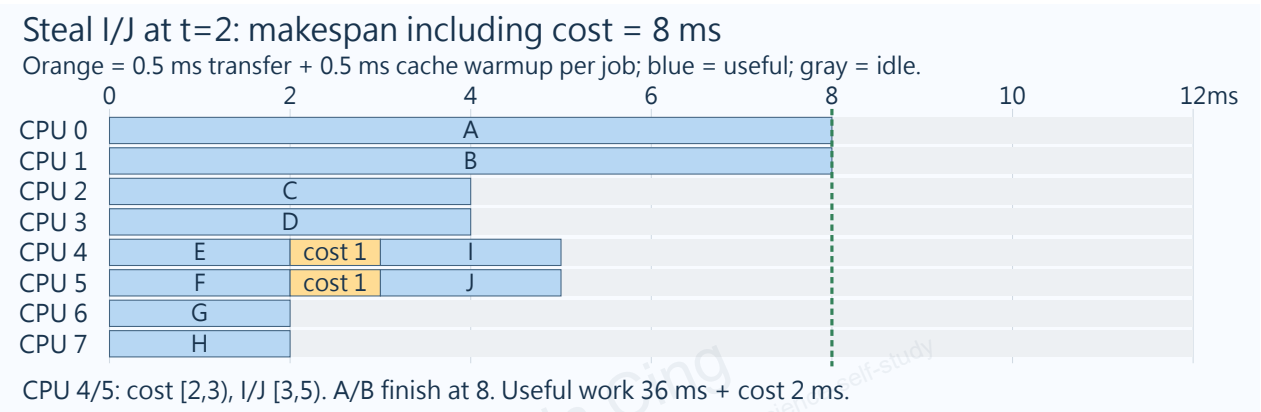
Without migration CPU 0 runs A[0,8), I[8,10), J[10,12), so makespan is 12 ms. At t=2 CPU 4/5 finish E/F. Under queue synchronization they each remove one still-queued job I/J and transfer its unique ownership. Charge each migration 0.5 ms for queue/transfer and 0.5 ms for cache warmup. Both destination cores pay overhead [2,3), then run I/J [3,5). CPU 0 finishes A at 8; makespan becomes 8 ms, speedup=12/8=1.5. Useful work remains 36 ms; added CPU work is 2 ms. These are specified model costs, not hardware measurements.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Before migration: locate waiting work



After migration: include its cost



Job A takes 8 ms, so $T \geq 8$; this schedule attains that bound. With migration cost c, $T=\max(8,4+c)$, improving on 12 only if $c<8$. Affinity and dependencies may change the result.

Real-time scheduling: distinguish tasks, jobs, and deadlines

A periodic task τ_i releases successive jobs. Let C_i be its execution-time upper bound, T_i its period, and D_i its relative deadline. Unless stated otherwise, worked timelines use actual demand C_i for each job. With synchronous release at zero and job index k starting at zero,

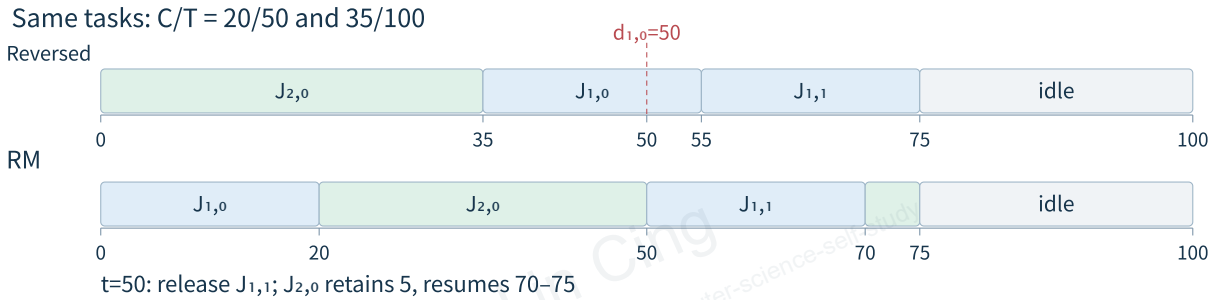
$$r_{i,k} = kT_i, \qquad d_{i,k} = r_{i,k} + D_i.$$

Base model: one CPU, immediate preemption, independent tasks, no lock blocking, I/O, self-suspension, switching or interrupt costs; $D_i=T_i$ except on the final page. A release adds a job to the ready set without necessarily changing the current job. Completion exactly at a deadline succeeds. At an event time t, account for execution up to t before processing new releases.

Quantity	Question answered	Distinction
C_i	How much CPU can each job require?	Not response time
T_i	How far apart are successive releases?	Not remaining time to deadline
D_i	How long after release may completion occur?	Absolute deadline is $r_{i,k}+D_i$
$U=\Sigma C_i/T_i$	What is the long-run CPU demand fraction?	Does not describe every short interval

RM gives shorter-period tasks higher fixed priority

Rate-monotonic (RM) assigns **higher fixed priority to a shorter period**, with preemption on a higher-priority release. Set τ_1=(C_1=20, T_1=D_1=50) and τ_2=(35,100,100); all times are milliseconds.



Reversed priorities delay $J_{1,0}$ to [35,55), missing50. Under RM, $J_{2,0}$ executes 30 in [20,50), is preempted, then completes its **remaining five units** in [70,75).

RM completions 20,70,75 meet deadlines 50,100,100. At 100 no old job remains and synchronous releases recur, proving repetition every 100 ms.

The RM utilization bound is sufficient, not a rejection rule

For n independent preemptible tasks with $D_i=T_i$ on the ideal single CPU, the classical **sufficient condition** is

$$U = \sum_i C_i/T_i \le n(2^{1/n} - 1).$$

For n=2 the bound is $2(\sqrt{2}-1)\approx 0.8284$. The preceding U=0.75 passes. The limit as n grows is $\ln 2 \approx 0.6931$. These are workload-independent guarantee bounds, **not a utilization ceiling for every RM workload**.

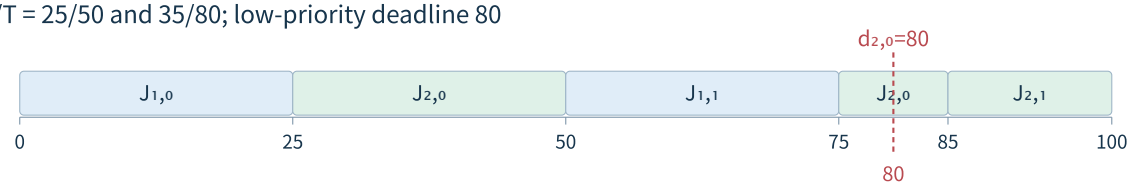
Counterexample: τ_1=(C=1, T=D=2), τ_2=(2,4,4) gives U=1. RM executes $J_{1,0}$ in [0,1), $J_{2,0}$ in [1,2), $J_{1,1}$ in [2,3), then the final unit of $J_{2,0}$ in [3,4). Completion times 1,3,4 meet deadlines 2,4,4. At t=4 the empty carry-in state and synchronous releases recur, so this schedule repeats.

Check assumptions, then apply the sufficient test. If it fails, use tighter interference analysis or a schedule whose recurring state is proved. U>0.8284 alone does not prove failure, and $U \le 1$ does not certify an arbitrary fixed-priority policy.

RM can miss a deadline even below full utilization

Now set τ_1=(25,50,50), τ_2=(35,80,80). Utilization is 15/16=0.9375. RM still gives τ_1 higher priority.

RM: C/T = 25/50 and 35/80; low-priority deadline 80



At 80: old $J_{2,0}$ still needs 5; new $J_{2,1}$ is separate

$J_{2,0}$ executes 25 units in [25,50), leaving ten. $J_{1,1}$ preempts it in [50,75). It receives only five more units in [75,80), so **five remain at deadline 80**. The miss has already happened. If late jobs are retained, this job finishes at 85. The new $J_{2,1}$ released at 80 is a separate job.

Long-run spare capacity does not prevent fixed priorities from crowding a particular interval. The pending demand at 80 proves failure; exceeding a sufficient utilization bound does not.

Count interference to check the RM completion time

For two synchronous tasks, τ_1 higher priority, $C_1 \le T_1$ and no blocking, $[0, R)$ contains $\lceil R/T_1 \rceil$ higher-priority releases at $0, T_1, 2T_1, \ldots$. **A release exactly at R is excluded.**

$$w^{(0)} = C_2, \qquad w^{(k+1)} = C_2 + \left\lceil w^{(k)}/T_1 \right\rceil C_1.$$

For the synchronous sequence where each job actually needs its bound C_i, iterate from C_2 to the earliest fixed point; stop if $w > D_2$. This is a worst-case sequence, not a claim that every lower-demand run misses its deadline.

Monotonicity makes iterates nondecreasing. Any fixed point $R^* \le D_2$ bounds every iterate by induction; an iterate $> D_2$ excludes it. Only the first synchronous job is covered; other phases, tasks or blocking need further analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three workload sets: evaluate each iteration

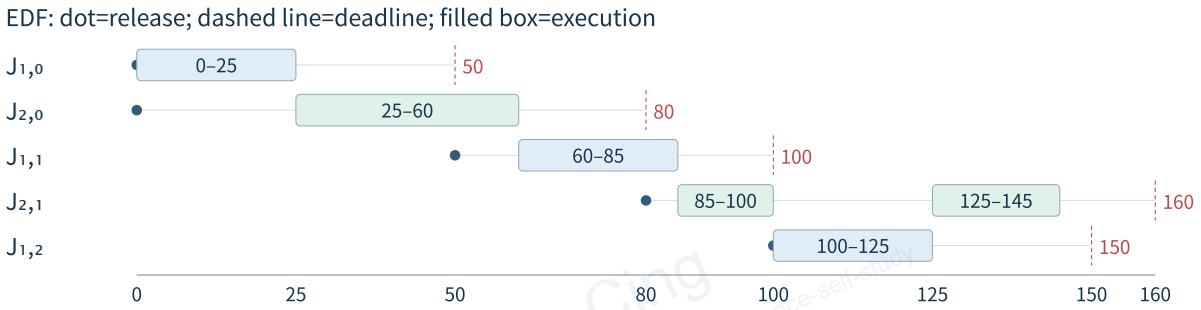
Task pairs (C, T)	$w^{(0)}$	$w^{(1)}$	$w^{(2)}$	$w^{(3)}$	Check
(20,50),(35,100)	35	55	75	75	$75 \le 100$
(25,50),(35,80)	35	60	85	85	$85 > 80$
(1,2),(2,4)	2	3	4	4	Equality succeeds

For row 2:35+ceil(35/50)×25=60, then 35+ceil(60/50)×25=85>80, beyond the deadline. Row 1 stabilizes at $75 \le 100$. The preceding interference argument gives these iterations their meaning.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

EDF compares absolute deadlines of individual jobs

EDF selects the **earliest absolute deadline** among ready jobs. Use $\tau_1=(25,50,50)$,$\tau_2=(35,80,80)$; ties retain current, otherwise task/job order.



At 50, current $J_{2,0}$'s80 beats new $J_{1,1}$'s100: **continue to 60**. At 100, new $J_{1,2}$'s150 beats current $J_{2,1}$'s160: preempt.

The first 150 ms alone cannot certify every future job; the theorem below does. Preemption preserves remaining execution.

Verify each EDF choice against the ready set

Each row is after processing that event and before executing the next interval. Parentheses contain (remaining CPU, absolute deadline); completed jobs are absent.

t (ms)	Ready jobs	Next execution	Reason
0	$J_{1,0}$(25,50), $J_{2,0}$(35,80)	$J_{1,0}$:0–25	50<80
25	$J_{2,0}$(35,80)	$J_{2,0}$:25–50	Only ready job
50	$J_{1,1}$(25,100), $J_{2,0}$(10,80)	$J_{2,0}$:50–60	80<100; continue
60	$J_{1,1}$(25,100)	$J_{1,1}$:60–80	Only ready job
80	$J_{1,1}$(5,100), $J_{2,1}$(35,160)	$J_{1,1}$:80–85	100<160; continue

At both 50 and 80 a new release changes the ready set while current remains earliest-due. Release alone does not imply preemption.

The same EDF rule does preempt at 100

Continue with (remaining CPU, absolute deadline) for ready jobs.

t (ms)	Ready jobs	Next execution	Reason
85	$J_{2,1}$(35,160)	$J_{2,1}$:85–100	Only ready job
100	$J_{1,2}$(25,150), $J_{2,1}$(20,160)	$J_{1,2}$:100–125	150<160; preempt
125	$J_{2,1}$(20,160)	$J_{2,1}$:125–145	Only ready job
145	Empty	Idle:145–150	No ready job

The first three τ_1 completions are 25,85,125 against 50,100,150. The first two τ_2 completions are 60,145 against 80,160. **Release, priority reevaluation and switching are distinct events.** Releases at 50 and 80 do not change current.

Why does ideal EDF need only U≤1 when D=T?

Use the stated one-CPU model, starting without carry-in at t=0; tasks may have different nonnegative release phases. If U>1, consider the legal sequence in which every job actually needs its upper bound C_i. Demand due by time t is Ut−O(1), eventually exceeding available CPU time. No policy can guarantee all legal demand sequences meet every deadline; an actual run with lower demands may still succeed.

For sufficiency suppose U≤1, yet EDF first misses a deadline b. Consider only jobs with deadlines at most b. Let [a, b) be their last continuously backlogged interval, of length L=b−a. None of these jobs was pending immediately before a, so the relevant jobs all release at or after a. Throughout the interval EDF executes only such jobs.

Since $D_i=T_i$, at most $\lfloor L/T_i \rfloor$ jobs of task i can both release and become due in this interval: the first cannot be due before $a+T_i$ and later deadlines are separated by T_i. Thus their total demand satisfies

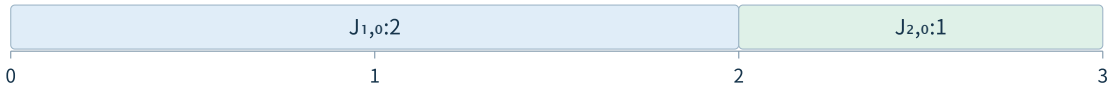
$$\sum_i N_i C_i \le \sum_i \lfloor L/T_i \rfloor \, C_i \le L \sum_i C_i/T_i = LU \le L.$$

A miss after L units of uninterrupted service instead requires demand **greater than L**, a contradiction. Hence U≤1 is sufficient and, with the necessity argument, necessary and sufficient to guarantee deadlines for all legal demand sequences in this model. The proof bounds every relevant interval rather than relying on an average alone.

Shorter relative deadlines can defeat low utilization

Let both tasks have C=2, T=10, D=3, released together at 0. U=0.4, yet four CPU units must complete by 3. A single CPU supplies only three in [0,3). **Every policy, including EDF, must miss at least one deadline.**

C=2, T=10, D=3: both release 0; four units required by 3



Capacity 3< demand 4; $J_{2,0}$ still needs 1 at 3

For $D_i \leq T_i$ the maximum demand both released and due within length L is

$$\mathrm{dbf}_i(L) = \max\left(0, \left\lfloor \frac{L - D_i}{T_i} \right\rfloor + 1\right) C_i.$$

The first job needs D_i time; each additional T_i permits another. At L=3 total demand 4>3 proves impossibility. Passing just one length cannot certify all intervals. Priority inheritance handles specific lock-induced inversion; it cannot fix this capacity contradiction.

A system name is not a scheduling rule

Fix the following historical versions and event models: Solaris TS/IA, Windows 7 and Linux 3.2 CFS. These are mechanism comparisons, not a modern-product benchmark.

Case	Selection mechanism	Required conditions
Solaris TS/IA	Class priority mapped to global priority; expiry and wakeup change levels	Dispatch table, quanta and return rules
Windows 7	32 ready priorities, equal-level rotation and dynamic boosts	Base level, boosts and tick units
Linux 3.2 CFS	Vruntime tree with fairness-bounded candidate hints	Weights, timer and wakeup thresholds

Do not reduce all three to a universal RR quantum. The following CFS case fixes its event model before deriving the timeline.

A fixed Linux 3.2 event model

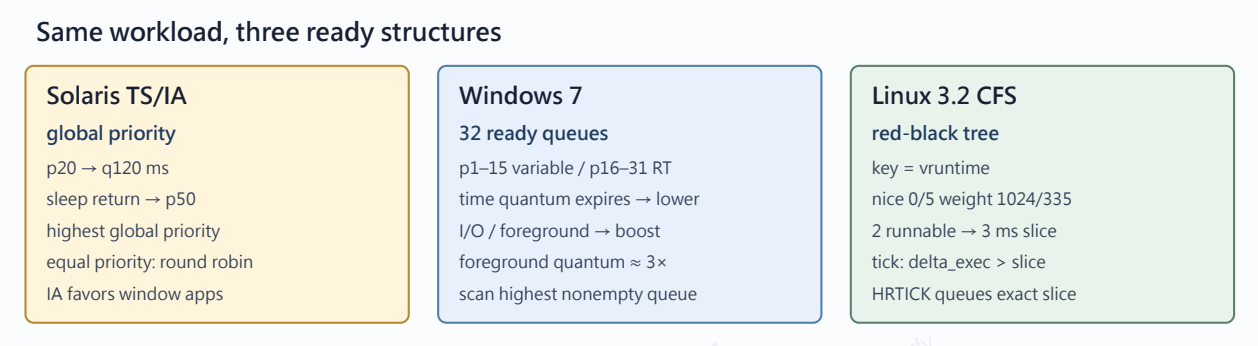
One CPU: P1 and P2 are ready at 0 and require 8 and 4 ms; P3 requires 2 ms and wakes at 9. All weights are 1024, initial vruntime is normalized to 0, and P1 wins the initial tie. Ignore switching, interrupt and timer-jitter costs.

Set target latency=6 ms, minimum granularity=0.75 ms, wakeup granularity=1 ms and GENTLE_FAIR_SLEEPERS=true. Compile with CONFIG_SCHED_HRTICK=y; enable the runtime HRTICK feature, use an active high-resolution timer, and set DOUBLE_TICK=false. These assumptions do not imply HRTICK is enabled on every Linux 3.2 system.

For $n \leq 8$ runnable tasks, period=6 ms; otherwise period=0.75n ms. Equal weights give an ideal slice of period/n: 3 ms for two tasks and 2 ms for three.

At t=9, process P3's enqueue before the old HRTICK expiry handler. Ordinary ticks may occur, but do not cause additional switches in this model.

Three ready structures use different selection rules



Current is outside the ready tree. A sleeping entity is ineligible until its wakeup event; selection and sleep placement must be tracked separately.

CFS event trace

States are after event processing and selection. Ready excludes current; slice units are ms.

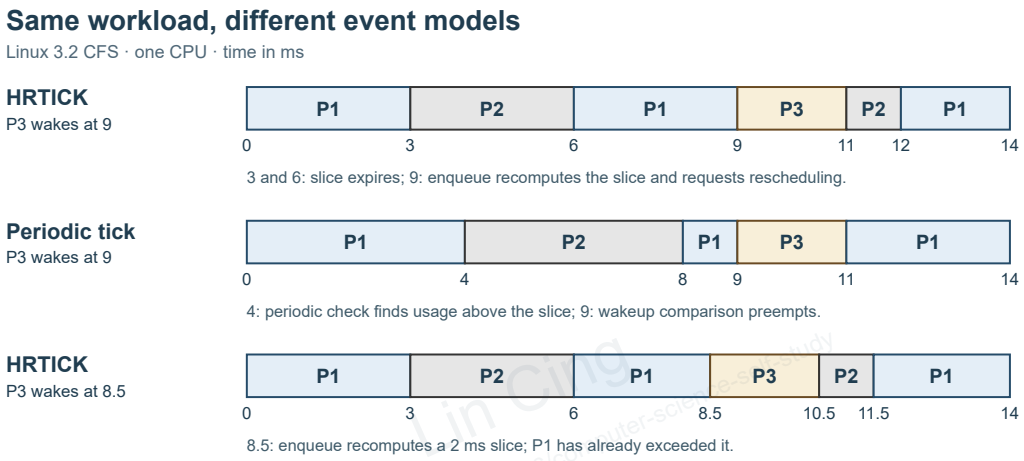
Time/event	Current/slice	Other ready	min_vruntime	Selection reason
0 initial	P1/3	P2@0	0	P1 inserted first on tie
3 HRTICK	P2/3	P1@3	0	P1's slice expires
6 HRTICK	P1/3	P2@3	3	Existing leftmost wins tie
9 P3 enqueue	P3/2	P2@3, P1@6	3	Recomputed slice is 2; P1 has run 3; enqueue update requests rescheduling
11 P3 complete	P2/3	P1@6	3	Ready key 3 precedes 6
12 P2 complete	P1/6	none	6	P1 alone, 2 ms remaining
14 P1 complete	none	none	8	No work remains

CFS event trace (continued)

With GENTLE_FAIR_SLEEPERS, P3 is placed at max(0,3−6/2)=0. Its key may be below min_vruntime, but the baseline does not move backward. The enqueue-triggered HRTICK update has already requested rescheduling; the 6−0>1 wakeup comparison is not the cause of the switch in this event order.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Compare the timer-dependent timelines



P1 / P2 / P3 CPU demands: 8 / 4 / 2 ms. Numbers mark event times.

Tick, HRTICK, and wakeup sensitivity

Settings	Timeline (ms)	Completion P1/P2/P3
HRTICK; wake at 9	P1 0–3, P2 3–6, P1 6–9, P3 9–11, P2 11–12, P1 12–14	14/12/11
No HRTICK; 1 ms tick; wake at 9	P1 0–4, P2 4–8, P1 8–9, P3 9–11, P1 11–14	14/8/11
HRTICK; wake at 8.5	P1 0–3, P2 3–6, P1 6–8.5, P3 8.5–10.5, P2 10.5–11.5, P1 11.5–14	14/11.5/10.5

The ordinary tick's slice comparison is strict: 3>3 is false, so P1 first leaves at t=4 without HRTICK. In the final row P3's enqueue reduces P1's ideal slice to 2 ms after P1 has run 2.5 ms; hrtick_update requests rescheduling. In the second row, the wake comparison 5−2>1 requests it instead. These are different triggering paths.

Tick checks, wakeup comparison and buddy hints

After minimum granularity, the ordinary tick may also compare the current task's vruntime with the leftmost candidate. Arrival or wakeup alone does not guarantee preemption.

Linux 3.2's wakeup-preemption path checks whether rescheduling has already been requested. If P3's enqueue has already triggered hrtick_update, the wakeup comparison is not the cause of that switch; if not, the wakeup rule must be evaluated separately.

Last/next/skip buddy are candidate hints for locality and scheduling overhead, constrained by fairness. They are not absolute priorities, and selecting P3 does not by itself prove that P3 was nominated next buddy.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Nice weights and historical comparisons

With only A nice 0 and B nice 5 runnable, total weight=1359 and period=6 ms.

Task	Weight	Ideal runtime	vruntime per real ms	CPU share
A	1024	$6\times1024/1359\approx4.521$ ms	1 ms	75.35%
B	335	$6\times335/1359\approx1.479$ ms	$1024/335\approx3.057$ ms	24.65%

Weights are not linear nice percentages. Above eight runnable tasks the period grows to preserve minimum granularity instead of shrinking slices without bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why the same workload need not give one system timeline

System	Selection mechanisms	Inputs required to reconstruct a timeline
Solaris TS/IA	Class/global priorities, expiry and wakeup adjustments	Initial priorities, dispatch table, quanta and event ordering
Windows 7	Multiple ready priorities, equal-level rotation and dynamic boosts	Initial priorities, quantum, tick unit, boosts and queue order
Linux 3.2 CFS	Vruntime, weights, timer and sleep placement	Weights, timer configuration, wake time and tie rules

CPU demands and wake time alone do not determine a Solaris or Windows Gantt chart. Without the missing parameters, completion times cannot be derived. The Linux timeline above follows from its stated assumptions; it is not a performance ranking of the three systems.

Comparing policies requires comparable evidence

Method	What it can establish	What it cannot establish alone
Deterministic trace	Exact decisions and metrics for specified jobs	The same outcome under other arrivals
Queueing model	Means under arrival and service assumptions	Tail behavior outside those assumptions
Simulation	Behavior under modeled costs and distributions	Costs omitted from the model
Measurement	Observations on a fixed version and machine	All machines and future workloads

Use the same inputs and account for unfinished jobs. Reporting only completed short jobs biases a comparison that omits a long job still waiting.

Waiting, first response and throughput answer different questions

One CPU; all arrivals 0; P1/P2/P3 bursts5/2/3 ms; no I/O or switching cost. Response here means first CPU. Nearest-rank p99 for three observations is the batch maximum, not a population-tail estimate.

Policy	Completion P1/P2/P3	Waiting P1/P2/P3	Mean waiting	First response P1/P2/P3	Batch p99 response
FCFS, initial P1/P2/P3	5/7/10	0/5/7	4	0/5/7	7
SJF, P2/P3/P1	10/2/5	5/0/2	7/3	5/0/2	5
RR q=2, initial P1/P2/P3	10/4/9	5/2/6	13/3	0/2/4	4

Throughput is 3/10 jobs/ms for every policy. SJF minimizes this batch's mean wait; RR lowers its maximum first-CPU delay. Three observations do not estimate population p99. Random-workload claims would require distributions and independent replications.

A busy CPU can still leave jobs waiting longer

Keep RR q=2, all arrivals 0 and initial order P1/P2/P3. Multiply each burst by k. Run the queue head for min(2, remaining), then append it only if unfinished. Ignore switching cost; units are ms.

k; bursts	Completion P1/P2/P3	Waiting P1/P2/P3	Throughput jobs/ms	Batch p99 first-CPU response	Maximum waiting
1; 5/2/3	10/4/9	5/2/6	3/10	4	6
2; 10/4/6	20/10/16	10/6/10	3/20	4	10
3; 15/6/9	30/16/25	15/10/16	1/10	4	16

All three batches first run at 0/2/4 and keep the CPU 100% busy. Maximum waiting still grows6→10→16. Fast first dispatch does not imply fast completion. A finite-batch comparison does not prove starvation freedom under continuing arrivals.

Replay every slice for k=2

Completion 20/10/16 minus demand 10/4/6 gives waiting 10/6/10; no I/O. Each cell follows chronological order.

Time	Jobs	Remaining	Outcome
0–2; 2–4	P1; P2	8; 2	Both requeue
4–6; 6–8	P3; P1	4; 6	Both requeue
8–10; 10–12	P2; P3	0; 2	P2 completes; P3 requeues
12–14; 14–16	P1; P3	4; 0	P1 requeues; P3 completes
16–18; 18–20	P1; P1	2; 0	Same job continues to completion

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 6. Linux 3.2 scheduler source; Linux 3.2 scheduler features.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Deadlocks: what this chapter explains

- Distinguish necessary from sufficient deadlock conditions
- Prove how a global lock order excludes circular waiting
- Apply the Banker's safety test using complete resource vectors

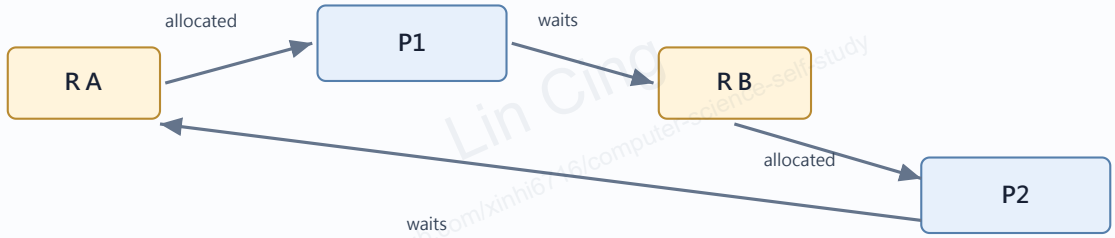
Central question. When can resource waiting become permanent, and which prevention or recovery strategy is justified by available information?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

System model

- A resource type has a defined number of instances. Processes request, use, and release instances; an unsatisfied request may block.
- In a resource-allocation graph, process-to-resource edges represent requests and resource-instance-to-process edges represent allocations. The graph records specified ownership and waiting, not every possible future action.

Single-instance cycle: A → P1 → B → P2 → A



Two single-instance owners wait for each other and cannot be preempted: deadlocked.

Resource state and concrete deadlocks

A resource type groups interchangeable instances. Two printers with incompatible location/function requirements belong to different types. Tables record type totals, free/allocated instances, owners, and each outstanding requester/type. In a deadlocked set every member waits for an event that only a waiting member of the same set can cause, with no permitted outside way to unblock it. The member can be itself: a thread holding a nonrecursive mutex that blocks trying to acquire it again cannot reach the release. This is a one-thread deadlock.

With three CD-RW drives, P1/P2/P3 each holds one and requests one more. Available=0 and none can finish, so all three deadlock. With one printer and one DVD, P1 holds DVD and requests printer; P2 holds printer and requests DVD. Neither can release, so their distinct resource types still deadlock.

Exact graph criteria and a multiple-instance counterexample

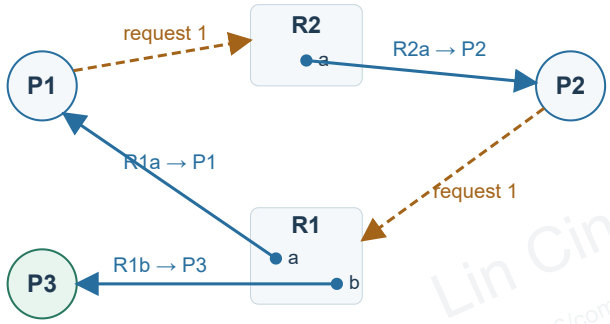
Mutual exclusion makes at least one resource exclusive. Hold-and-wait keeps an allocation while another request waits. No preemption forbids forced reclamation. Circular wait gives a dependency chain returning to its start. Circular wait already implies hold-and-wait; the four conditions are not logically independent.

A process node is a circle, resource type a box, instances dots. Request Pi→Rj becomes assignment Rj→Pi when granted; release removes the assignment. Assume a consistent snapshot: each request waits for one instance, cannot time out or be cancelled, and a blocked owner cannot be preempted or release its holdings early. No cycle implies no resource deadlock. If every resource type in the entire graph has one instance, there is a cycle if and only if there is a deadlock. Even in a mixed system, one particular cycle whose resource types all have one instance is sufficient to prove its participants deadlocked. A cycle involving a multiple-instance type alone does not decide deadlock: another instance holder may finish and release it. Not every deadlock in a mixed system must contain a cycle consisting solely of single-instance resource types.

Why can a cycle still make progress?

R1 has instances R1a/R1b; R2 has R2a. P3 needs nothing more. Each process finishes and returns all holdings after its current request is granted. Available=(0,0).

Initial state: Total=(2,1), Available=(0,0)



Can finish; no more requests

Read arrow direction

Solid: instance → owner

Dashed: waiter → resource type

R1 contains two distinct instances.

Cycle: P1 → R2 → P2 → R1 → P1

P3 holds the other R1 instance.

Let P3 finish to unlock progress.

P3 can release R1b even while P1 and P2 wait on the cycle.

Finish P3, then unblock P2 and P1

Completion order P3 → P2 → P1; vector order (R1,R2)

Event	R1a owner	R1b owner	R2a owner	Available
Initial: P1 and P2 wait	P1	P3	P2	(0, 0)
P3 finishes; returns R1b	P1	free	P2	(1, 0)
Grant R1b to P2	P1	P2	P2	(0, 0)
P2 finishes; returns R1b, R2a	P1	free	free	(1, 1)
Grant R2a to P1	P1	free	P1	(1, 0)
P1 finishes; returns R1a, R2a	free	free	free	(2, 1)

A grant decreases Available; an actual completion returns holdings and increases it. The completion order P3→P2→P1 alone omits the grants to P2 and P1. The figure includes both intermediate deductions.

Detection's Work is a simulation, not live inventory

Vectors use order (R1,R2). Current outstanding requests and existing allocations give:

Process	Allocation	Request	Work before	Work after simulated completion
P3	(1,0)	(0,0)	(0,0)	(1,0)
P2	(0,1)	(1,0)	(1,0)	(1,1)
P1	(1,0)	(0,1)	(1,1)	(2,1)

Check Request≤Work before each step, then set Work←Work+Allocation. This combines granting Request with later returning Allocation+Request, for a net increase of Allocation only. Work may include returns by processes simulated as finished. **The simulation does not execute them; initial Available remains (0,0).**

This test uses **current outstanding Request**, not the Banker's maximum remaining Need. Section 7.6 gives the complete detection algorithm.

Change only P3's next action to obtain deadlock

Keep every allocation and the requests of P1 and P2. Now require P3 to obtain one R2 before it can finish, adding P3→R2.

Initial state	P3 Request	Eligible at Work=(0,0)	Result
Original	(0,0)	P3	Completion order P3→P2→P1
Modified	(0,1)	None	P1, P2, P3 are deadlocked

P1 and P3 now both wait for R2a, held by P2. Both instances of R1 needed by P2 are held by those waiting processes. Every instance is trapped within the waiting set, with no owner able to return one first. This assumes no preemption, cancellation, extra instances, or early release before completion. Merely delaying P3's CPU service does not change resource feasibility in the original case; scheduling starvation and resource deadlock are different questions.

A single-instance lock cycle

Thread A acquires `first` then `second`; B uses the reverse order.

Time	Action	Holdings and waits
t0	A acquires first	A holds first
t1	B acquires second	A holds first; B holds second
t2	A requests second	A holds first and waits for B
t3	B requests first	B holds second and waits for A

Both locks have one instance. Neither owner can proceed or release, and no external holder can help. This interleaving deadlocks. If A finishes before t1, that schedule avoids the cycle, but one successful stress schedule cannot exclude the bad schedule.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Lock-order proof

- If a thread may request a new lock only above every lock it already holds, each dependency follows an increasing rank. A cycle would require rank(A)<...<rank(A), a contradiction.
- Callbacks, lock upgrades, condition-wait reacquisition, and exceptional paths must obey the same rule. Checking only the normal path is incomplete.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Policy information and cost

Policy	Information	Guarantee	Cost
Prevention	Restricted request protocol	Removes a necessary condition	Underutilization, refused work, rollback
Avoidance	Trustworthy declared maxima	Every grant preserves a safe sequence	Claims and runtime safety checks
Detection/recovery	Current ownership/request snapshot	Finds and repairs allowed deadlocks	Delay, scanning, lost victim work
Ignore	No general deadlock protocol	No general guarantee	Application timeout or restart

For one R1 and one R2, prevention orders F(R1)=1<F(R2)=2; a holder of R2 cannot request R1. Avoidance uses prior claim edges and refuses a grant that would create a cycle. Detection grants first and scans the wait-for cycle later. Their decision times differ.

Sharing read-only files removes exclusion only for those resources. Acquiring everything initially removes hold-and-wait but can reserve an idle printer until the final stage. Releasing everything before a new request may destroy uncommitted state. Preemption is useful only for restorable resources; it can reclaim a blocked process's allocations and restart when all needed resources become available. Strict ordering removes circular wait.

Policy information and cost (continued)

With injective F:R→N, a cycle would require F(R0)<F(R1)<…<F(Rn)<F(R0), hence F(R0)<F(R0), a contradiction. Sort dynamic account locks by stable ID before acquiring them; from/to parameter order can reverse across transactions. Callback, upgrade, wait reacquisition, and error paths participate in the same proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Claim edges and the complete Banker algorithm

For single instances, a claim Pi→Rj means Pi may request Rj later. A request changes claim to request, then trial assignment. Grant only if the resulting graph remains acyclic. Claims must be known before start, or added only while that process holds no assignments. Missing claims invalidate the guarantee. An adjacency-matrix cycle method costs $O(n^2)$.

For n processes and m resource types, Need=Max−Allocation and comparisons are componentwise. Legal inputs have nonnegative integers and $0 \le \text{Allocation} \le \text{Max} \le \text{Total}$.

```
Work = Available
Finish = [false] * n
while there is unfinished i with Need[i] <= Work:
    Work += Allocation[i]
    Finish[i] = true
safe iff all Finish
```

Once $\text{Need}_i \le \text{Work}$, lending Need_i and receiving $\text{Allocation}_i+\text{Need}_i$ has net effect $\text{Work}-\text{Need}_i+(\text{Allocation}_i+\text{Need}_i)=\text{Work}+\text{Allocation}_i$. It neither manufactures Need nor double-counts the temporary loan. Assume participants finish and return everything once their declared remaining needs are met.

Claim edges and the complete Banker algorithm (continued)

The greedy choice needs no backtracking. If a safe order exists and a different currently feasible Pi is chosen first, move Pi to the front and keep all other processes in their original order. Returning Pi's nonnegative allocation only increases resources faced by every remaining step. If the algorithm stops unfinished despite a supposed safe order, take the first unfinished Pj in that order. Every predecessor has already been removed by the algorithm; current Work contains their allocations and possibly more. Thus $\text{Need}_j \le \text{Work}$, contradicting the absence of a candidate. Completion provides a safe-order certificate; stopping unfinished proves unsafety within this model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Resource-request transaction

Validate an m-component nonnegative integer Request_i. If any component exceeds Need_i, reject as exceeding the claim. Otherwise if any component exceeds Available, wait. Negating componentwise $\leq$ means some component is greater, not all components.

Trial Available−=Request, Allocation_i+=Request, Need_i−=Request. Run safety. Commit only if safe; otherwise restore all three arrays and wait. The update is transactional. Typical safety complexity is $O(mn^2)$, at most n scans of n rows with m-component tests.

Counterexample: Total=(2,1), Allocation=(1,0), Need=(1,1), Available=(1,1) is legal. Request=(2,0) exceeds Need and Available in the first component. It must error before trial. Requiring both components to be greater would wrongly allow Available=(−1,1).

Banker T0 and every Work state

Total=(10,5,7), Available=(3,3,2).

Process	Allocation	Max	Need
P0	(0,1,0)	(7,5,3)	(7,4,3)
P1	(2,0,0)	(3,2,2)	(1,2,2)
P2	(3,0,2)	(9,0,2)	(6,0,0)
P3	(2,1,1)	(2,2,2)	(0,1,1)
P4	(0,0,2)	(4,3,3)	(4,3,1)

Each next Need fits. Work proceeds (3,3,2)→P1 (5,3,2)→P3 (7,4,3)→P4 (7,4,5)→P0 (7,5,5)→P2 (10,5,7). Thus ⟨P1, P3, P4, P0, P2⟩ is safe.

P1 requests (1,0,2), within Need(1,2,2) and Available. Trial gives Available=(2,3,0), Allocation1=(3,0,2), Need1=(0,2,0). The same order yields Work (2,3,0)→(5,3,2)→(7,4,3)→(7,4,5)→(7,5,5)→(10,5,7), so grant.

After that grant, P0 requests (0,2,0). Although available, its trial leaves Work=(2,1,0). No remaining Need fits, including P0's updated Need=(7,2,3), so rollback and wait. Unsafe means the declared worst-case completion guarantee is missing. Voluntary early release or smaller actual demands can still avoid a real deadlock.

Five resource types: find the smallest safe stock

Four processes share five non-preemptible, reusable resource types. Once a process receives its maximum remaining demand, it finishes in finite time and releases all holdings. Vectors are ordered by resources A, B, C, D, E.

Process	Allocation	Max	Need = Max − Allocation
P1	(1,0,2,1,1)	(1,1,5,1,2)	(0,1,3,0,1)
P2	(2,0,1,1,0)	(5,2,2,1,0)	(3,2,1,0,0)
P3	(1,1,0,1,0)	(3,1,3,1,0)	(2,0,3,0,0)
P4	(1,1,1,1,0)	(1,1,2,2,1)	(0,0,1,1,1)

Available = (0,0,x,1,1), where x is a nonnegative integer. Each x corresponds to Total = ΣAllocation + Available = (5,2,4+x,5,2). Find the smallest x giving a safe state. Different x values specify different candidate totals, not a changing inventory in one fixed system.

Follow Work to find the smallest x

x = 2: check Need ≤ Work at each step

Finishes	Old Work	Its Need		New Work
P4	(0,0,2,1,1)	(0,0,1,1,1)	→	(1,1,3,2,1)
P1	(1,1,3,2,1)	(0,1,3,0,1)	→	(2,1,5,3,2)
P3	(2,1,5,3,2)	(2,0,3,0,0)	→	(3,2,5,4,2)
P2	(3,2,5,4,2)	(3,2,1,0,0)	→	(5,2,6,5,2)

New Work = old Work + the process's Allocation

At x=0 no Need fits the initial Work. At x=1 only P4 can finish first; after its simulated release Work=(1,1,2,2,1). P1 lacks C, P2 lacks A and B, and P3 lacks A and C, so no next process fits.

At x=2, check Need ≤ Work along the diagram to obtain the safe sequence ⟨P4,P1,P3,P2⟩. Thus the smallest integer is **x=2**. Work is a simulation ledger: passing the check does not mean processes have finished or that real Available has become the final Work.

One resource type: a sufficient safety bound

There are m identical reusable instances and a finite set of n processes. Process i declares $1 \le Max_i \le m$. Its holdings are not preempted before completion; it blocks on a resource request only when insufficient instances are available, and completes in finite time and releases all holdings once its required resources are provided.

For unfinished processes, set $Need_i = Max_i - Allocation_i$. If

$$\sum_{i=1}^{n} Max_i < m + n,$$

every legal allocation has an unfinished process with $Need_i \le Available$. Completing such a process and repeating gives a safe completion sequence.

For m=10 and four processes each with Max=3, ΣMax=12 < 14, so every legal allocation is safe in this model. The condition is **sufficient, not necessary**: with five processes each having Max=3, ΣMax=15 is not less than m+n=15, yet the state in which nobody holds any instances and Available=10 is safe.

Why does every round have a candidate?

Let v be the current number of available instances. Suppose none of the n unfinished processes fits its maximum remaining demand. Then each integer $Need_i > v$ implies $Need_i \ge v+1$. Since $\Sigma Allocation_i = m - v$,

$$\sum_i Max_i = \sum_i Allocation_i + \sum_i Need_i \ge (m - v) + n(v + 1) = m + n + (n - 1)v \ge m + n,$$

contradicting the bound. A candidate therefore exists. This argument covers v>0 as well as the fully allocated case.

On completion, subtract its $Max_i \ge 1$ and reduce the process count by one; the strict inequality still holds for the remaining processes. Repeating yields a safe sequence. This is a resource-model existence guarantee: actual progress still requires sensible granting and scheduling, and the inequality alone cannot prevent starvation under an unfair scheduler.

Same free resources, different safety decisions

Use resource types A and B, Total=(4,3), and initial state S with Available=(1,1). Every vector is ordered A then B. The two trials below start independently from S.

Process	Allocation	Max	Need=Max−Allocation
P0	(1,0)	(3,2)	(2,2)
P1	(1,1)	(2,2)	(1,1)
P2	(1,1)	(3,2)	(2,1)

Allocated totals (3,2), so adding Available recovers Total. Each row satisfies Allocation+Need=Max and Max≤Total. Resources are nonpreemptible and returned together at completion. A process finishes in finite time once its actual demand is supplied.

S is safe: P1 fits Work=(1,1), then returns its allocation to give (2,2). P0 gives (3,2), then P2 gives (4,3). This is a simulated safety certificate, not a required CPU schedule.

Who receives the request changes the result

Each trial starts from S and requests (1,0). Both requests fit the requester's Need and Available. The second trial does not follow the first.

Same starting state and request; different safety outcomes

S: Available=(1,1); each branch starts independently from S

P0 requests (1,0)

Trial: Available=(0,1)

Allocation0=(2, 0)

Need0=(1, 2)

No Need fits (0,1)

unsafe: roll back all three updates

P1 requests (1,0)

Trial: Available=(0,1)

Allocation1=(2, 1)

Need1=(0, 1)

Simulated order: P1 → P0 → P2

safe: commit the trial state

Both trials leave (0,1), but ownership and remaining claims differ.
A safety order is a simulated certificate, not an executed schedule.

Availability alone is insufficient. Giving A to P1 leaves it needing only one B to finish. Giving the same A to P0 leaves nobody able to finish against its maximum remaining claim.

Reject P0: identify every failing component

Call P0's trial state U. Available=(0,1); only P0 changes, to Allocation0=(2,0), Need0=(1,2). Initialize Work=(0,1) and all Finish entries false.

Candidate	Trial Need	Failing component
P0	(1,2)	A: 1>0; B: 2>1
P1	(1,1)	A: 1>0
P2	(2,1)	A: 2>0

No candidate exists, so safety fails immediately. Restore S: add (1,0) to Available, subtract it from Allocation0, and add it to Need0. Leave other rows and Max unchanged.

Restoring only Available double-counts A. Available+ΣAllocation would be (5,3), exceeding Total=(4,3). Other requests must not observe a partial transaction. Protect check/commit with a lock, or validate the snapshot version at commit and retry after interference.

Grant P1: safety does not execute processes

Restart at S. Trial P1's (1,0) gives Available=(0,1), Allocation1=(2,1), Need1=(0,1). This table updates only simulated Work and Finish.

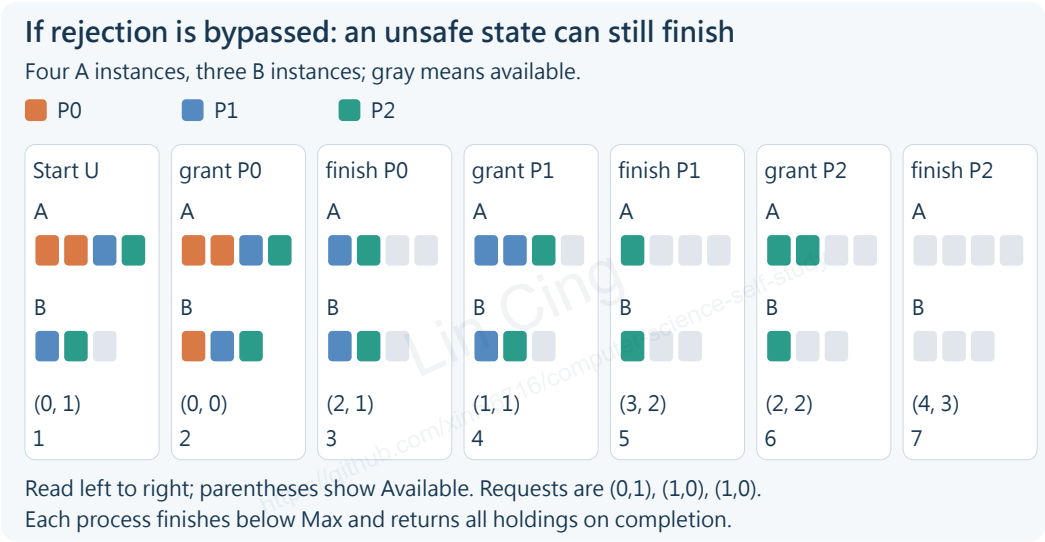
Simulated finish	Need≤old Work	Return Allocation	New Work	Finish: P0/P1/P2
Initial	—	—	(0,1)	F/F/F
P1	(0,1)≤(0,1)	(2,1)	(2,2)	F/T/F
P0	(2,2)≤(2,2)	(1,0)	(3,2)	T/T/F
P2	(2,1)≤(3,2)	(1,1)	(4,3)	T/T/T

Commit the **trial state**, whose Available remains (0,1). Do not copy final Work=(4,3) into Available or treat simulated Finish as actual termination. Real instances return only after actual completion.

Work adds Allocation because granting Need and later returning Allocation+Need has net change −Need+(Allocation+Need)=Allocation. Need creates no resources.

Unsafe does not mean deadlocked: an actual completion path

Suppose P0's rejected trial U is committed anyway. Max bounds demand; it need not be exhausted.



Requests (0,1), (1,0), (1,0) fit the claims in U. This is **one** successful future, not a worst-case guarantee.

The same U also permits deadlock

At U, suppose P0/P1/P2 instead request their entire remaining Need: (1,2), (1,1), (2,1). Each request respects Max, but Available=(0,1) satisfies none. All block while retaining A. With no cancellation, preemption, or early return, this is deadlock.

The two futures share Allocation, Max and Available; their actual requests differ. Safety uses maximum remaining Need to certify a conservative completion order. Detection uses current outstanding Request. Do not silently turn a possible future Need into a present Request.

An unsafe state has no safe sequence, yet can have a successful actual execution. Safe sequence is defined against maximum claims. An observed completion order need not be a safe sequence in that technical sense.

Detection reduction and recovery states

For single instances, replace Pi→Rq→Pj by Pi→Pj. A cycle is deadlock exactly under the exclusive single-instance assumptions. A cycle through a multiple-instance type needs allocation/request analysis; a concrete cycle containing only single-instance types still proves its members deadlocked under the stated blocking assumptions.

```
Work = Available
Finish[i] = (Allocation[i] == zero_vector)
while there is i with not Finish[i] and Request[i] <= Work:
    Work += Allocation[i]
    Finish[i] = true
remaining false holders form the detected blocked set
```

Detection uses current Request, not maximum Need. In the optimistic reduction model, a process whose current request is met completes in finite time, returns all its allocation, and issues no additional requests before completion. Initializing zero-allocation processes to true means they hold no resources needed by others; it does not assert that they actually finished or are not waiting. New requests need another snapshot. A consistent snapshot is required; reduction costs $O(mn^2)$.

Detection: reduce the baseline snapshot

Available=(0,0,0). The complete base snapshot is:

Process	Allocation	Request
P0	(0,1,0)	(0,0,0)
P1	(2,0,0)	(2,0,2)
P2	(3,0,3)	(0,0,0)
P3	(2,1,1)	(1,0,0)
P4	(0,0,2)	(0,0,2)

Work transitions P0→(0,1,0), P2→(3,1,3), P3→(5,2,4), P1→(7,2,4), P4→(7,2,6). All Finish become true: under the stated optimistic model, this snapshot admits a simulated order that clears the current requests. This does not guarantee that future real requests cannot cause deadlock.

Detection: alter one request

Change only Request2 to (0,0,1). Simulating P0 gives Work=(0,1,0); no other Request fits, so Finish=(true, false, false, false, false) and the unresolved holders are {P1, P2, P3, P4}. With no cancellation, preemption or early return, they form a deadlocked set in this model. Simulation does not actually complete P0 or change Available. If P0 has not finished, aborting P1 and safely reclaiming Allocation1=(2,0,0) changes actual Available from (0,0,0) to (2,0,0).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recovery: rerun reduction after termination

Restart reduction with Work=Available, excluding the terminated P1:

Simulated completion	Current Request	Work after returning Allocation
Initial; P1 terminated	—	(2,0,0)
P0	(0,0,0)	(2,1,0)
P3	(1,0,0)	(4,2,1)
P2	(0,0,1)	(7,2,4)
P4	(0,0,2)	(7,2,6)

Check Request≤Work before each row and add only the existing Allocation. Assuming each survivor finishes in finite time without new requests after its current Request is granted, this certifies a possible completion order; actual Available rises only when they really return resources. Starting at actual Available=(2,1,0) is valid only if P0 has already completed outside the simulation.

Frequent scans reduce detection delay and can identify the latest requester, but cost more. Periodic scans can see several cycles together. A snapshot does not guarantee every future state.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recovery unit and victim fairness

Abort all deadlocked processes to break every cycle at the cost of all unfinished work, or abort one, reclaim its resources, rerun detection, and stop once survivors can progress. Victim policy considers priority, interactivity, executed/remaining time, held/needed resources, reclamation difficulty, and how many aborts are needed.

Files and devices can retain partial external effects. Recovery needs undo, journals, checkpoints, or device reset, not merely unlocked mutexes. Preemption must specify victim, consistent rollback point, and protection against repeated victimization.

T2 has checkpoint cost 2 and rollback count 0; T4 cost 3/count 0. Choose T2, roll back and release 2 resources. Rerun detection; if T4 now completes, stop aborting. Increment T2's rollback count. A cost such as execution_i+k×rollback_count_i penalizes repeated selection, but a formal finite bound also needs workload assumptions. A limit on consecutive selections alone does not ensure fairness: A and B can alternate forever.

A single-instance P1/P2 wait cycle is deadlock. Separately, a ready P3 selected zero times while P4 completes five admissions is evidence of possible starvation; only an unbounded execution or policy argument proves no finite bound. No fabricated wait cycle is needed. Ordering can remove deadlock while a CPU policy still starves the lock holder.

Distinguish the evidence for each conclusion

Observation	Conclusion	Additional evidence
Four conditions	Deadlock possible	No remaining completion transition
Single-instance graph no cycle	No deadlock in snapshot	Validated graph
Single-instance cycle	Deadlock	All cycle resources truly single-instance
Multi-instance cycle	Deadlock possible	Availability/reduction
Unsafe	No worst-case avoidance guarantee	Actual current requests
One long wait while others finish	Possible starvation	Admission policy and waiting bound

Chapter conclusions

- Necessary conditions alone do not establish every deadlock claim
- Avoidance depends on valid maximum claims
- Detection, recovery, and data consistency must be designed together

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 7.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Main memory: what this chapter explains

- Derive base-and-limit, segment, and page translations
- Distinguish internal and external fragmentation
- Calculate TLB costs and explain page-table levels

Central question. How can a logical address space map safely onto finite, physically scattered memory?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Address protection: check before relocation

In a single contiguous-region model, base is the start of a process's physical region and limit is its length. Given CPU offset d, hardware checks `0 ≤ d < limit` before calculating physical address `base + d`; otherwise it raises a protection exception. Privileged software controls base and limit.

For base=`0x8000`, limit=`0x2000`, offset `0x1FFF` maps to `0x9FFF`; `0x2000` equals the length and is rejected. A multi-byte access must have its entire range inside the region, not just its starting address.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Swapping

- Classical swapping moves an entire process image to backing storage to free RAM. Restoration needs the saved image, address-space metadata, and an appropriate placement or relocation mechanism.
- Demand paging selectively loads pages rather than requiring the whole image. Its granularity and cost differ from whole-process swapping.

For an 80 KiB process and 100 KiB available backing store, preserve its required state on swap-out. A DMA buffer must remain valid until I/O completes, for example by pinning its frame.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

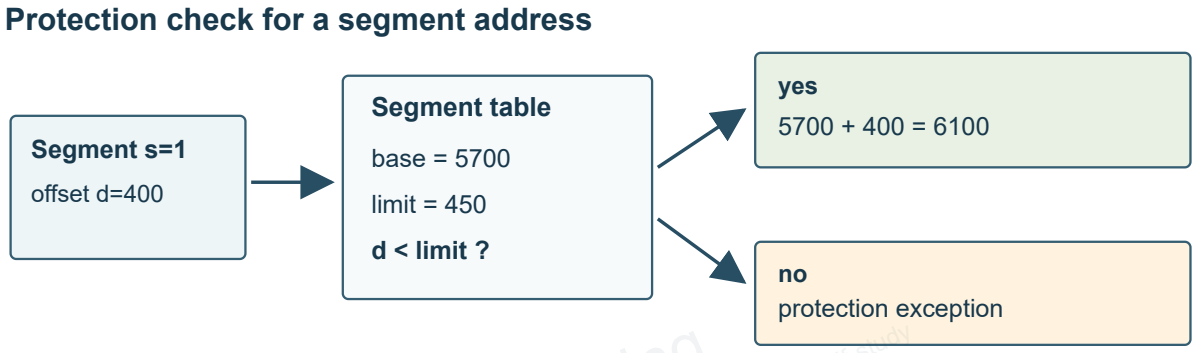
Contiguous allocation: enough total space can still fail

Holes in address order are 100, 500, 200 KiB; a new contiguous request needs 180 KiB. First-fit selects 500 and leaves 320; best-fit selects 200 and leaves 20; worst-fit selects 500 and leaves 320. Each policy is independently applied to the same initial state.

Four separated 5 KiB holes total 20 KiB, yet none accommodates a contiguous 10 KiB request: this is **external fragmentation**. Allocating two 4 KiB units for a 6 KiB request leaves 2 KiB inside the allocated units: **internal fragmentation**. Compaction requires relocation and coordination with active I/O and address references.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Segmentation: each segment has its own limit



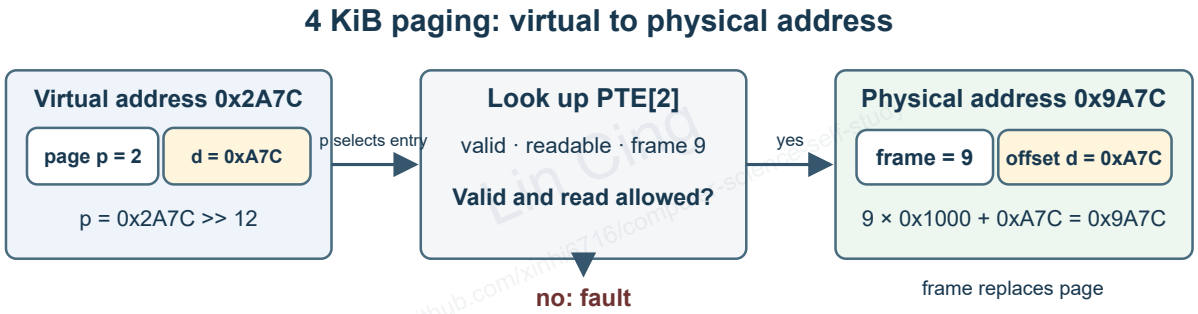
Offset 449 is valid; offset 450 is out of bounds.

Let segment 0 have base=1000 and limit=339, segment 1 base=5700 and limit=450, and segment 2 base=6300 and limit=1280, all in bytes. `<0,330>` is valid because `330<339` and maps to 1330; `<1,400>` maps to 6100; `<2,1380>` raises an exception because `1380≥1280`.

The limit is a length, not the last legal offset: segment 1 permits offset 449 and rejects 450. Segments may have different lengths, permissions and physical bases; each segment occupies a contiguous physical region in this model.

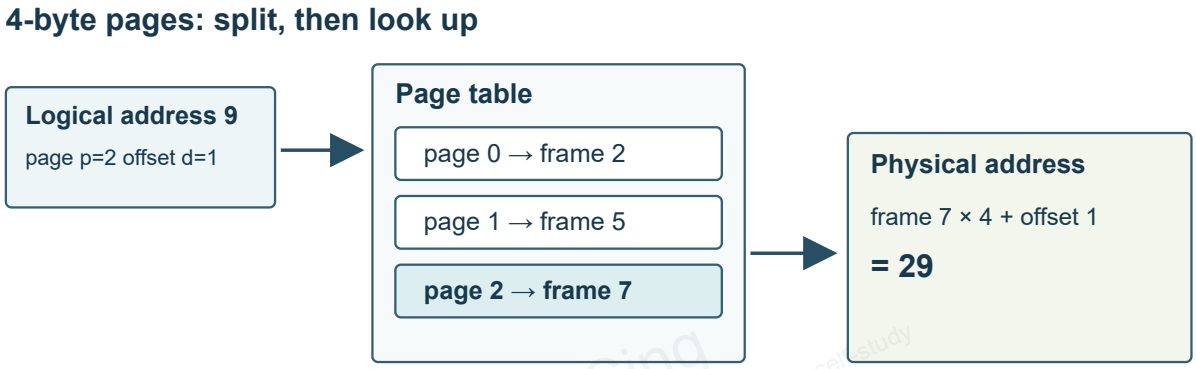
Paging

- Split a virtual address into page p and offset d. If an authorized resident entry maps p to frame f, physical address=f*page_size+d. The offset is unchanged.
- Smaller pages reduce tail waste but increase the number of entries for a fixed range. Larger pages increase TLB coverage per entry and may reduce table work, while increasing allocation and transfer granularity.



Original vector diagram for this course.

Translate address 9 to address 29



The offset is unchanged; the table maps a page number to a frame.

Page size is 4 bytes. For logical address 9, `p = floor(9/4) = 2` and `d = 9 mod 4 = 1`. The table maps page 2 to frame `f = 7`; the physical address is `7×4+1 = 29`. The offset is preserved. Page and frame numbers need not match.

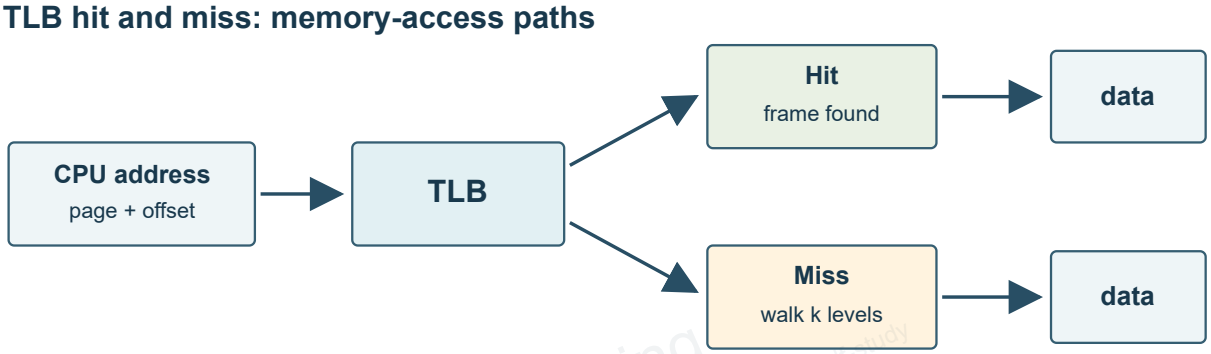
Only valid pages 0, 1 and 2 are shown. An invalid page-table entry does not specify a frame; the system raises a fault or handles a missing page according to its policy.

Address bits and fragmentation

With 1 KiB = 2^{10} bytes per page and 8 = 2^3 pages per address space, the logical address has 3 page bits plus 10 offset bits, **13 bits** total. With 32 = 2^5 physical frames, the physical address has 5 frame bits plus the same 10 offset bits, **15 bits** total. Addresses are byte-based and the full encoding ranges are usable.

A contiguous 32 KB logical region in 10 KB allocation units needs four pages, or 40 KB. If the first three are full, the last uses 2 KB, leaving 8 KB of internal fragmentation. Here KB means 1000 bytes for this capacity example; 1 KiB in the binary address example is 1024 bytes.

TLB hit and miss timing



Serial model: hit = TLB + data; miss = TLB + k table reads + data.

On a TLB hit, use the frame to access data. On a miss, consult the page table first. Assume serial lookup, equal memory-access costs, fixed walk depth, and no cache, page-walk cache, replacement or page fault.

For one level, TLB lookup = 10 ns, RAM access = 200 ns and hit rate = 0.8. A hit costs `10+200=210 ns`; a miss costs `10+200+200=410 ns`. EAT is `0.8×210 + 0.2×410 = 250 ns`. This is an average under the stated model, not a measured latency for every CPU.

Adding a page-table level

Now use two page-table levels, a 20 ns TLB lookup, 200 ns RAM access and hit rate 0.8; keep the other assumptions. A hit costs `20+200=220 ns`. A miss reads two entries and then data, costing `20+2×200+200=620 ns`.

EAT is `0.8×220 + 0.2×620 = 300 ns`. Both TLB lookup time and walk depth changed between these examples, so the entire 50 ns difference cannot be attributed to the extra level. Real hardware may overlap lookups or cache intermediate entries.

In this serial model, hit rate h, lookup time t, memory time m and k table reads on a miss give EAT = h(t+m)+(1−h)(t+km+m).

Protection and sharing

- Entries may encode read, write, execute, privilege, and presence rules. Different address spaces can share a physical frame through explicitly authorized mappings.
- Read-only code sharing saves frames. An accidentally writable or executable mapping can break isolation. Presence alone does not grant every operation.

For example, P1 and P2 can map the same frame as user-readable and read-only. Both can read the same contents; user writes fail. Removing one mapping leaves the other intact. Before reclaiming the frame, account for every reference and invalidate affected TLB entries.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two-level tables: storage depends on mappings

Use 32-bit byte addresses, 4 KiB pages and 4-byte entries. A 4 KiB table holds $1024 = 2^{10}$ entries, so the address splits into `10-bit root index | 10-bit leaf index | 12-bit offset`. Each leaf table covers 1024 pages, or 4 MiB of virtual address space.

To map one page, a complete root plus one leaf needs `4 KiB + 4 KiB = 8 KiB` of table storage; this excludes the data page. Mapping all 2^{20} virtual pages needs 1024 leaves: `4 KiB + 1024×4 KiB = 4 MiB + 4 KiB`. Sparse mappings avoid unused leaves; 8 KiB is not a general maximum.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Inverted tables: count physical frames

For 1 GiB physical memory and 8 KiB pages/frames, there are $2^{30} / 2^{13} = 2^{17}$ frames. If each per-frame record is exactly 4 bytes, these records occupy $2^{17} \times 4 = 2^{19}$ bytes = 512 KiB.

This counts only the frame-indexed records. A hash/index for (address-space ID, virtual page), collision chains and records for multiple mappings of a shared frame can require extra space. A real record must also hold its required identifiers and permissions; 4 bytes is an example assumption, not a universal size.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

IA-32 linear-address walk

Fix IA-32 protected mode with CR0.PG=1, CR4.PAE=0, ordinary 4 KiB pages, and linear address 0x12345678 after segmentation. The selector first locates an LDT/GDT descriptor; segment base/limit and rights produce the linear address. Paging translates that result, so the given number is not the pre-segmentation logical address.

A 4 KiB directory/table has 1024 four-byte entries, giving 10/10/12 bits. For linear L,

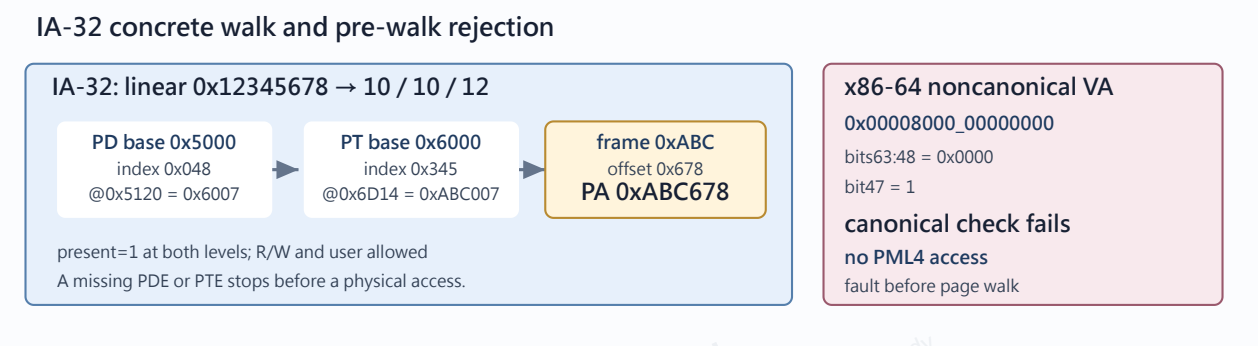
$$PDI = (L \gg 22)\ \&\ 0x3FF, \quad PTI = (L \gg 12)\ \&\ 0x3FF, \quad d = L\ \&\ 0xFFF.$$

CR3 base 0x5000 and next PT base 0x6000 yield:

Level	Index	Entry address	Entry value	Meaning
Directory	0x048	0x5000+4×0x048=0x5120	0x00006007	Present, next base 0x6000, RW/user
Table	0x345	0x6000+4×0x345=0x6D14	0x00ABC007	Present, frame 0xABC, RW/user

Offset 0x678 gives PA0xABC000+0x678=0xABC678. This path uses directory, table, and target page, each 4 KiB. Table-allocation costs are separate from an individual walk.

IA-32 linear-address walk (continued)



An absent PDE stops before reading the PT; an absent PTE cannot supply a usable frame. On a CPU supporting 4 MiB pages, `CR4.PSE=1` permits PDE `PS=1` to terminate at the directory and use 22 offset bits. With `CR4.PSE=0`, 32-bit paging ignores PS and treats the PDE as a reference to a 4 KiB page table, not a 4 MiB mapping. This differs from the fixed trace.

Canonical x86-64 addresses and four levels

Fix 48-bit canonical virtual addresses, four levels,4 KiB pages,8-byte entries. Index fields are[47:39],[38:30],[29:21],[20:12], with offset[11:0]. Each table has 512 entries. First require 0≤v<2^64, then require every bit 63:48 to repeat bit 47.

$$canonical(v) \iff (v \gg 48) = \begin{cases} 65535, & ((v \gg 47)\ \&\ 1) = 1, \\ 0, & ((v \gg 47)\ \&\ 1) = 0. \end{cases}$$

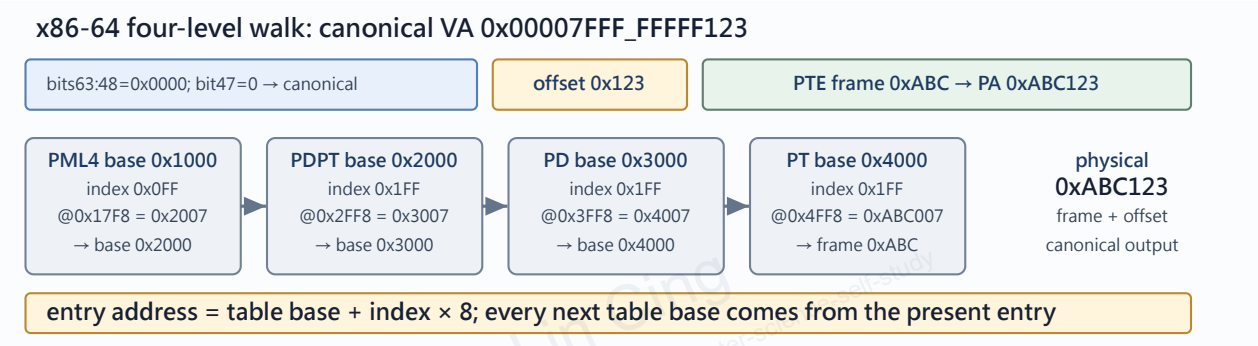
Comparing only bit 63 to bit 47 is insufficient. v=0x0001_0000_0000_0000 has both 0 but upper 16 bits=1, so is noncanonical. Flipping any one sign-extension bit of a legal address invalidates it. Ranges 0…0x00007FFFFFFFFFFF and 0xFFFF800000000000…0xFFFFFFFFFFFFFFFF are legal; the interval between is illegal.

For VA 0x00007FFF_FFFFF123, bit 47=0 and upper 16 bits=0. CR3=0x1000; subsequent bases are 0x2000,0x3000,0x4000.

Level	Index	Offset	Entry address	Entry value	Next base/frame
PML4	0x0FF	0x7F8	0x17F8	0x2007	PDPT 0x2000
PDPT	0x1FF	0xFF8	0x2FF8	0x3007	PD0x3000
PD	0x1FF	0xFF8	0x3FF8	0x4007	PT0x4000
PT	0x1FF	0xFF8	0x4FF8	0xABC007	Frame 0xABC

Canonical x86-64 addresses and four levels (continued)

Every address is base+index×8; every next base comes from the present entry. Offset 0x123 is retained, so PA=0xABC123.



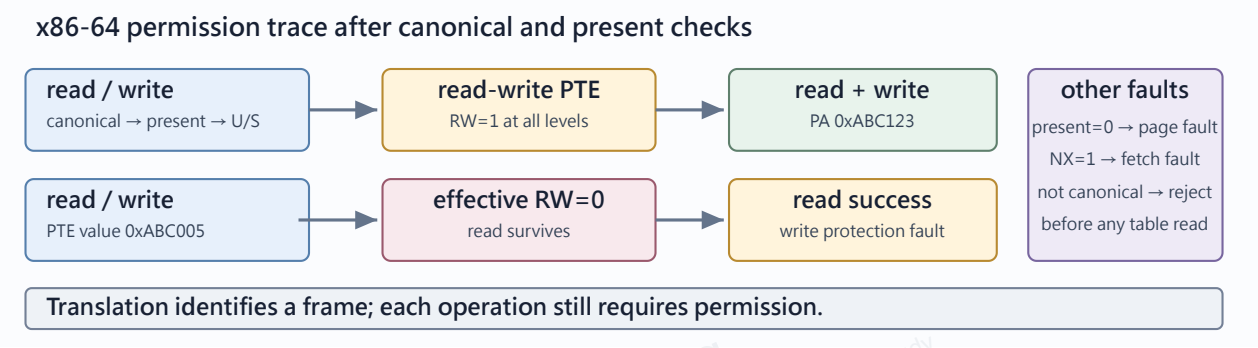
2 MiB and 1 GiB encodings end higher in the walk and require other formats/alignment and larger offsets. The 9/9/9/9/12 split applies only to this 4 KiB configuration.

Mapping and operation permission

Use four-level 48-bit x86-64 paging, user-mode `CPL=3`, NX-capable hardware with `IA32_EFER.NXE=1`, and no intervening SMEP, SMAP, PKE or other added restriction. Canonical check precedes all table memory accesses. Present establishes usable entries, U/S restricts user access at every level, effective RW restricts writes, and NX restricts instruction fetch.

Case	Canonical/mapping	Permissions	Read	Write	Fetch
Normal trace	Canonical, four present levels	All user/RW, NX0	0xABC123	0xABC123	Allowed
PTE 0xABC005	Same present path	User, RW0, NX0	0xABC123	Protection fault	Allowed
PML4[0x0FF] absent	Canonical, no mapping	No effective mapping	Page fault	Page fault	Instruction page fault
PTE NX1	Canonical, present	User/RW, NX1	Allowed	Allowed	Fetch fault
VA 0x00008000_00000000	Noncanonical	No walk	Reject	Reject	Reject

Mapping and operation permission (continued)



The last address has bit 47=1 but upper 16 bits=0. The canonical address 0x00007FFF_FFFFF000 still page-faults if any required level is absent. The illegal interval ends at 0xFFFF7FFFFFFFFFFF, immediately below the upper canonical range. For a user write, effective RW=0 causes a protection fault; supervisor writes also depend on `CR0.WP` and are outside this table. If `EFER.NXE=0`, an entry with XD/NX set has a reserved bit and causes a reserved-bit page fault, rather than simply preventing fetch. A root, frame, and valid canonical input do not alone authorize every operation. PAE, five levels, large pages, or another physical width require the matching masks and entries. Updated entries need required TLB invalidation; a stale cached permission can outlive a changed PTE.

ARMv7-A short descriptors

Fix AArch32, short descriptors, TTBCR.N=0, TTBCR.PD0=0, SCTLR.AFE=0, SCTLR.TRE=0, PL1 table construction, and 4 KiB small pages. P1 and P2 use distinct, unreused ASIDs, e.g. 1 and 2. This is not AArch64 long-descriptor translation. N=0 selects TTBR0 for all input VAs. The complete L1 has 4096 four-byte entries,16 KiB, aligned 16 KiB. Sparse use does not permit allocating only the populated entries; other entries need valid architectural encodings, including invalid ones.

$$L1addr = TTBR0 + ((VA \gg 20)\ \&\ 0xFFF) \times 4.$$

A coarse L1 descriptor points to a256-entry L2,1 KiB, aligned 1 KiB; bits 31:10 supply its base.

$$L2addr = L2base + ((VA \gg 12)\ \&\ 0xFF) \times 4, \qquad PA = frame + (VA\ \&\ 0xFFF).$$

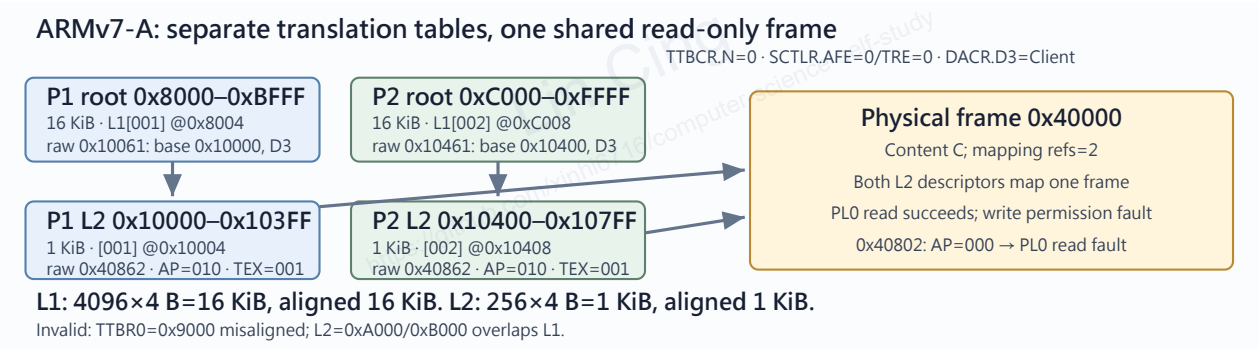
Storage	Base	Inclusive range	Size/alignment
P1 root	0x8000	0x8000–0xBFFF	16 KiB, base mod 0x4000=0
P2 root	0xC000	0xC000–0xFFFF	16 KiB, base mod 0x4000=0
P1 L2	0x10000	0x10000–0x103FF	1 KiB, base mod 0x400=0
P2 L2	0x10400	0x10400–0x107FF	1 KiB, base mod 0x400=0

All four ranges are distinct. Only the final data frame is shared.

ARMv7-A short descriptors (continued)

Process/VA	L1 index/address	L2 base	L2 index/address	Frame	PA
P1 0x00101000	0x001 /0x8004	0x10000	0x001 /0x10004	0x40000	0x40000
P2 0x00202000	0x002 /0xC008	0x10400	0x002 /0x10408	0x40000	0x40000
P1 0x00101A5C	0x001 /0x8004	0x10000	0x001 /0x10004	0x40000	0x40A5C

Offset[11:0]=0xA5C does not enter either table index. Keeping only 8bits would incorrectly give 0x4005C.



Raw ARM words, domains, AFE and AP

Both L1 descriptors use domain 3; DACR. D3=01 means Client. Domain is in L1 and inherited by L2. With AFE=0, small-page AP[2] is bit 9 and AP[1:0] bits 5:4, not section bits 11:10.

Descriptor	Raw	Decode
P1 L1	0x10061	Type 01 coarse, base 0x10000, domain 3
P2 L1	0x10461	Type 01 coarse, base 0x10400, domain 3
Both L2	0x40862	Frame 0x40000, type 10, XN0, nG1, AP010, TEX001, C=B=0

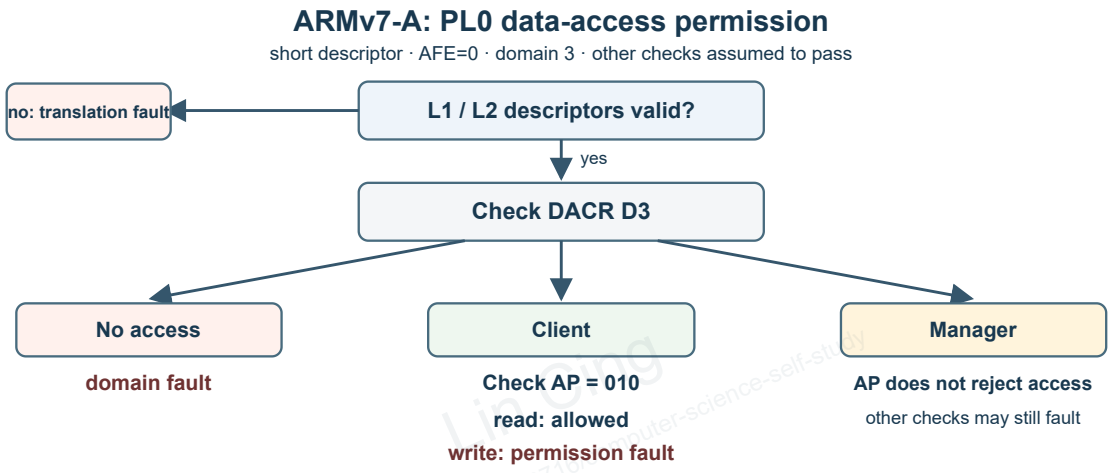
Decode field=(raw>>shift)&mask. AP2=(raw>>9)&1, AP1=(raw>>5)&1, AP0=(raw>>4)&1, TEX=(raw>>6)&7, C=(raw>>3)&1, B=(raw>>2)&1, XN=raw&1. AP010 gives PL1 read/write and PL0 read-only. With TRE0, TEX001/C0/B0 means Normal non-cacheable. Fetch additionally assumes WXN=UWXN=0, readable mapping, Client domain, and no other restriction. XN0 alone cannot prove executable Strongly-ordered memory.

0x40802 instead decodes AP000 and denies PL0 read/write even though the frame is valid. 0x40022 has AP010 but TEX/C/B=0/0/0, Strongly-ordered; it only illustrates data AP, not guaranteed fetch.

Raw ARM words, domains, AFE and AP (continued)

Mode	PL0 read	PL0 write	Reason
AFE0, Client, AP010	Success	Permission fault	Read-only PL0
AFE0, No access domain	Domain fault	Domain fault	Domain rejects first
AFE0, Manager domain 11	Not rejected by AP	Not rejected by AP	Manager bypasses AP; other faults remain possible
AFE1, Client, same 0x40862	Access-flag fault	Access-flag fault	AP0 is now access flag 0; software-managed flag model
AFE1, Client,0x40872	Success	Success	Flag 1, AP[2:1]=01 grants PL0 RW

Raw ARM words, domains, AFE and AP (continued)



A valid mapping is not enough: the domain and access permissions also matter.

Changing DACR or AFE changes the interpretation, so the AFE0 read-only result cannot be reused unchanged.

ARM unmap, mapping counts and stale translations

Initially the frame contains C and has exactly these two mappings, with no other references or pins; mapping count is 2, both entries 0x40862, Client parents, TTBCR.PD0=0, distinct unreused ASIDs 1 and 2 for P1 and P2, and no cached translation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ARM unmap, mapping counts and stale translations (continued)

State/operation	Translation/permission	Result	Content/references
S0 P1 read	0x00101000→0x40000, AP permits	Success	C, refs 2
S0 P1 write	Valid mapping, AP denies	Permission fault	C unchanged
S0 P2 read	0x00202000→0x40000, AP permits	Success	C, refs 2
S0 P2 write	Valid mapping, AP denies	Permission fault	C unchanged
S1 remove P1	L2 invalid, required TLB maintenance complete	P1 translation fault	P2 remains, refs 1
S2 P2 read	Independent tables still valid	Success	refs 1
S3 remove P2	L2 invalid, maintenance complete	P2 translation fault	refs 0; reclaim only after all stale translations are unusable

ARM unmap, mapping counts and stale translations (continued)

An absent mapping is not a valid-but-read-only mapping. After descriptor changes, perform architecture-specific VA/ASID TLBI and required barriers on every relevant core that might hold the old translation; wait for completion before reclaiming the frame. nG1 associates translations with their ASIDs; ASID does not determine domain or AP. Clearing a PTE alone cannot authorize frame reuse while an old TLB can still access it.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Table storage must be valid before sharing is analyzed

Invalid layout	Failure
P2 TTBR0=0x9000, N0	0x9000 mod 0x4000=0x1000, root misaligned
P1 root 0x8000, P2 root 0x9000	Ranges 0x8000–BFFF and 9000–CFFF overlap
P1 L2=0xA000	Inside P1 root
P2 L2=0xB000	Inside P1 root
Only 4 KiB allocated for L1	4096×4 entries require 16 KiB

Legal sharing has independent roots/L2 storage and one shared frame. Verify page identity, storage, domain/AP/XN, references, and TLB state together. A writable P1 mapping would modify the frame P2 reads; that is writable sharing, not read-only code. Preserving P2's old data while P1 changes requires COW or a separate frame, not merely changing permissions in one entry.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter conclusions

- A mapping and its permissions are distinct parts of access validation
- Classify fragmentation before comparing allocation schemes
- Effective access time is a model with explicit cache and fault assumptions

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 8. Intel 64 and IA-32 SDM, Vol. 3A; Arm Architecture Reference Manual ARMv7-A/R.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Legality, mapping and residency

Virtual memory separates logical ranges, software mappings, and physical residency. A legal virtual page need not be resident. Present/resident bits have architecture-specific meanings; valid is not one universal cross-architecture name. A handler must separately establish legal range/backing, translation state, and read/write/execute/user permission.

Fix 4 KiB pages and processes A/B with identical virtual layouts.

Virtual range	Legality/rights	Content source	A mapping	B mapping
[0x0000,0x1000)	Legal RX, not W	Same program code	Shared F0	Shared F0
[0x1000,0x2000)	Legal RW private	Zero-fill-on-demand	Nonresident	Nonresident
[0x2000,0x3000)	Legal RW private	Separate data	F1	F2
[0x3000,0x4000)	Guard, no mapping	None	None	None

Each has 3 legal pages and 2 resident mappings. Four resident mappings consume only 3 unique frames because code F0 is shared. Adding per-process resident counts double-counts shared frames.

A first read 0x1004 can allocate zeroed storage and restart. Writing 0x0004 violates code permission. Reading 0x3004 is guard access, not an invitation to create a mapping merely because a free frame exists. Nonpresent PTEs need VMA/backing metadata to distinguish these outcomes.

Legality, mapping and residency (continued)

Reservation, committed backing, and residency are different quantities. A VMA can reserve addresses before policy promises RAM/swap. A virtual range larger than RAM does not guarantee all pages can be written simultaneously or that allocation can never fail.

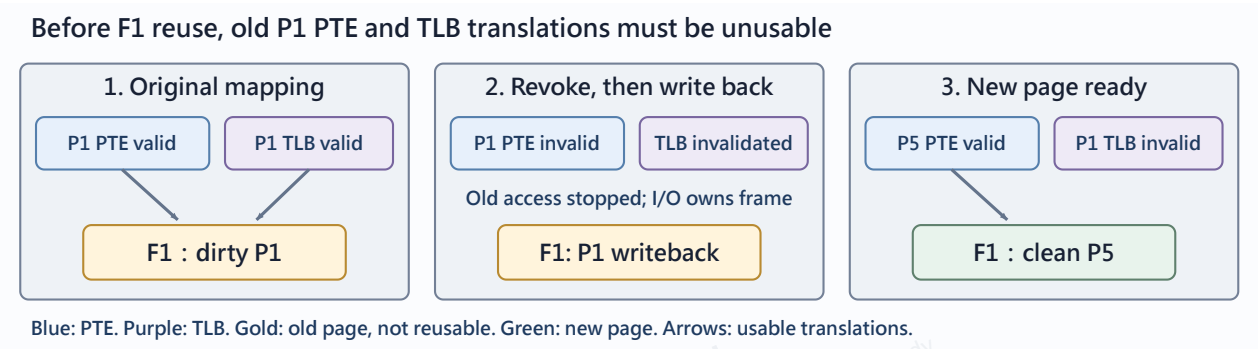
Lin Cing https://github.com/xinhi6716/computer-science-self-study

Demand-fault service and frame reuse

Fix 4 KiB pages and full frames F0→P0, F1→dirty P1, F2→P2. P5 is readable in a legal VMA/backing object but nonresident. Replacement chooses F1.

Step	State/action	Can P5 be read?
0	CPU accesses nonpresent P5 and traps	No
1	Validate VMA, backing, and read rights	No
2	Revoke P1 PTE; complete old TLB invalidation and prevent further writes to F1	No
3	Write back dirty P1; I/O still owns F1, so cannot overwrite it	No
4	Load 4096 bytes of P5 into F1; mapping not yet published	No
5	Publish P5→F1, perform required translation maintenance, restart instruction	Yes

Demand-fault service and frame reuse (continued)



This is1 fault,1 disk write,1 disk read, and one old-translation maintenance phase. On one CPU that can be local invalidation; other CPUs caching the old translation require completed shootdown. Installation may require additional architecture-specific invalidation/barriers, so this is not a claim of exactly one TLB instruction everywhere. Nonoverlapped 8 ms write plus 8 ms read costs 16 ms without other overhead. A clean victim removes the write, leaving 8 ms.

Let m be the total cost of a normal reference and F the total cost of a faulting reference through successful retry; $0 \le p \le 1$. The bound below assumes $F > m$ and $EAT_max \ge m$, and counts retry only once.

Demand-fault service and frame reuse (continued)

$$EAT = (1-p)m + pF, \qquad p \le \frac{EAT_{max} - m}{F - m}.$$

For m=200 ns and clean F=8,000,000 ns, EAT≤400 requires p≤200/7,999,800≈2.5001×10⁻⁵. Dirty write+read F=16,000,000 reduces it to≈1.2500×10⁻⁵. The same fault ratio can therefore have different service cost. Another fixed baseline m=100 ns, F=8 ms, EAT≤200 gives p≤100/7,999,900≈1.25×10⁻⁵; keep its denominator separate.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fault classification and stale-TLB counterexample

The handler checks legal VMA/backing, software mapping versus nonresident hardware translation, requested rights or authorized COW exception, free/victim eligibility including dirty/writeback/pin state, then complete contents/permissions before retry.

CPU0 clearing P1 PTE does not revoke CPU1's cached writable P1→F1. Loading P5 into F1 immediately lets CPU1's next P1 store corrupt P5. Reuse requires old translation revocation, necessary shootdown acknowledgments, and completed old I/O/ownership. Conversely, loading P5 without installing its mapping makes the restarted instruction fault again.

Read-ahead may bring P5/P6 in one transfer, changing fault counts and frame pressure. Every prefetched page still needs legality, backing, a frame and a consistent mapping. A batch read cannot authorize an adjacent guard page.

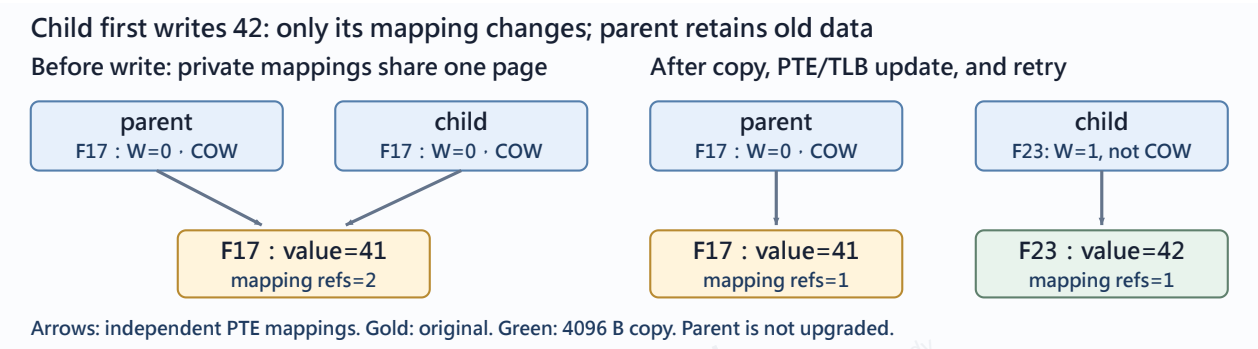
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

COW is a write-permission fault

Fork preserves separate tasks and page-table metadata while parent/child PTEs initially share F17. Hardware marks both read-only; software records COW. The translation exists, so the child's failed store is not a missing-backing-page fault.

State/event	Parent PTE	Child PTE	F17	F23
S0 fork complete	F17, W0, COW	F17, W0, COW	41, refs 2	Absent
S1 child tries write	Unchanged	Unchanged	41, refs 2	Absent
S2 allocate/copy 4096 B, not published	F17 RO	Still F17	41, refs 2	41, handler-owned
S3 publish, maintain TLB, retry 42	F17, W0, COW	F23, W1, non-COW	41, refs 1	42, refs 1, dirty

COW is a write-permission fault (continued)



Validate COW→allocate→copy→publish child PTE/TLB→retry. Making both mappings writable or clearing only the parent's COW lets the child change the parent's original data. Here refs counts user mappings, not the handler's temporary hold.

The child handler need not visit or immediately upgrade the parent's PTE. On the parent's later write fault, an exclusively owned private anonymous page with no other sharing/pin restrictions may be made writable without copying. One mapping alone does not authorize writing a shared file-cache page in place. Fork followed by exec can unmap untouched COW pages without copying. Zero-fill must clear a reused frame before exposing it to another process; demand paging, COW, and zero-fill have different fault causes and data sources.

Replacement policies and full trace

FIFO uses load order; LRU last use; OPT farthest next use, requiring an offline future; CLOCK gives referenced pages another chance. Exact LRU ordering on every access has hardware/concurrency cost. Reference-bit histories or clock scans approximate it and have their own sampling/hand rules.

Fix 3 frames, initially empty, references 1,2,3,4,1,2,5,1,2,3,4,5. H/F means hit/fault. FIFO is load order, LRU oldest→newest, CLOCK physical slots; these are policy metadata rather than one shared frame numbering.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Replacement policies and full trace (continued)

t/page	FIFO	LRU	OPT	CLOCK
1/1	[1]F	[1]F	[1]F	[1]F
2/2	[1,2]F	[1,2]F	[1,2]F	[1,2]F
3/3	[1,2,3]F	[1,2,3]F	[1,2,3]F	[1,2,3]F
4/4	[2,3,4]F	[2,3,4]F	[1,2,4]F	[4,2,3]F
5/1	[3,4,1]F	[3,4,1]F	[1,2,4]H	[4,1,3]F
6/2	[4,1,2]F	[4,1,2]F	[1,2,4]H	[4,1,2]F
7/5	[1,2,5]F	[1,2,5]F	[1,2,5]F	[5,1,2]F
8/1	[1,2,5]H	[2,5,1]H	[1,2,5]H	[5,1,2]H
9/2	[1,2,5]H	[5,1,2]H	[1,2,5]H	[5,1,2]H
10/3	[2,5,3]F	[1,2,3]F	[2,5,3]F	[5,3,2]F
11/4	[5,3,4]F	[2,3,4]F	[5,3,4]F	[5,3,4]F
12/5	[5,3,4]H	[3,4,5]F	[5,3,4]H	[5,3,4]H

Totals FIFO 9, LRU 10, OPT 7, CLOCK 9. Victims, with dash for free-frame insertions/hits:

Replacement policies and full trace (continued)

Policy	Victims
FIFO	−,−,−,1,2,3,4,−,−,1,2,−
LRU	−,−,−,1,2,3,4,−,−,5,1,2
OPT	−,−,−,3,−,−,4,−,−,1,2,−
CLOCK	−,−,−,1,2,3,4,−,−,1,2,−

FIFO hits do not reorder; LRU hits move to most recent. OPT evicts the latest next use or never-used-again page. Aging can shift the reference bit into a history byte each sampling period, with smaller values suggesting older usage. Changing sample periods, hand positions, or bit clearing changes the approximation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trace three replacement policies (steps 1–10)

Use the same sequence `7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1` with three initially empty frames, demand loading only, and no prefetch or external invalidation. Slots are fixed physical positions; `-` is empty, H is a hit, F is a fault, and `F→x` evicts page `x`.

FIFO evicts the earliest loaded resident page; LRU evicts the least recently used page; OPT evicts the page whose next use is farthest away, with no future use treated as infinity. OPT ties choose the smallest page number.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trace three replacement policies (steps 1–10) (continued)

t	page	FIFO : slots ; event	LRU : slots ; event	OPT : slots ; event
1	7	7,–,– ; F	7,–,– ; F	7,–,– ; F
2	0	7,0,– ; F	7,0,– ; F	7,0,– ; F
3	1	7,0,1 ; F	7,0,1 ; F	7,0,1 ; F
4	2	2,0,1 ; F→7	2,0,1 ; F→7	2,0,1 ; F→7
5	0	2,0,1 ; H	2,0,1 ; H	2,0,1 ; H
6	3	2,3,1 ; F→0	2,0,3 ; F→1	2,0,3 ; F→1
7	0	2,3,0 ; F→1	2,0,3 ; H	2,0,3 ; H
8	4	4,3,0 ; F→2	4,0,3 ; F→2	2,4,3 ; F→0
9	2	4,2,0 ; F→3	4,0,2 ; F→3	2,4,3 ; H
10	3	4,2,3 ; F→0	4,3,2 ; F→0	2,4,3 ; H

Trace three replacement policies (steps 11–20)

Continue from the previous page. Every cell gives the three fixed slots **after** the access and the event. A FIFO hit does not alter load order; an LRU hit updates recency.

t	page	FIFO : slots ; event	LRU : slots ; event	OPT : slots ; event
11	0	0,2,3 ; F→4	0,3,2 ; F→4	2,0,3 ; F→4
12	3	0,2,3 ; H	0,3,2 ; H	2,0,3 ; H
13	2	0,2,3 ; H	0,3,2 ; H	2,0,3 ; H
14	1	0,1,3 ; F→2	1,3,2 ; F→0	2,0,1 ; F→3
15	2	0,1,2 ; F→3	1,3,2 ; H	2,0,1 ; H
16	0	0,1,2 ; H	1,0,2 ; F→3	2,0,1 ; H
17	1	0,1,2 ; H	1,0,2 ; H	2,0,1 ; H
18	7	7,1,2 ; F→0	1,0,7 ; F→2	7,0,1 ; F→2
19	0	7,0,2 ; F→1	1,0,7 ; H	7,0,1 ; H
20	1	7,0,1 ; F→2	1,0,7 ; H	7,0,1 ; H

Across 20 accesses FIFO incurs **15**, LRU **12**, and OPT **9** faults. At step 8, LRU compares pages 2, 0 and 3 last used at steps 4, 7 and 6, so it evicts 2. OPT sees 0 next used at step 11, later than 2 at step 9 and 3 at step 10, so it evicts 0.

Every CLOCK hand and reference bit

Start hand 0. Fill free slots 0/1/2 with R1 without moving hand. Hits set the page's R1 without moving hand. When full, clear each encountered R1 and advance; replace first R0, set new R1 and advance once.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every CLOCK hand and reference bit (continued)

t/page	Slots 0/1/2	R0/R1/R2	Next hand	H/F, victim
1/1	[1]	[1]	0	F,–
2/2	[1,2]	[1,1]	0	F,–
3/3	[1,2,3]	[1,1,1]	0	F,–
4/4	[4,2,3]	[1,0,0]	1	F,1
5/1	[4,1,3]	[1,1,0]	2	F,2
6/2	[4,1,2]	[1,1,1]	0	F,3
7/5	[5,1,2]	[1,0,0]	1	F,4
8/1	[5,1,2]	[1,1,0]	1	H,–
9/2	[5,1,2]	[1,1,1]	1	H,–
10/3	[5,3,2]	[0,1,0]	2	F,1
11/4	[5,3,4]	[0,1,1]	0	F,2
12/5	[5,3,4]	[1,1,1]	0	H,–

At4 it clears a full circle then evicts 1 in slot 0; at 7 another circle evicts 4. The initialization and hit behavior are necessary to reconstruct these states.

FIFO anomaly with both capacities

t/page	3-frame queue	4-frame queue	Cumulative faults 3/4
1/1	[1]F	[1]F	1/1
2/2	[1,2]F	[1,2]F	2/2
3/3	[1,2,3]F	[1,2,3]F	3/3
4/4	[2,3,4]F	[1,2,3,4]F	4/4
5/1	[3,4,1]F	[1,2,3,4]H	5/4
6/2	[4,1,2]F	[1,2,3,4]H	6/4
7/5	[1,2,5]F	[2,3,4,5]F	7/5
8/1	[1,2,5]H	[3,4,5,1]F	7/6
9/2	[1,2,5]H	[4,5,1,2]F	7/7
10/3	[2,5,3]F	[5,1,2,3]F	8/8
11/4	[5,3,4]F	[1,2,3,4]F	9/9
12/5	[5,3,4]H	[2,3,4,5]F	9/10

FIFO anomaly with both capacities (continued)

After 7 the smaller set contains 1 but the larger does not; access 8 faults only with 4 frames. FIFO's capacity-dependent fault history determines load order, so increasing capacity can raise faults. This Belady anomaly does not apply to every policy.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

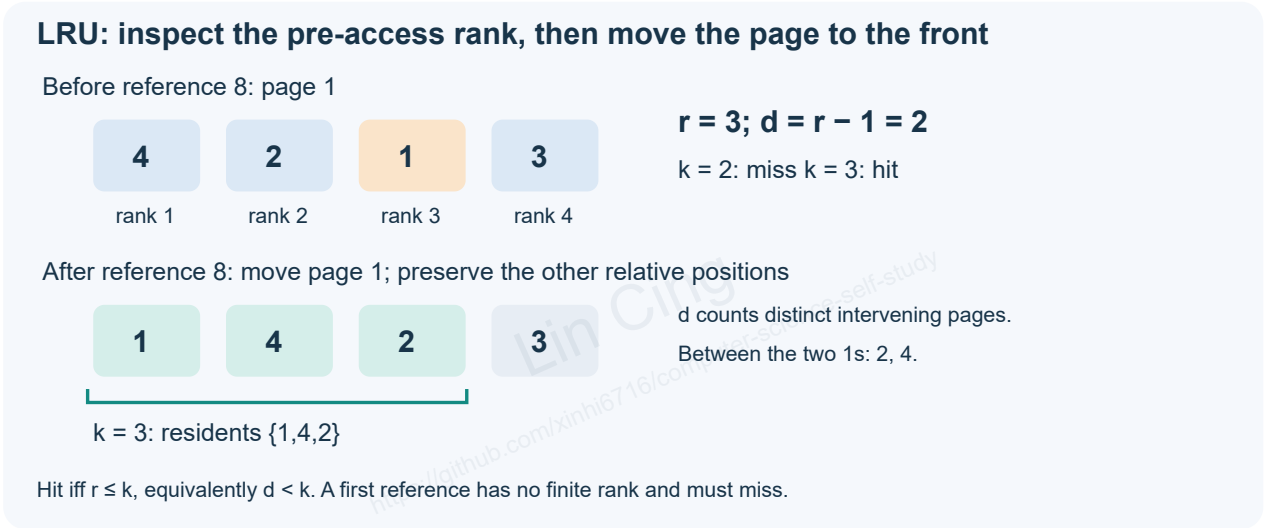
General LRU inclusion proof

Form one recency stack of every seen page from newest to oldest, independent of capacity. On every access move that page to the front. Inductively, exact k-frame LRU holds the first k entries, or all entries when fewer exist. Initially empty; a hit reorders without changing membership of those k; a miss inserts at front and pushes out old entry k, exactly the LRU victim.

Thus $R_k(t) \subseteq R_{k+1}(t)$. If k+1 faults at time t, its requested page was absent from R(k+1) before access and therefore absent from subset Rk. Fault indicators satisfy the inequality pointwise, and summing gives $F_{k+1} \leq F_k$. The finite 12-access example cannot replace this induction. FIFO load histories and CLOCK hand/reference scans depend on capacity and do not supply that same stack invariant.

Predict LRU hits with stack distance

Start empty with equal-sized pages and demand paging; exclude prefetching, pins, and external invalidations. Rank r is the 1-based position before access, newest first.



Reuse distance d counts distinct intervening pages: d=r−1. Some texts call r the stack distance; check whether indexing starts at 0 or 1.

One trace predicts faults at two capacities

t	Page	Pre-access r	Post-access order: newest → oldest	LRU 2	LRU 3
1	1	First	1	F	F
2	2	First	2,1	F	F
3	1	2	1,2	H	H
4	3	First	3,1,2	F	F
5	1	2	1,3,2	H	H
6	2	3	2,1,3	F	H
7	4	First	4,2,1,3	F	F
8	1	3	1,4,2,3	F	H

Faults: 6 with two frames; 4 with three. F denotes a fault and H a hit. Before reference 8, page 1 has rank 3: two-frame LRU has evicted it, whereas three-frame LRU retains it.

Why is the hit condition d < k?

Immediately after using page p, it is first. Each distinct other page subsequently referenced places one page ahead of p; repeating a page already ahead does not increase that number. Thus r=d+1 before the next use. Exact k-frame LRU retains precisely the first k pages, so a hit is equivalent to $r \leq k$, or $d<k$. First references must fault.

A working-set window counts references; an LRU capacity counts distinct pages. At t=7 the four-reference window contains four pages. Three-frame LRU holds only {4,2,1}; page 3 remains in the working set but has been evicted. WS(t,4) is not automatically the resident set of four-frame LRU.

OPT exchange proof and its cost boundary

Fix an offline finite string, identical initial residents, unit miss cost, and no dirty cost difference. At a full-frame miss OPT evicts q with latest next use, treating never again as infinity. Take an optimal schedule's first disagreement, evicting p instead. Modify it to retain p and evict q. Before next(p), q is not requested earlier, so other replacements can track the original with only p/q differing. If the original evicts q sooner the states can align. Otherwise at next(p), the modified schedule already has p while the original faults. Use that saved transfer to bring q in and remove the original's chosen victim, aligning states without more transfers. If neither page is reused, the difference costs nothing.

Any early transfer in that alignment can be delayed until the page's first request without increasing cost, giving a legal demand schedule. Hence an optimal schedule agrees with this OPT step; eliminate first disagreements successively. This is a unit page-in proof, not a proof for heterogeneous page sizes, dirty page-out, or unequal I/O weights. More frames can simulate any smaller-capacity optimum by leaving space unused, so OPT's minimal cost cannot rise. This simulation does not require every OPT tie-break to produce nested residents.

Fault service is trap+dirty swapout+swapin. Half dirty victims gives an expectation between read-only and write+read paths. Minimum fault count need not minimize a dirty-weighted cost.

LFU/MFU frequency lifetimes

Start 3 empty frames, count 1 on load, increment on hit, discard count on eviction, and break ties by earliest load. References 1,1,1,2,3,4 give counts{1:3,2:1,3:1} before the final access. LFU evicts 2→{1:3,3:1,4:1}; MFU evicts 1→{2:1,3:1,4:1}. Old high counts can preserve 1 after locality shifts to 2/3/4. Aging counts can reduce stale history but adds a decay parameter. MFU's belief that low counts are newly loaded future candidates is a workload assumption, not universal optimality.

Quotas, rounding and victim pools

Equal allocation first gives floor(F/m) each and needs a remainder rule. Proportional allocation floors $a_i = Fs_i / \sum s_j$, then distributes remaining frames by specified remainder/priority/order.

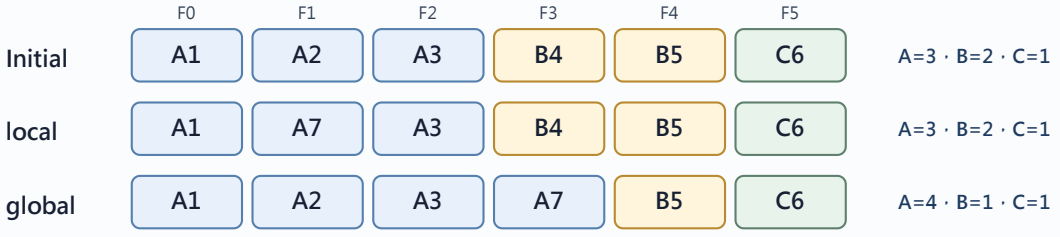
For F64 and sizes A100/B200/C300, raw shares 10.667/21.333/32, floors 10/21/32 total 63. Largest remainders 0.667/0.333/0 give the last frame to A, final 11/21/32 total 64.

In a separate six-frame model, A={1,2,3}, B={4,5}, C={6}; last-use A1=10, A2=5, A3=8, B4=2, B5=7, C6=9. A faults 7 with no free frame.

Policy	Eligible victim	Victim	New residents
Local	Only A	A2	A={1,7,3}, B={4,5}, C={6}
Global	All processes	B4	A={1,2,3,7}, B={5}, C={6}

Quotas, rounding and victim pools (continued)

One A7 fault: local/global each reuses one frame



All rows have six frames. After global eviction, the next B4 access faults.

Global raises A's residency 3→4, lowers B2→1, and retains 6 total. It neither deletes A3 nor invents B8. B's next access 4 now faults, spreading pressure. Local limits interference but cannot borrow an idle other's frame. Allocation defines quotas/bounds; replacement defines eligible victims. Global need not mean unrestricted admission.

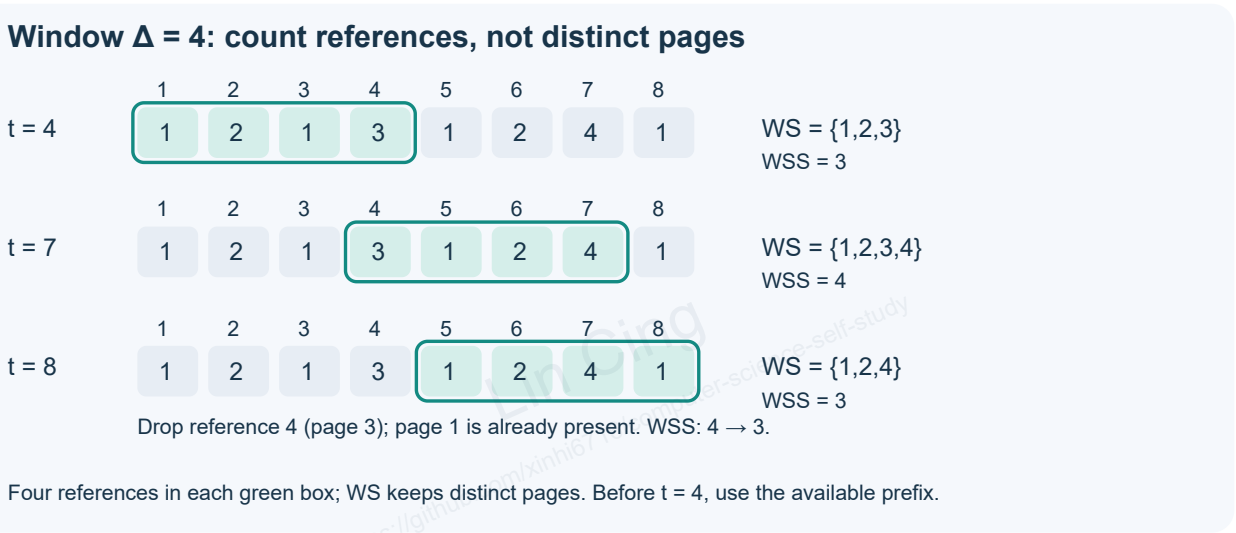
Minimum frames versus a working set

Instruction code/source/destination pages and restart rules create a minimum progress requirement. Evicting an earlier required operand to load the last can fault forever when an instruction cannot partially complete. That is different from WSS, a performance-demand estimate over a chosen window. Fault-handler and table kernel memory are separately guaranteed system costs.

If lower bounds li apply, require Σli≤F. Grant them first, then split F−Σli with explicit rounding. Taking max(proportional_share, li) independently can exceed F. For F6 and three processes each requiring 3 unshared frames, total minimum 9 is impossible regardless of fairness. Suspend at least one, change execution, or add frames. Pins and kernel reserves also reduce the user pool.

A moving working set is not a fixed-capacity cache

Let p_t be the page at reference t, and choose $\Delta = 4$.



$WS(t,4) = \{p_j : \max(1, t-3) \le j \le t\}$, with $WSS = |WS|$. At t=8 the window [1,2,4,1] contains three distinct pages, not four.

An expired reference need not remove its page

Maintain a count for each page: increment for the incoming reference and decrement for the outgoing one. Remove the page from WS only when its count becomes zero.

t	Incoming	Outgoing reference	Window	Working set	WSS
4	3	None	1,2,1,3	{1,2,3}	3
5	1	Reference 1: page 1	2,1,3,1	{1,2,3}	3
6	2	Reference 2: page 2	1,3,1,2	{1,2,3}	3
7	4	Reference 3: page 1	3,1,2,4	{1,2,3,4}	4
8	1	Reference 4: page 3	1,2,4,1	{1,2,4}	3

Do not remove page 1 at t=5: only one reference expires, and two references to page 1 remain. At t=8 the final reference to page 3 expires, so WSS decreases.

Working sets and evidence of thrashing

Thrashing means paging dominates useful execution. With a four-reference window:

Process	References	Working set	WSS
A	1,2,3,1,2,4,5,1	{1,2,4,5}	4
B	6,7,6,8,7	{6,7,8}	3
C	9,9,9	{9}	1

Without sharing, demand D=8 exceeds six available frames. Active localities cannot all remain resident. For shared pages, count unique backing pages rather than summing WSS.

Excess demand predicts pressure; it does not measure thrashing. A short window misses reused pages; a long one retains stale demand. Check fault rates, useful CPU utilization, and paging queues. A lock or one slow I/O can also leave the CPU idle.

PFF hysteresis controller

For fixed 5-access windows and upper 0.4/lower 0.1, FFFFF gives 1.0 and requests another frame; FHHHH gives 0.2 and holds; HHHHH gives 0 and may release a spare under pressure. Two thresholds avoid allocation/reclaim oscillation near one threshold.

```
for each process every sample_window:
    if accesses_in_window == 0: continue
    rate = faults_in_window / accesses_in_window
    if rate > upper and free_frames > 0:
        grant_one_frame(process)
    else if rate > upper and free_frames == 0:
        choose one process to suspend
    else if rate < lower and memory_pressure and quota > minimum_required:
        reclaim_one_spare_frame(process)
```

High PFF with no free frame calls for reduced multiprogramming, swapping/reallocation, or rejected admission. More CPU priority cannot manufacture resident pages. Short windows are noisy, long windows react slowly; thresholds, window and suspension rule jointly define the controller.

Mapped files, visibility and durability

MAP_SHARED processes map the same file-cache page; shared updates still require application synchronization. MAP_PRIVATE writes COW into private pages without writing the original file. Creating a mapping is not residency; a store is not persistence.

These experiments independently start at 7, with no external file modification.

Shared event	P1	P2	Cache F40	File
S0	F40	F40	7, clean	7
S1 P1 writes 8 and synchronizes	F40	F40	8, dirty	7
S2 P2 reads after synchronization	F40	F40	Reads 8, not durable	7
S3 completed required flush	F40	F40	8, clean	8

Private event	P1/value	P2/value	Cache	File
V0	F40/7, COW	F40/7, COW	F40/7	7
V1 only P2 writes 9	F40/7, uncopied	New F53/9	F40/7	7
V2 P1 writes 8	New F52/8	F53/9	F40/7	7

V1 copies 4096 B; cumulative V2 copies 8192 B. P2's write does not copy the still-unwritten P1 mapping. MAP_PRIVATE does not guarantee whether later external file changes appear, so this is not an automatic file snapshot.

Mapped files, visibility and durability (continued)

msync/platform counterparts trigger writeback/order under their actual API/filesystem/cache/flush contracts. Dirty shared file pages need safe writeback before reclaim. Dirty private COW pages use swap/backing rather than overwriting the source file. Failed writeback must retain error/dirty state instead of declaring clean and reusing data.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

mmap and read/write costs

Fix cold cache, one complete 4 KiB page, no read-ahead, successful operations. Disk→cache I/O is different from cache↔user CPU copy.

Path	Calls	CPU copies	Translation	Write durability
read	1 syscall	Cache→user 4096 B	Ordinary	No write promise
write	1 syscall	User→cache 4096 B	Ordinary	Requires specified sync/commit
First mapped read	Earlier mmap call; current fault/trap	No cache→user copy; disk still reads 4096 B	Install/maintenance	No write
Mapped shared store	No syscall until flush	Zero bytes in this model	Access/dirty tracking	Specified msync/flush needed

read updates file offset; mapped loads use MMU after faults create PTEs. A page fault is an exception, not an explicit user syscall. Both write and mapped store can finish with only dirty cache data.

Kernel allocation and buddy splitting

Kernel interrupt/I/O paths may need nonsleeping, contiguous, or specially owned memory, so ordinary user demand-paging assumptions do not suffice. Buddy allocators split power-of-two blocks; slabs cache fixed-size objects.

A256 KiB region with 4 KiB minimum rounds 21 KiB up to 32. Ranges below use KiB.

Step	Split/result
0	Free[0,256)
1	[0,128),[128,256)
2	Split left into[0,64),[64,128)
3	Split left into[0,32),[32,64)
4	Allocate[0,32),21 used and 11 internal waste

Free blocks[32,64),[64,128),[128,256) total 224 KiB. A64 KiB request can use an existing 64 block; coalescing[0,64) specifically requires releasing its allocated 32 first. Blocks have unique order/buddy ownership and never overlap. For appropriately aligned a and block size 2^k bytes, buddy=a xor 2^k. Freeing[0,32) merges with[32,64), then[64,128), then[128,256). Each step checks only the matching-order buddy; maximum tree height log2(256/4)=6.

Slab object counts and the denominator

Ignoring metadata, a4096 B slab page holds 16 objects of256 B. This is a model, not an exact count for every slab implementation. Allocate 2 and free 1: live objects 1, free objects 15, live bytes 256, retained slab bytes 4096. Process RSS cannot be inferred from kernel object count because it uses process mappings as its denominator.

Retaining free objects saves initialization and page-allocation work, so footprint need not shrink with live count. NUMA caches add node locality and remote-reclaim/allocation cost.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Page size, prepaging and pins

One MiB needs 256 PTEs with 4 KiB pages or16 with 64 KiB pages. Larger pages give 16 times coverage per TLB entry but larger tails, transfer sizes, and read amplification. The number of faults does not decrease monotonically with page size; page-table cost, TLB coverage, transfer size and tail waste require separate comparisons.

An8-page workload uses locality{P1, P2, P3}. Prepaging those 3 does 3 reads and the later first 3 accesses fault 0 times. Pure demand does the same 3 reads but 3 faults. The difference is trap/latency placement, not eliminated data I/O. Guessing unused P4 adds a needless read.

DMA stage	F40 pin/state	Reclaimable?
t0 user buffer	Pin 0	Yes if otherwise eligible
t1 device queued	Pin 1, DMA pending	No
t2 device completes	DMA 0, pin not released	No
t3 completion releases pin	Pin 0, eligible LRU	Can be considered

Reusing at t1 lets DMA overwrite a different page. Device completion alone does not end the completion protocol; pins need normal/error release paths. A pin, lock bit and mlock are different API mechanisms, but none allows duplicate allocation of a still-used frame. Long pins shrink the evictable pool.

Page size, prepaging and pins (continued)

Counterexample: larger pages need not reduce faults. Fix a 256 KiB frame budget for this workload, initially empty, exact LRU, demand paging, no prefetch and no competing references. Cycle through five distinct 64 KiB regions, touching the same single 4 KiB subpage in each. With 4 KiB pages there are 64 frames: five initial faults then hits. With 64 KiB pages there are only four frames: cyclic LRU faults on every access. Fix both the byte budget and the reference pattern before comparing.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

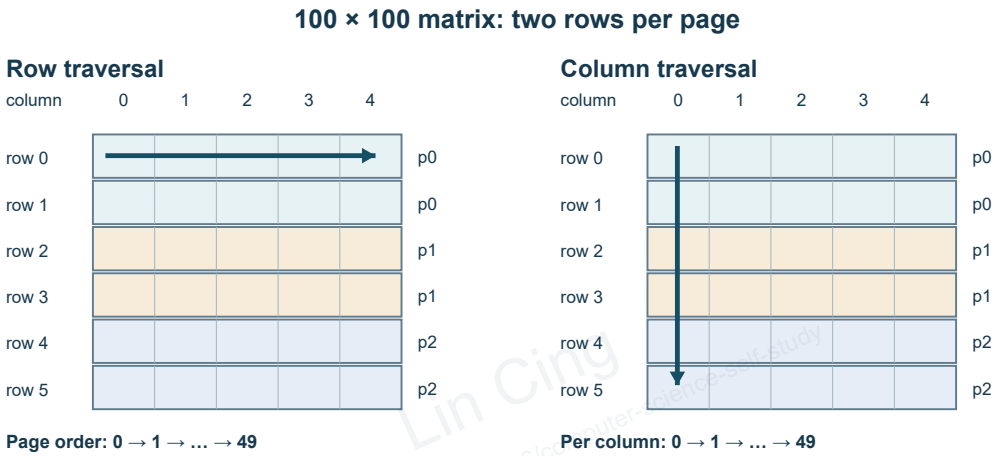
Access order changes page locality

C/C++ row-major int A[4][1024],4-byte int,4 KiB-aligned base,4 KiB pages makes one row per page. Count only array references,2 initially empty exact-LRU frames; code/stack/cache-line costs are separate.

Loop order	Page references	Faults
Rows outside, columns inside	Each 0,1,2,3 repeated 1024 times	4, first access per row
Columns outside, rows inside	0,1,2,3 repeated 1024 rounds	4096, each reuse already evicted

First 8 column states oldest→newest are[0]F,[0,1]F,[1,2]F,[2,3]F,[3,0]F,[0,1]F,[1,2]F,[2,3]F, then repeat. Identical 16 KiB and operation count produce different fault counts under this residency model. Real OS frame grants/prefetch/policies can change that difference; it is not a measured universal loop speed ratio.

Matrix traversal changes the page sequence



First 6 rows and 5 columns shown; each pair of rows shares a page.

The matrix has 100×100 one-byte elements in row-major storage, starting at a 200-byte page boundary. Each page holds exactly two rows, so there are 50 pages. Count only matrix data; code and other data do not compete for the data frames.

Pages are numbered from 0. Row i (i=0…99) is on page `floor(i/2)`. Each traversal starts with an empty frame set.

Why does column traversal incur 5,000 faults?

Traversal	Data frames, policy	Page order to check	Faults
Row by row	2, LRU	Use each page for 200 consecutive accesses before the next	50
Column by column	2, LRU	Each column visits pages 0,1,…,49; two adjacent accesses per page	100×50 = 5,000
Column by column	50, LRU	First column loads all pages; later accesses hit	50

Before page 0 is reused in the next column, the traversal touches 49 other distinct pages; two frames cannot retain it. Within each column the two rows of one page are adjacent: the first access faults and the second hits. With 50 frames the whole matrix remains resident. Page size, alignment, frame capacity and policy all affect the result; traversal order alone does not determine the fault count.

Linux 3.2 reclaim branches

Fix asynchronous order-0 Linux v3.2. Free pages below the zone low watermark can wake kswapd; scanning does not guarantee every candidate frees memory. It scans inactive anon/file lists and balances active/inactive. page_check_references distinguishes PTE references and PageReferenced; shrink_page_list then tests unmap, dirty, writeback, mapping/refcounts. This is not modern multi-gen LRU.

Fix current_is_kswapd=true, priority 9, DEF_PRIORITY12, may_writepage=true, filesystem/I/O/unmap allowed, successful page locks, no concurrent rereference/mutation, removable P2/P3 mapping/refcounts, successful asynchronous P3 pageout, non-lumpy reclaim for P1, and PageReferenced=0 for all five candidates. The PTE-referenced column is a distinct state. P4 is an already-isolated candidate that became mlocked before its unevictable cull completed; normally an mlocked page is not a stable ordinary inactive-LRU candidate.

Linux 3.2 reclaim branches (continued)

Page	Type	PTE referenced	Dirty	Writeback	mlocked	Action	Free delta
P1	Anon	1	0	0	0	Activate	0
P2	File	0	0	0	0	Reclaim after final removal	+1
P3	File	0	1	0	0	Start async writeback	0
P4	Anon	0	0	0	1	Cull to unevictable	0
P5	File	0	1	1	0	Defer	0

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Linux 3.2 reclaim branches (continued)

Linux 3.2: outcomes for five candidate pages

Teaching snapshot: order-0, kswapd, asynchronous writeback; P4 isolated before culling

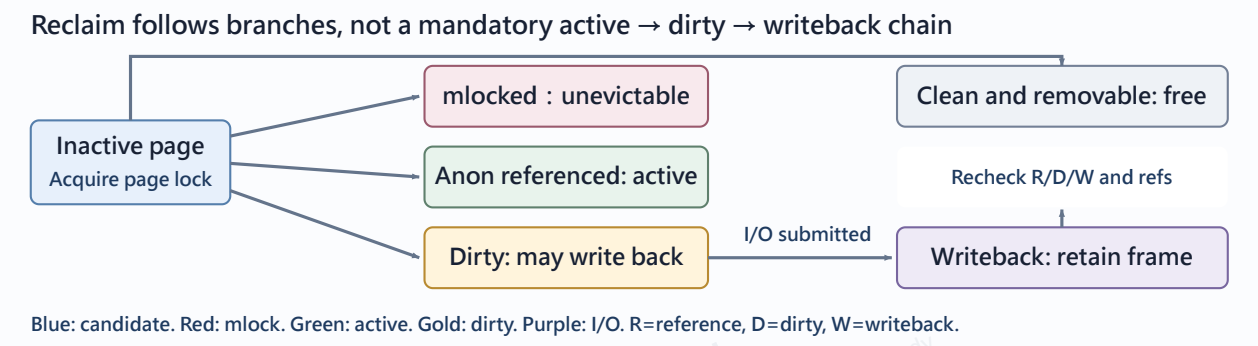
Page	Conditions driving the decision		Action in this scan	Freed
P1	Anonymous; PTE referenced=1	→	Activate to active LRU	0
P2	File; unreferenced, clean; removable	→	Reclaim and free frame	+1
P3	File; dirty; PageReferenced=0	→	Submit writeback; retain	0
P4	mlocked; isolated before culling	→	Move to unevictable	0
P5	PageWriteback=1	→	Defer	0

P3 pageout starts successfully. After writeback, recheck dirty and references before reuse.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Only P2 frees a frame now. P3 is a selected candidate but remains owned by I/O. An extra DMA pin/reference is not mlock; it may remain on ordinary LRU and fail final mapping/reference removal rather than become unevictable.

Recheck after writeback and changed conditions



Writeback completion does not guarantee no subsequent dirty write. Before reuse recheck dirty 0, writeback 0, removable mappings/references. Mlocked pages require authorized unlock; time alone cannot clear ownership. Free count increases only after actual reclaim.

Recheck after writeback and changed conditions (continued)

Changed condition	v3.2 branch/result
P1 instead file, first PTE reference, PageReferenced 0	Set PageReferenced, PAGEREF_KEEP; remain inactive rather than unconditional activate
P3 priority 10	priority≥DEF_PRIORITY−2; retain/mark reclaim, other writeback path
P3 direct reclaim, not kswapd	Dirty file-cache restriction blocks this pageout
P3 PageReferenced=1, PTE referenced=0	Non-swap-backed file page yields PAGEREF_RECLAIM_CLEAN; a still-dirty page is kept, not sent to the original pageout branch
P3 may_writepage=false	shrink_inactive_list may use ISOLATE_CLEAN and exclude dirty pages before isolation; an already isolated page reaching shrink_page_list is kept by its may_writepage check. In either case this path does not page out P3 or free it
P2 extra pin/reference	Final removal fails, retain; not synonymous with unevictable
P5 completed I/O then redirtied	Dirty branch again; no clean reuse

Recheck after writeback and changed conditions (continued)

A single if-referenced-activate/if-dirty-writeback sketch omits page type, reference source, scan context, and final ownership. Windows working-set limits, Solaris lotsfree/desfree/minfree, and Linux watermarks are distinct version-specific controls.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Historical Windows trimming

Use the OSC 9e simplified Windows working-set model, cross-read with Chapter 19's Windows 7 snapshot. It is not a guarantee for every current patch. Min/max control residency; low free memory can trigger trimming above minima.

Set threshold 3, free 1; A(min 2, resident 4, max 4), B(2,3,4), C(2,2,4). All selected pages are clean/unlocked; visit A→B→C and stop only once free>3.

Action	A	B	C	Free
Initial	4	3	2	1
Trim 2 from A	2	3	2	3
Trim 1 from B	2	2	2	4
Do not trim C	2	2	2	4

A later fault can grow its set below maximum if free pages exist; at maximum this model uses local replacement. Clustered paging can add neighbors, so exactly one resident page per fault belongs only to the earlier single-page model.

Historical Solaris two-hand window

In the OSC 9e snapshot free<lotsfree starts the scanner, whose scanrate increases from slowscan under pressure. desfree/minfree prompt stronger reclaim/swap measures. These are not Linux's same low watermark.

Front clears R; back revisits after h/s seconds for hand distance h pages and steady rate s pages/s. h4, s100 gives 0.04 s, illustrative rather than a default.

Page	After front clears R	Back observes	Action
Q1	Read again, R1	R1	Retain locality
Q2	Unused, clean	R0, clean	Unmap, then reclaim/cache list
Q3	Unused, dirty	R0, dirty	Write back; cannot reuse during I/O

Raising s to 400 with h4 shortens the window to 0.01 s, making absence of a recent reference more likely. An unoverwritten clean cache-list page may be recovered; invalid free-list contents are not a retained-data guarantee.

Complete access decision and cost model

Stage	Required evidence	Invalid inference
Range/backing	Authorized mapping and content source	Every nonpresent fault authorizes page-in
Rights/COW	Requested operation allowed or valid COW	Every forbidden write gets a new frame
Residency/PTE	Frame, standby, or storage requirement	Every fault means one disk I/O
Victim/writeback	Unmap eligibility and safe dirty handling	Pinned/dirty frames can be reused immediately
Restart	Contents, mapping, rights coherent	This page can never refault

Capacity counts unique frames; COW/unmap/replacement must track final mapping, TLB, and I/O users. With the fixed 12 references,3 frames,100 ns hits, and 8 ms total fault service including that access:

Complete access decision and cost model (continued)

Policy	Faults	Hits	Total cost
FIFO	9	3	72 ms+300 ns
LRU	10	2	80 ms+200 ns
OPT	7	5	56 ms+500 ns
CLOCK	9	3	72 ms+300 ns

OPT uses future information for a lower bound. Different fault costs, prefetch, or clustering invalidate this table's common multiplier. Working sets, quotas, PFF, and scanning jointly affect residency; evaluate useful throughput and waits as well as fault counts.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 9. Linux v3.2 vmscan.c; Linux v3.2 memory.c; mmap(2).

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

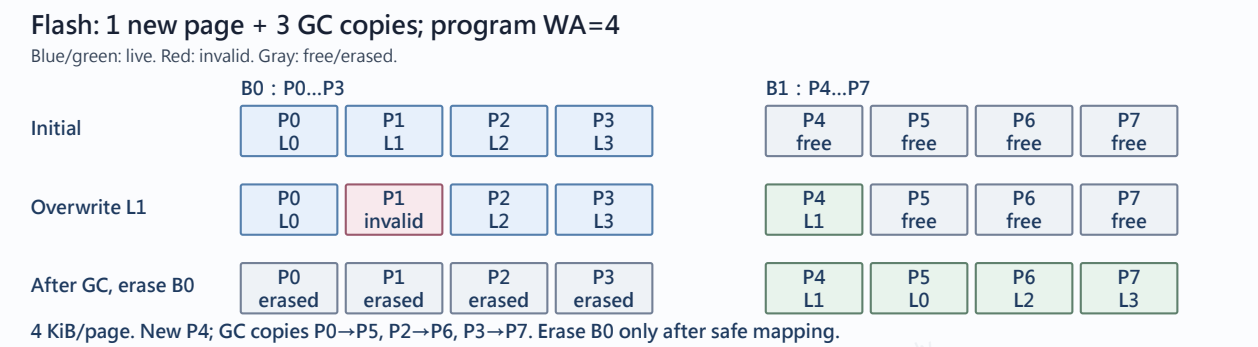
https://github.com/xinhi6716/computer-science-self-study

Media mechanisms and metric denominators

HDDs use magnetic surfaces, rotating platters and moving heads. SSDs use nonvolatile semiconductor media with controllers/FTLs. Tape supports streaming backup/archive with long positioning costs. Hosts address logical blocks rather than controlling every physical location. SSD costs cannot be modeled as head seek plus rotation.

Metric	Denominator/boundary	Distinct quantity
Bandwidth	Completed bytes/elapsed seconds	Interface/media peak need not equal end-to-end
IOPS	Completed requests/second	Different request sizes have different byte rates
Service time	Actual device service interval	Excludes time not yet selected
Response latency	Arrival→specified completion	Can include queue, scheduling, copy
Durability	Failures tolerated after acknowledged success	Completion may still be volatile

Flash out-of-place writes and GC



Fix 4 KiB NAND pages and four-page 16 KiB erase blocks. B0 P0/P1/P2/P3 hold L0/L1/L2/L3; B1 P4…P7 are erased. Overwriting L1 programs new P4, sets L1→P4, and invalidates P1. An unerased NAND page cannot be arbitrarily overwritten in place.

GC copies live L0:P0→P5, L2:P2→P6, L3:P3→P7. Only after live contents and necessary mapping are safe can it erase B0. One host page causes 1 new+3 copied physical page programs, write amplification 4 in this crash-free model. If L2 is discarded first, only 2 copies remain,3 programs and WA=3. Mapping writes and other controller overhead are excluded.

Flash out-of-place writes and GC (continued)

Discard/TRIM says logical data is no longer needed; it does not promise immediate erase or secure deletion. Wear leveling distributes erase load. GC, over-provisioning, and controller caches change latency. Power-loss correctness additionally needs recoverable mapping/data commit order; a normal-path diagram does not prove it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Geometry, LBA and access costs

Tracks at the same radius form a cylinder. Seek changes radius, rotation brings a sector under the head, and transfer moves bytes. Zones and remapping alter geometry. LBA is a one-dimensional logical index, not a permanent physical cylinder; SSDs have no head radius for SSTF to optimize.

With 512-byte logical sectors, LBA 4096 starts at 2 MiB. With 4 KiB blocks, the same index starts at 16 MiB. Unsigned 32-bit LBA addresses $2^{32} \times 512 = 2^{41}$ bytes=2 TiB; this is an address-space calculation, not a universal partition/API limit.

A512-byte upper write on a4 KiB physical/atomic unit may need read-modify-write or device translation. Filesystem block size does not establish atomic sector size; alignment and crash granularity follow device/driver contracts. Bad-sector remapping can preserve the LBA while adding latency.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Disk capacity: expand the geometry and units

Assume a disk assembly has 10 double-sided platters, with the top and bottom outer surfaces unused. There are `10×2−2=18` usable surfaces. Each surface has 2,048 tracks, each track 4,096 sectors, and each sector 8,192 bytes. Geometry is uniform and no space is deducted for spare sectors or formatting.

$$18 \times 2{,}048 \times 4{,}096 \times 8{,}192 = 1{,}236{,}950{,}581{,}248 \text{ bytes}.$$

Dividing this same quantity by `2³⁰` gives **1,152 GiB**; by `2⁴⁰` it gives **1.125 TiB**. In decimal units, `1 GB = 10⁹ bytes`, so it is approximately **1,236.95 GB**. State the units when comparing values; this is the capacity under the specified geometry, not a guarantee of formatted usable space.

1000 completed 4 KiB requests/s gives 3.90625 MiB/s; 1000 of64 KiB gives 62.5 MiB/s. The same IOPS does not mean the same bandwidth, and this arithmetic is not a product measurement. Comparisons fix request size, random/sequential pattern, R/W mix, queue depth, cache, and synchronization contract.

Derive timing from rotational speed and track size

Assume 6,000 rpm, 1,024 sectors per track, and 4 KiB per sector. Adjacent sectors are continuously readable; ignore sector gaps, controller, cache, and queueing costs.

Step	Calculation	Result
Revolutions per second	6,000 ÷ 60	100 rev/s
Revolution time	1 ÷ 100	0.01 s = 10 ms
Data per revolution	1,024 × 4 KiB	4 MiB
Ideal sequential rate	100 × 4 MiB/s	400 MiB/s
Average rotational wait	Uniform target phase, half a revolution	5 ms

With a fixed 3-second seek, reading 4 MiB laid out continuously on one track takes one revolution after the target sector arrives: `4 MiB ÷ 400 MiB/s = 0.01 s`. Average service time is `3 + 0.005 + 0.01 = 3.015 s`. This is a specified model, not a measured device speed; real geometry, gaps, caches, and overhead can change the result.

A7200 RPM,64 KiB read

Fix seek 5 ms, uniform rotational phase,7200 RPM, transfer 128 MiB/s, request 64 KiB, no cache/controller overhead. A rotation is60,000/7200=25/3 ms; mean rotation 25/6 ms; transfer 64 KiB/(128 MiB/s)=125/256 ms.

$$service = 5 + 25/6 + 125/256 = 7415/768 \approx 9.65495 \text{ ms}.$$

With queue wait 15 ms, response≈24.65495 ms. One-at-a-time service without overlap gives 1000/9.65495≈103.574 IOPS, not a multiqueue/cache/adjacent-track prediction. Sequential 1 GiB has transfer-only lower bound 1024/128=8 s, with positioning/queue added. Random small-I/O positioning dominates differently from streaming bandwidth.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Host, NAS and SAN responsibilities

Path	Client abstraction	Filesystem metadata owner	Additional failure boundary
Host-attached block	Local driver/controller blocks	Local host filesystem	Controller/device
NAS	Remote file/directory operations	File server	Network, server, cache/retries
SAN block	Logical blocks over storage fabric	Host using the LUN	Fabric, target, multipath

For a4 KiB NAS read, the client sends handle/offset/length, the server resolves filesystem metadata and issues block I/O. For SAN, the local filesystem first maps offset to LBA, then block transport reaches the target. The same array can sit below both while pathname/cache/retry responsibility differs.

A SAN LUN shared by two hosts does not provide filesystem distributed locking/coherence. Concurrently mounting an ordinary exclusive-host filesystem for writes can corrupt metadata. A remote mount success proves neither all future I/O availability nor durable completion.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Host, NAS and SAN responsibilities (continued)

Paths A/B to one target can survive A's link loss while B/target remain; target loss defeats both. Two paths are not two independent data copies. Retries preserve request identity/completion semantics; an unknown prior write commit cannot authorize blindly repeating nonidempotent upper operations. Latency depends on protocol, size, network, cache and queues, not the attachment name alone.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Scheduling on one fixed batch

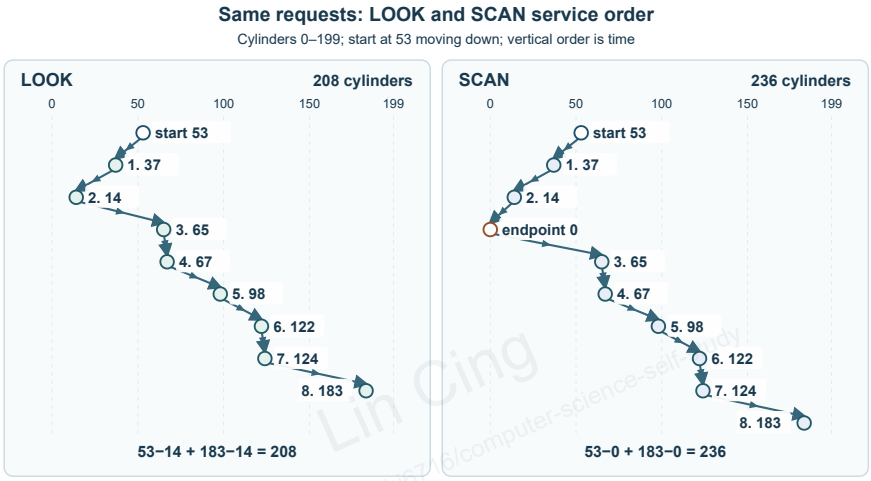
FCFS follows arrivals; SSTF chooses nearest pending cylinder; SCAN serves one direction to a physical endpoint then reverses; C-SCAN wraps and serves only one direction. LOOK reverses at the last request; C-LOOK wraps between extreme requests.

Fix cylinders 0…199, head 53, all arrivals 0, requests 98,183,37,122,14,124,65,67. Cost is absolute head movement until the last request completes, including turn/wrap, excluding rotation, transfer and final empty travel. SSTF ties choose smaller cylinder.

Policy	Path including turn/wrap	Movement
FCFS	53→98→183→37→122→14→124→65→67	640
SSTF	53→65→67→37→14→98→122→124→183	236
LOOK down	53→37→14→65→67→98→122→124→183	208
SCAN down	53→37→14→0→65→67→98→122→124→183	236
C-SCAN up	53→65→67→98→122→124→183→199→0→14→37	382
C-LOOK up	53→65→67→98→122→124→183→14→37	322

Circular returns count physical movement but serve no requests during wrap. A problem excluding that cost must explicitly use another accounting model.

Scheduling on one fixed batch (continued)



0 is a physical endpoint, not a request. Both paths stop after serving request 183.

LOOK=|53−14|+|183−14|=39+169=208. SCAN=|53−0|+|183−0|=53+183=236. SSTF's236 is coincidental, not the same path. FCFS differences 45+85+146+85+108+110+59+2=640; SSTF 12+2+30+23+84+24+2+59=236. LOOK 208 disproves SSTF's global-batch optimality.

The same requests with an upward initial direction

Use cylinders 0–199, start at 53, and note the previous position was 50. This example explicitly starts toward larger cylinder numbers. All eight requests are pending initially: `98, 183, 37, 122, 14, 124, 65, 67`. Charge absolute movement for every leg and stop after the last request; rotation and transfer are excluded.

Policy	Route (199 is a turn point, not a request)	Leg distances	Total
Upward SCAN	53→65→67→98→122→124→183→199→37→14	12, 2, 31, 24, 2, 59, 16, 162, 23	331
Upward LOOK	53→65→67→98→122→124→183→37→14	12, 2, 31, 24, 2, 59, 146, 23	299

Under this convention, SCAN reverses at the physical endpoint 199; LOOK reverses at the last request in that direction, 183. Neither route wraps around. With the **downward** initial direction in the other worked example, SCAN and LOOK instead total 236 and 208. The request batch alone does not determine the number: state the direction, endpoint rule, and stopping condition.

Arrival assumptions and waiting guarantees

A finite batch has no infinite starvation. SSTF's starvation risk needs continuing nearby arrivals that indefinitely defer a distant request. SCAN/LOOK reduces that bias, but finite wait needs bounded range, per-sweep service and admission/track rules; an unbounded stream at one position cannot silently be served in finite time.

Controller reordering, merging and parallel queues alter actual device behavior. SSD erase/parallelism differs from seek geometry. Movement/request is an average positioning proxy, not actual response time. Deadlines, fairness and throughput require their own cost model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Disk layout and management

Physical formatting organizes media sectors/error controls, often by device manufacturing/management. Partitioning divides logical address ranges. Logical formatting creates filesystem metadata/free-space structures. Mount attaches a recognized filesystem to a namespace. Formatting is neither secure erase nor automatic mount.

For a teaching 1024 MiB disk with 512-byte sectors, reserve[0,1) MiB for abstract boot/partition metadata. A=[1,101) MiB, B=[101,301) MiB.

Partition	First LBA	Sector count	Exclusive end
A100 MiB	2048	204800	206848
B200 MiB	206848	409600	616448

Ranges do not overlap. This is not a complete GPT specification. B mounting does not establish A's formatting/data integrity. A common mountpoint does not prove device/UUID identity. Bootloader kernel discovery and kernel root-mount need early driver dependencies from Chapter 2.

Disk layout and management (continued)

Controller remapping keeps an LBA while moving a defective physical sector to a spare. It cannot reconstruct already unreadable bytes by itself. Checksums/redundancy can detect/rebuild only with trustworthy data and error propagation. Boot-block damage, filesystem-metadata damage, and data-block damage require different recovery objects/versions. A partition existing is not proof that the disk is repaired. Scrubbing competes with foreground I/O and depends on spare/detection capacity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Swap backing and slot lifetime

Anonymous pages may need swap; clean file pages can be reread from their file. Swap partition/file are allocation and access-path choices. Prebuilt extent maps can reduce normal filesystem lookup, so an old swap-file overhead is not universal. There is no unconditional twice-RAM sizing formula.

Fix 4 KiB pages, slot S7/value 41, and two private-COW logical pages backed by it. Count only surviving swap-PTE references, not Linux swap_map encodings or all swapcache holds.

Event	P1	P2	S7 swap-PTE refs
Initial	Swap S7	Swap S7	2
P1 reads/private-writes 42	Resident F23/42	Swap S7, expects 41	1
P2 reads back	F23/42	Resident F24/41	0
No I/O/cache ownership remains	Resident	Resident	Only now reclaim S7

P1's new value cannot overwrite S7 while P2 needs 41; allocate new backing if required. This does not fix Linux fault counts or swapcache-sharing choices. Existing swap PTEs, I/O or cache holds prevent early slot reuse after one process exits. Counts need explicit scope/increment/decrement rules. Swap extends current execution's memory, not backup; hibernation has a different recovery contract. More swap can help commit but does not fix physical working-set thrashing.

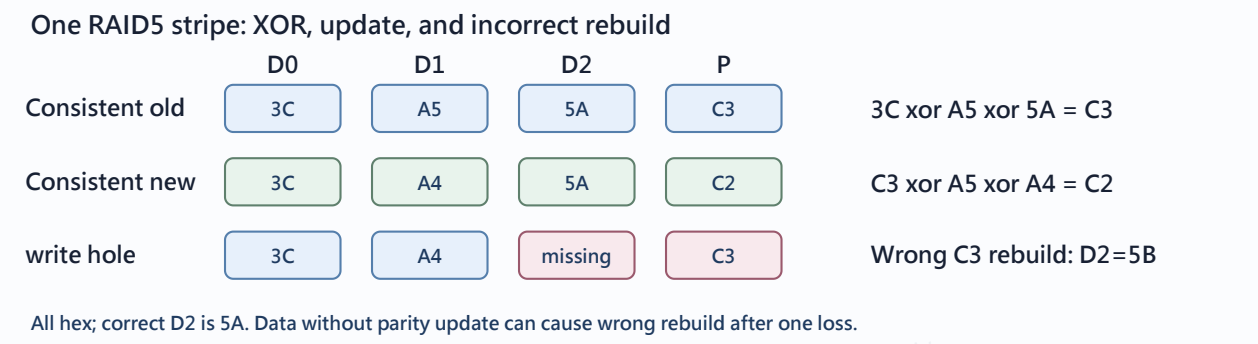
RAID capacities and device-loss assumptions

For N equal disks of capacity D, excluding metadata and spares:

Level	Layout	Capacity	Disk-loss guarantee
0	Stripe	ND	No redundancy
1	Two-copy pairs	D per pair	One loss per pair
2	Bit striping, ECC	Code-dependent	Depends on ECC
3	Fine stripe, dedicated parity	(N−1)D	One disk
4	Block stripe, dedicated parity	(N−1)D	One disk
5	Block stripe, rotating parity	(N−1)D	Any one disk
6	Two independent parity relations	(N−2)D	Any two, with suitable code

Two two-disk mirrors use four disks and yield 2D; one four-copy mirror yields D. Rebuild adds load. RAID cannot replace backups against deletion or common failures.

RAID5 XOR and the write hole



One stripe has hex D0=3C, D1=A5, D2=5A, P=3C xor A5 xor 5A=C3. Missing D1 reconstructs C3 xor 3C xor 5A=A5. Two unknown data values cannot be uniquely solved from one XOR relation.

Change D1 to A4. New parity C3 xor A5 xor A4=C2. Cold uncached read-modify-write reads old D1 and P, writes new D1/P',2 reads+2 writes excluding log/cache. A full three-data stripe can calculate parity directly and do 4 writes without old-stripe reads.

If A4 is durable but parity remains C3, all data disks can still be directly read but the relation is inconsistent. Then losing D2 and rebuilding with old parity yields C3 xor 3C xor A4=5B, not 5A. Single-device tolerance assumes coherent redundancy; it does not include arbitrary interrupted parity update plus a later loss.

RAID5 XOR and the write hole (continued)

Properly protected write-back cache, journaling/parity protocols or COW can protect that transition under implemented commit/recovery rules. Mirrors, checksums or a RAM flag alone do not solve it. Redundancy and persistence are separate contracts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why RAID6 needs an independent second equation

Over a suitable binary field, P=ΣDi, Q=ΣαiDi. Missing locations a/b have distinct coefficients. Subtract known values to get A+B=Pmissing and αaA+αbB=Qmissing. Substitute A=Pmissing+B:

$$(\alpha_a + \alpha_b)B = Q_{missing} + \alpha_a P_{missing}.$$

Distinct coefficients make their sum nonzero/invertible, so B then A is unique. Duplicating the same XOR gives dependent equations and cannot solve two unknowns. Codes specify field polynomial, layout, and error/erasure model.

Use GF(2^8), modulus 0x11D, coefficients 1/2/4, data 3C/A5/5A. Q=3C xor(2 • A5) xor(4 • 5A)=3C xor 57 xor 75=1E. Lose D1/D2: Pmissing=FF, Qmissing=22. Then 6B=22 xor(2 • FF)=22 xor E3=C1. Inverse 6=7A; B=7A • C1=5A, A=FF xor 5A=A5. Multiplication is carryless with field reduction, not integer multiplication.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

RAID 6: check each pair of erasures

Fix one stripe with each symbol on a distinct disk. Its data symbols are D_i, $P=\oplus D_i$ and $Q=\oplus(\alpha_i D_i)$. Arithmetic is in $GF(2^8)$, where addition is XOR. The coefficients α_i are **nonzero and pairwise distinct**. Assume the failed locations are known, every surviving symbol is correct, and at most two symbols in this stripe are erased.

Erased symbols	Information and reconstruction
D_a, D_β	Subtract known data from P and Q; the two-equation determinant $\alpha_a \oplus \alpha_\beta$ is nonzero, so the solution is unique.
P, D_a	Subtract the other data contributions from Q to obtain $\alpha_a D_a$; multiply by α_a^{-1}, then recompute P.
Q, D_a	Subtract the other data from P to obtain D_a, then recompute Q.
P, Q	All data survive, so recompute both parity symbols.

Nonzero coefficients make the P-plus-data case recoverable through Q; distinct coefficients make the two-data equations independent. This is a known-location erasure guarantee. It does not by itself identify arbitrary silent corruption or cover common-cause failures outside the model.

Recover D_1 when P and D_1 are erased

Use $GF(2^8)$ with reduction polynomial `0x11D`; all values below are hexadecimal bytes. Let D_0=`3C`, D_1=`A5`, D_2=`5A`, with coefficients `01, 02, 04`. Initially `P=C3` and `Q=1E`.

P and D_1 are erased, while D_0, D_2 and Q survive. Subtract the known contributions from Q; subtraction is XOR in this field:

$$02D_1 = Q \oplus D_0 \oplus (04D_2) = 1E \oplus 3C \oplus 75 = 57.$$

The multiplicative inverse of `02` is `8E`, so D_1=`8E·57=A5`. Recompute `P=3C⊕A5⊕5A=C3`. Check `02·A5=57` and `04·5A=75` to verify Q. Multiplication is carryless and reduced modulo `0x11D`; it is not ordinary integer multiplication.

0+1 and 1+0 failure sets

0+1 stripes(A, B) and(C, D), then mirrors the stripes. 1+0 mirrors(A, B) and(C, D), then stripes the pairs. After A fails,0+1 depends on complete(C, D); 1+0 uses B and C/D. A second C loss breaks 0+1 but leaves 1+0 withB/D. A second B loss instead leaves 0+1's complete CD, but destroys 1+0's AB mirror. Neither layout tolerates every arbitrary two-device loss.

With independent per-device loss probability p over one interval and no repair, a two-device mirror survives $1-p^2$. Four-disk RAID5 survives$(1-p)^4+4p(1-p)^3$. Common power/firmware/environment and repair/rebuild violate those independence/no-repair assumptions. These are model probabilities, not measured lifetimes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Checksums, storage pools and reservations

The book's ZFS example stores child addresses/checksums in parent metadata and validates down the tree. A checksum inside the same corrupt block is not equivalent to a trusted parent's end-to-end check. Detection requires the stated corruption model; repair additionally requires a correct redundant source. A checksum is not malicious-authentication protection.

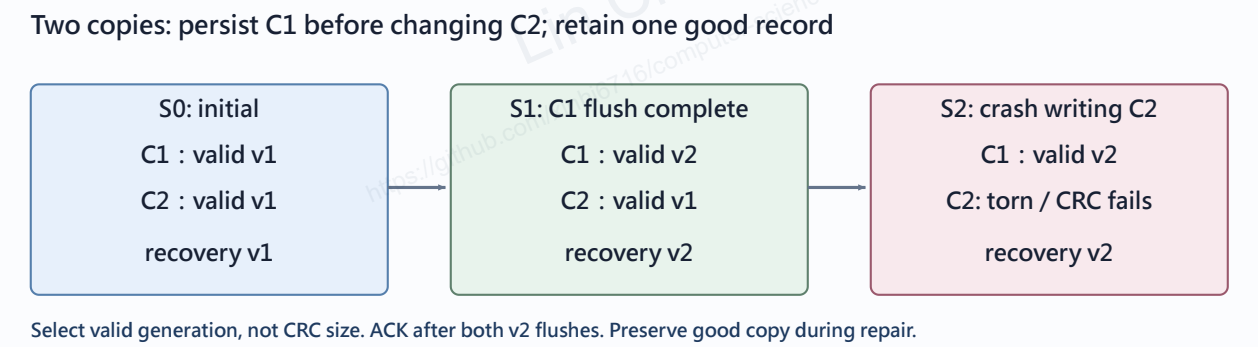
A storage pool supplies several filesystems; quotas limit growth, reservations protect shares. In an abstract 100 GiB pool, A uses 30, B20, free 50, A quota 60, B reservation 40 still needs 20 protected. Unreserved free=30, so A can grow min(quota remaining 30, unreserved 30)=30. Snapshots, dedup and compression are excluded; this is not all ZFS accounting or unbounded capacity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two-copy stable storage

Ideal stable storage never loses information; implementations approximate it within a failure model. Fix serialized 512-byte records, two device copies, nonwrapping generation, CRC detecting the injected tears, and trustworthy durable flush. Write only one copy at a time. A crash/torn write affects the currently written copy; the untouched and previously flushed copies stay valid. Common loss of those copies is outside this invariant.

Record format is4-byte big-endian generation+504-byte payload+4-byte CRC32 over the first 508 bytes. V1 payload is ASCII V1 then 502 zeros, CRC BF4CA2E8; V2 similarly gives 4A112E80. Recovery validates CRC then selects highest generation, not the numerically largest CRC. CRC is neither collision-free arbitrary-corruption proof nor authentication.



Every crash prefix and the invariant

Write/flush C1, then write/flush C2, then ACK.

Crash prefix	C1	C2	Recoverable version
Initial	Valid v1	Valid v1	v1
During C1, no confirmed flush	v1/v2/torn	Valid v1	At least v1; complete valid v2 may win
C1 v2 flushed, before C2	Valid v2	Valid v1	v2
During C2	Valid v2	v1/v2/torn	v2, not v1
C2 v2 flushed, then ACK	Valid v2	Valid v2	Acknowledged v2

Induction starts with two valid copies. During C1 the untouched C2 is valid old; before C2 is modified, C1 is durable new. Hence at least one valid copy survives every prefix. Recovery preserves the selected good copy, repairs/flushes the other first, and still retains a good copy if repair crashes. Concurrently overwriting both, untrustworthy flush, or common loss breaks the premises.

An unacknowledged operation may recover old or complete new; no reply does not mean no execution. One-record recovery does not make a multi-record filesystem transaction atomic.

Flush, FUA, ordering and atomicity

Volatile device cache can complete before nonvolatile media. Flush asks specified prior writes to reach persistence; FUA makes that request's completion satisfy the durable contract. Native support, emulation, and propagation through layers follow driver/device rules. Modern Linux REQ_PREFLUSH/REQ_FUA are concrete examples, not every historical OSC 9 device's flag names.

CPU memory barriers are not device-cache flushes. Ordered writes can still tear. Two-copy record recovery is distinct from RAID5 parity consistency. The guarantee must connect host buffer, device cache, persistent media, and ACK under the same failure model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Positioning, redundancy and durable completion

HDD 53→14 versus 53→0 changes positioning. FTL maps L1 to a new page and moves live data. RAID reconstructs under a specified loss set. Stable-write ordering decides which version survives interrupted updates. Success in one layer cannot replace another layer's contract.

Keep queue wait, seek/rotation, transfer and completion scheduling separate. State request bytes with IOPS; host/physical bytes with WA; topology with RAID capacity; failure correlation/repair with reliability; flush/atomicity with ACK. Changing media, topology or cache strategy requires revisiting that layer's premises. Detection, repair sources and historical backups each serve a different need.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 10.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Chapter route

How does a file-system interface express persistent-data access through composable objects, names, and rights?

Prerequisites are block storage, processes and file descriptors, and basic access control.

- Distinguish file metadata, data, and handle state.
- Compare sequential, direct, and indexed access.
- Analyze boundaries between mounting, sharing, and protection.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Files, names, and descriptors have different identities

Use ordinary local Linux/POSIX regular-file opens, excluding O_PATH and extra LSM revocation. Multiple names can refer to one live (filesystem,inode) object; one name can refer to different objects over time. After reclamation an inode number can be reused.

Layer	State	Sharing
Directory entry	Name → inode; the name lives here	A hard link adds another entry
Inode/file object	Size, owner, mode, data mapping, link count	All opens of that object
Open file description, OFD	Current offset, access mode, status flags	Descriptors shared through dup/fork
Process descriptor table	Integer fd → OFD; FD_CLOEXEC	Each process has its own table in this fork example

Open checks permission and creates an authorized OFD. Subsequent read/write still checks the descriptor's access mode. Changing DAC permissions with chmod does not automatically invalidate an already open local descriptor. Other protection mechanisms have their own rules.

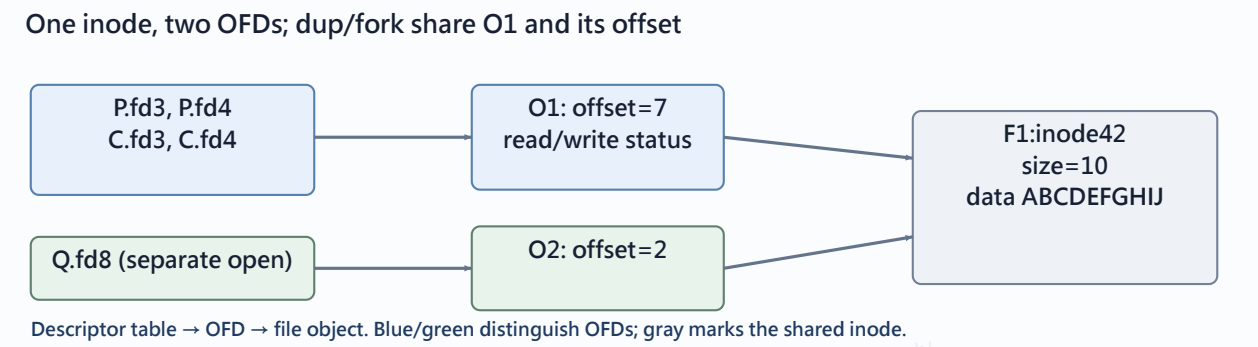
Shared offsets with data ABCDEFGHIJ

P first obtains O1 through `fd3=open(F)`, then calls `fd4=dup(fd3)`. Q independently obtains O2 through `fd8=open(F)`. P then forks C, whose fd3/fd4 also refer to O1. Operations below execute in the specified sequence; this avoids claiming historical Linux concurrent-offset behavior as a guarantee.

Step	Returned data	O1.offset	O2.offset
Initial	None	0	0
P read(fd3,3)	ABC	3	0
P read(fd4,2)	DE	5	0
Q read(fd8,2)	AB	5	2
C read(fd3,2)	FG	7	2
P pread(fd3,2, offset=1)	BC	7	2

A successful read of k bytes advances its OFD offset by k. Pread uses an explicit position without changing O1. Sharing an inode does not combine independently opened offsets.

Shared offsets with data ABCDEFGHIJ (continued)



What survives unlink, truncate, and rename?

Regular object F1:42 initially has names `/a` and `/b`, link count 2, and open description O1. Unlinking a reduces the count to 1; unlinking b reduces it to 0, but O1 can still read the object. Reclamation requires the last close and no other references. Keep name references separate from open references; real systems may also have mmap references.

`truncate(F,0)` retains the object and names but changes size to zero. Existing OFD offsets need not reset. Reading an empty file at offset 7 returns zero bytes without moving the offset back to zero.

Assume tmp and target are distinct objects in the same mount, permissions and types permit replacement, and `rename(tmp, target)` succeeds. It atomically replaces the name. New lookups of target reach the new inode; descriptors already open on the old target keep the old inode. If the names are already hard links to the same inode, Linux rename succeeds without changing them. Crossing different mounts can fail with EXDEV even when the underlying filesystem is the same. Atomic name visibility during execution is separate from surviving power loss.

Filename extensions are application conventions. They prove neither content format nor permission to execute. Linux execution additionally checks permissions and the executable/interpreter format. Renaming arbitrary bytes to `.exe` does not create a valid executable.

Logical access methods do not specify physical positions

Method	Caller specifies	Required state
Sequential	The next portion	Current position and rewind/seek rules
Direct	Relative record/block number	Fixed length or computable record boundaries
Indexed	Key	A consistent key-to-record/block index

For fixed record length L and zero-based record j, the starting byte is jL. Relative block numbers start at the beginning of the file. They are not disk LBAs; Chapter 12 handles physical mapping.

With L=32 B and blocks of 1024 B, each block holds 32 records. Record 77 starts at byte 2464, file block 2, displacement 416. A thousand records occupy 32000 B, requiring $\lceil 32000/1024 \rceil = 32$ blocks. The final block holds eight records, or 256 B, leaving 768 B of internal fragmentation. Exclude sparse files, compression, and inline data in this example.

Sparse indexes and I/O counts

The thousand records are sorted by unique integer keys 0 through 999, still at 32 records per block. The index has 32 entries of (first key, block pointer), each 16 B, totaling 512 B. For key 77, choose the greatest first key no larger than 77. Entry 64 identifies block 2; inspect keys 64 through 95 there.

If block i covers $[k_i, k_{i+1})$, sorted keys and a correct index restrict lookup to that candidate block. A gap still requires comparing the actual key. A predecessor-index hit does not prove that a record exists.

With the index in RAM, binary search takes at most $\lfloor \log_2 32 \rfloor + 1 = 6$ comparisons plus one data-block I/O. If the uncached entire index occupies one block, the bound is one index read plus one data read. This is neither a universal fixed two-I/O cost nor constant-time CPU search.

Short transfers preserve a completed prefix

Write 6000 bytes from initial file offset 100, with successful results 2500,1500,2000:

Call	Buffer start	Requested	Returned	Total done / next offset
1	0	6000	2500	2500 / 2600
2	2500	3500	1500	4000 / 4100
3	4000	2000	2000	6000 / 6100

Maintain $0 \le done \le N$: buffer[0:done] has been delivered in order, and only [done:N] remains. A successful k>0 advances done by k and reduces the remainder. If every iteration makes positive progress, at most N one-byte successes complete the request. Errors do not guarantee eventual success. Retry EINTR only under its no-data-transferred condition; wait for readiness on EAGAIN; never spin indefinitely on zero progress.

A read near EOF can return a short result; a later positive-count request returns zero. A zero-count request returning zero does not prove EOF. After a read error, the offset may not be inferable. The table assumes successful outcomes and does not invent offset advances for errors.

Append atomicity and positioned I/O

Separate `lseek(SEEK_END)` and write calls are not one atomic step. P and Q can both observe size 100 and write at offset 100, allowing the later write to overwrite the earlier one. O_APPEND combines EOF selection with that write within the interface's implementation scope. In particular, an NFS emulation need not provide the same promise as a local filesystem.

Even if each append write is atomic, a record split into A 1 and A 2 can interleave as A 1, B1, A 2. Whole-record guarantees require framing and locking or a single writer. Regular files are not pipes, so PIPE_BUF is not their atomicity theorem.

Pread/pwrite carry an explicit offset and avoid another thread changing a shared cursor between lseek and read. They do not serialize overlapping data writes. The pwrite scenario here excludes O_APPEND; Linux has a special append behavior for their combination.

Directory structures and their costs

A directory maps names to objects. Names are unique within one directory but can repeat across directories. Lookup, create, unlink, rename, and enumeration are interfaces, not a requirement to use a linear list.

Structure	Capability	Cost or limitation
Single level	One shared naming domain	Collisions and no grouping
Two level	A directory per user	Awkward sharing across users
Tree	Paths and subtree grouping	No direct shared child under a unique-parent rule
DAG	Multiple paths/names to one object	Reference counting and duplicate visits
General graph	Cycles	Termination, reclamation, cycle detection

Directory structures and their costs (continued)

A successful hard link adds an entry to the same inode within a mounted filesystem, with one copy of the data. Linux can reject a link across different mounts with EXDEV even when the underlying filesystem is the same; permissions and object type must also permit it. A symlink is a separate object storing a pathname; it can cross filesystems, dangle, or cycle. Ordinary users cannot create arbitrary directory hard links. A conceptual DAG does not authorize unrestricted Linux directory hard links.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why reference counts cannot reclaim cycles

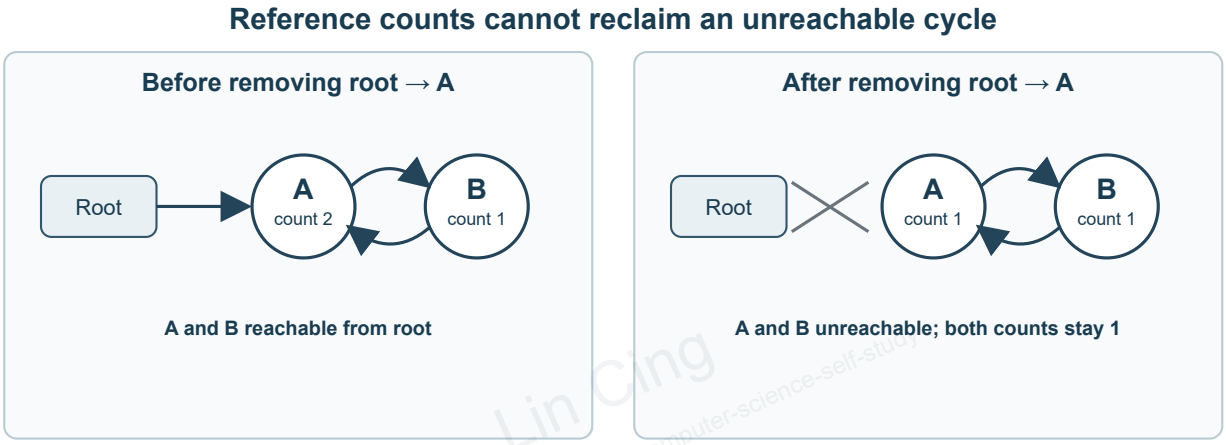
Consider only the abstract name/edge graph, excluding held opens. Let A→B, B→A, and root→A. Counts are A=2, B=1. Removing root→A leaves A=1, B=1. Both objects are unreachable, but neither count becomes zero.

For a finite DAG, an unreachable subgraph remains acyclic and has an indegree-zero vertex. It cannot have an incoming edge from a reachable vertex, or it would be reachable. Its reference count is therefore zero. Remove it and apply induction to reclaim every unreachable vertex. A cycle lacks that starting vertex.

A visited set prevents repeated traversal. DFS on a finite graph takes O(V+E), but traversal alone is not reclamation. Mark/sweep marks from roots and reclaims unmarked objects, requiring coordination with concurrent graph changes. A symlink traversal limit terminates one resolution; it does not prove all data reachable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why reference counts cannot reclaim cycles (continued)



Counts include graph edges only, not open or mmap references.

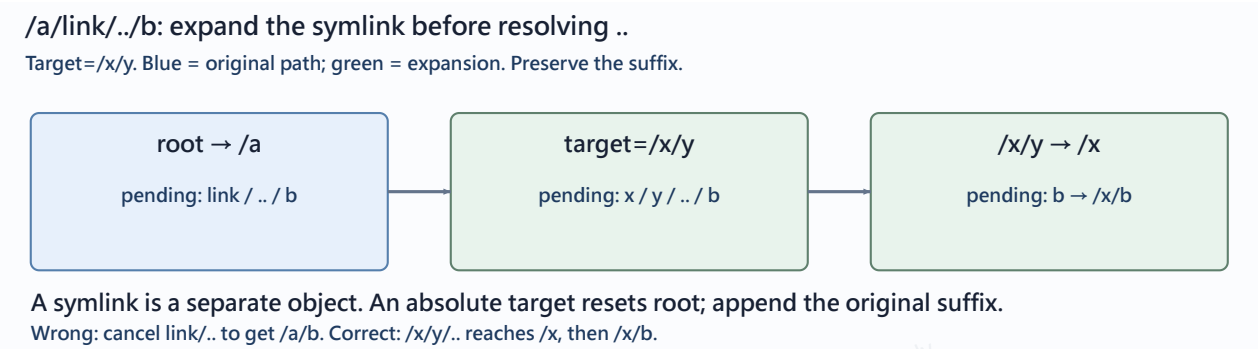
Path resolution cannot start with textual cancellation

Let `/a/link` store absolute target `/x/y`; all required directories exist and are searchable. Resolve `/a/link/../b`:

Step	Current object	Pending components	Action
0	Root	a, link,.., b	Start at process root
1	/a	link,.., b	Read link as a symlink
2	Root	x, y,.., b	Absolute target resets root; preserve suffix
3	/x/y	.., b	Resolve x and y
4	/x	b	Apply .. to resolved object
5	/x/b	Empty	Complete

Canceling `link/..` first incorrectly gives `/a/b`. Resolve a relative symlink target from the directory containing that link, not the process cwd. Each next component requires search permission on the current directory. Whether to follow a final symlink depends on the operation.

Path resolution cannot start with textual cancellation (continued)



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

TOCTOU can change the object behind a name

P checks that `/tmp/out` is a safe ordinary file. An attacker replaces it with a symlink to a sensitive file before P opens the same string. Two resolutions can reach different objects because the name graph changed; symlinks themselves are not universally invalid.

A trusted directory descriptor with openat fixes the relative starting point. O_NOFOLLOW constrains only the final symlink and does not authenticate intermediate components. Later Linux openat2 resolution restrictions can constrain the whole walk; do not assign that later interface to Linux 3.2.

Complete the operation through the same protected reference or appropriately restricted resolution. After open, fstat examines the opened object. Still define acceptable owner/type and allowed open side effects. Checking safety after O_TRUNC is too late to prevent truncation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

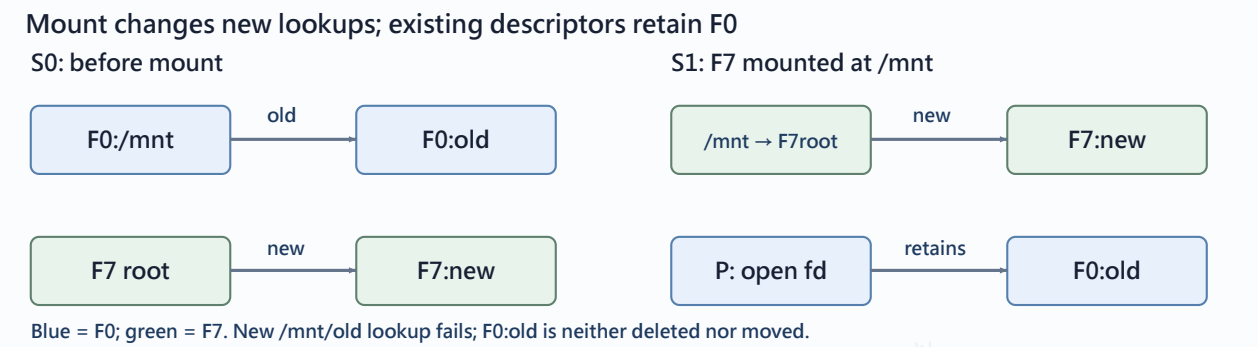
Mount changes a resolution edge

F0's `/mnt` initially contains old but not new; F7's root contains new but not old. All path ancestors and relevant directories are searchable, without other permission restrictions that prevent resolution. After mounting F7 at /mnt, a new lookup of `/mnt/new` reaches F7:new, while `/mnt/old` returns ENOENT. F0's old remains allocated and becomes visible after unmount.

Observation	Before mount	After mount
New lookup /mnt/old	F0:old	ENOENT in F7
New lookup /mnt/new	ENOENT	F7:new
Previously opened F0:old descriptor	F0:old	Still F0:old

The namespace selects a mount map. The same string can reach different objects in different namespaces without copying their data. Blue denotes F0, green F7, and gray the old content hidden from new lookups.

Mount changes a resolution edge (continued)



Mount crossings, .., and busy references

In this Linux-style model, crossing a mount point enters the mounted root. Applying `..` there returns to the mount point's parent, rather than arbitrarily remaining at the underlying filesystem root. At the process root itself, `..` stays at root.

Let `/a/link` target `../b` and mount F7 at `/b`. Resolving `/a/link` follows root→a→link target→a/..=root→b→F7 root, then stops. Without an input filename, it cannot invent `/file` as a suffix.

Normal unmount can report busy for active cwd, open-file, or submount references under the system's rules. Linux lazy detach first removes the namespace attachment, then reclaims after references disappear. It neither immediately invalidates every descriptor nor guarantees pending I/O success.

A mount point is not a symlink. The namespace mount map controls mounting; a symlink is a directory object storing a target string. Both can occur in one path walk without becoming the same entry type.

Identity, ordering, visibility, and durability

The same user name on two hosts need not map to the same UID. Inconsistent AUTH_SYS-style numerical mappings can authorize the wrong principal. A network address or declared UID does not replace trusted authentication. NIS, LDAP, and Active Directory are naming/identity infrastructure examples. LDAP is a directory protocol; its use alone proves neither encryption nor correct authentication.

A file protocol must separately define who may operate, operation order, when another client rereads new data, and what survives a crash. NFSv3 basic file operations do not need OPEN state to identify each READ/WRITE, but still have caches and stable-write semantics. NFSv4 adds stateful mechanisms. Stateless does not mean no state of any kind or automatic security.

Equal paths can reach different mounted objects. Clients of one object can temporarily observe different cached/session versions. Successful write, successful lock acquisition, another client's visibility, and media durability are four different events.

One history under immediate and session visibility

Start at value 0. A and B open, A writes 1, B reads before A closes, A closes, B reads within its existing session, then newly opened C reads. Execute in order and ignore errors/data races. The immediate column is a single coherent image abstraction, not a claim about every network cache.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One history under immediate and session visibility (continued)

One history: immediate visibility and session snapshots

Event	One shared image	Session snapshot
A, B open	A, B see 0	A and B each hold 0
A writes 1	image becomes 1	A private 1; B still 0
B reads	returns 1	returns 0
A closes / publishes	image remains 1	published image becomes 1
B reads, same session	returns 1	old snapshot returns 0
C opens / reads	returns 1	new snapshot returns 1

Ordered example only; multi-writer conflicts need a separate rule. Not a claim about all network filesystems.

Session snapshots do not change an already open instance when another closes. Multiple-writer conflict and last-close rules need separate definitions. Immutable sharing prohibits changing contents or reusing the unique name, simplifying caching but expressing updates as new objects/names.

Advisory locks provide conditional exclusion

A holds an exclusive lock on [0,100). Cooperating B waits for a shared or exclusive lock on the same range; a nonoverlapping lock on [100,200) can proceed. Half-open ranges overlap exactly when $\max(start) < \min(end)$, so these two ranges do not overlap.

Time	A	Cooperating B	C without a lock
0	Acquires [0,100) X	None	None
1	Writes version 8	Waits for overlapping lock	Can directly write the same data
2	fsync succeeds	Still waits	May violate A/B expectations
3	Unlocks	Acquires, then validates/reads under cache protocol	None

Assume every conflicting operation first obtains a compatible lock. If two writers executed inside the same range simultaneously, both would hold incompatible X locks, contradicting the lock service. C's uncooperative access breaks that premise. Locks alone establish neither cache invalidation nor durability; the shared-filesystem contract must provide those rules.

Atomic publication is separate from durable publication

Assume a local Linux filesystem supporting same-mount atomic replacement and directory fsync, and a device honoring flush. Tmp and target are distinct objects in one directory; permissions, types and other conditions allow replacement. Preserve the old target while preparing the new version:

1. Create a unique tmp, write all data with short-write handling, and set required metadata.
2. Successfully fsync(tmpfd) to persist contents and necessary metadata.
3. Successfully rename(tmp, target). New lookups reach the new inode; existing old descriptors retain the old version.
4. Successfully fsync the parent-directory descriptor before promising that the new name survives a crash. Handle every error.

Step 3 avoids a runtime interval in which target is absent. Step 4 concerns the durable name. Before it finishes, this chapter does not claim that every filesystem must recover to a specified old or new name. Two fsync calls are not a cross-filesystem transaction; rename can fail with EXDEV.

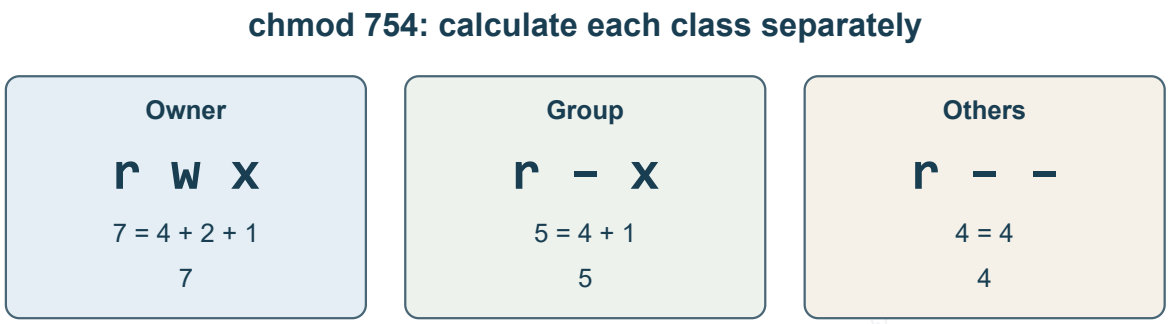
File and directory rwx govern different operations

First exclude root/capability privilege, ACLs, sticky rules, and other controls. Linux compares filesystem UID (fsuid), filesystem GID (fsgid), and supplementary groups; assume here fsuid/fsgid equal the effective UID/GID. Select one mode class: owner if matched, otherwise a matching filesystem or supplementary group, otherwise other. Do not combine all three classes. With r=4, w=2, x=1, every requested bit must be present in the selected class.

Right	Regular file	Directory
r	Read data	Enumerate names
w	Change data	Add, remove, rename entries, normally also requiring x
x	Request execution with valid format and other conditions	Search/traverse a known name

With D mode0711 and F mode0644, a nonowner can traverse D and read known F but cannot list D. With D mode0733, other has wx and can add/remove known names without listing. Unlink primarily checks parent-directory wx, not the file's w bit. Sticky directories add identity restrictions.

chmod 754: three groups of permission bits



`chmod 754 target` sets the basic mode bits to owner `rwx`, group `r-x`, and others `r--`. Within each group `r=4`, `w=2`, and `x=1`: group `5=4+1`, so it has read and execute but not write.

Access follows the applicable owner, group, or others class; the three columns are not simply added together. For the examples below, assume an ordinary unprivileged user and a local Linux file, with no additional ACL, LSM restrictions, or mount restrictions. ACLs, special permissions, and privileges can change the result.

The meaning of rwx depends on the object

Object	r	w	x
Regular file	Read contents	Write contents	Request execution, subject to format, interpreter, and other checks
Directory	List entry names	Modify entries, usually also requiring x	Search the directory and resolve names inside it

Let `/work` be a 754 directory and assume all ancestors are searchable. A user in the applicable group class has `r-x`: they may list and search, but cannot create an entry using those permissions alone. An others-class user has `r--`: they can read entry names but cannot resolve `/work/a` without directory search permission. The owner has `rwx`; creation or deletion remains subject to other conditions such as the sticky bit and mount restrictions.

Suppose a process successfully opened a separate regular file for writing before its mode was changed. On a local Linux regular file, removing that process's write mode bit later does not automatically close its existing descriptor; a new open is evaluated against the permissions then in force. This example excludes additional LSM revocation, network protocol and device-specific behavior. The existing descriptor's access mode still limits later operations.

Linux POSIX-style ACL mask and selection order

Use Linux access ACLs, not Windows deny ACLs, excluding privilege/LSM. Here filesystem UID/GID (fsuid/fsgid) equal effective UID/GID, and group matching also includes supplementary groups. Owner alice and file group dev have entries user::rw-, user:bob:-w-, group::r--, group:qa:-w-, mask::rw-, other::r--.

1. If fsuid matches owner, check user:: directly without the mask.
2. Otherwise, a matching named-user entry must contain the entire requested set, as must the mask. Failure denies without falling back to groups/other.
3. Otherwise consider matching owning/named groups. At least one matching entry must contain the whole requested set, and the mask must contain it. A matched but insufficient group class denies without falling back to other.
4. Only if no preceding identity matches, use other without the mask.

Bob has only w even if he belongs to dev; group r does not supplement his named entry. Changing mask to r leaves him no rights, without falling back to other r. Alice keeps rw. With a mask entry, displayed mode group-class bits represent the mask, not necessarily group::.

Multiple groups do not authorize every combined request

Carol belongs to dev and qa. Here dev=r, qa=w, mask=rw. A read request passes through dev; a write passes through qa. A combined rw request, such as O_RDWR, fails because no single matching group entry contains both rights.

Subject/request	Selected path	Decision
Alice/rw	Owner rw	Allow without mask
Bob/r	Named user w	Deny without group fallback
Bob/w	Named user w and mask rw	Allow
Carol/r	dev r and mask rw	Allow
Carol/w	qa w and mask rw	Allow
Carol/rw	Each matching group lacks a right	Deny
No matching identity/r	Other r	Allow

Changing mask to r also denies Carol's write. Apply the algorithm to the complete current request rather than OR-ing every displayed entry. Named-user precedence is not a general deny-entry composition rule.

Default ACLs, umask, and existing handles

Without a default ACL, creating with mode0666 and umask0027 gives $0666 \;\&\; \sim 0027 = 0640$. Umask removes rights and does not add unrequested execute permission.

A directory default ACL is inherited as the new object's access ACL and restricted by the supplied create mode for owner/group-class/other. Do not additionally apply umask unconditionally. A default ACL initializes new objects without retroactively changing old files or replacing the ordinary access-check algorithm.

Opening a writable descriptor before chmod0000 generally retains write authority for an ordinary local Linux regular file, while a new open is denied by DAC. Immediate revocation requires a separately defined mechanism or closing/revoking references. Changing displayed permissions does not recall every authorized reference. See acl(5).

Four traces for the same file

Trace	Record	Typical mistake
Names	Root/cwd, components, symlink suffix, mount map	Textually cancel link/..
Objects	Filesystem/inode, links, open references	Free after unlinking the last name
Access	OFD offset/access mode, actual transfer results	Assume dup offsets are independent
Authority	Directory search, selected ACL class, mask, authorization time	Treat a displayed w as unconditional write access

Atomic publication adds a persistence trace: complete tmp data→file fsync→rename→directory fsync. The name graph and durable storage have different publication points; one success label cannot represent all of them.

Choose operations from their counterexamples

Requirement	Condition that matters
Read at a position	pread plus short-read/error handling
Append a whole record	framing and whole-record lock or one writer
Remove or confine a name	parent permissions; constrain the entire path
Coordinate client updates	cooperating writers and a visibility protocol
Preserve new name after crash	file and directory persistence under the stated flush contract

Chapter 12 implements these name, object, and handle contracts.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 11. Linux open(2); rename(2); acl(5); path_resolution(7); credentials(7).

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Chapter route

How can a filesystem maintain accessible, recoverable persistent objects using finite blocks, metadata, and recovery protocols?

Prerequisites are the file interface, logical/physical blocks, and basic journaling.

- Trace inodes, directories, open tables, and page caches.
- Compare contiguous, linked, and indexed allocation.
- Analyze metadata/data consistency across failures.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Logical files and recoverable disk structures

Use a teaching volume with 4 KiB blocks:

Block	Persistent structure	Contents
0	Superblock	Block size/count, inode/free-space locations
1	Inode bitmap	Allocated inodes
2	Data-block bitmap	Free data blocks
3	Inode table	Type, size, mode, direct/indirect pointers
4	Root directory	home→inode 3
5	/home directory	u→inode 5
6	/home/u directory	notes.txt→inode 7
8–10	File data	Three 4 KiB blocks of notes.txt

The basic filesystem translates logical blocks into device blocks; file organization manages free space and logical-to-physical mapping; the logical filesystem manages directories, FCBs/inodes, and APIs; I/O control and drivers perform transfers. Caches may reside above these layers, but the system contract must define the durable commit point.

An append must preserve

$$\text{name} \to \text{valid inode} \to \text{consistent block set}, \qquad \text{allocated} \cap \text{free} = \varnothing.$$

Logical files and recoverable disk structures (continued)

A normal write return does not make the directory entry, inode, bitmap, and data simultaneously durable. Unmount, successful flush, and transaction completion are different events.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

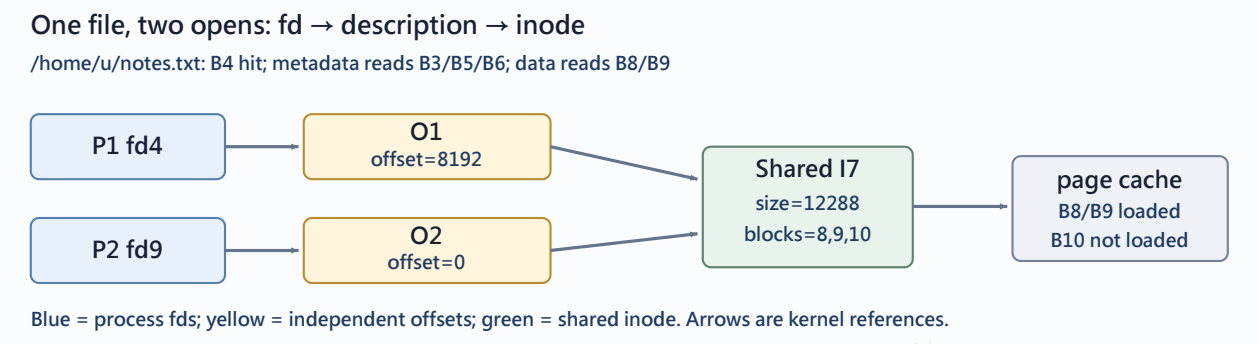
Complete path walk, open, and read states

Use `/home/u/notes.txt`, initially caching only root-directory block 4:

Step	Lookup	Read or hit	Result
1	home in /	Root block 4 hit	Inode 3
2	Inode 3	Cold inode-table block 3	Directory block 5
3	u in /home	Cold directory block 5	Inode 5
4	notes.txt in /home/u	Cold directory block 6; inode table cached	Inode 7
5	Open inode 7, permission succeeds	No further disk read	O1→I7, P1 fd4→O1, offset 0
6	P1 reads 8192 B	Data blocks 8,9	Data loaded, offset 8192
7	P2 independently opens same file	Inode/cache hit	New O2→I7, P2 fd9→O2, offset 0

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete path walk, open, and read states (continued)



The cold walk costs three metadata reads, inode table plus directories 5/6, and two data reads, totaling five. With everything cached it costs zero disk reads. Directory cache, inode cache, system-wide open-file table, and page cache are different layers. A hit in any layer proves no durability property.

The process fd table maps integers to descriptions and contains descriptor flags such as close-on-exec. An OFD is the kernel object created by open, containing offset, access/status flags, and an inode reference. An inode/FCB contains identity, size, permissions, and mapping and is associated with shared page-cache state. Independent opens do not merge offsets merely because the inode is shared.

Complete path walk, open, and read states (continued)

The textbook's teaching representation places some fields in per-process tables and has a system-wide FCB table. That abstraction must still identify the object owning a shared offset. This example uses the Linux/POSIX open/dup three-layer representation. Two descriptions now refer to I7; this is not one description with reference count two.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Independent-open and dup offset invariants

Continue with P1 fd4→O1, P2 fd9→O2, I7 size 12288. Reads succeed completely, without signals or other writers.

Event	Descriptor relation	O1 offset	O2 offset	Returned range
After opens	P1:4→O1; P2:9→O2	0	0	None
P1 read(fd4,8192)	Unchanged	8192	0	[0,8192)
P1 fd6=dup(fd4)	fd6 also points to O1	8192	0	No read
P1 read(fd6,4096)	fd4/fd6 share O1	12288	0	[8192,12288)
P2 lseek(fd9,4096, SEEK_SET)	Only O2 changes	12288	4096	Contents unchanged
P1 read(fd4,1)	O1 is at EOF	12288	4096	Zero bytes

For a successful r-byte read, $o' = o + r$, with $0 \le r \le \min(n, \max(0, size - o))$. Dup and fork inheritance of the same description share o; independent opens do not. Closing fd6 decreases O1's references while fd4 remains usable. Closing the last description does not remove a directory name. Unlink and final-open-reference release have different lifetimes.

Mount and VFS dispatch

Mount filesystem B's root inode 2 at A's /mnt. Resolving /mnt/log/x first finds A's mount point, crosses to (B, inode 2), then uses B's lookup for log and x. A and B may both have inode number 2, so identity requires filesystem/mount context as well as inode number.

VFS resolves dentry/inode and mount relationships, then dispatches a regular-file read through that filesystem's operations. A local filesystem maps blocks and submits block I/O; NFS forms RPCs. A common interface does not require every lower layer to use the same on-disk index block. Handle permissions, symlinks, .., and mounts during each resolution step. The five-read example explicitly excludes those extra traversals and permission-data I/Os.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Directory lookup costs

For equally likely queries over the n stored names and equal cost per name comparison, a linear list averages $(n+1)/2$ comparisons on successful lookup. If name i is queried with probability p_i, the mean is $\Sigma_i p_i i$. A hash table maps a name to a bucket and compares entries there. Fix

$$h(name) = \left(\sum_i code(char_i) \right) \bmod 8.$$

Bucket	Entries	Chain length
0	Empty	0
1	Empty	0
2	delta, epsilon, eta	3
3	gamma	1
4	beta, zeta	2
5	Empty	0
6	alpha	1
7	Empty	0

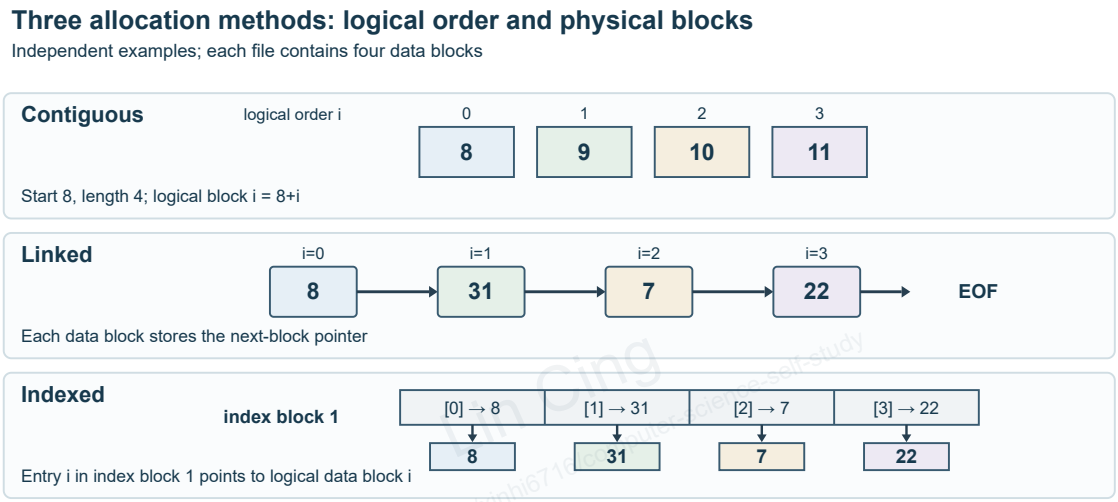
Directory lookup costs (continued)

Load is $\alpha = 7/8 = 0.875$. Uniform queries over these seven stored keys give $(1+1+1+1+2+2+3)/7 = 11/7 \approx 1.571$ comparisons. This averages query targets for a fixed function, not random functions.

Under an additional uniform pair-collision model over m buckets with fixed insertion order, insertion i has expected successful cost $1 + (i-1)/m$. Uniformly averaging stored keys gives $1 + (n-1)/(2m)$, or 1.375 for n=7, m=8. The common $1 + \alpha/2$ omits the finite-n correction. Observed 1.571 and expected 1.375 use different randomness and denominators. All keys colliding still yields O(n) worst case.

Eight chaining buckets can store more than eight names; a ninth name does not inherently overflow the table. Resize policy can depend on load and latency. Open addressing instead has at most one entry per slot. A visible intermediate rehash can omit an old entry or duplicate a name. Locks, versions, or a specified incremental migration protocol must preserve a coherent lookup result.

Three block allocation layouts



Direct, linked, and indexed access costs

Fix 4 KiB blocks and four-byte pointers, giving 1024 entries per pointer block. Retrieve zero-based logical block 2:

Method	Mapping	Disk reads	Main cost
Contiguous	Start11, length3	One data read	Fragmentation and relocation on growth
Linked	11→12→13	Three block reads	Follow links for random access
Indexed, index cached	Index entry→data13	One data read	Index space
Indexed, index cold	Index block, then data13	Two reads	Large-index memory/TLB costs

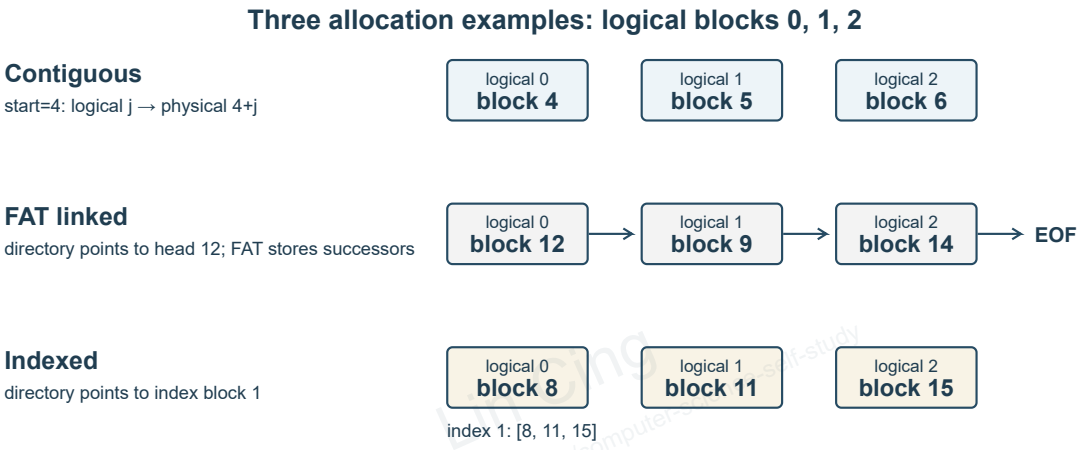
Contiguous mapping is start+i. Linked access starts at the head. Indexed access selects pointer i, with cost determined by its direct/single/double/triple level.

Direct, linked, and indexed access costs (continued)

Region	Logical blocks	Data capacity
Direct	0–11	48 KiB
Single indirect	12–1035	4 MiB
Double indirect	1036–1,049,611	4 GiB
Triple indirect	Following region	4 TiB

Offset 5 MiB exceeds the direct-plus-single region and requires double indirection. A sparse hole need not allocate a data block and reads as zero even though logical size covers it. A zero inode pointer is therefore not automatically corruption.

Three allocation methods: locate a file block



These are independent configurations. Numbers denote physical block IDs.

These are three independent examples, each with three data blocks numbered logically from zero. For contiguous allocation, `start=4, length=3` maps logical block 2 to physical block `4+2=6`. For FAT, the directory stores head 12; follow `12→9→14→EOF`, so logical block 2 is physical block 14. For indexed allocation, the directory points to index block 1, whose entries are 8, 11, 15; logical block 2 is physical block 15.

Two index levels: address calculation and I/O

Let data and index blocks both be 4 KiB, with 4-byte block pointers. Each index block holds `N=4096/4=1024` entries. Two levels can identify `N²=1,048,576` data blocks; logical block j, numbered from zero, uses `outer=floor(j/N)` and `inner=j mod N`.

For `j=1280`, outer=1 and inner=256. Read entry 1 of the first-level index to locate a second-level block, then its entry 256 to locate the data block. If the inode is already in memory, both index blocks and the target data block miss all relevant caches, each block requires one read, and other metadata and device caches are excluded, the model costs **2 index-block reads + 1 data-block read = 3 reads**. A cached first level saves one index read; a cached data block can save another. Three is a result of these assumptions, not a universal fixed cost.

Capacity of an inode index tree

Assume an inode has 12 direct pointers and one each of single, double, and triple indirect pointers. With 4 KiB blocks and 4-byte pointers, each index block contains `N=1024` pointers. A direct pointer identifies one data block; the three indirect forms identify up to `N`, `N²`, and `N³` data blocks, respectively.

Path	Addressable data blocks
12 direct	12
Single indirect	1,024
Double indirect	1,048,576
Triple indirect	1,073,741,824
Total	**1,074,791,436**

The data capacity is `1,074,791,436 × 4096 = 4,402,345,721,856 bytes`, about **4.00391 TiB**. This is the structural addressability limit of this index tree. It does not subtract physical storage for index metadata, nor does it establish that the filesystem's file-size field, block-address format, or available capacity permits such a file. Reading a hole within file size returns zero-valued data; a read beyond EOF returns 0 bytes instead.

Derive indirect indices from a byte offset

Let $q = \lfloor offset/4096 \rfloor$, $d = offset \bmod 4096$. Twelve logical blocks are direct, the next 1024 single-indirect, then 1024^2 double-indirect and 1024^3 triple-indirect. Use half-open ranges to avoid an extra endpoint.

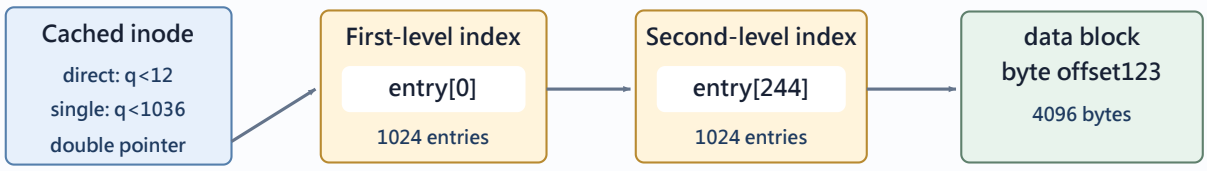
For offset=5 MiB+123=5,243,003 within the file size, q=1280, d=123. Subtract 12 direct and 1024 single blocks to obtain double-region index 244:

$$244 = 0 \cdot 1024 + 244.$$

Follow inode.double→first-level entry0→second-level entry244→data frame, then byte123. With the inode resident and both index levels/data cold, this costs two index reads plus one data read. A cached first index saves one metadata I/O.

Derive indirect indices from a byte offset (continued)

offset=5 MiB+123 → logical block1280 → double indirect [0][244]



244=1280−12−1024=0×1024+244. Cold access reads each yellow index, then the green data block.
Arrows are block pointers. Add 123 only after locating the final data block.

Total data addressability is $4096(12 + 1024 + 1024^2 + 1024^3)$ bytes, excluding index blocks from user data. Actual file limits also depend on size fields, block-number ranges, and filesystem rules. Within file size, a hole pointer can supply zeros without allocating a data block; returned zeros do not prove a zero-filled disk block exists. A regular read starting at EOF returns zero bytes, not an unbounded stream of zeros.

Allocation invariants must admit sparse holes and reflink/COW sharing. A hole has no data block. Reflink lets different inodes temporarily reference a COW-protected block, tracked by reference/version metadata and split on modification. Duplicate allocation means treating one block as independently writable without a sharing protocol. Removing one inode reference cannot free a block still used by a snapshot/reflink.

FAT versus pointers embedded in data blocks

For chain 11→12→13→EOF, embedding a four-byte next pointer in every 4096 B data block leaves 4092 B payload. Reaching the third chunk reads 11 and 12 for links, then 13, totaling three reads. Do not simultaneously count 4096 B payload and ignore the embedded pointer.

FAT stores FAT[11]=12, FAT[12]=13, FAT[13]=EOF in a separate table. If resident, locating the third block still needs two pointer lookups but only one data I/O. A cold FAT adds the table blocks actually read. FAT improves chain-metadata locality without changing CPU traversal from O(q) to O(1). Indexed allocation directly selects entry q.

An extent stores (logical_start, physical_start, length). Extent (0,8,6) maps logical 4 to physical12 by displacement. Occupied adjacent space can require another extent rather than moving the entire file, but extent-tree lookup and metadata costs remain separate.

Bitmap convention and a concrete allocation

Use 1=free,0=allocated, with block 0 first:

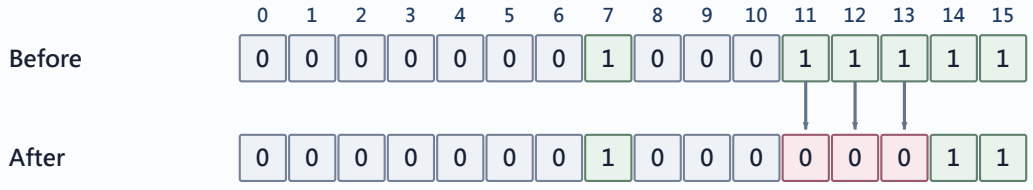
```
block:  0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
bits:   0 0 0 0 0 0 0 1 0 0  0  1  1  1  1  1
```

The string 0000000100011111 represents free set {7,11,12,13,14,15}. Inode7 already owns data8–10. Appending 12 KiB needs three blocks. A single contiguous extent starts at 11, because block 8 prevents a three-block run starting at 7.

```
after: 0000000100000011
free: {7,14,15}
```

Bitmap convention and a concrete allocation (continued)

First-fit allocates consecutive blocks 11, 12, 13: free → used



inode7: blocks [8,9,10] → [8,9,10,11,12,13]; size 12 KiB → 24 KiB

Green 1 = free; gray 0 = already allocated; red 0 = newly claimed. Free set = {7,14,15}.

Preserve that allocated blocks are not free and valid allocated inode pointers identify allocated blocks. Selecting volatile bits and updating disk bitmap/inode separately can crash into duplicate allocation or an orphan. A memory lock prevents concurrent races, not power-loss inconsistency.

$$\#bits = \#blocks, \qquad \#bytes = \left\lceil \frac{\#blocks}{8} \right\rceil .$$

A 1 TiB volume has 2^{28} four-KiB blocks, requiring 2^{25} bitmap bytes, or 32 MiB. Storage I/O still operates in blocks.

Four representations of the same free set

For {7,11,12,13,14,15}, a bitmap uses 16 bits. A free list can be head7→11→12→13→14→15→null. Allocating one block pops head7 and sets head=11; it need not scan the entire list. Finding a contiguous three-block run needs additional search/organization.

Grouping places several free addresses in a management block. With three addresses per group, the first two identify allocatable pages and the third links the next group. One read obtains two candidates. The format must define when the next-group management page itself becomes free.

Counting stores (start, length), here (7,1),(11,5). Allocating three from (11,5) returns [11,14), leaving (14,2). Freeing [11,14) merges it with (14,2) into (11,5). Free intervals are disjoint, adjacent intervals coalesced, and their union exactly the free set. A balanced offset tree supports O(log r) insertion/coalescing for r runs. Finding a sufficiently large run additionally needs a size index or subtree maximum; an offset tree alone does not make every query logarithmic.

The textbook ZFS space-map model logs ordered extent allocations/frees. Starting at (11,5), replay alloc(11,3) to obtain (14,2), then free(12,1) to obtain (12,1),(14,2). Rebuild from a checkpoint and remaining log instead of rewriting an entire bitmap in place. Preserve event order and transaction identity; replaying one allocation twice is not two new allocations.

Cache, read-ahead, and writeback

The cold example costs three metadata plus two data reads. Fully cached directory/inode/data state gives zero disk reads. Report hit rates by layer instead of replacing the model with a generic reduction percentage.

1. A cache hit avoids disk transfer but still costs lookup, copying, locking, and memory work.
2. Read-ahead loads consecutive blocks 8/9 to reduce seeking/latency; incorrect predictions perform extra reads.
3. Asynchronous writeback can return from write while data remains in dirty page cache. Flush/commit records determine the later durability point.

Fix 10,000 files, each exactly 33 KiB, without compression, deduplication, or holes. Four-KiB units allocate nine units, 36 KiB per file, wasting 3 KiB. Sixty-four-KiB units allocate one unit, wasting 31 KiB. The difference is 28 KiB per file, or 280,000 KiB=273.4375 MiB total. An average size alone cannot justify taking a ceiling after averaging because generally $E[\lceil S/B \rceil] \neq \lceil E[S]/B \rceil$.

In the simplified model of one four-byte mapping entry per unit, entries decrease from nine to one, saving only 32 bytes/file and 320,000 bytes total. These pointer bytes are separate from the 28 KiB data-space difference. Actual inode/extent/index metadata must follow its own structure.

Pointer sizes, unified cache, and synchronous writes

A four-byte pointer addresses 2^{32} block positions, theoretically $2^{32} \cdot 4\ \mathrm{KiB} = 16\ \mathrm{TiB}$. Actual file size also depends on indirection, block counts, and filesystem fields. Eight-byte pointers expand addressing but halve pointers per index block, so they do not automatically save space.

A unified page cache shares pages between mapped files and read/write. Consistency depends on PTEs, dirty tracking, writeback, and locking. Synchronous writing waits for its specified durability point. An asynchronous write may return in cache; its success alone does not prove survival of a later power loss.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Journal recovery at each failure point

Transaction T7 appends 12 KiB to a file initially sized 12 KiB. Blocks 11, 12, and 13 are exclusively reserved or locked from selection until commit or abort, so concurrent transactions cannot allocate them; old data blocks are not overwritten. After a crash, journal recovery completes before allocation or serving the new state resumes:

```
bitmap 11,12,13: free→used
I7.blocks += 11,12,13
I7.size = 24 KiB
data: blocks 11,12,13
```

Use a redo-only, no-steal ordered metadata journal. Descriptors, metadata after-images, and commit identify the transaction. A complete commit is atomically recognizable; torn/incomplete records do not count. Durable ordering is descriptor→data completion→metadata records→commit→checkpoint→transaction clearing. Before commit, uncommitted metadata cannot reach home locations. Device volatile caching must honor the required flush/barrier order.

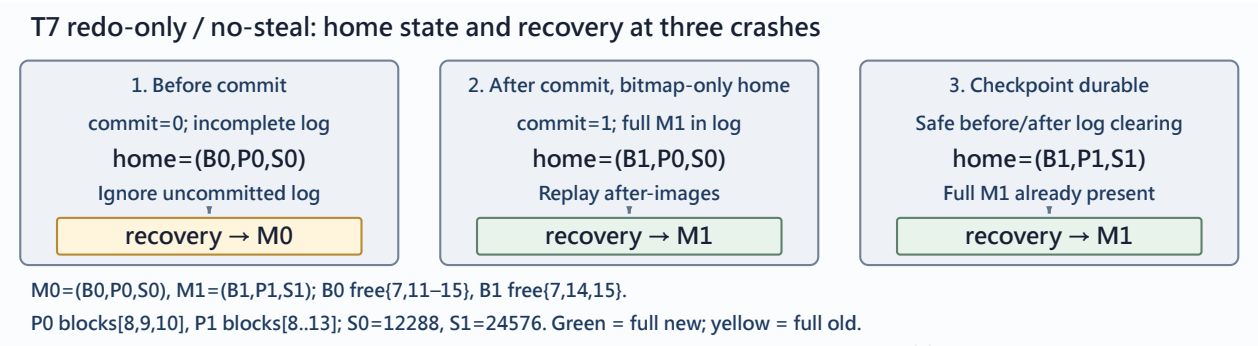
Journal recovery at each failure point (continued)

Crash point	Recovery evidence	Action	Visible state
Before descriptor	No T7	Discard	Old, size 12 KiB
After descriptor, before commit	Uncommitted	Ignore	Old
After commit, incomplete home	Committed	Idempotent redo	New, size 24 KiB
Home complete, log not cleared	Committed, possibly applied	Redo or recognize completion	Still new

Written data alone is not the commit criterion. This ordered model completes data before commit so metadata never publishes unwritten data. Metadata journaling alone does not give all user data the same transaction atomicity/durability.

Fsck scans directory/inode/bitmap relationships for orphan or duplicate blocks. It cannot reconstruct a user write that never persisted.

Journal recovery at each failure point (continued)



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every legal crash prefix recovers to old or new

Let metadata $M = (bitmap, pointers, size)$. Old M_0 has pointers [8,9,10] and size 12288; new M_1 has [8,9,10,11,12,13] and size 24576.

Before commit, no-steal keeps home at M_0. New data may exist in blocks 11–13 but no published inode points there. Ignoring uncommitted records preserves M_0, and those free blocks can be allocated again.

After commit but before checkpoint completion, full after-images are already durable. They overwrite any partially updated home into M_1. If bitmap is new while inode pointers are old, recovery must redo all three fields before mounting. Redo assigns size←24576 and the entire new pointer array; repeating it still yields M_1, rather than appending again to 36 KiB.

Log clearing is allowed only after durable checkpoint, so a cleared transaction has full M_1 at home. These branches cover every legal crash prefix and prove atomic metadata visibility. Allowing precommit home changes would require undo or another protocol; discarding the uncommitted log would then be insufficient.

Journal, snapshot, and backup solve different failures

A journal repairs interrupted updates, not legitimate deletion followed by overwrite. A snapshot retains old reachable blocks but can fail with the same destroyed device. A backup stores a separate copy; recovery requires that copy to exist.

Let Sunday full backup be B0, Monday incremental B1 contain changes since Sunday, and Tuesday B2 changes since Monday. Recovering Tuesday after Wednesday failure requires B0→B1→B2, including deletion/rename records. B0+B2 omits files changed only Monday. Only a differential B2 containing all changes since B0 permits B0+B2. Untrusted mtimes or a changing live source require a consistent snapshot/point and reliable change records.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NFSv3 handles, validation, and retries

Fix NFSv3, RFC1813, without importing NFSv4 OPEN/session state. Mount supplies a remote root handle. LOOKUP(directory_handle, name) obtains opaque child handles; READ carries handle, offset, count. NFSv3 has no remote OPEN/CLOSE procedure. Client fd/offset management is not a server POSIX OFD. A handle identifies an object, not an arbitrary resolvable filename.

Given /export handle H0, LOOKUP(H0, u)→H1, LOOKUP(H1, notes.txt)→H7, then READ(H7,4096,4096). Retrying READ adds no repeated mutation, but another client may have changed its bytes. Idempotent effects do not imply eternally identical results. If REMOVE's response is lost and another client recreates the same name, executing the old REMOVE as a new request can delete the wrong object.

Step	Client	Server	Evidence
1	Sends create X, id12	Creates X; reply lost	Request ID/effect/result
2	Times out, retries id12	Finds prior record	Returns prior result without another create

This requires a duplicate-request cache or the corresponding version's transaction/session mechanism, with explicit retention and crash behavior. Expiring finite caches do not establish permanent exactly-once execution. Client caches must validate relevant version/mtime/handle state; a cache hit is not a server commit.

UNSTABLE writes, COMMIT, and server restart

NFSv3 WRITE returns its actual stability level and a write verifier. UNSTABLE can leave data only in volatile memory. A client requiring durability sends COMMIT and compares its verifier with prior WRITE. A restart losing volatile data changes the verifier, requiring retransmission of unconfirmed data.

Event	Server state	Client obligation
WRITE offset 0, A returns UNSTABLE, v7	A may only be in RAM	Retain A and uncommitted range
Restart changes verifier to v8	A may be lost	Prior WRITE success proves no durability
COMMIT returns v8 instead of v7	A's persistence unproven	Retransmit A, then request stability
Retransmitted write completes stably with matching verifier	Protocol-stable storage reached	Release retransmission obligation

Attribute/data caches can still be stale. Close-to-open and timeout validation are client policies, not global linearizability implied by the NFSv3 name. A remote mount does not automatically import the server's other mounts; clients resolve their own mount tables.

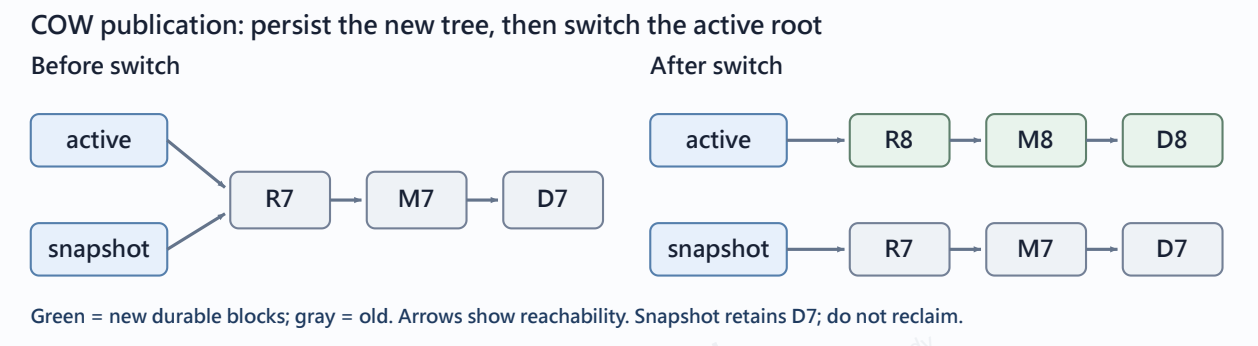
COW root publication and snapshot sharing

Version7 has R7→M7→D7, reachable from both active and snapshot roots. Counting two retained roots is a reachability model, not a claim that WAFL stores integer refcount2 in every block. The textbook's snapshot bitmap records snapshot usage.

Stage	Active root	New blocks	Old-tree reachability
v7	R7	None	Active/snapshot reach R7/M7/D7
Prepare v8	R7	D8, M8, R8	Transaction reserves unpublished R8; old tree retained
Durable	R7	D8/M8/R8 durable	Crash before switch still selects v7
Atomic switch	R8	Published	Active reaches new tree; snapshot reaches old

Write new data and all new metadata before recoverable atomic root publication. This does not assume arbitrary-size hardware root writes are intrinsically atomic. A snapshot retaining R7 prevents reclaiming D7. Only removal of all retaining roots permits reclamation. Transactions also reserve unpublished new blocks so other allocators cannot reuse them merely because no public root reaches them yet.

COW root publication and snapshot sharing (continued)



One data-block update can require new data, metadata, and root blocks. Reclamation must account for active and snapshot references instead of freeing a block merely because it is old.

Four mappings from names to blocks

Mapping	Invariant	Failure
Path/directory→inode	Namespace reaches intended object	Textual symlink-suffix cancellation
fd→OFD→inode	Correct offset/reference ownership	Treat dup offsets as independent
Logical→physical block or hole	Mapping agrees with allocation	Treat hole as disk block 0
Pointers/size↔free space	No incompatible reallocation	Mark free while old pointer remains usable

Memory locks coordinate concurrent updates. Crash consistency of several metadata fields additionally requires journaling/COW. Valid, dirty, writeback, and device-completed cache states are not one written state.

Journal and COW publication points

T7 uses redo-only/no-steal; complete after-images can be replayed after durable commit.

Crash prefix	Recovery
Commit not durable	Home remains old; no uncommitted home writes
Commit durable, home partial	Replay every after-image into consistent new
Home/checkpoint durable	Reclaim log only under the proved truncation rule

After-image assignments are idempotent. Precommit home writes require undo or another mechanism. COW instead persists new data/metadata before recoverable root publication; retained snapshots forbid premature reclamation.

Metadata consistency alone is not a data transaction, backup, or cross-client coherence.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 12.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Chapter route

How can an OS control heterogeneous devices through standard interfaces while controlling latency, throughput, isolation, and errors?

Prerequisites are interrupts, DMA, memory mapping, blocks, and files.

- Trace an application request through device completion.
- Distinguish blocking, nonblocking, asynchronous, and buffered I/O.
- Locate bottlenecks beyond the CPU.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three I/O timelines

A read(fd, buf,4096) can hit page cache or start a device. The thread can be running, blocked, or ready; the request queued, in flight, or complete; and the buffer owned by CPU or device. These are separate state machines.

Assume one blocking thread, cache miss, and successful DMA. At t=0 the syscall starts; at 2 µs validation/submission finishes; at 3 the thread sleeps; at 102 DMA completes; at 104 the handler records completion and wakes the thread; at 110 scheduling runs it; at 112 it returns to user mode. Device service takes about100 µs, call latency112 µs, and wakeup-to-execution wait6 µs.

Before successful data is observed, it must be CPU-readable. Ready does not mean returned. Scheduling, notification, and visibility have separate synchronization conditions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One interface, several contracts

A descriptor denotes an opened object and access authority, not a hardware address. Regular files support seek; pipes/sockets normally do not. Terminals can apply line discipline; ioctl performs type-specific control. A common entry point does not enable every operation.

Distinguish acceptance into a buffer, driver submission, controller completion, media persistence, and remote application processing. These are not one universal linear sequence: a local file may reach durability under its filesystem/device synchronization contract, whereas network transmission and remote application processing have protocol-specific meanings. A write returning 4096 does not universally establish all five. Durability needs a contract supported by filesystem and device; transmission success does not prove a receiving application committed. Numerical cases here are explicit teaching models, not machine measurements.

Ports, MMIO, controllers, and buses

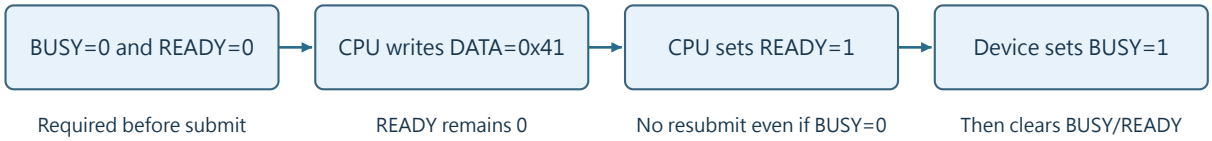
A device is the peripheral; its controller decodes commands, retains status, and transfers data. CPUs access ports through dedicated instructions or MMIO registers through protected mappings. MMIO is not ordinary RAM; caching, combining, and reordering follow architecture/driver I/O-accessor and barrier rules.

The teaching controller has STATUS, COMMAND, and DATA. STATUS bit0=BUSY, bit1=ERROR; COMMAND bit0=READY. Require BUSY=0 and READY=0, write DATA=0x41, select output, then publish READY. Publishing READY first can expose old data. Multiple CPU submitters additionally need a lock/queue; BUSY alone is not mutual exclusion.

A bus arbitrates shared transfers. Daisy-chain position/topology affects access and interrupt routing without removing permission checks.

Polling handshake and invariant

PIO output 0x41: prepare data before publishing



Required before submit | READY remains 0 | No resubmit even if BUSY=0 | Then clears BUSY/READY

can_submit = !BUSY && !READY. Do not overwrite DATA before completion.

Step	Actor	BUSY	READY	DATA	Next action
0	Initial	0	0	Old	Driver may submit
1	CPU writes data	0	0	0x41	Publish command
2	CPU sets READY	0	1	0x41	Controller accepts
3	Controller accepts	1	1	0x41	Output data
4	Completion clears state	0	0	0x41	CPU checks ERROR

can_submit=(BUSY==0 && READY==0). READY=1, BUSY=0 means published but not yet accepted. Even a single producer immediately attempting another byte must not overwrite this window. Testing only BUSY can replace0x41 with 0x42 before consumption.

Polling handshake and invariant (continued)

The correct trace is (0,0, old)→publish(0,1,0x41)→reject another submit, retaining0x41→accept(1,1,0x41)→complete(0,0,0x41)→publish(0,1,0x42). Under one producer, both-bit checking, and required ordering, can_submit stays false from publication through completion, preserving DATA. A device that never completes defeats termination; timeout does not prove it stopped using buffers.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Interrupts notify without moving every byte

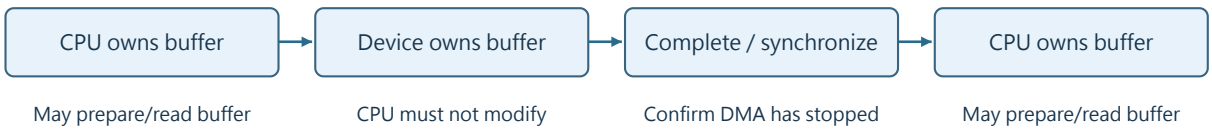
The controller raises an IRQ. At a permitted boundary the CPU saves required state and dispatches by vector. The handler checks source/status, acknowledges, updates the request, queues deferred work or wakes waiters, and returns. Shared IRQs require source identification rather than blindly completing the queue head.

Maskable interrupts, NMI, and synchronous exceptions differ; a low vector range is not uniformly NMI. Short handlers and sleepable later processing follow OS rules. An interrupt saves the required CPU control state and enters its handler; a privilege-mode transition occurs if it interrupted user mode, not necessarily when already in kernel mode. Switching threads additionally requires a scheduling context switch.

Lost wakeup occurs when a waiter reads done=false, IRQ sets true and wakes nobody, and the waiter subsequently sleeps. Under one lock, check the predicate, enqueue, and atomically release the lock while sleeping. Recheck in a while loop after waking; the IRQ publishes done under corresponding synchronization. The predicate stores completion evidence, while notification alone need not accumulate.

DMA addresses, coherence, and lifetime

4096 B receive: address mapping and ownership



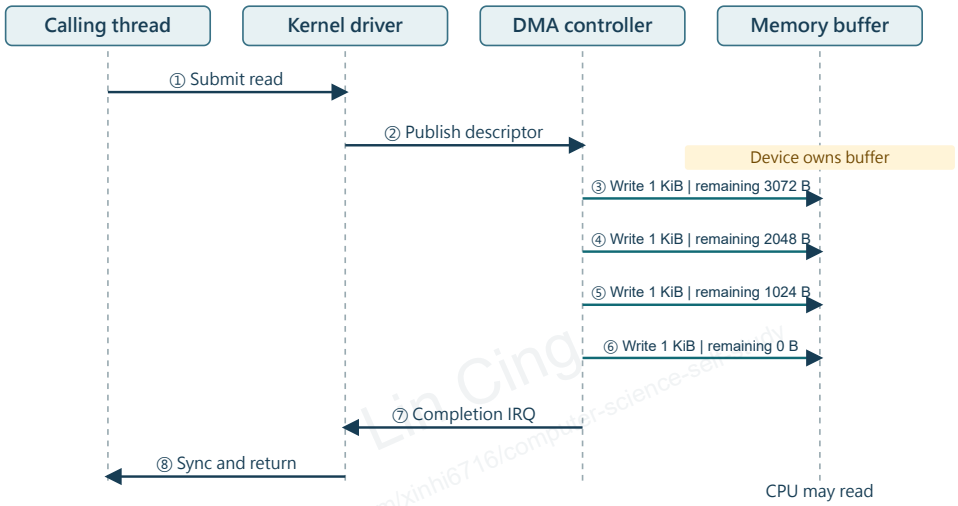
Before submit: map/cache maintenance → descriptor/ownership → doorbell

After terminal: synchronize → publish → unmap/free only after hardware stops accessing

Prepare descriptor {device_addr, dma_addr, length=4096, direction=FROM_DEVICE}, mapping, and required cache maintenance before publication. Hardware transfers and notifies completion. IOMMU translation means a DMA address need not equal user VA or physical address. IOMMU confinement does not replace descriptor authorization.

After handing the receive buffer to the device, CPU must not access it until completion and synchronization. Retain descriptor, mapping, and pages until hardware can no longer access them. Unmapping user VA does not cancel DMA. Noncoherent systems especially require the specified DMA API; a CPU barrier does not automatically invalidate caches.

One DMA read: separate control and data paths



This example reads 4096 B in four 1024 B transfers and raises one completion IRQ; neither is a universal DMA rule. The driver maps and publishes a descriptor. The device owns the buffer during DMA; CPU use resumes only after completion and required synchronization. Waking a thread does not run it immediately.

Transfer and notification costs

For 4 KiB, PIO moving 4 B per 80 ns needs 1024 operations, or 81.92 µs CPU time. DMA submission 2 µs plus IRQ/cleanup 1 µs costs 3 µs CPU time, while device transfer still takes 100 µs.

Periodic checks costing 100 ns each, every 5 µs, perform about 20 checks during service, assuming the CPU does other work or sleeps between checks, costing 2 µs CPU, with 0–5 µs detection delay, averaging 2.5 µs under uniform phase. At 2000 completions/s and 1 µs/IRQ, demand is 0.002 s/s=0.2% CPU. Periodic 5 µs checks need 200000 checks/s, or 2% CPU for the checks alone. Spinning continuously between checks would occupy nearly 100% of a core.

DMA still consumes memory/bus bandwidth. Overlapped CPU work does not remove system cost. Small transfers can favor PIO's lower setup cost. Compare fixed bytes, latency, and CPU denominators.

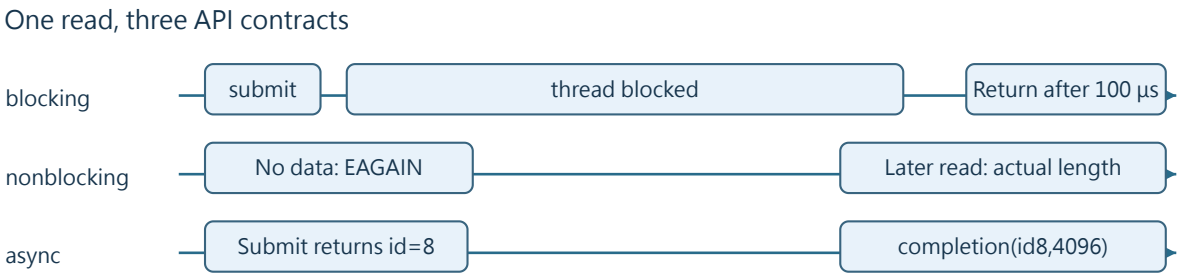
Device classes, mapping, and timers

Block devices address blocks and can support random access. Character devices provide sequential data with device-specific record/seek rules. File mmap maps contents through faults/page cache; controller MMIO accesses registers.

Raw/direct I/O bypasses selected services while retaining permission, alignment, lifetime, and isolation requirements. Sockets add endpoints/connections/transports. Streams do not preserve write boundaries; datagrams have message/truncation rules. Readiness transfers no bytes and does not prevent a competing thread consuming them first.

Timers provide clocks, interrupts, and deadlines. A deadline queue at now7 ms contains10,13,13. Program10, remove that expiration, program13, then remove both equal deadlines. A trigger deadline does not promise immediate callback execution.

Blocking, nonblocking, and asynchronous control flow



Device service time is unchanged; call control flow and notification timing differ.

Blocking can sleep and returns an ended operation, possibly short. For a positive-length nonblocking pipe or stream socket read, EAGAIN applies when no data is available, the writing side remains open, and the operation would otherwise wait; EOF can instead return 0. Regular-file O_NONBLOCK behavior depends on the object and system contract. A read after poll/select still handles EAGAIN if a competing reader consumes the data. Async submission and completion differ; completion carries ID, status, actual length. Failed submission may create no request.

Blocking, nonblocking, and asynchronous control flow (continued)

A 512 B request with 200 B available can return 200. Then done=200, remaining=312, dst=buf+200. Another 112 gives done=312, remaining=200; subsequent zero means EOF. Deliver the short 312 B or report protocol truncation. Never add -1 to done. Handle EINTR/EAGAIN under the API and progress already made.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Partial vector I/O

For writev lengths[100,200,300], successful250 B consumes100 then150. Retry [base1+150,50],[base2,300], totaling350 B. Reusing the original vector duplicates250 B.

Vectors reduce calls and copies into contiguous staging storage. They do not universally guarantee device zero-copy or arbitrary-length atomicity. Atomicity, order, and durability are independent. Delayed async buffers live through completion; a kernel copy changes cost/lifetime. Failed cancellation or an already-completed response does not authorize freeing memory hardware may still use.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Queues, buffers, caches, and spools

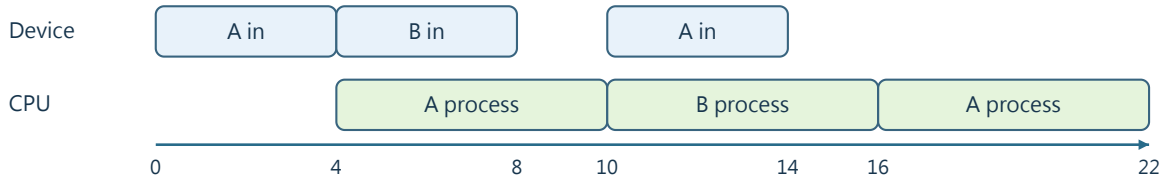
Scheduling selects service order; buffering absorbs rate/unit mismatches and preserves copy semantics; caching retains reusable data; spooling queues complete jobs for relatively exclusive devices. One region can be buffer and cache, with distinct correctness conditions.

Reads arrive for 40,10,41. Reordering10→40→41 can reduce HDD movement but cannot cross dependencies/barriers. This mechanical-seek model is not SSD latency. Fairness/deadlines/throughput can conflict.

A dirty write-back cache can hold the only newest copy. Cache is not always an identical duplicate elsewhere. Accepted buffered write, device completion, and durability differ.

Double buffering with occupied slots

Alternate two slots; hand only a free slot to the device



ms. Input 4 ms/block; CPU 6 ms/block. B waits at 8–10; A waits at 14–16.

Fill 0–4; overlap 4–16; drain 16–22. Three blocks finish at 22 ms, not 18 ms.

Input takes 4 ms/block and CPU6 ms. One buffer takes 30 ms for three blocks. With two, A inputs0–4 and processes4–10. B inputs4–8, waits8–10, processes10–16. A inputs10–14, waits14–16, processes16–22. Total22 ms; steady period=max(4,6)=6 ms.

At4–8 the units overlap, but at 8 hardware cannot overwrite A while CPU owns it. For N blocks with full overlap and two sufficient slots,

$$T = t_{in} + t_{cpu} + (N-1)\max(t_{in}, t_{cpu}).$$

Finite batches retain fill and drain time.

Copy semantics, printing, and errors

An application writes ABC then changes its buffer to XYZ. If write copies/finishes consumption before returning, output remains ABC. Zero-copy async may forbid modification until completion. State the applicable contract.

The spooler persists three-page job J17 and prints. Page2 prints but acknowledgment is lost. Resending can duplicate pages; deleting can omit pages. A persistent host job ID prevents duplicate host job creation, not necessarily duplicate physical effects. Without device progress/deduplication, report outcome unknown.

Transient failures can permit retry, while non-idempotent operations cannot be blindly resent. Length/failure conventions determine valid errno use; stale errno does not make a successful call fail. Controller sense/error logs provide detail.

Permissions and kernel objects

P fd4→process table→open-file object{offset 8192, readable=true, ops=regular_file_ops}→inode{logical 2→physical70}→block-driver queue. Some descriptors share an open offset; independent opens need not.

Validate fd/mode/range/user pointer, then VFS dispatches by type. Page-cache hit copies directly; miss enters block/driver. Socket ops receive network bytes rather than using block 70. Name resolution and read dispatch differ; read need not walk the pathname anew.

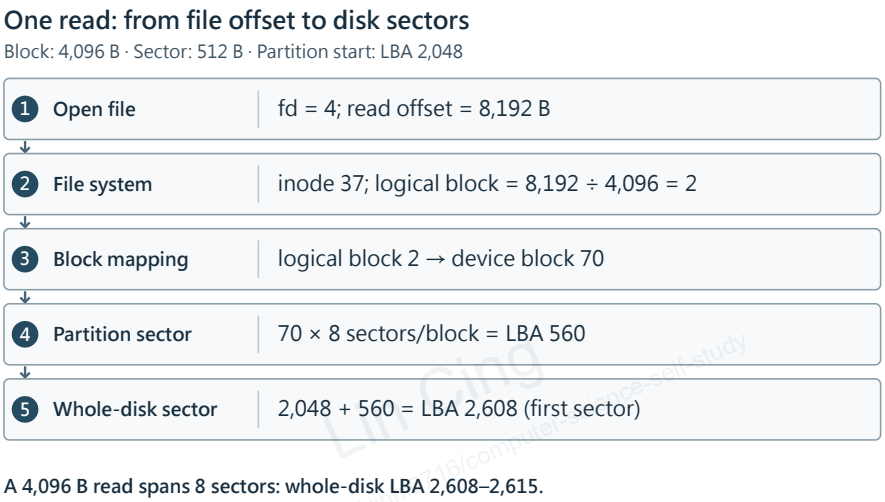
Privilege/mapping permissions protect ports, MMIO, and control registers. Validate/appropriately pin user buffers. Device authority does not authorize arbitrary-memory DMA. Reservations can deadlock when A holds scanner waiting printer while B holds printer waiting scanner. Ordered acquisition, joint allocation, or recovery must break the cycle.

Complete address translation

Mount /data selects device(8,2); /data/log.bin has inode 37; open creates fd4. Read4096 B at offset 8192 gives logical 2, displacement0 for 4096 B blocks. Inode maps logical 2 to device block 70. With512 B sectors, partition LBA=560, count8. Major 8/minor 2 selects driver/partition; startLBA 2048 gives whole-disk LBA 2608. The controller additionally uses a DMA mapping. Fd, inode, user VA are not LBAs. This fixed teaching model does not describe every UNIX implementation identically.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Address translation for one read



A 4,096 B read spans 8 sectors: whole-disk LBA 2,608–2,615.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Completion, timeout, and abort races

NEW→QUEUED→IN_FLIGHT→DONE or ABORTED publishes at most once; a device that never responds need not reach a terminal state. Timeout requests abort without proving ABORTED. Release mappings/pages only after normal completion or an abort acknowledgment confirming hardware quiescence, followed by the platform-required DMA synchronization.

Request 8 is submitted at t=0, times out at t=100, requests abort at t=101, completes at t=102, and receives an already-completed abort reply at t=103. A lock/CAS protects terminal state. Completion at t=102 wins; the abort reply at t=103 only releases bookkeeping without waking/unmapping twice. A timeout may be an early status notification, not terminal completion. If DMA targets a user buffer, the API must require the caller to retain it and not access or reuse it until final completion/quiescence; pinning pages alone cannot prevent reuse. Alternatively, DMA may use a kernel-owned bounce buffer so that the user buffer is no longer accessed by the device. Specify notification, terminal completion, and reclamation separately.

Only a branch atomically changing nonterminal to terminal obtains publication authority; unmapping also requires hardware quiescence and the required synchronization. All others see terminal and refrain. Two winners would require a forbidden second terminal transition, a contradiction. This proves at-most-once completion, neither eventual response nor exactly-once external effects.

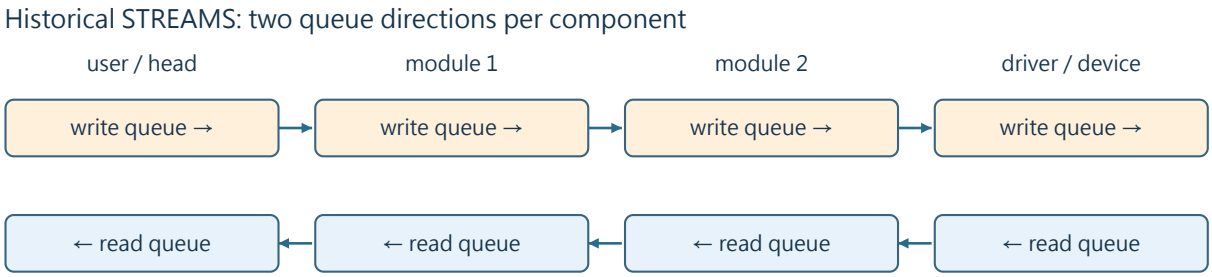
Buffer and control-block lifetimes

Stopped DMA permits buffer synchronization/unmapping, but timers/IRQs/abort callbacks may still read request state. The winner cannot immediately free its control block. Every queued/active callback already holds a stable reference and releases on exit. Canceling a timer cannot drop its reference without proving callback exit.

At101 owner, IRQ, abort hold count3. IRQ wins102, cleans DMA, exits3→2; owner delivers result and exits2→1; abort103 reads DONE, cleans bookkeeping, exits1→0, then free. An in-flight timer adds a reference. Acquisition itself requires lifetime protection; incrementing through a freed raw pointer is unsafe.

While any callback can read control state, a positive reference preserves it. Together with terminal checking this prevents double completion and use-after-free. Slot/ID reuse additionally requires generation or stable event identity against late events.

Bidirectional message queues



Write moves right; read moves left. Full ordinary data queues propagate backpressure upstream.

Use historical System V/Solaris STREAMS. Head meets users, driver meets devices, ioctl pushes/pops modules. Every component has read/write queues; read moves toward head, write toward driver. Data/control messages do not require every module to copy payload.

Put procedures can process/forward immediately or enqueue. Service procedures drain runnable queues. Read/write expose byte streams; getmsg/putmsg can preserve control/data portions. Blocking at the user head does not mean synchronous waiting through every module.

Numerical backpressure

Q capacity 1000 B, high 800, low 300, messages 200. Starting600, one accepted message gives800 and FULL, stopping upstream. Service yields600,400,200; below300 back-enables upstream. Hysteresis avoids switching repeatedly at one threshold.

Two upstream messages still consume memory. Full upstream queues eventually block the head writer. An unpausable hardware source may drop when all fill; IRQ cannot sleep indefinitely. Actual STREAMS priority/accounting is richer. This model covers ordinary fixed200 B data messages only.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Resource throughput limits

Requests4 KiB; device100000/s; CPU5 µs/request gives200000/s on one core. Memory1 GiB/s with 12 KiB traffic/request, 4 KiB DMA plus copy read 4/write 4, gives87381.33/s. Thus overall throughput≤min(100000,200000,87381.33)=87381.33/s, a bound rather than achieved performance.

Truly removing that copy gives4 KiB traffic and262144/s memory bound; device100000/s becomes limiting. Count any remaining traffic. More cores cannot overcome the original memory bound.

Batching eight arrivals10 µs apart adds70 µs for first,0 for last, average35 µs. At1 µs/IRQ, CPU cost/request falls from 1 to0.125 µs at a latency cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Latency and Little's relation

For finite steady-state means over one boundary, $L = \lambda W$. At80000/s and250 µs queue-plus-service response, L=20. With100 µs service, in-service count8, queue12, waiting150 µs. Queue count must not be paired with total response.

Minimum100 µs latency and100000/s throughput require average10 in flight, but queue depth10 does not guarantee that rate. Beyond saturation, more queueing can raise W without raising λ. At110000 arrivals/s and at most100000 completions/s without dropping/throttling, queue grows and finite steady-state L is unavailable.

This explicit conservation-model extension assumes neither exponential service nor the M/M/1 formula $W = 1/(\mu - \lambda)$. The latter needs additional distribution assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One read across layers

Fd4, offset 8192→inode 37 logical 2→partitionLBA 560→whole-diskLBA2608→DMA4096 B. Thread blocked, request in flight, device owns buffer. Completion establishes CPU visibility, publishes state, readies thread; scheduling later returns it.

Cache hits skip lower layers. Nonblocking can return EAGAIN without bytes; async can return submission first. Short reads, errors, permissions, and lifetime remain. Check pre/postconditions along naming/authorization→queue→device→publication→use→reclamation rather than infer durable effects from one callback.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Costs and observable guarantees

PIO/DMA choose transfer, poll/IRQ detection, blocking/async call contract. These can combine. Buffer/cache/spool solve different problems; copies can improve performance while separating return from durability.

State bytes/request, CPU time/request, bus bytes/request, service time, and observation boundary. Compare equal denominators; retain finite fill/drain and steady-state stability. Exercises retain intermediate states and general arguments rather than equating a few traces with proof.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 13.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Chapter route

How can enforceable structures limit a subject's authority over objects and preserve provable consistency after revocation?

Prerequisites are mode bits, access control, graphs, and matrices.

- Distinguish subjects, objects, domains, and rights.
- Compare ACLs and capabilities.
- Analyze revocation of existing handles.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Who may perform which operation on an object?

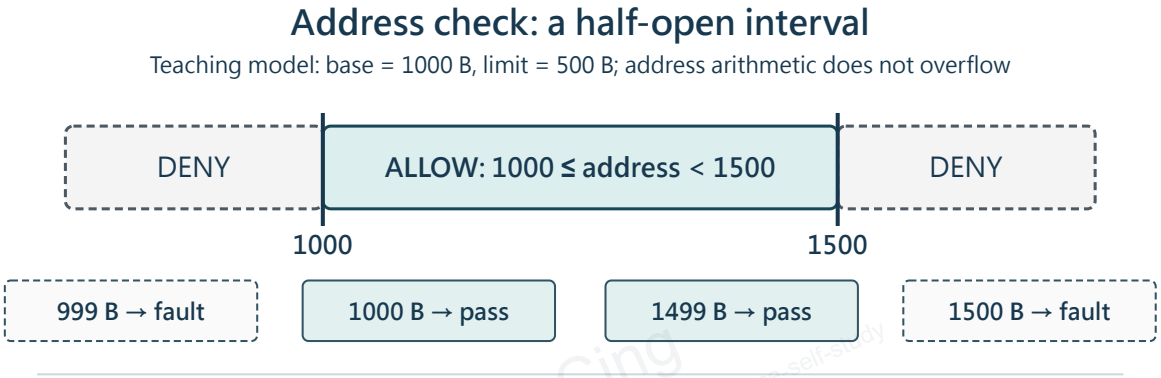
A **subject** is an actor issuing requests, such as a process. Its current **protection domain** is the set of object-right pairs it can exercise; an authorized domain switch can change that set. An object is a resource, and a right denotes an operation. Subject and domain are distinct concepts.

A policy may let assistants read submissions and only the instructor change grades. ACLs, capabilities, and protected system calls are mechanisms that enforce it. Complete mediation requires effective authorization for each use. Checking an ACL at open and a protected handle's rights on each read/write also requires a defined revocation policy for issued handles.

A reference monitor must resist bypass, protect its own state, and be small enough to inspect. Hardware modes and address-space isolation support protection but do not prove application policy correct. Security also addresses authentication, attacks, faults, auditing, and trust; an obscure pathname is not access control.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Base and limit: enforcing an OS policy



Mechanism: compare base ≤ address < base + limit; otherwise raise a fault.

Policy: the OS decides the permitted range and securely sets base and limit.

This byte-addressed example checks a physical address against `[base, base + limit)` and assumes the addition cannot overflow. The OS chooses the permitted range and protects its configuration; hardware performs the comparison and raises a fault for an out-of-range address. Actual hardware may check a relative address before relocation, so this diagram is a model, not a universal implementation.

Least privilege includes scope and time

A backup job needs read access to /course/submissions and write access to its backup target, not grade changes or module loading. Running the whole parser with full authority lets a malicious input exploit every permission. A low-privilege parser plus a small authorized service limits exposure, provided that service validates parameters and scope.

Granted rights R(t) must include currently necessary N(t), while minimizing the excess $R(t) \setminus N(t)$. Counting permissions alone is inadequate. Reading every student's personal data can be riskier than writing one temporary file. Scope, duration, and delegation affect the damage bound.

Fail-safe defaults deny absent explicit authorization. Audit subject, object, operation, decision, time, and correlation ID; logging does not itself block an attack. Separation of duty can require two roles to publish grades, but fails if one person can freely activate both roles.

A domain is a set of rights

Let D={⟨F1,{read}⟩,⟨F2,{read, write}⟩}. Two processes of one user can have different domains; a protected call within one process can switch domains. User/kernel modes alone do not separate user-mode processes, which also need address spaces and authorization.

A static domain unions phase requirements, such as reading source during compilation and writing binaries during linking, exposing both rights throughout. A dynamic domain narrows each phase, but its switches must be authorized and preserve/restore caller context correctly while validating arguments.

A traditional UNIX setuid executable can change effective UID to its owner on successful exec under specified conditions. It does not necessarily change real UID or act like an ordinary function call. With real UID1001 and owner0, an allowed setuid exec can produce euid0 while retaining real UID1001. The program must validate input/environment and drop privileges correctly. OS-specific suppression conditions still apply.

Multics rings and controlled gates

In the historical abstraction, ring 0 is more privileged. An access bracket (b1, b2) and gate limit b3≥b2 allow direct execution from ring i in [b1, b2]. For i>b2, entry into a more privileged ring requires i≤b3 and a designated gate. For i<b1, transfer toward lower privilege must not let parameter references expose memory unavailable to that callee.

With bracket[1,3], limit5, and gate entryA, ring 2 may use an ordinary entry and remain ring 2. Ring4 may enter ring 3 through entryA, but entryB is denied. Ring6 is denied even at entryA. Return restores caller ring/stack. Full Multics also has read/write brackets beyond this model.

Nested privilege cannot express incomparable requirements. If one domain needs only F and another only G, forcing either to contain the other grants an unnecessary object. A general access matrix supports this separation.

The initial matrix

Access matrix and domain-switch reachability

domain	F	G	P	D2	D3
D1	r*	ø	ø	switch	ø
D2	w	r	ø	ø	switch
D3	ø	ø	print	ø	ø

Switch path: D1 → D2 → D3

Current rows are not unioned

r* = read plus same-object delegation; ø = no rights. Switch paths do not authorize arbitrary entry points.

A[D, O] is a set of operations; a blank is ∅. The process uses its current row. D1 may read F, not write it; D2 may write F and switch to D3; D3 may print P. Because D1 can switch to D2 then D3, its initial row does not prove it can never print.

Switch is a right over a domain object. Switching selects another row rather than unioning both. A controlled service also needs entry/parameter restrictions against jumping into arbitrary privileged code.

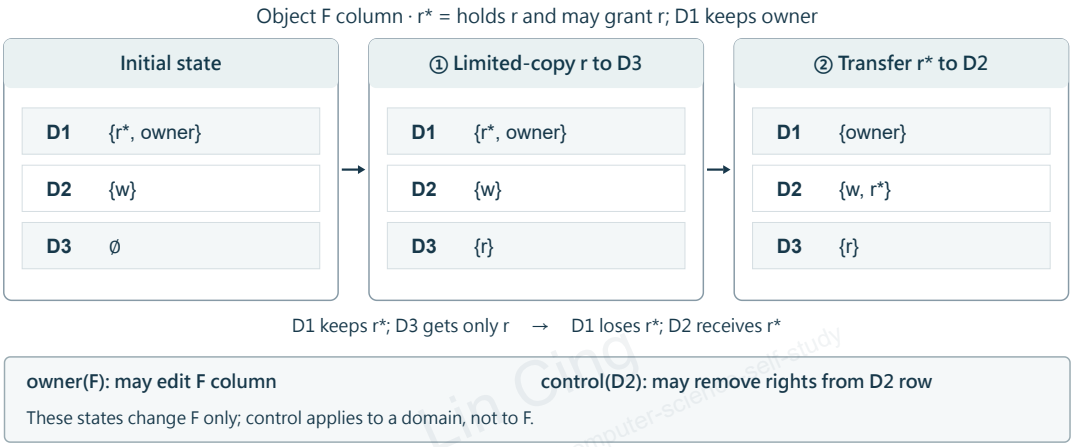
Copy, transfer, limited copy, owner, and control

Right `r*` grants r and permits its propagation within the **same object column**. F:`r*` does not authorize G:r. Ordinary copy retains the source; this model's transfer removes `r*` from the source and gives it to the target; limited copy grants r without propagation authority.

Independent initial state. This example does not continue the preceding matrix: A[D1,F]={`r*`, owner}, A[D2,F]={w}, A[D3,F]=∅. After D1 limited-copies r to D3, D3={r} and cannot delegate that right using this grant. As F's owner, D1 may add or remove specified rights in F's column, including D2's w. Ownership of F does not imply ownership of every object.

Control applies to a domain object. With control in A[D1,D2], D1 may remove rights from D2's row, but not freely add them. Removing D2's switch to D3 breaks the path D1→D2→D3; check separately for alternative paths.

Track copying and transfer in one column



This is an independent initial state; control and owner are distinct rights.

Rights propagation does not prevent information flow

D3 without copy(F) can still read F and write each byte to G if it holds those data rights. Copy rights govern delegation, not information copying. Revoking F:read also cannot ordinarily recall bytes already read into user memory.

Consider a restricted system where the only grant rule attenuates existing source rights in the same column, with no ownership or other creation of rights. Every right in every successor state originates in an initial right for that same object. The initial state satisfies the claim. Inductively, each new r has a source r, whose provenance reaches the initial state. This proves the claim for arbitrary finite transition sequences. It neither prevents data leakage nor applies when an owner can create rights freely.

Sparse representations

Representation	Index	Convenient query	Protection/update requirement
Global triples (D, O, R)	Global or dual indexes	Any pair	Protect table and avoid a full scan per use
ACL(O)=[(D, R)]	Object column	Who may use O?	Central deletion; explicit matching/default rules
Capability list(D)	Domain row	What authority does D hold?	Unforgeable references; controlled transfer/attenuation
Locks/keys	Object locks and domain keys	Is a key accepted?	Protect lists; change keys to revoke

A capability is a protected object reference with rights. A numeric fd indexes the current process's table; guessing another process's number does not acquire its capability. Kernel tables, tagged hardware, or another protected representation can ensure unforgeability. Capabilities need not universally be absent from user address spaces.

Convert columns to rows without changing rights

F has ACL[D1:r, D2:rw]; G has[D2:r, D3:w]. Corresponding lists are D1=[F:r], D2=[F:rw, G:r], D3=[G:w]. Both represent one matrix at one instant, but delegation/revocation costs differ. Hash-indexed pair queries can have expected O(1) cost; a linear list of length k costs O(k). The representation's name does not imply constant time.

A hybrid checks ACL at open and issues a kernel handle{F, read}; write later fails its mode check. If the ACL subsequently removes read, continued use depends on revocation policy. UNIX generally does not promise immediate invalidation of every open fd. Per-use generation checks can change this contract, with mappings/in-flight operations handled separately.

This chapter's simplified ACL uses allow entries with union and default denial. Deny ACEs, ordering, and group rules in real systems need their own algorithm. Do not substitute this model for Windows DACL evaluation.

Users, roles, and task permissions

Solaris10 separates roles, rights profiles, authorizations, and process privileges. A role groups administrative tasks; a profile can specify commands/execution attributes; an application can check authorizations; the kernel checks privileges. Role authority is not merely a UID, and an authorization string is not itself a kernel privilege.

In the original teaching model, U assumes BackupRole and executes BackupCmd with only source-read and target-write rights, unable to change GradeDB. V assumes GradeRole but cannot approve its own publication. Role activation still needs authentication/auditing. User-to-role assignment alone does not allow every request.

Formally $UA \subseteq Users \times Roles$, $PA \subseteq Roles \times Permissions$, and $activeRoles(session) \subseteq assignedRoles(U)$. A request passes iff some active role contains the required permission, subject to activation constraints such as exclusion. This simplified RBAC equation is not all Solaris privilege-set computation.

Four independent revocation dimensions

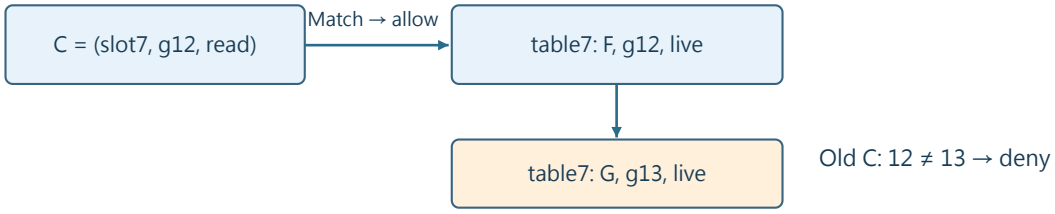
Immediate/delayed specifies when revocation takes effect; selective/general specifies holders; partial/total specifies operations; temporary/permanent specifies future regranting. Even immediate revocation needs a boundary: denying later operations does not automatically cancel in-flight I/O or erase previously read data.

An ACL centralizes removal, but issued handles or caches may delay its effect. Distributed capabilities can use reacquisition/leases, back-pointers, indirection, or keys/versions. Back-pointers must track all copies; indirection centralizes checks at a revocable slot. Changing a shared master key revokes all capabilities using it; selective revocation needs separate key groups or proxies.

Lease example. Use a consistent trusted time base. A grant begins at t=0 and expires at t=10 seconds; every new operation checks expiry. At t=3 the authority revokes the grant and refuses renewal. Even if notification is delayed, operations starting at $t \geq 10$ are denied by expiry, so the delay is at most 7 seconds. Unlimited caching, use after expiry, or renewal after revocation invalidates this bound. A separate cancellation contract governs operations already in flight.

Indirection and generation numbers

Reusing a slot: index 7 alone is insufficient



Change generation on reuse. Synchronize checking and retaining the object during resolution.

Capability C={slot7, generation12, rights r}; table[7]={F, g12, live}. Resolve checks liveness, generation equality, and rights, then obtains a stable object reference under the same synchronization. Revocation clears live; reuse for G increments generation to13. Old C=(7,12) fails against(7,13).

Assume generations are never reused while old tokens may exist, and revocation/reallocation cannot interleave between checking and retaining the object. Then every accepted C identifies the object version for which it was issued. Finite wrapping counters with indefinitely retained old tokens eventually permit ABA. Prevent reuse or prove old-token quiescence before wrapping. Checking only slot7 demonstrably grants G to F's old capability.

Hydra and restricted amplification

Historical Hydra supports auxiliary rights for user-defined types. A client capability for Account A contains deposit, not raw representation read/write. Calling trusted deposit(A,10) checks type and operation rights; the protected procedure temporarily obtains representation access, changing balance40→50. The client still holds only deposit afterward.

Only approved procedures, types, and rules amplify authority. Arbitrary routines that turn read-only arguments into writable ones would violate callers' protection. Mutually distrustful subsystems must constrain amplifiable rights, retained references, and parameter scope; encapsulation alone is insufficient.

Cambridge CAP data and software capabilities

Machine/microcode enforces data-capability segment read/write/execute rights. Protected procedures interpret software capabilities' custom meanings; the lower mechanism prevents forgery/modification. Seal/unseal-style authority limits representation inspection to the matching type/environment.

Both CAP and Hydra give clients abstract operations while implementations require lower-level access through a protected typed boundary. Their rights interpretation/support libraries differ. Running arbitrary user code does not itself prove safety. A Bank procedure that corrupts balance still breaks Bank semantics; isolation limits damage to its environment without proving business invariants. Account values here are an original model, not executable Hydra/CAP APIs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

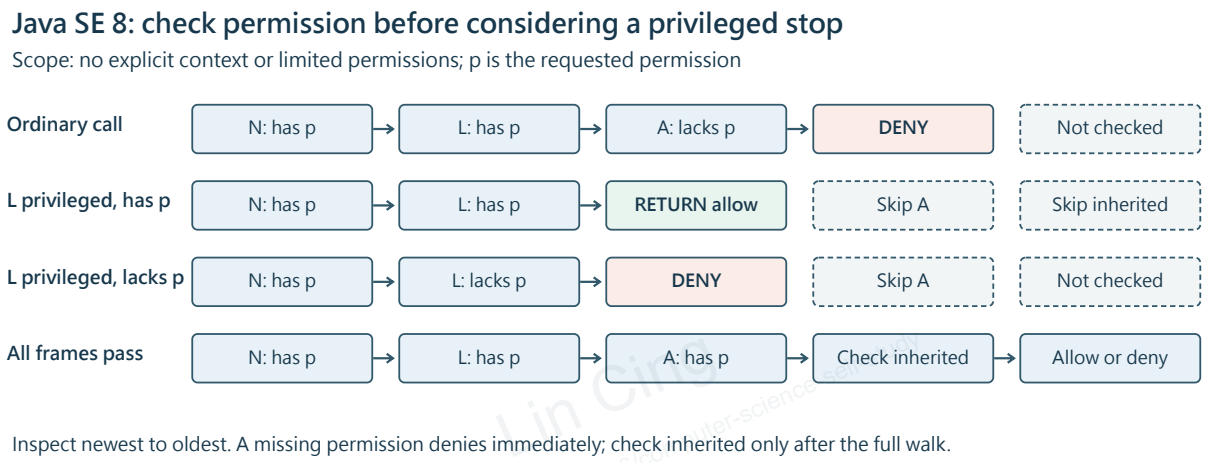
Compile-time, load-time, and runtime guarantees

Types/modules/interfaces declare access boundaries. Compilers reject statically invalid operations; runtimes check dynamic indices, types, and capabilities. If an attacker can alter machine code, forge references, or bypass checks through native entries, source-level private fields alone cannot isolate it.

Type safety constrains memory/type behavior, not business authorization. A valid Account reference can still be misused to allow withdrawals or leak information. Kernel isolation separates processes; language isolation can separate classes/objects within a process, adding runtime, loader, and native interfaces to the trusted computing base.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Historical Java SE 8 stack inspection



In the Java SE 8 model, doPrivileged has neither explicit context nor limited permissions. Stack N→L→A is newest first; N and L have permission p, A lacks it. Ordinary inspection denies at A. A privileged L permits immediately after N and L pass, skipping A and inherited context; if L lacks p, it still denies.

Only when inspection reaches the end without such an early return and all frames pass does it check the thread’ s inherited context. Other overloads have additional rules. This is a historical security model, not a modern sandbox recommendation.

Confused deputies and validated targets

Loader can connect to a fixed proxy for a low-privilege applet within a controlled privileged block. Passing arbitrary applet addr directly into that block lets the caller use Loader to scan an internal network. Restrict validated destination/port, operation, and returned data; do not execute an entire untrusted callback with elevated authority.

Inspection maintains that every checked frame holds p. A privileged stopping frame must also hold p. Thus this authorization never exceeds that boundary caller's existing p. Correct object/argument selection still depends on the caller; the stack theorem does not prove the service policy.

Equivalence from matrix to implementation

The matrix states current permission; ACLs/capabilities index it; dynamic operations change it. Preserve object identity, right types, and domain context with unforgeable, non-bypassable checks. Cached decisions require explicit revocation semantics.

Owner changes a column; control deletes from a domain row; switch changes current domain; copy/transfer/limited copy change same-object grants. Their composition changes reachable domains, so one static row is not a whole-program authority bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three distinct safety claims

Authorization safety asks whether each operation has a right. State safety asks whether revocation/reuse/races identify the wrong object. Information/business safety asks whether legal reads/writes or privileged services still leak data or violate policy. Each needs different evidence.

State allowed transitions before proving a general claim. Attenuation induction excludes arbitrary owner grants; generation proofs exclude finite wrap with everlasting old tokens; Java privilege does not excuse parameter validation. Exercises use full columns, versions, and stacks to expose these conditions.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz et al., Operating System Concepts, 9th ed., Ch. 14. Oracle: Java SE 8 AccessController. Oracle: Solaris 10 RBAC.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Chapter route

Which cryptographic and system mechanisms provide verifiable guarantees when identities, programs, and networks can be attacked?

Prerequisites are protection, basic networking, probability, and hashing.

- Distinguish confidentiality, integrity, availability, and authentication.
- Analyze symmetric/asymmetric encryption and hashes.
- Evaluate defenses under explicit attack models.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Assets, adversaries, and goals

Confidentiality restricts unauthorized reading; integrity restricts unauthorized modification; availability requires authorized service under stated conditions. A threat can violate a goal, a vulnerability is an exploitable weakness, and an attack is an actual attempt. Theft followed by encryption/deletion can violate several goals.

The teaching grade service protects grades, credentials, and availability. The adversary controls the network and submits arbitrary forms, but cannot modify the kernel or obtain the server's private key. If key theft is later allowed, earlier authentication conclusions no longer follow. Every guarantee depends on its adversary, trust roots, keys, and endpoint integrity.

Physical, human, OS, and network factors affect the boundary. TLS does not protect input captured by an endpoint keylogger. ACLs restrict unauthorized use without proving every valid-account action follows business rules.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Impersonation, replay, and intermediaries

Masquerading impersonates an identity; replay resends an old valid message; a man-in-the-middle controls what both endpoints observe. A source IP is not a cryptographic identity, and an unauthenticated public key is not bound to a service.

An attacker can replay a valid MAC on transfer(A,B,100,id17) without understanding its contents. Assuming MAC unforgeability, an uncompromised key, and the stated key-holder trust model, successful verification provides computational evidence of origin from some key holder; it does not prove fresh authorization. Authenticate the session, sequence, nonce, or operation ID, and maintain receiver state. Returning an earlier result on retry requires durable deduplication atomically linked to the transaction.

Preventing new forgeries and preventing repeated execution of one authorization are distinct goals. Encryption alone establishes neither combined claim.

Authority and propagation

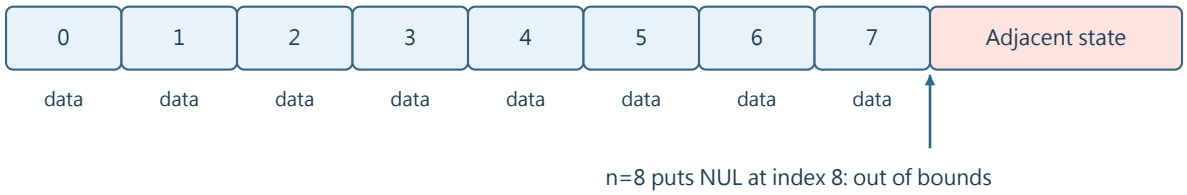
A Trojan abuses execution authority behind apparent useful behavior. A backdoor bypasses normal entry; a logic bomb triggers under a condition. Viruses infect host files/programs, while worms propagate through services/networks. Categories overlap and do not restrict viruses to one OS vendor.

An untrusted directory in PATH can substitute a namesake tool. Malicious code acquires the victim program's actual rights, not automatic root authority from its label. A compromised compiler can produce malicious output from inspected source. Executables, loaders, compilers, and update channels belong in the trust boundary.

File, boot, macro, and source viruses describe infection locations. Polymorphic, encrypted, stealth, and armored describe evasion; multipartite affects several locations. Byte signatures and behavior analysis complement one another. No signature match does not prove absence.

Buffer bounds precede control-flow claims

Abstract byte layout: the string terminator occupies a byte



Capacity 8 holds at most 7 data bytes + NUL. Binary buffers use explicit lengths.

An eight-byte buffer has indices 0–7. A C string with n data bytes requires $n+1 \leq 8$ for complete copying. At n=8 the data fits, but its NUL at index 8 overflows. Actual layout depends on ABI/compiler; this abstract figure illustrates adjacent corruption without platform addresses.

One safe contract rejects overlength input. Another explicitly truncates to k=min(n,7), copies k bytes, and sets buf[k]=0. Strncpy(...,7) alone need not append NUL. Binary data requires explicit lengths rather than strlen, with range and integer-overflow checks.

Different defenses prove different properties

Bounds/type/lifetime checks address invalid accesses. Canaries detect some overwrites before return; NX/W⊕X restrict execution from data pages; ASLR complicates address prediction; control-flow restrictions constrain targets. NX neither prevents out-of-bounds writes nor alone excludes reuse of existing executable code.

Corrupting is_admin, length, or an object pointer can violate authorization without changing return addresses. Least privilege limits a damaged component's reach but cannot replace fixing the overflow. A managed safe subset eliminates some memory errors while native/unsafe code, runtime flaws, and business policy remain boundaries. Type safety does not prove every security goal.

Worm propagation and service exposure

The 1988 Morris worm exploited service weaknesses and remote trust to seek reachable hosts. Repeated infection consumed resources; an event need not steal data to harm security.

In this discrete model, each infected host infects **at most** two distinct susceptible hosts per round, with no recovery. Thus $I_{t+1} \leq 3I_t$. Starting from $I_0=1$, the sequence 1, 3, 9, 27 holds only if every host infects exactly two distinct new hosts each round and enough susceptible hosts exist. A closed network of 100 hosts obeys $I_{t+1} \leq \min(100,3I_t)$. This is an ideal bound, not a measurement of Morris.

Scanning reveals reachable services, not necessarily vulnerabilities, and banners need not be truthful. Authorization, patching, and disabling unnecessary services reduce exposure; removing one entry does not fix input bugs elsewhere.

Resource exhaustion and defensive side effects

A teaching service has 1000 pending slots. Each incomplete handshake occupies one for 10 seconds with no early cleanup. Starting empty, accept one request every 0.01 seconds from t=0 through t=9.99 seconds: all 1000 slots are occupied before the first expires. This specified timing does not imply every arrival pattern with the same average fills the slots simultaneously.

In steady-state Little's law $L=\lambda W$, λ is the **accepted throughput** across the same observation boundary. With finite capacity and rejected requests, it need not equal offered arrival rate. If accepted throughput averages 100/s and average residence is 10 s, then average occupancy is 1000. This does not guarantee all later offers are accepted.

Legitimate demand of 80/s plus attack demand of 40/s exceeds 100/s service capacity by at least 20/s. Cookies, quotas, and delayed allocation change state costs, not upstream bandwidth. Rate limits must name their key; IP limits can penalize users sharing NAT. A password lockout can hinder guessing while allowing attackers to deny service to others. High load alone does not distinguish DDoS from a legitimate flash crowd.

Primitives and key roles

Tool	Confidentiality	Integrity/source	Creation and verification
Symmetric encryption	With a suitable scheme	Not from encryption alone	Shared-secret holders
Public-key encryption	With a suitable scheme	Does not establish sender	Anyone encrypts; private key decrypts
Hash	No	Mismatch against trusted digest	Anyone computes
MAC	No	Shared-key origin/integrity	Key holders generate/verify
Signature	No	Private-key origin/integrity	Private signs; public verifies

Correctness means $D_k(E_k(m))=m$. Security is a computational claim against specified distinguishing/forgery attacks, not logical impossibility of guessing m. Algorithms can be public; secret keys remain secret. Public keys need authenticated integrity of their identity binding.

Reused keystream algebra

For $c=m \oplus s$, $m=c \oplus s$. Let $m_1=0x41$, $m_2=0x42$, $s=0xA6$; $c_1=0xE7$, $c_2=0xE4$. Reuse gives $c_1 \oplus c_2=0x03=m_1 \oplus m_2$. Knowing m_1 reveals s and then m_2. This identity does not break the underlying cipher.

Block ciphers process fixed blocks; longer messages need correct modes and nonce/IV rules. AES's 128-bit block differs from 128/192/256-bit keys. DES, 3DES, RC4, WEP are historical context, not new-deployment recommendations. Use a complete analyzed confidentiality/authentication scheme. An unprotected public hash appended to ciphertext can be recomputed after tampering.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A complete small RSA computation

This number-theory example is not secure encryption. Set p=5, q=11, n=55,ϕ=40, e=3, d=27, ed=81=1+2ϕ. Public key=(55,3), private=(55,27). For m=12, 12^2 mod55=34 and 12^3 mod55=34 • 12 mod55=23.

Decrypt with square table $23^1 \to 23, 23^2 \to 34, 23^4 \to 1, 23^8 \to 1, 23^{16} \to 1$. Since 27=16+8+2+1, 23^{27} mod55=1 • 1 • 34 • 23 mod55=12. Raw RSA is deterministic and algebraically structured. Deployable encryption/signatures additionally require specified encoding/padding, key sizes, and purpose separation. These small primes provide no confidentiality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

RSA correctness for every message

For every m∈{0,…, n−1}, ed=1+k(p−1)(q−1). Modulo p, if p divides m, both m and m^ed are 0. Otherwise Fermat gives m^(p−1)≡1, so

$$m^{ed} = m(m^{p-1})^{k(q-1)} \equiv m \pmod{p}.$$

The same branches hold modulo q. Distinct primes and CRT yield m^ed≡m modpq, including messages not coprime with n.

Euler applied only when gcd(m, n)=1 would omit valid multiples of p/q. This proves mathematical decryption correctness, not RSA hardness, chosen-ciphertext security, or side-channel resistance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hashes, MACs, and signatures

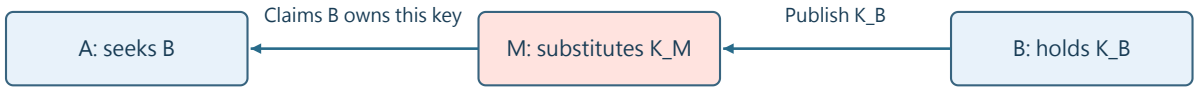
An n-bit hash has 2^n outputs, so collisions exist by the pigeonhole principle for a larger message space. Collision resistance concerns finding any distinct pair with equal digest; second-preimage resistance fixes one message first. Digest equality is computational evidence, not a theorem that messages are equal.

Anyone can replace (m, H(m)) with (m′, H(m′)); public hashes do not authenticate. MAC verifiers also know the secret, so MAC alone cannot prove to a third party that only the other participant generated a tag. Signatures use private signing and public verification, with trusted key binding, complete encoding, and key protection. They are not arbitrary encryption run backward.

HMAC is a standard keyed construction, not arbitrary H(key ∥ m). Include context, roles, session, and sequence against cross-purpose reuse.

Key distribution and substitution

Public keys need trusted identity binding, not secrecy



Without binding: A encrypts under K_M → M decrypts. Correct cipher, wrong identity.

Validate chain, name, purpose, validity, and handshake proof of private-key possession.

A seeking B receives an unauthenticated claim "B, key K". M substitutes K_M; A encrypts to M, which can decrypt although the cipher works correctly. Certificates bind service identity and key. Validate signature chains to trusted roots, target name, purpose, time, and relevant revocation policy. Certificate presence alone is insufficient.

Pairwise symmetric keys for N users require N(N−1)/2 keys. Six users need 15; adding a seventh needs six more. A KDC reduces long-term secrets per user but centralizes trust. Public-key techniques reduce secret distribution without eliminating public-key authentication or private-key protection.

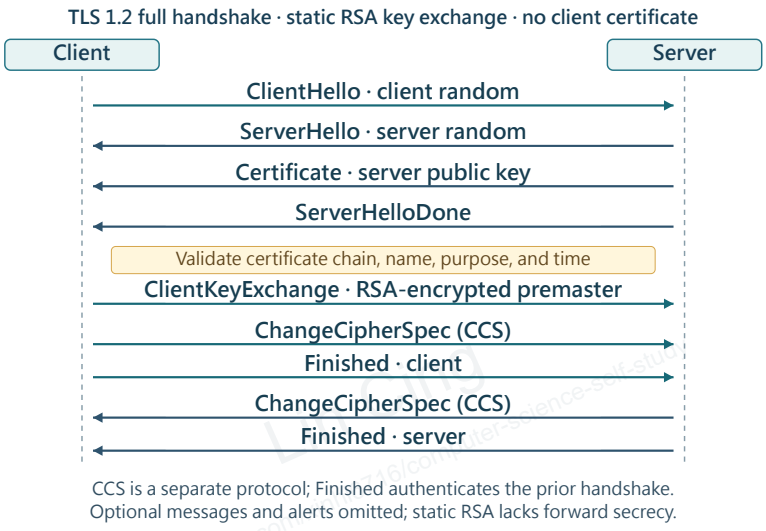
Endpoints determine protection scope

IPsec protects IP traffic between its network endpoints; TLS protects its session endpoints; encrypted mail can span store-and-forward hops. TLS termination at a reverse proxy exposes plaintext there and does not automatically protect proxy-to-backend traffic.

Link/network protection does not directly identify an application user. Disk encryption protects offline media, not a logged-in process authorized to read plaintext. For each layer state key holders, plaintext observers, and trusted components. Layer counts do not sum into unconditional safety.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Historical TLS 1.2 RSA handshake



Use the historical RFC 5246 full handshake with static RSA key exchange, not TLS 1.3. The client sends ClientHello(n_c); the server sends separate **ServerHello(n_s)**, **Certificate**, and **ServerHelloDone** messages. After validating chain, name, purpose, and time, the client encrypts a 48-byte premaster secret, consisting of two version bytes and 46 random bytes, under the server public key and sends it in **ClientKeyExchange**. Both derive the master secret and directional traffic keys with the PRF.

Historical TLS 1.2 RSA handshake (continued)

The client sends ChangeCipherSpec and Finished; after verification, the server sends its own ChangeCipherSpec and Finished, which the client verifies. If the figure omits ChangeCipherSpec, it remains required by the protocol; optional messages such as client certificates depend on the selected case. Finished authenticates the preceding handshake transcript. The certificate signature alone does not prove the current peer possesses the private key. Authentication also requires correct name binding and endpoint assumptions; a live relay remains possible.

Static RSA key exchange lacks forward secrecy: later server private-key compromise and a recorded ClientKeyExchange can expose the old premaster even after traffic keys are deleted. SSL 3 and TLS are not synonyms; RFC 7568 prohibited SSL 3 in 2015. This is a historical teaching model.

Freshness and atomic nonce consumption

Server issues nonce 42 with pending[42]=(U, expiry). Client returns MAC_k("login" ∥ U ∥ n ∥ serverID). Receiver validates tag and pending/expiry, and atomically changes n to used. First succeeds; an identical valid replay fails because n is used.

MAC integrity cannot reject a copied tag without freshness state. Consume only after successful verification and make checking/consumption atomic; otherwise two threads can both observe pending. Random nonces need entropy/nonreuse; counters need restart persistence or a new session. Payments additionally link consumption and business effects transactionally or through recoverable deduplication, preventing crash-induced loss/repetition.

Credentials and sessions

Knowledge, possession, and inherence are distinct factor types. Two passwords remain one type; MFA uses at least two types, not necessarily all three. A stolen authenticated session token can bypass the original factors, so session protection, invalidation, and reauthentication matter.

An abstract verifier record is {user, salt, cost, KDF(password, salt, cost)}. Login recomputes and compares. Public salt prevents sharing one precomputed dictionary across records, not guessing one record or mathematically eliminating collisions. Stolen verifiers still permit offline guesses and need access protection. Historical eight-character UNIX limits belong to particular old schemes, not every UNIX or a modern policy.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Guess spaces and computation

Uniform random four-digit decimal PINs, including 0000, have 10000 possibilities. Distinct unbiased guesses have mean rank 5000.5. At 1 ms/guess, mean time is 5.0005 s. This is not every four-character password or a human-biased dictionary.

For 1000 distinct salts and 100000 candidates, testing every record costs about $1000 \cdot 100000$ KDFs rather than 100000 reusable hashes. At 5 ms each, one serial work unit takes $500000\ \text{s} \approx 5.79$ days. Parallel resources, memory constraints, and prior distributions change actual cost. Online limits do not constrain stolen-verifier offline work; lockouts can become DoS.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

OTP and biometric errors

OTP/challenge-response adds freshness. One-time use is a receiver acceptance rule, not merely changing displayed digits. TOTP windows need explicit replay/used-code handling and remain vulnerable to real-time phishing relay.

Biometric thresholds trade false acceptance and rejection. FAR uses impostor attempts; FRR uses genuine attempts. With 100000 impostor attempts, FAR 0.1% means 100 false accepts; with 10000 genuine attempts, FRR 2% means 200 false rejects. Threshold, population, device, and environment change estimates.

Biometrics are not freely replaceable secrets. Templates, liveness/attack detection, lost devices, and fallback policy affect security. Factors sharing a compromised endpoint/recovery path can fail together; multiplying failure probabilities requires independence.

Policy, assessment, and layers

Policy specifies assets, operations, responsibilities, and response. Assessment checks configuration, code, and exposure; authorized penetration tests examine exploit paths. A passed scan supports its time/scope/tools/known checks, not universal absence of vulnerabilities.

Layers sharing a stolen key or bad identity header have common failure. Patching, least privilege, memory/input safety, segmentation, authentication, monitoring, and recovery affect different probabilities/damage. Name the path each blocks.

An original model has loss 100000 currency units and annual probability 0.02, expected loss 2000/year. Reducing probability to 0.005 leaves 500, saving expected 1500/year. Tail disasters, correlated events, and regulatory obligations are absent and can still determine policy.

IDS/IPS and base rates

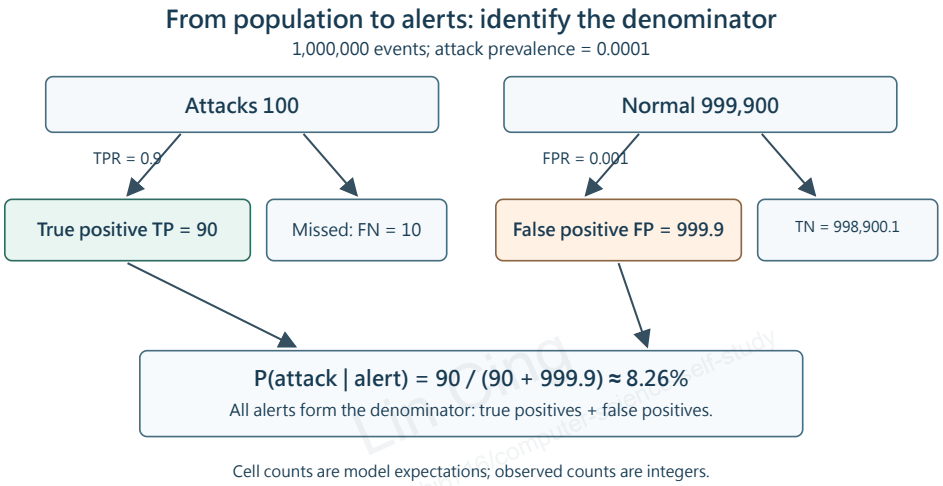
Signatures detect encoded patterns; anomaly detectors find baseline deviations. Unknown attacks need not all evade signatures; anomalous events need not be malicious. IDS usually alerts; IPS can block, directly risking availability on errors.

Of 1000000 events, 100 are attacks. TPR=0.9 gives TP=90, FN=10. Normal=999900 with FPR=0.001 gives expected FP=999.9, TN=998900.1. Alarm precision=90/(90+999.9)≈8.26%, not 90%. TPR divides by attacks; precision divides by alarms.

$$\text{precision} = \frac{\pi TPR}{\pi TPR + (1-\pi)FPR}.$$

At π=0.0001, TPR=0.9, precision≥0.5 requires FPR≤0.000090009…. Baseline contamination, drift, and correlated-event aggregation affect estimation. Development data cannot be relabeled independent acceptance data.

IDS alerts: build the denominator first



The model yields 90 expected true positives and 999.9 expected false positives. The population conditional probability of attack given an alert is about 8.26%. Fractional cell counts are expectations, not observed event counts.

Integrity monitoring and recovery

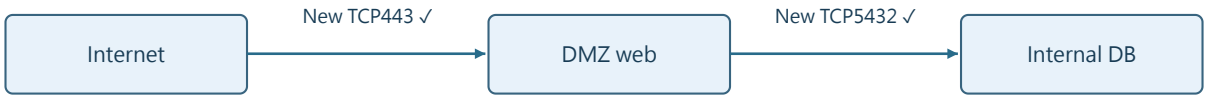
Compare content/metadata with a trusted baseline. Compromising both checker and baseline removes that trust root. Hash comparisons require suitable collision resistance and protected original digests; equality is not absolute identity proof.

Tripwire-style monitoring distinguishes authorized updates from unexpected changes. Changing logs still need protected remote/append-only records and correlation. A honeypot must not become an unrestricted internal attack relay. Antivirus signatures, behavior, and sandboxes have limited coverage; a clean scan does not prove supply-chain innocence.

Response includes isolation, evidence preservation, root-cause repair, trusted-backup recovery, and credential rotation. Audit subject, object, action, decision, time, and correlation ID while protecting log integrity/availability.

Allowed connections before filter rules

Zones are not full trust: allow only specified directions/services



Deny new Internet → DB and web → DB22. Validate return packets against existing state.

Compromised web still reaches DB5432. DB needs authentication and minimum query authority.

Allow new Internet→DMZ web TCP443, web→internal DB TCP5432, and internal→Internet TCP443. Deny other new connections. Permit returns only if they match existing state. DMZ does not gain all internal access. A compromised web server can abuse its required DB5432 path, requiring database authentication/query authorization.

Stateless filters inspect headers; stateful filters track connections; proxies terminate/understand applications; host firewalls govern local ingress/egress. Without TLS termination, packet filters cannot inspect encrypted application meaning. Allowing 443 does not make its payload safe.

Rule order and limits

Rules2–4 implement the three new-connection paths: Internet→web, web→DB, and internal→Internet. Rule1 admits subsequent packets in either direction of an existing allowed connection. New TCP service rules match destination ports.

Order	Match	Action
1	Packet matching an existing allowed connection state, either direction	Allow
2	New TCP, Internet→DMZ web, destination443	Allow
3	New TCP, DMZ web→internal DB, destination5432	Allow
4	New TCP, internal→Internet, destination443	Allow
5	All remaining packets	Deny

Rule order and limits (continued)

Packet	First matching rule	Result
New Internet→web TCP443	2	Allow
New web→DB TCP5432	3	Allow
New internal→Internet TCP443	4	Allow
New Internet→DB TCP5432	5	Deny
New web→DB TCP22	5	Deny
DB→web return actually matching existing state	1	Allow
web→DB subsequent data in an existing allowed connection	1	Allow

Match source/destination addresses, zones, destination port, direction, and state. An ACK bit or a claimed established status without a matching state-table entry does not satisfy rule1.

Putting deny-all first makes later permits unreachable. Actual products have specific chain/priority semantics. Firewalls do not repair allowed-protocol bugs, authenticate hosts merely by IP, or restore saturated upstream bandwidth through local dropping. Tunnels, egress, and dependencies such as DNS need explicit policies.

Historical TCSEC assurance

The DoD Orange Book is not a universal modern product score. D fails higher classes. C1/C2 emphasize discretionary access/accountability; C2 adds individual control, auditing, and object reuse. B1–B3 add mandatory labels and stronger structure/analysis. A 1 adds formal design/verification and development assurance to B3 functionality.

B2 covert-channel requirements identify/analyze paths bypassing intended information flow, rather than support exfiltration. The TCB includes hardware, software, and firmware enforcing policy. If an outside component can break that policy, the boundary is wrong.

Evaluation applies to its version, configuration, policy, and environment. A 1 does not eliminate business errors/all attacks or automatically certify later versions. Formal proofs depend on their models, assumptions, and specifications.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Tokens and security descriptors

The historical primary token holds user/group SIDs and privileges. An impersonating server thread can use client context. Object descriptors contain owner, DACL, SACL; DACL grants/denies, while SACL can hold auditing/mandatory labels. Existing handles have granted access; not every operation reruns an identical open check.

For ordinary specific rights only, excluding owner/privilege exceptions, restricted-token checks, and object ACEs, set remaining=requested. In ACE order, matching deny intersecting remaining rejects. Matching allow removes granted bits. Empty remaining succeeds; reaching the end with bits left denies. This is not a universal union-all-allows-minus-all-denies algorithm.

Inheritance, copy/move, and propagation to existing objects depend on filesystem/API options. Inspect the final descriptor at creation. Parent ACL changes do not universally rewrite existing handles.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DACL mask trace

Request={r, w}; token U includes enabled Students. DACL=[deny(Students, w), allow(U, rw)]. Initially remaining={r, w}; the first intersects w and immediately denies. Without Students, skip it and allow U removes both bits, succeeding.

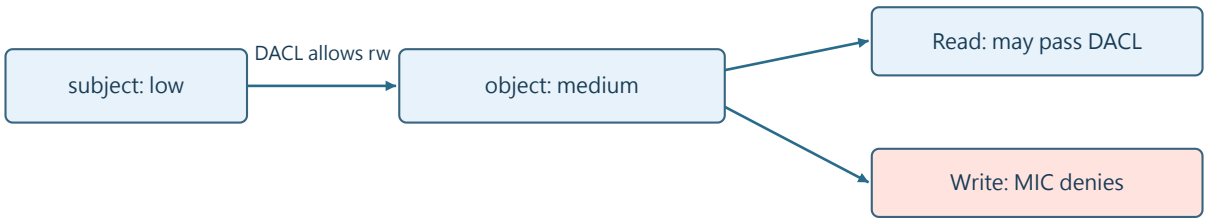
Deliberately placing allow(U, rw) first satisfies the request before reaching later deny. Deny does not override every allow regardless of order. Preferred canonical order places relevant explicit denies before allows; inheritance needs more ordering rules.

NULL DACL is absent and allows in this ordinary model. Empty DACL exists with no ACEs and grants nothing. Neither bypasses mandatory policy or special privilege rules.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MIC and UAC

One DACL grant; mandatory policy still distinguishes read/write



Assume NO_WRITE_UP only. Adding NO_READ_UP also denies the read branch.

Low-integrity subject, medium object, DACL allows r, w. With only NO_WRITE_UP, write fails while read can pass DACL/other checks. NO_READ_UP additionally denies read. Integrity primarily resists low-trust modification; it is not a full confidentiality-clearance model.

Windows 7 UAC can run an administrator's ordinary applications under a filtered medium token; controlled elevation obtains the corresponding higher token. A prompt neither creates rights the user never held nor identifies malicious programs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Composing security arguments

Authentication binds credentials/keys to subjects; authorization limits operations; cryptographic schemes protect messages; freshness/receiver state prevent replay; transaction/recovery connects authorization to a recoverable effect. One layer cannot supply another's missing premise.

Valid MAC replay still needs denial. TLS ending at a compromised proxy does not guarantee end-to-end secrecy. MIC can reject DACL-authorized writes. NX does not prevent authorization-data corruption. State premises, failure consequences, and observable checks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Proofs, examples, and configuration evidence

Fermat/CRT proves RSA correctness for all messages; a modular-power table checks one. Keystream leakage follows an XOR identity. Bayes connects IDS denominators; TPR alone is not precision. Historical cases retain versions, endpoints, and omissions. One replay test, DACL trace, or scan supports that observation without replacing general transition/threat-model reasoning.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz et al., Operating System Concepts, 9th ed., Ch. 15. RFC 5246. RFC 7568. Microsoft: DACL. Microsoft: MIC.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Chapter route

How can multiple guests remain isolated while preserving instruction semantics, device interfaces, and resource bounds?

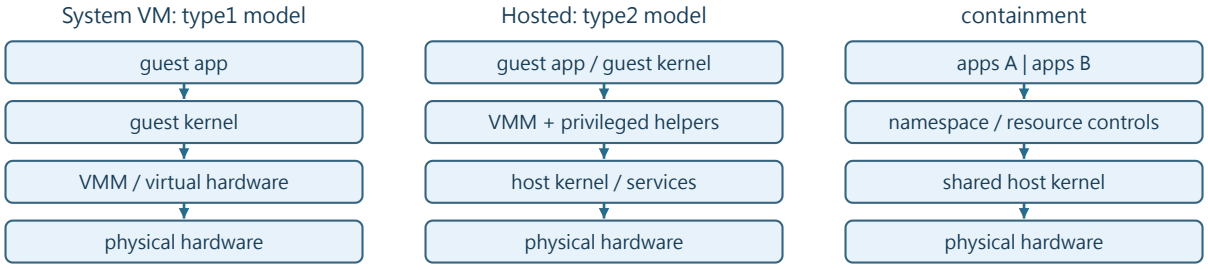
Prerequisites are OS structure, traps, address translation, and I/O.

- Distinguish type 1 and type 2 hypervisors.
- Analyze trap-and-emulate and binary translation.
- Explain memory/I/O virtualization safety and costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Host, guest, and VMM boundaries

Resource-control layers differ: VMs and shared-kernel containers



The VMM/hypervisor manages physical resources and implements virtual interfaces. Guest applications call their guest OS, which controls virtual CPUs/devices. A guest syscall need not immediately enter the host kernel.

A vCPU retains guest registers, PC, state, and virtual interrupts until scheduled on a pCPU. Two VMs can both name physical page5 while second-stage mappings select different host frames. The VMM isolates shared CPUs, memory, devices, and management interfaces.

System VMs, process VMs such as JVM, cross-ISA emulators, and OS containment expose different abstractions. Identify the provided interface, kernel ownership, and physical-resource controller rather than relying on a type number.

Motivation and design goals

IBM VM/370 in the 1970s presented separate physical-disk regions as guest minidisks for multiple OSs on one mainframe. The abstraction preserves ranges/isolation rather than exposing unrestricted disk registers.

Fidelity preserves specified guest-visible behavior except explicit differences. Resource control prevents guests seizing VMM-controlled resources. Efficiency permits most safe work to execute directly. Neither goal promises a fixed slowdown or identical timing for every workload.

Reliable interception of sensitive instructions needing mediation is a common sufficient condition for traditional trap-and-emulate. Otherwise use translation, paravirtualization, or hardware assistance. Full software emulation can interpret an ISA without dedicated VM instructions; hardware changes technique/efficiency, not the logical possibility of emulation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Isolation and shared failure

VMs support OS development, multiversion testing, and low-utilization server consolidation. Hypervisor/device-emulator vulnerabilities or a shared failed host can still affect several VMs. CPU, memory, I/O quotas are needed against interference; separate addresses alone do not prevent slow neighbors.

An eight-core host reserves one for VMM. A/B/C peak at 3/3/2 cores, totaling8>available7. Average demand of one each cannot guarantee no queueing at simultaneous peaks. Overcommit depends on workloads/service goals and creates no compute capacity.

Shared networks/disks introduce trust boundaries. Management can clone, snapshot, read disk images, making its authority part of the TCB even when guests are isolated.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

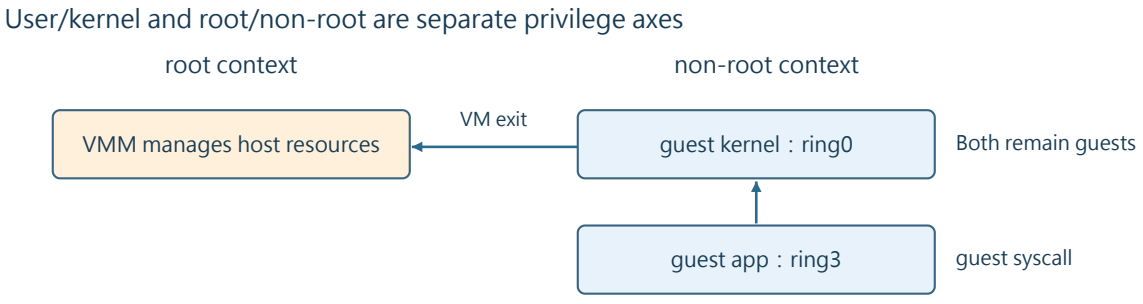
Suspend, snapshots, clones, and backups

Suspend retains resumable CPU/RAM/device state; a disk snapshot retains block versions at a point; a clone creates an independently runnable copy. Copying active disk files need not capture one consistent instant or matching RAM/disk state. Use shutdown, supported consistent snapshots, or guest application quiescence.

Crash consistency resembles recoverable power-loss disk state; application consistency additionally coordinates flush/transactions. External mail, payments, databases outside the snapshot are not undone by rollback. Clones must address duplicate IP/MAC, machine identities, and credentials.

Snapshots sharing the source's failure domain are not independent backups. Templates replicate both configurations and embedded vulnerabilities/credentials. Cloud/VDI APIs retain these isolation/external-state constraints.

Virtual modes versus physical modes



Hardware-assisted model: non-root ring0 ≠ host root. Selected events exit; not every guest syscall.

In a traditional model without a separate VM mode, the VMM holds physical privilege while guest kernel/user execute with reduced physical rights. VMM retains virtual guest mode. A guest-kernel privileged operation traps, is checked against virtual authority, and is emulated. An illegal guest-user operation must produce a guest exception rather than gain authority by entering VMM.

When a guest kernel disables interrupts, VMM clears vCPU.virtualIF and delays that guest's maskable interrupt injection. It must not permanently disable host interrupts serving other guests/devices. Re-enabling permits pending events according to the virtual architecture while retaining host control.

Sensitive instructions that do not trap

An instruction may silently ignore some changes at reduced physical privilege, leaving a guest kernel's expected flags wrong without notifying VMM. Historical x86 POPF behavior under particular flags/privileges illustrates the gap. Lack of trap does not imply harmlessness.

Binary translation reads guest blocks, replaces mediated operations with safe equivalent VMM behavior, and caches executable translations. It must preserve indirect branches, self-modifying code, page-permission changes, and exception-visible state. Changed source code invalidates stale translations.

Paravirtualization instead changes a cooperative guest to use hypercalls. It can address similar problems with different guest ABI/modification requirements. Hardware assistance does not remove scheduling, device-emulation, and I/O costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hardware-assisted execution

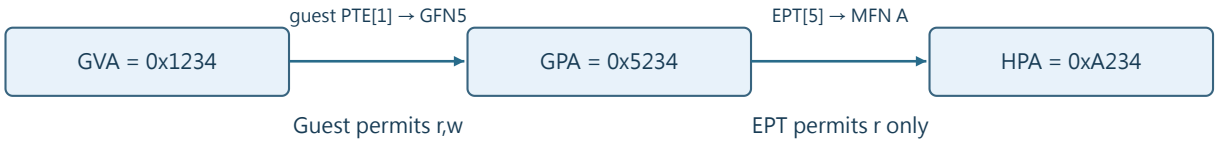
Root/non-root or host/guest context adds an axis beyond user/kernel. Guest kernel can run non-root ring 0, app non-root ring 3, while VMM manages root context. VM entry loads guest state; configured events exit, save state/provide reason, and let VMM handle/resume.

Architecture/controls determine exits; not every instruction or syscall exits. Fidelity/control require correct intercepts, state retention, event injection. IOMMU and interrupt remapping isolate/route device activity separately from CPU virtualization. EPT alone does not constrain all DMA.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Shadow tables and hardware two-stage translation

Compose addresses and permissions at both stages



4096 B pages preserve offset 0x234 at both stages. Read passes; second stage denies write.

Shadow maps VPN1 → MFN A directly, requiring synchronized guest PTE updates and invalidation.

Guest tables map GVA→GPA and VMM maps GPA→HPA. Hardware two-stage EPT/NPT combines both during walking. A shadow table is a VMM-maintained GVA→HPA composition used by an ordinary MMU, requiring interception/tracking of guest table changes. Extra tables do not make them one algorithm.

With4096 B pages, GVA0x1234=(VPN1, offset 0x234); guestPTE[1]=(GFN5, rw) gives GPA0x5234. EPT[5]=(MFN0xA, r) gives HPA0xA234. Read needs permission at both; EPT denies write despite guest w. Guest-table denial follows guest fault semantics; second-stage violation uses VMM handling. They are not one identical fault.

Why uncached nested walks are not twice as expensive

Assume m guest levels, n second-stage levels, resident page tables and data, valid permissions, and no TLB/walk caches or large pages. Each guest PTE read requires n second-stage PTE reads plus its own read, totaling m(n+1). Translating the final data GPA adds n. Thus the number of **table reads** is m(n+1)+n=(m+1)(n+1)−1; add one data read.

For m=n=4 this is 24 table reads plus one data read, not 4+4. It is a cold model, not the cost of every load; TLB hits, walk caches and huge pages can reduce actual reads. Shadow tables have a shorter hardware walk but require VMM maintenance after guest updates.

The count excludes writes: assume accessed/dirty bits need no updates. Real hardware may additionally write those bits. Mapping and permission changes require architecture-specific translation-cache invalidation; correct tagging or flushing prevents reuse of another VM's mapping.

Trace a two-stage page walk

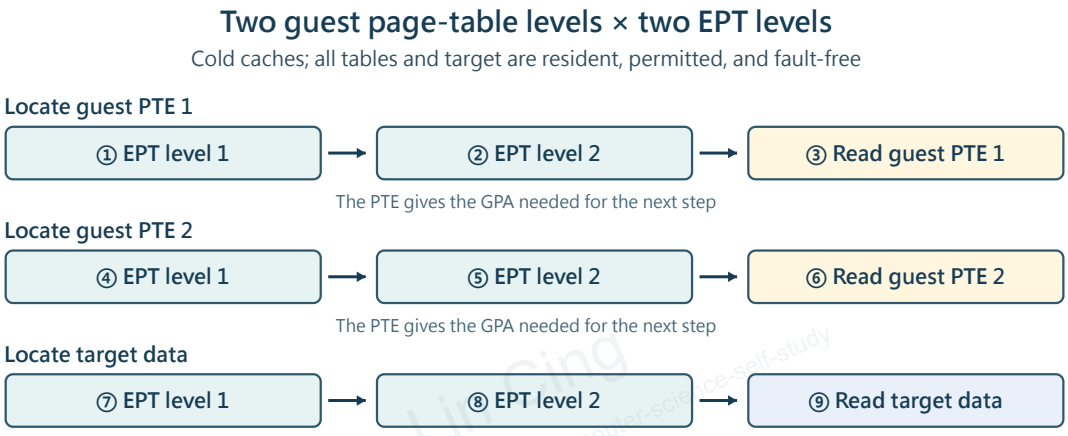


Table reads: 2 + 1 + 2 + 1 + 2 = 8; data reads: 1. A/D-bit updates and other writes are excluded.

With two guest levels, two EPT levels, cold caches, resident pages and valid permissions, each guest PTE GPA needs two EPT reads before the PTE read. The final data GPA needs two more EPT reads before the data read. The total is eight table reads and one data read; accessed/dirty-bit updates and other writes are excluded.

Lifecycle and type responsibilities

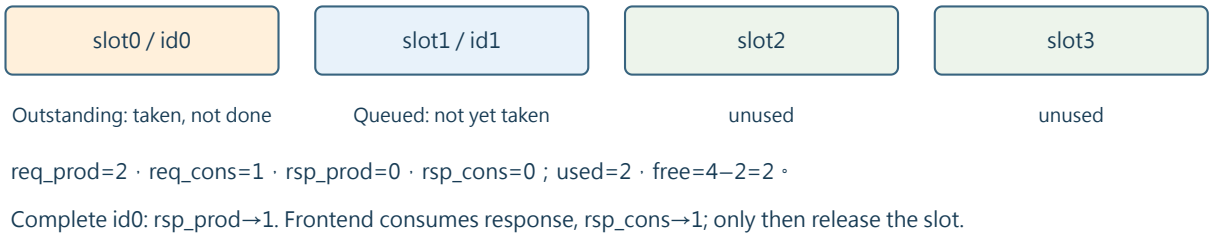
Creation validates CPU, memory, disk, network needs and policy, allocates vCPUs, address spaces, devices, then loads the guest. Running VMs may pause, resume, snapshot, migrate. Deletion first stops devices/events before resource reclamation; deleting management metadata while DMA/network identity remains active is insufficient.

Type0 partitions via firmware/hardware with dedicated/shared resources. Type1 directly manages host resources, including VMMs in some general-purpose kernels. Type2 uses host process/device/memory services, often with privileged helpers/drivers and hardware assistance. Type2 does not mean no host awareness or no VT-x.

Types do not universally rank performance/functionality. Nesting depends on VMM, CPU, configuration, not solely type 0. Compare management scope, TCB, and resource paths.

Paravirtual I/O and a four-slot ring

Four slots, ordered completion: backend consumption does not free a slot



req_prod=2 · req_cons=1 · rsp_prod=0 · rsp_cons=0 ; used=2 · free=4−2=2 。

Complete id0: rsp_prod→1. Frontend consumes response, rsp_cons→1; only then release the slot.

Historical reference is Xen SOSP2003 cooperation, hypercalls, shared rings. This simplified single-producer/consumer model is not every Xen ABI. Counters increase monotonically; slot=index mod4. With req_prod2, req_cons1, rsp_prod0, rsp_cons0, queued=1, outstanding=1, response=0, used=2, free=2.

Backend completing id0 advances rsp_prod1, making outstanding0, response1. Frontend consumes it, advances rsp_cons1, leaving used=req_prod−rsp_cons=1, free3. Backend taking a request does not permit descriptor reuse while completion is pending.

Ring synchronization and validation

Write each descriptor before release-publishing req_prod; the consumer acquire-observes prod before reading it. Write a response before publishing rsp_prod; the frontend consumes it before advancing rsp_cons to release the slot. A doorbell is notification, not a substitute for the counter predicate.

A recheck alone can lose a wakeup: consumer observes empty, producer publishes and notifies before the consumer registers, then consumer sleeps. In a teaching lock/condition model, the producer publishes the predicate and wakes under the same lock. The consumer holds that lock, checks in a while loop, then **atomically registers and releases the lock while sleeping**; it rechecks after waking. Where domains cannot share this lock, an event-count protocol must ensure that publication in the check-to-sleep window remains observable; ordinary notifications alone do not suffice.

With nonwrapping counters and ordered completion, $0 \le req_prod - rsp_cons \le N$ and $rsp_cons \le rsp_prod \le req_cons \le req_prod$ keep the four regions disjoint. Backend validates ID, operation, block range, length and buffer grants; shared memory does not authorize arbitrary guest DMA. Out-of-order completion requires a different ID/slot model.

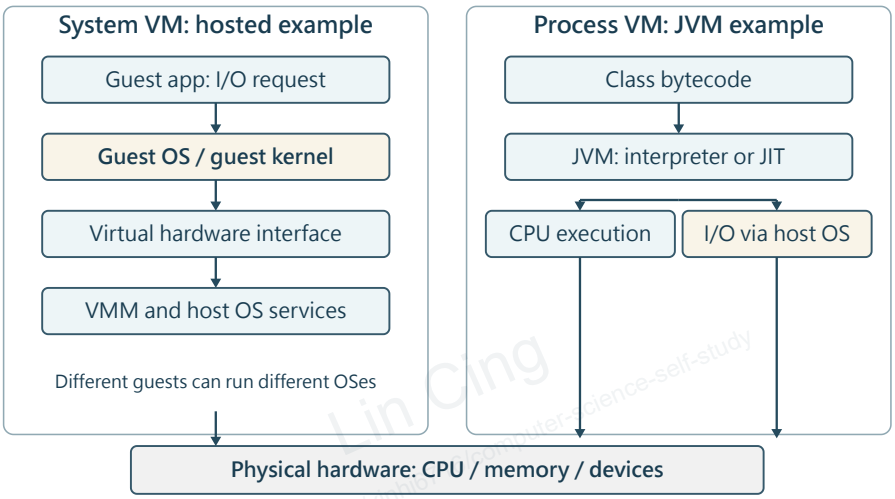
Process VMs, emulation, and containment

JVM/.NET expose bytecode/runtime rather than a full guest OS. Cross-ISA emulation interprets/translates instructions/machine state to run incompatible binaries. Neither is classified solely by owning hardware ring 0.

Solaris10 zones/BSD jails share the host kernel while separating namespaces, resources, and security contexts. Different userland/network views do not create independent kernels. Shared-kernel failure differs from VM isolation. Solaris global zone manages non-global zones; services/resource controls need configuration rather than an independent kernel scheduler per zone.

Testing another kernel needs a system VM/emulator; separating same-kernel services can use containment; bytecode needs its runtime. Choose an appropriate interface rather than a universal winner.

System VM and JVM: interfaces and kernels



Hosted example: isolation needs management and resource controls.

A system VM exposes virtual hardware to a guest OS. A JVM runs bytecode and still uses the host kernel. The host OS path shown handles services such as I/O; not every instruction is a system call. Isolation also depends on the VMM and management plane, and shared CPU, memory, and I/O can cause interference.

Two scheduling layers

Two pCPUs serve A 0/A 1 and B0/B1, ignoring host overhead. A runs0–10 ms, B10–20, A 20–30, B30–40. A thread needing20 ms CPU on A 0 completes at 30; B's counterpart at 40. Equal long-run halves do not make every20 ms budget have40 ms response.

Guest-ready threads wait while their vCPU lacks pCPU time. Descheduling a guest lock holder can waste another vCPU's spinning allocation. Separate virtual clock, wall clock, vCPU runtime. Expired virtual timers may await vCPU scheduling. Guest priorities alone cannot provide host-level real-time admission/isolation/scheduling guarantees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Balloon direction and capacity conservation

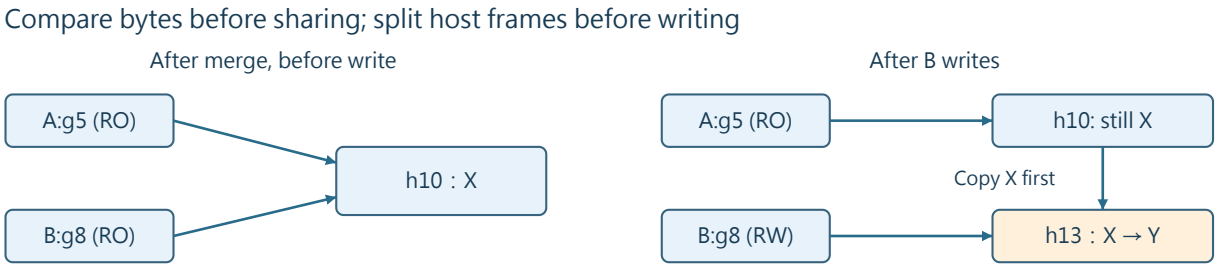
In Waldspurger's ESX OSDI 2002 design, a guest balloon driver reserves guest pages and reports PFNs; the VMM can reclaim their host backing while guest policy selects other pages to reclaim or swap. Inflation reduces workload-usable guest memory; it creates no RAM.

A 4096 MiB host reserves 512, leaving 3584 for guests. A and B each demand 2304 MiB of resident backing: 4608 total, not yet fully backed, exceeds availability by 1024. Balloon A by 256 and B by 768 to target A=2048 and B=1536, totaling 3584; adding VMM 512 gives 4096. This conserves demand and allocation targets, not an impossible prior allocation of 4608 MiB of guest RAM. Nominal guest sizes can stay fixed while usable backing changes at a cost.

Balloon pages are reserved by the driver and should not be used by the normal guest workload. For this teaching isolation model, assume an erroneous access to an unbacked mapping faults, so the VMM rejects it or installs safe backing; this is not a universal promise about every balloon protocol. Deflation must restore safe use; slow or uncooperative drivers may require other paging or limits.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sharing, COW, and double paging



B write fault: allocate/copy → change B mapping and invalidate → allow write. A stays unchanged.

A:g5→h10 and B:g8→h12 contain X. A hash selects candidates. Lock the state, write-protect both pages, invalidate writable TLB/EPT entries, and exclude DMA writes. Under this protection compare all bytes and map equal pages to read-only h10. Reclaim h12 only after references, pins, and in-flight accesses are gone.

B's write causes a COW fault: copy X to private h13, change B:g8→h13, invalidate old translations as required, then permit the write of Y; A still sees X. Guest and VMM swapping can cause double paging. Ballooning may improve victim selection; scanning and synchronization have costs, and sharing must respect isolation policy.

I/O paths and virtual networking

Emulation validates virtual-register operations before host-driver service. Paravirtual devices use cooperative queues. Passthrough lets a device serve a guest directly, still requiring IOMMU isolation, interrupt routing, and safe reset/ownership transfer.

If DMA0x5000 maps to host 0xA000, IOMMU must confine the permitted range. CPU EPT alone is not DMA confinement. Passthrough may require extra device-state support for snapshots/migration; RAM copying alone is insufficient.

Bridging joins an L2 network. NAT maps internal addresses/ports to external tuples. A 10.0.2.15:5000 and B10.0.2.16:5000 can use host:40001/40002 and share one public IP. Firewalls still define guest-to-guest/external channels.

Storage mappings and snapshot chains

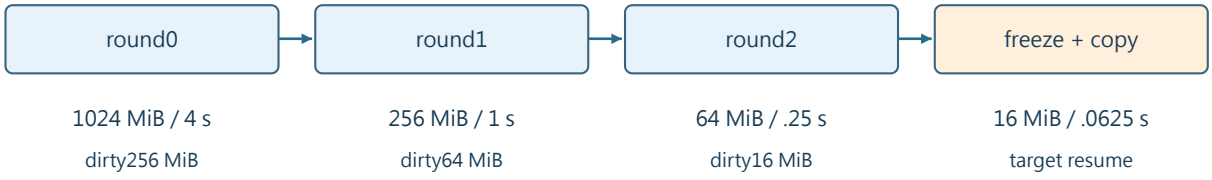
Guest LBA maps through an image/host filesystem to storage, or directly to a block volume; a virtual disk need not be one ordinary file. Thin provisioning postpones allocation without guaranteeing all logical capacities can be filled together.

Base={0:A,1:B,2:C}; after snapshot, active delta={1:B′}. Read1 finds B′ in delta, read 2 falls through to C. Selecting base restores B at 1 but does not retract sent mail. A shared base cannot be deleted while clones reference it. Backups need the required chain, metadata, and consistent point.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pre-copy migration round by round

Normal pre-copy: total time differs from downtime



B=256 MiB/s, d=64 MiB/s. Total 1360 MiB / 5.3125 s; pause .0625 s, excluding control cost.

Assume compatible VMM/CPU, shared storage, no unmigratable device, effective copy B=256 MiB/s, distinct dirtied pages d=64 MiB/s, and ignore protocol/CPU-state cost. R0=1024 copies in 4 s, creating R1=256; next1 s creates R2=64; next0.25 s creates R3=16. Freeze and copy16 in 0.0625 s. Total1360 MiB over5.3125 s; pause0.0625 s.

Generally R(i+1)=(d/B)R(i), geometrically shrinking if d/B<1. At d≥B it does not shrink; cap rounds, reduce dirtying, extend pause, or change method. Repeated dirty pages, variable bandwidth, and finite RAM make reality richer than this explicit model.

Stop-and-copy and one active owner

Freeze includes state-changing DMA/devices/events, not only user code. Save vCPU/device/dirty RAM and required network state. Source must stop externally executing the guest before ownership transfer; reclaim source after target operation is established.

If target starts but acknowledgment is lost, source restarting can create split brain. A safe protocol needs explicit commit/ownership and failure recovery. Waiting for acknowledgment alone is not a proof of all failure cases. This simplified normal path proves a consistent handoff, not transparent availability under arbitrary partitions.

Maintain external address/MAC/routing reachability. TCP tolerating a short pause does not make downtime zero. Shared storage is this model's premise; block migration exists elsewhere, so it does not prove all live migration excludes disk copying.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Workstation, Xen, and ESX

The book's circa2013 hosted Workstation diagram routes guest app→guest kernel→virtual device/CPU→VMM/host services→hardware. Ordinary instructions can execute directly; I/O can use host services. It is an architecture example, not a release's complete implementation or measurement.

Xen SOSP2003 uses cooperative guests/hypercalls/shared rings. ESX OSDI2002 manages overcommit with ballooning/content sharing. Their periods/interfaces differ. Early Xen PV is not every later Xen mode; one paper benchmark does not establish a universal performance ratio.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

JVM verification and execution

In a Java SE 7-era abstraction, class loading/verification checks types, operand stacks, and legal control flow. Execution interprets or JIT-compiles; runtime supplies heap/GC/security. JVM has no separate general-purpose guest kernel and still uses host memory, threads, I/O.

For iconst_2;iconst_3;iadd;istore_1, stack is[]→[2]→[2,3]→[5]→[], local1=5. Starting empty with iadd lacks its two integers and can be rejected by verification before a hardware underflow. This does not prove business correctness.

JIT can use hotness/tiering rather than compile a whole method on first call. GC follows reachability/collector rules rather than deciding semantic future usefulness. Separate language specifications from implementation choices.

Two required properties

Fidelity preserves specified guest-visible transitions; resource control prevents escaping CPU, address, device management. Traps, translation, hypercalls, hardware assistance implement these properties rather than prove themselves.

Permission intersection, ring publication/reuse, COW-before-write, and single-owner migration keep guest-visible and host-managed state consistent. A successful demonstration does not cover every transition.

VCPU count is not pCPU capacity; nominal RAM is not current host backing; logical disk size is not allocated bytes; snapshots do not roll back the outside world; migration transfer time is not downtime. Identify the layer of every number. Exercises reconstruct modes, addresses, walk counts, ring counters, balloon capacity, COW, and migration recurrences.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz et al., Operating System Concepts, 9th ed., Ch. 16. Xen SOSP 2003. ESX memory management OSDI 2002. JVM SE7 verifier.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Chapter route

How can services remain understandable and recoverable without shared memory, synchronized clocks, or failure-free nodes?

Prerequisites are network stacks, RPC, processes, and filesystem interfaces.

- Compare network and distributed operating systems.
- Analyze naming, routing, RPC, and failure semantics.
- Check DFS cache consistency and availability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

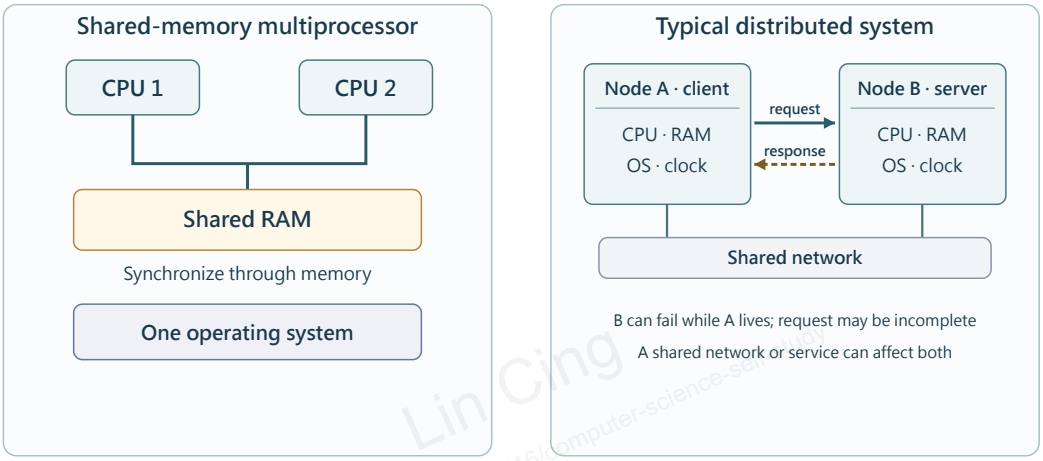
Message passing and its limits

In this chapter's typical message-passing model, nodes have local memory and OS instances and communicate by messages; no shared physical memory or perfect global clock is assumed. Distributed shared memory can provide an abstraction above this model, and synchronized clocks still have error. Local and remote are relative to the requester. Sharing, speedup, reliability and communication are goals, not automatic consequences of distribution.

For a fixed 100 s job, 20 s is serial, 80 s divides evenly across four nodes, and coordination costs 5 s. $T_4=20+80/4+5=45$ s, so speedup is $100/45\approx2.222$. Ignoring coordination, the infinite-node Amdahl bound is $100/20=5$. More nodes cannot remove serial work and can increase coordination costs.

At **one fixed read instant**, three interchangeable replicas have independent unavailability events, each with probability $p=.01$. If networking and metadata are available and any available replica can return valid data, the probability of an unavailable read is $p^3=10^{-6}$. Shared power, rack or a required metadata leader can invalidate independence. Each replica having failed at some point during an interval does not imply that all were unavailable simultaneously.

Shared memory and message passing



Models only: performance and reliability depend on load and failure domains.

Processors coordinate through shared memory in the first model. In the second, nodes exchange requests and responses across a network. A client may not know whether its request executed after a server failure or network interruption. Shared services and networks can fail together; distributed shared memory is another possible abstraction.

Transparency and machine boundaries

Network OSs expose remote hosts/copies explicitly. Distributed OSs/middleware can unify naming, access, and migration. Transparency packages location/invocation and failure semantics; it does not remove physical latency/failure.

Remote login executes a remote session; transfer creates another file copy; DFS provides shared remote-file behavior. SSH/SFTP still need host authentication/user authorization; ordinary FTP is not inherently encrypted. Anonymous services need constrained namespaces/operations. Naming, identity mapping, network faults, and conflicting copies require rules.

Moving data or computation

For 1 GiB input, 1 MiB result, 100 MiB/s throughput, and 20 ms RTT, moving all input costs at least 1024/100+.020=10.260 s before local compute. Sending computation to a remote CPU taking .500 s and returning 1 MiB takes .500+.010+.020=.530 s, with negligible request. Include CPU queues, startup, serialization, and output size before generalizing.

Whole-file caching pays upfront for later local access. On-demand caching transfers requested blocks. Reading only 1 MiB makes a whole 1 GiB transfer waste 1023 MiB; reading all blocks through unpipelined RTT-bound RPCs can instead favor bulk transfer.

Process migration moves/reconstructs PC, registers, memory, open files/sockets, and environment. Sending code for fresh remote execution is computation placement, not migration of running state. Cross-ISA/ABI execution needs compatibility rather than copying register bits blindly.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Scope and latency components

LANs commonly cover smaller administrative/geographic areas; WANs connect sites. Capacity/latency depend on links, distance, queues, load. Switches, routers, and APs have different roles. Switched Ethernet is not universally one shared collision bus; a congested LAN can underperform a high-capacity WAN.

Latency≈L/R+propagation+queue+processing. A 1500 B packet on 1 Gbit/s serializes in 12 µs. One-way 5 ms propagation dominates. A tenfold rate increase reduces serialization to 1.2 µs without dividing propagation by ten.

100 Mbit/s equals 12.5 decimal MB/s before headers; MiB/s uses 2^{20} B. Wireless nominal rates differ from goodput. These are explicit models rather than current product maxima.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bandwidth-delay product

100 Mbit/s and 40 ms RTT give BDP=4×10^6 bits=500000 B≈488.28 KiB. A 64 KiB outstanding-data limit bounds throughput at 65536 • 8/.040=13.1072 Mbit/s despite 100 Mbit/s capacity.

This assumes no loss, fixed RTT, and no other bottleneck. TCP congestion/receive windows, ACK behavior, slow start, retransmission further constrain results. Send buffers remove at most one bound, not congestion/server CPU. A 16 B RPC can be dominated by RTT rather than serialization, explaining the importance of placement/cache.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Names, addresses, ports, and objects

`store.lab.test` is a domain name, potentially resolving to IPv4 such as 198.51.100.20 or to IPv6. An IP address is not a user or a permanent machine identity. A service also involves transport protocol, port and application naming. A TCP connection is typically distinguished by source/destination addresses and ports and protocol; a port is not a PID.

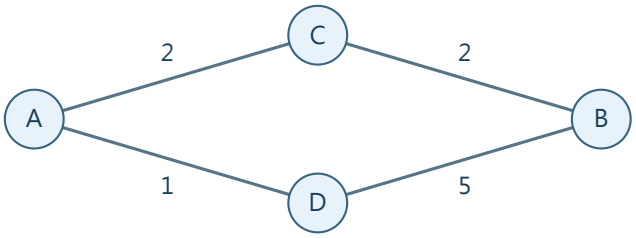
DNS uses resolvers, queries, caches and delegation; a label does not require its own server. Assume root delegates test and test delegates lab.test. On cache miss, the resolver follows known-root referrals to an authoritative A record. This example assumes a resolver that obeys TTL, with no serve-stale behavior or local override; that is not a universal implementation rule.

At t=0 an A record .20 has TTL 600 s; at t=100, 500 s remains. A change at the authority to .30 at t=200 need not replace the cached .20 before t=600. After expiry, a successful new query obtains .30 if the authoritative record has not changed again. Failure to query does not make a stale value a verified new answer. DNS integrity and endpoint authentication need corresponding mechanisms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Routing on a fixed graph

One nonnegative weighted graph: every distance update



From A: D=1, C=2, B=∞

Settle D: B=1+5=6

Settle C: B=min(6,2+2)=4

Settle B: A → C → B, cost4

If C–B fails, recompute A→D→B, cost6. Dynamic protocols need detection/convergence time.

Edges are A–C=2, C–B=2, A–D=1, D–B=5. Dijkstra starts with d(D)=1, d(C)=2, d(B)=∞. Settling D gives d(B)=6; settling C lowers it to 4; settling B yields A→C→B, cost 4. Losing C–B gives A→D→B, cost 6 after recomputation.

Static routing uses configured/admin rules; virtual circuits select session paths; dynamic protocols update metrics/reachability. Dynamic does not mean every packet chooses the emptiest link or converges without loops/drops. A path preserves order only with suitable link/queue behavior; transport supplies end-to-end stream ordering.

Control plane computes routes; data plane forwards from tables. SDN can separate them but still needs consistent updates, failure handling, and protected management.

Switching and connections

Frames, packets, and datagrams have layer-specific headers/limits and can vary in length. Application messages can span packets or share a TCP read. TCP reserves no dedicated physical circuit; applications can implement reliability over UDP.

Circuit switching reserves path resources; message switching stores/forwards whole messages; packet switching pipelines fragments with statistical multiplexing. Across two 1 Mbit/s links, ignoring headers/propagation/queues, a 4000 B message takes 64 ms. Four 1000 B packets each take 8 ms/hop; pipeline completion is (2+4−1) • 8=40 ms.

With 40 B per-packet headers, it becomes 5 • 1040 • 8/10^6=41.6 ms. One 40 B whole-message header gives 64.64 ms. These are store-and-forward assumptions, not all cut-through/circuit networks.

Layer responsibilities

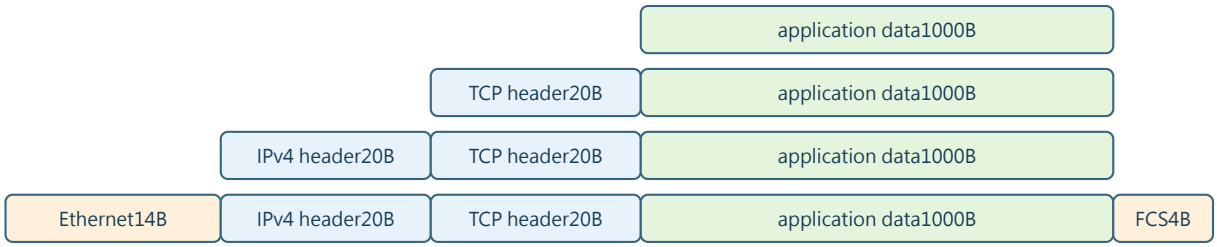
Physical carries signals/bits; link carries frames; network addresses/forwards; transport serves process endpoints; session manages dialogue; presentation encodes representations; application defines service operations. TCP/IP often places session/presentation in libraries/applications. Fewer named layers does not prove efficiency.

TCP offers ordered reliable bytes and flow/congestion control without message boundaries. UDP preserves datagrams without equivalent built-in delivery/order. IP is best effort. NIC offload can move checksums/segmentation into hardware, so no universal three-hardware/four-software split exists.

Serialization specifies lengths, byte order, encoding, versions. Raw C structs with padding/pointers are not portable protocols.

Encapsulation length accounting

Each layer adds headers/trailers; account for every length



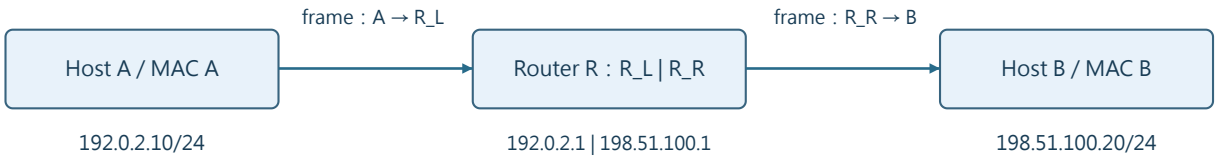
Widths are not to scale. IP total1040 B; MAC frame1058 B. Preamble/SFD and IFG are separate.

1000 B application data plus 20 B TCP gives 1020 B segment; 20 B IPv4 gives 1040 B IP total. Ethernet II 14 B header+1040+4 FCS=1058 B. Add 8 B preamble/SFD and specified 12 byte-times IFG for 1078 wire byte-times.

With IPv4 MTU 1500, this fits one frame. MSS subtracts TCP header from available IP payload; 1500 is not all application data. VLAN/options/tunnels/security change accounting. TLS endpoints are not every forwarding router. A lower checksum detects accidental corruption rather than authenticating malicious changes with a secret key.

Two Ethernet LANs

Across LANs without NAT: each hop uses different MAC addresses



A ARPs next-hop 192.0.2.1, not remote B on the left LAN.

IP stays192.0.2.10 → 198.51.100.20. Router decrements TTL64→63; recompute checksum/FCS.

No NAT/tunnel. A=192.0.2.10/24, MAC A. Router left=192.0.2.1, MAC R_L; right=198.51.100.1, MAC R_R. B=198.51.100.20/24, MAC B. DNS gives B; /24 comparison selects next-hop 192.0.2.1.

Without cached R_L, A broadcasts ARP for 192.0.2.1 on its LAN. R answers. Frame 1 has src=A, dst=R_L, IP src=A, dst=B. A does not ARP remote B directly. Router looks up route, decrements TTL 64→63, updates IPv4 checksum, resolves B on the right LAN, and sends frame 2 src=R_R, dst=B with unchanged A→B IP endpoints.

B decapsulates and demultiplexes by transport port/connection. Routers normally do not forward ARP broadcasts across L2 domains; ARP replies do not cryptographically prove identity.

Header changes and minimum frames

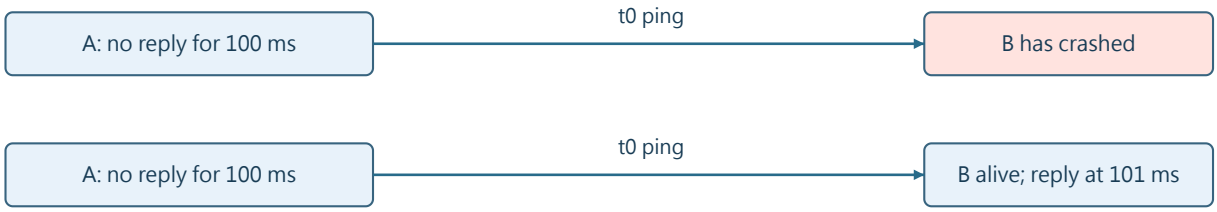
Each hop rebuilds Ethernet addresses/FCS. IPv4 TTL/checksum change; source/destination IP, TCP endpoints, payload remain under the stated exclusions. Saying only Ethernet changes is false. TTL reaching zero prevents normal forwarding; error reports can themselves be lost/filtered.

Untagged Ethernet II has 6 B destination, 6 B source, 2 B EtherType, 46–1500 B data/pad, 4 B FCS, minimum 64 B from destination through FCS. Padding is not included in IP total length; preamble/SFD is outside that frame denominator.

A 4 B TCP payload plus 20+20 headers is 44 B IP, requiring 2 B pad. Frame 14+46+4=64 B. Add 8 B preamble/SFD and 96-bit IFG=12 byte-times to 84. At 100 Mbit/s, it occupies 6.72 µs. IFG is idle time, not 12 bytes appended to IP; high-speed PHY tolerances are outside this model.

Failure-detector indistinguishability

Same local observation, different remote states



A has the same history through 100 ms. Without delay bounds, timeout does not prove crash.

In an asynchronous system without known message or scheduling delay bounds, A pings at t=0 and sees no reply by 100 ms. B may have crashed, a link may be partitioned, a message may be lost, or B may simply be slow. Compare B crashed with B alive but replying at 101 ms: A's local history through 100 ms is identical. An algorithm declaring a crash solely from that observation errs in the second execution.

Failure-detector indistinguishability (continued)

For a bounded teaching model, B remains operational; heartbeat **nominal send times** are H=1 s apart, each actual send occurs 0–s after its nominal time, with s=.2 s; every message arrives with one-way delay in 0–δ, δ=.1 s. Receiver processing adds no delay and its elapsed-time clock has no drift. The previous heartbeat can arrive immediately at its nominal time and the next no later than H+s+δ=1.3 s after it. A 1.5 s timeout measured from the last arrival does not falsely suspect this live node under those assumptions. Loss, partition, an exceeded delay bound or clock error invalidates this guarantee.

New leaders need old-leader fencing

A writes shared storage. After a partition B times out and declares itself leader while A still writes, creating split brain. Heartbeats and local declarations alone do not guarantee exclusion.

In the teaching fencing model, a correct coordinator issues monotonic epochs and storage installs currentEpoch=43 before B operates. **Each write's token check and commit must be atomic with respect to epoch installation, or equivalently serialized in the same linearization order.** A write with epoch 42 that linearized before 43 was installed belongs to the old term; storage must reject one ordered after installation. Checking once before commit is insufficient: an old write can pass under 42, be delayed, and otherwise reach storage after 43 is installed.

Every write path must enforce the token and issuance and installation must be correct; partitions cannot independently increment numbers and call that legitimate ownership. Confirmed hardware or access isolation can also fence old nodes. Before A rejoins as a follower, it synchronizes committed state, membership/epoch and unfinished requests. IDs or generations prevent stale callbacks and replies from matching new requests after restart.

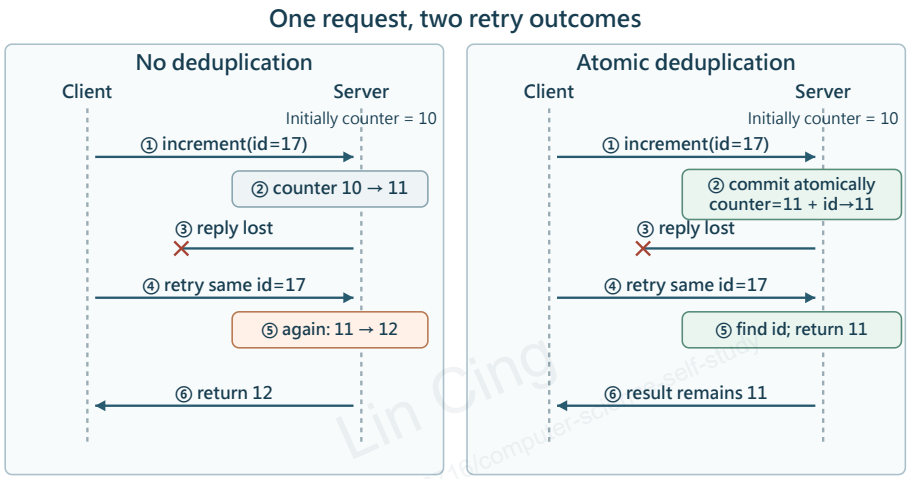
Lost RPC replies and deduplication

Counter 10 receives increment(id17), commits 11, and loses reply. Naive retry yields 12. TCP retransmission supplies connection-byte delivery, not cross-reconnection application deduplication.

In the explicit at-most-once model, a server transaction checks durable table[id]. Existing IDs return saved results; new IDs atomically persist counter++ and table[id]=result before replying. Retrying id 17 returns 11. Updating counter before recording ID permits duplicate increment after crash; recording ID first permits lost effect.

The proof requires shared transaction/recovery scope, unique IDs, and retention long enough for retries. Permanent disconnection can still leave the client uncertain. External email/payment outside the transaction needs another protocol.

RPC retries: lost replies and deduplication



ID must be unique and retained throughout retries; external effects need a separate protocol.

Persist the counter update and `id→result` in one transaction. Retrying with the same unique ID returns the recorded result while its record remains available. External effects such as email or payment require a separate protocol.

Fault domains and failover capacity

Two active servers each handle 100/s with total load 160/s. Losing one leaves 100/s and at least 60/s unsustainable. Replicas being able to take over does not preserve SLO without spare capacity, shedding, or degradation.

Stateless worker retries can still duplicate effects. Stateful services additionally need logs, recovery, replication, locks. RAID addresses selected disk faults, not cross-site availability or backups of logical corruption. Recover durable committed state, reconcile in-flight operations, then serve. Missing commit reply does not imply uncommitted work.

Observable transparency and scaling

Location transparency hides placement;migration transparency preserves naming/operations after movement;replication transparency hides copies. Failure transparency cannot indefinitely hide a nonreturning operation. User mobility also needs identity, environment, and session consistency.

Ten workers at 100/s offer 1000/s, but a one-core coordinator spending 2 ms/request caps the system at 500/s. An eleventh worker does not lift it. Caching, batching, or partitioning can alter the bottleneck's per-request demand. Replication/rebuild traffic consumes capacity too.

Hadoop 1.2.1 HDFS NameNode manages namespace/block locations; DataNodes store/transfer blocks; client data bypasses NameNode. Large blocks, streaming, and a simplified write model favor bulk data rather than full POSIX sharing or arbitrary low-latency transactions.

HDFS payload and flow

Custom 64 MiB blocks and replication 3 split 130 MiB into 64, 64, 2. Logical 130 MiB yields 390 MiB replicated payload, excluding metadata, checksums, padding/allocation. The last 2 MiB need not transfer a full 64 MiB of payload but still needs block location metadata.

Client gets block/DataNode locations from NameNode, then reads DataNodes directly; writes can use a DataNode pipeline. Heartbeats/block reports detect missing replicas and trigger re-replication with delay/network/disk cost.

Secondary NameNode assists checkpointing;it is not an automatic hot standby. Fault-domain-aware placement helps data availability, while metadata and client reachability remain required.

Names, stable identities, and locations

Location transparency hides physical placement;independence preserves names after movement. ServerA:/x exposes server location. Client mounts can hide it differently per client;global namespaces add common naming/management.

Path /course/a.txt→FID(volume7, file 12, generation 3), volume7→serverA. Moving to serverB changes only the second map. Reusing deleted file 12 must change generation or stale FIDs can identify the new file. Path→FID and volume→location caches have different invalidation conditions.

Replicated/cached location maps still need versions/coherence. Indirection does not eliminate updates. Diskless boot requires firmware/initial protocols to load kernel before DFS exists; an unloaded DFS cannot load itself.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete cache-cost accounting

100 reads of 4 KiB at 5 ms remote cost total 500 ms uncached. Ten misses at 5 plus 90 hits at .1 ms cost 59 ms and 40 KiB network payload versus 400 KiB. This specified workload does not prove a universal hit ratio.

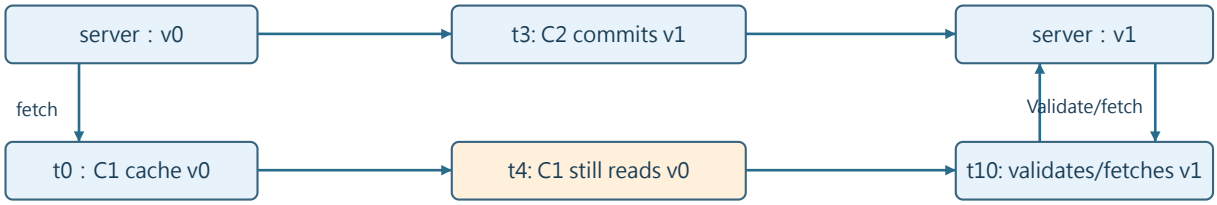
Per-read 1 ms validation adds 100 ms, for 159 ms. Cache hits need not avoid network RTT. Larger cache units can improve read-ahead or waste bandwidth on a one-byte read and reduce resident-block counts at fixed capacity.

Memory cache is volatile;disk cache retains durable bytes but needs metadata recovery/freshness checks. Servers cache too;server RAM acceptance is not stable storage. Write-through/back specify propagation timing, while the next layer's return defines durability separately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Cache consistency timelines

Server versions and client cache visibility have different timelines



Periodic checks permit staleness. Strong callback/lease models recall conflicting read rights before write completion.

Client-initiated validation: C1 reads/validates v0 at 0; C2 completes v1 at 3; C1 validating next at 10 can still return v0 at 4. This is not latest-completed-write visibility on each read. Open-time validation needs within-open semantics; every-access validation costs network and still needs coordination with the actual read.

Server-initiated coherence tracks cache/read/write rights and invalidates/recalls them, or disables cache on conflict. A strong model requires confirmation that conflicting read permissions are revoked/invalidated or their controlled leases expired before write completion. During partition, unlimited old-client reads cannot coexist with an immediate-visibility claim.

Cache consistency timelines (continued)

AFS/OpenAFS callbacks avoid checking every access, but do not rewrite application-owned bytes already read. Save/close and cache-manager refresh affect observations. Session, close-to-open, and stronger DFS guarantees are not synonyms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NFSv3 state and write stability

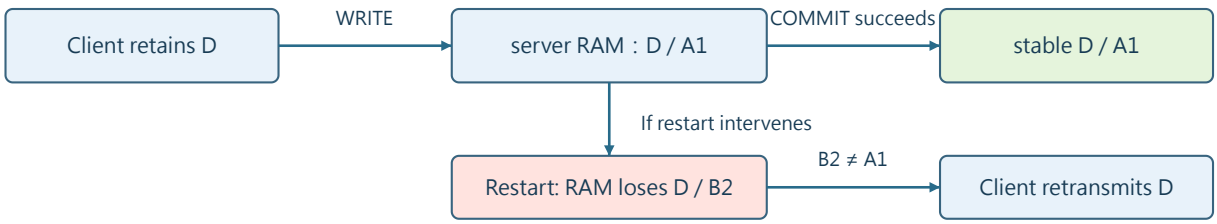
RFC 1813 READ/WRITE carry filehandle, offset, count rather than relying on equivalent server OPEN sessions. Server caches/duplicate-request state and separate lock protocols can still exist. Authentication, permissions, and handle lifetime remain relevant.

WRITE requests/replies distinguish UNSTABLE, DATA_SYNC, FILE_SYNC. UNSTABLE success is not durable data. Clients requiring later COMMIT/verifier checks retain retransmittable bytes. Weak cache-consistency attributes assist change detection without proving global linearizability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Restart after an unstable write

NFSv3: normal and restart paths after one successful WRITE



After UNSTABLE, retain data; a successful COMMIT requires a verifier check.
A retransmitted WRITE can confirm the bytes actually written at the required stability.

Client sends 4 KiB D at offset 0, UNSTABLE. Server receives into RAM and replies UNSTABLE, verifier A1; client retains D. Normal COMMIT succeeds with A1, establishing protocol-stable storage and permitting release of that retransmission obligation.

If restart loses volatile D, new instance uses B2. A COMMIT/WRITE returning B2≠A1 cannot certify the old data. Retransmit unconfirmed D and obtain the required stable result. COMMIT success without verifier comparison is insufficient.

This requires retained client bytes, correct server stability/verifier/range behavior, and does not give arbitrary concurrent writers application transactions. Durability, visibility, and RPC deduplication differ.

One remote operation across layers

Service name→DNS address→route→ARP→link/IP/transport→RPC ID→authorization/durable commit→client cache. Frame delivery does not prove RPC completion;TCP bytes do not prove one business execution;NFS UNSTABLE does not prove persistence.

Transparency offers a consistent interface inside stated boundaries without eliminating latency, partial failures, copies, or external effects. Insufficient observations require unknown/suspected rather than treating timeout as logical proof.

Split serial/parallel/coordination cost. Track bits/bytes, headers, RTT, windows. Draw reliability fault domains. Link RPC IDs to atomic effects;separate DFS names, identities, locations, visibility, durability. General guarantees depend on transitions:unbounded-delay timeout cannot prove crash, fencing needs resource-side epoch enforcement, and deduplication needs sufficient retention. Exercises include counterexamples, normal paths, and restart states.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz et al., Operating System Concepts, 9th ed., Ch. 17. RFC 1034, RFC 826, RFC 1813. Hadoop 1.2.1 HDFS. OpenAFS.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

One remote operation across layers (continued)

https://github.com/xinhi6716/computer-science-self-study

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter route

How does a UNIX-influenced kernel compose processes, resources, and modular interfaces?

Prerequisites are processes/threads, virtual memory, filesystems, and modules. This case fixes Linux v3.2 rather than describing unchecked current behavior.

- Distinguish historical design from present systems.
- Trace fork/exec, module loading, and page-cache paths.
- Analyze historical CFS/predecessor, reclaim, and filesystem boundaries.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kernel, libraries, and distributions

Linux v3.2 provides tasks, VM, VFS, IPC, networking, and drivers. GNU tools, shell, libc, and package managers mostly run in user space. A distribution integrates kernel, tools, configuration, installation, and updates; its name does not specify every CONFIG option.

Two teaching machines share v3.2/x86. An ELF requests PT_INTERP=/lib/ld-linux.so.2. A provides it;B does not. Both find the executable, but B fails obtaining the interpreter. Kernel-version equality is insufficient. Static/dynamic linking, ISA, ABI, and library symbol versions impose different compatibility conditions.

Component	Responsibility	Observable failure
Kernel image/CONFIG	Privileged syscalls/device support	Missing driver prevents device use
ELF interpreter/libc	Usually user-mode loading/API wrappers	Missing interpreter fails exec;missing symbol fails linking
Shell/utilities	User processes using services	Missing command does not remove fork/read
Distribution policy	Packages/default integration	Different permissions/startup behavior

Kernel, libraries, and distributions (continued)

From its 1991 beginnings, Linux expanded to SMP and many architectures. Modules, fine-grained locks, and standard interfaces address hardware diversity, multitasking, and compatibility. Concrete claims still depend on versioned source/configuration.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Protection boundaries inside a modular kernel

Scheduler, memory, VFS, and most drivers share one kernel protection domain. Loading independently does not create isolated user servers. Ordinary function calls cross module boundaries without necessarily switching address spaces.

For one synchronous buffered read, syscall enters kernel, VFS calls filesystem, then block/driver code. User/kernel crossings change mode;internal calls do not. Waiting can schedule another task, adding a distinct context/address-space switch. No universal read switch count follows.

A bad driver pointer can corrupt kernel data. ABI/type/signature checks do not create another isolation domain. A user-space file service can use page protection but requires IPC, data-transfer, and server-failure protocols. Chapter 2 compares fixed paths;this chapter connects Linux objects.

UNIX/POSIX APIs specify observable fd/process/signal semantics, not a unique internal structure. Read can serve disk, pipe, socket with different seeking, blocking, and EOF contracts.

Module loading as a resource transaction

The following is a custom publish-last driver protocol, not a visibility guarantee for all Linux 3.2 registrations. Driver M imports A/B and registers D. Validate format/version, allocate/relocate, resolve exports, initialize, then publish. Finding A without B cannot expose a callable half-built D. If init allocates a buffer then registration fails, unwind successful acquisitions in reverse order.

Stage	Visibility	Resources	Failure action
Before resolution	Not generally callable	Load region	Reject validation failure
Resolve A/B	Not LIVE	Relocated code/data	Release on unresolved import
Init	Service being built	Queues and buffers; registration not yet published	Undo completed acquisitions
LIVE	New calls retain references	Registered service	Enforce per-call lifetime

In Linux 3.2 some registered entry points may be called before init returns. Drivers must initialize externally visible state before registration, then revoke entry points and synchronize concurrent calls on init failure. The table's init row describes only private preparation under this example's publish-last protocol.

Zero references do not close entry points

This driver assigns one M reference per handle/request. Initially three handles plus two requests give 5.

Stage	New references?	Handles	Requests	References	May free code?
LIVE	Yes	3	2	5	No
GOING, entries withdrawn	No	3	2	5	No
Handles closed	No	0	2	2	No
I/O finished/legally canceled	No	0	0	0	Callbacks may remain
All callbacks/workers quiescent	No	0	0	0	Yes

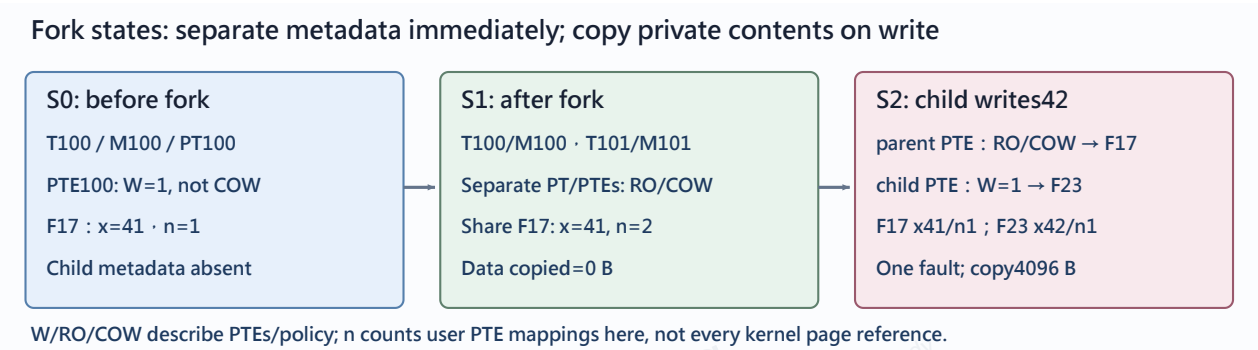
Checking zero before stopping entry lets another CPU retain and execute code about to be freed. State/registration protocol first prevents entry, then drains use. RCU callbacks need their grace periods;workqueues, timers, DMA need their own synchronous cancellation/quiescence. One RCU wait is not sufficient for every module.

Four layers of fork state

In v3.2, copy_process/dup_task_struct allocates child task/stack, then initializes PID, credentials, scheduler, and other execution context. Ordinary fork uses copy_mm/dup_mm for independent mm_struct/page tables. Copy_page_range may map shared private frames write-protected. Some VMA page-table entries can be established lazily by a later fault. A write fault copies only when separation is required; a safely exclusive anonymous page may be reused and made writable. There is no wait-until-first-metadata-write allocation of task_struct.

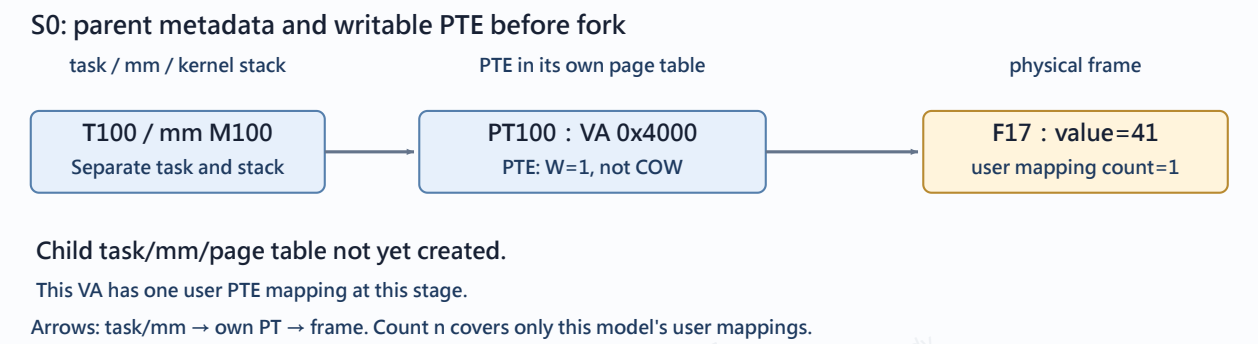
Layer	Ordinary fork	First private write
Task/kernel stack	Separate PID, state, stack objects	Not deferred by data COW
Mm/page tables	Separate objects	Child updates its PTE
Eligible resident private anonymous page	May share a temporarily write-protected frame	Copies if separation is needed; an exclusive page may be reused
Open-file descriptions	Existing descriptors can share objects	Offset sharing is unrelated to user-page COW

Four layers of fork state (continued)



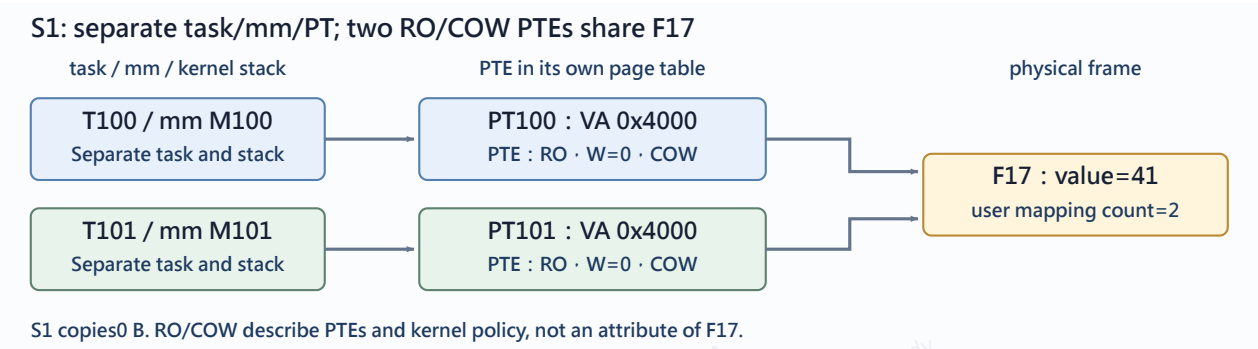
Clone parameterizes sharing. CLONE_VM shares mm;CLONE_FILES descriptor table;CLONE_FS filesystem context;CLONE_SIGHAND handler table. Thread-like sharing does not make ordinary fork share task metadata.

S0 before fork



T100/M100/PT100 maps VA 0x4000 to frame 17, x=41, W=1, user-PTE mapping count 1. Child task does not exist. This is the baseline, not a preexisting COW state.

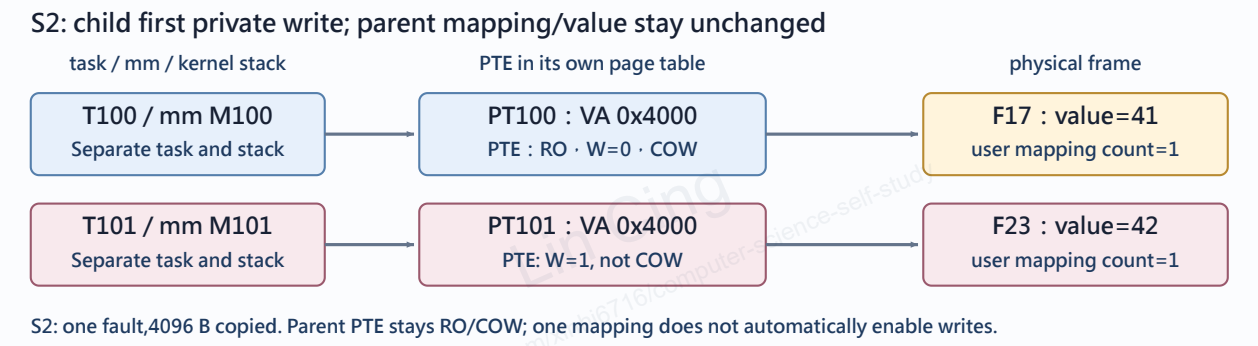
S1 separate metadata, shared frame



Child T101/M101/PT101 already has separate PID, state, kernel stack, mm, page table. Both PTEs map F17 with writes disabled under COW policy. F17 remains 41, mapping count 2, data copied 0 bytes.

S2 first child write

Assume one 4 KiB anonymous private page, logically writable VMA, x at offset 0 of VA 0x4000, initial 41. Exclude huge pages, other mappings, DMA, concurrent writes. COW is a policy label, not an x86 hardware PTE bit named COW. Counts cover this model's user PTE mappings rather than all Linux page references.



S2 first child write (continued)

State	Parent	Child	Frame contents/mappings
S0	T100/M100/PT100:F17, W1	Absent	F17:x41, n1
S1	T100/M100/PT100:F17, W0, COW	T101/M101/PT101:F17, W0, COW	F17:x41, n2;copied0
S2 writes 42	Parent unchanged	T101/M101/PT101:F23, W1	F17:x41, n1;F23:x42, n1;one fault,4096 B copied

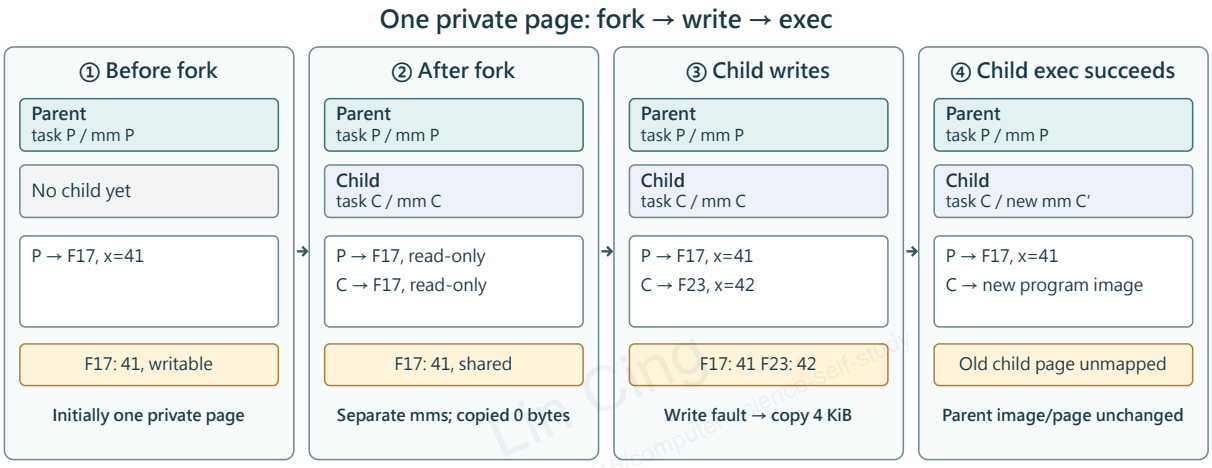
Observable counterexamples and CLONE_VM

If child and parent task were one object, setting child PID 101 would change parent PID too. Correctly, child ready→blocked leaves parent ready. Shared data pages do not imply shared tasks.

Successful exec in this single-thread child preserves process PID while replacing its program image. Early failure before destroying the old image can return an error. After v3.2 flush_old_exec crosses the point of no return, some later failures terminate by signal rather than restore the old image.

CLONE_VM is a different explicit option:two tasks share mm/tables. With writable F17, child writes 42, parent sees 42 without this private-COW fault. Ordinary fork instead yields parent 41/child 42. These finite models expose differences rather than claim live Linux tests;real verification needs corresponding source, fault traces, and frame/PTE counters.

Fork, write, and exec: track object identities



Model: private anonymous page; no concurrent writes, extra mappings, or DMA

In this model, fork creates separate tasks and mms while sharing one private anonymous page F17. If the child writes while both still share it, COW first copies the page to F23. Successful exec replaces the child image; the parent retains its own.

A fault copies only when necessary; an exclusive page may be reused. MAP_SHARED follows different rules, and inherited descriptors may share an open file description.

Clone flags are constrained

Mode	Task/TID	TGID	Mm	Fd table	Existing descriptions
Fork	New	New group	New, private pages may COW	Copied	Shared references
CLONE_VM only	New	Can be a new group	Shared	May be copied	References can share
VM/SIGHAND/THREAD	New	Shared group	Shared	Depends on CLONE_FILES	By table entries

V3.2 rejects THREAD without SIGHAND, and SIGHAND without VM. VM/FS/FILES/SIGHAND alone omit pthread-like thread-group/TGID/signal lifecycle semantics. Parent TID 100/TGID 100 can fork 101/101 or create thread 102/100. Both have independent tasks/stacks despite shared mm.

Real time converted to virtual runtime

For v3.2 fair class, greater weight means slower virtual accumulation and more CPU share. With nice 0 baseline 1024,

$$\Delta v = \Delta t \frac{1024}{w}.$$

Continuously runnable A, nice 0, w=1024 and B, nice 5, w=335 share an ideal 6 ms target period in ratios 1024/1359 and 335/1359, about 4.521/1.479 ms. Both virtual increments are about 4.521 ms. This weighted calculation does not promise exact per-period segments;tick, wakeup, minimum granularity, and other classes change traces.

The fair rbtree's handling of a running entity follows scheduling paths;not every task is always in one tree. RT FIFO/RR can outrank fair and do not compare the same vruntime. Historical predecessors explain design pressures without becoming v3.2's scheduler. Chapter 6's fixed HRTICK/periodic-tick Gantts provide intermediate selections.

How weights determine CPU share and vruntime

CFS weights: actual time and virtual runtime

One CPU; A and B remain runnable for an ideal 6 ms sharing interval

Total weight 1024 + 335 = 1359 $\Delta v = \Delta t \times 1024 / \text{weight}$

Weight share	Actual CPU time	Virtual runtime increase
A: nice 0, weight 1024 1024 / 1359 ≈ 75.35%	6 × 1024 / 1359 ≈ 4.521 ms	6 × 1024 / 1359 ≈ 4.521 ms
B: nice 5, weight 335 335 / 1359 ≈ 24.65%	6 × 335 / 1359 ≈ 1.479 ms	6 × 1024 / 1359 ≈ 4.521 ms

This is an ideal-share calculation, not a trace of every scheduling period.

In this ideal Linux 3.2 model, both tasks remain runnable on one CPU. The lower-weight task B receives less actual CPU time, yet its weighted virtual runtime, calculated before rounding, increases by the same amount as A. Real schedules also depend on ticks, wakeups, and other classes.

Local reentry versus SMP exclusion

A process context and same-CPU IRQ handler both acquire L. If the process holds L without masking that IRQ, the handler interrupts and spins forever because the holder cannot resume. Use the appropriate irq-save protocol and restore prior state rather than unconditionally enable IRQs.

Mechanism	Prevents	Does not prevent
Local preemption disable	Scheduler task switch here	IRQ, other CPUs
Local ordinary IRQ disable	Corresponding local handlers	Other CPUs, NMI
Atomic spinlock	Cooperating contexts on same lock	Nonparticipants, invalid DMA
Irq-save spinlock	Same-lock SMP races and specified local IRQ reentry	Mechanisms outside the protocol

Do not acquire a blocking mutex in nonsleepable paths. Keep spinlock regions short. NMI/softirq/configuration variations need separate analysis rather than one universal priority diagram.

Zones under a fixed x86-32 configuration

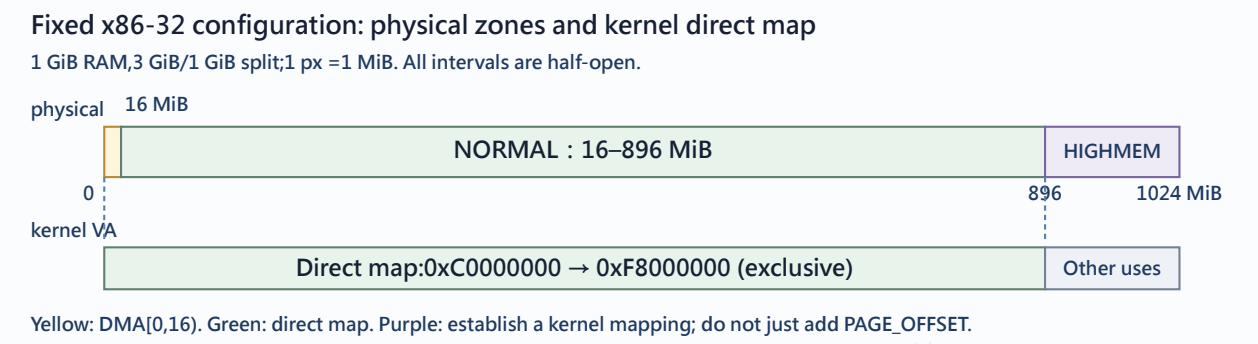
Fix v3.2, x86-32, 4 KiB pages, 3 GiB user/1 GiB kernel split, 896 MiB permanent direct-map coverage, and 1 GiB RAM. These are configuration-specific coverage intervals, not all-free memory.

Physical range	Zone	Constraint
[0,16 MiB)	DMA	Selected legacy ISA limits
[16,896 MiB)	NORMAL	Ordinary permanent direct map
[896 MiB,1 GiB)	HIGHMEM	Requires suitable temporary kernel mapping

PA0x01002000 plus PAGE_OFFSET0xC0000000 gives kernelVA0xC1002000 here. HighmemPA0x39000000 cannot simply be declaredVA0xF9000000, whose region has other uses. Establish a valid mapping. User PTEs can still map highmem:VPN0x40000→PFN0x39000 mapsVA0x40000123 toPA0x39000123.

The physical allocator follows zone/order constraints;buddy allocates power-of-two contiguous pages, slab smaller objects. Device masks, IOMMU, bounce buffers, and DMA API determine DMA addresses;not all DMA is limited to 16 MiB.

Zones under a fixed x86-32 configuration (continued)



ELF bytes and zero-fill

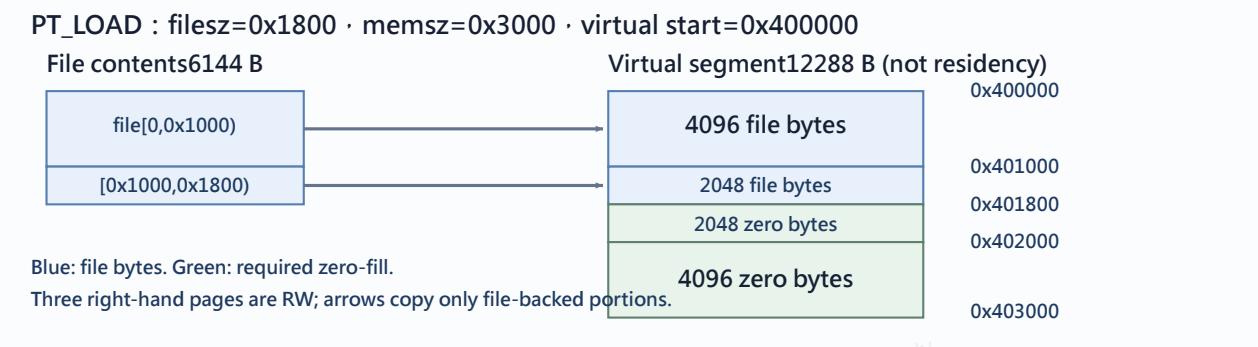
This separate example uses an NX-capable x86-64 ELF permission model, not the preceding x86-32 layout. PT_LOAD has p_offset=0, p_vaddr=0x400000, p_filesz=0x1800, p_memsz=0x3000, RW. Memory beyond filesz must read zero rather than later file garbage.

Virtual page	Contents	Permission
[0x400000,0x401000)	File[0,0x1000)	RW
[0x401000,0x402000)	File[0x1000,0x1800), then 0x800 zero bytes	RW
[0x402000,0x403000)	0x1000 zero bytes	RW

File contents 6144 B plus zero tail 6144 B equal 12288 virtual B. Creating a VMA does not make every page resident. Separate RX segments describe code;private writes use COW, connecting fork and fault handling.

PT_INTERP selects a user-mode dynamic interpreter. Kernel maps program/interpreter, builds initial stack, and enters it;dynamic linker resolves DT_NEEDED/symbols. Kernel need not resolve every function call, and libraries need not always map at one VA.

ELF bytes and zero-fill (continued)



Reclaim preserves backing and live references

Track mapping, backing, dirty state, and reclaim eligibility separately. Reuse cannot occur while valid CPU/device references still require a frame. Clean file-cache data is reconstructable only from valid backing. Dirty data may be the only latest copy;complete the required writeback before marking the covered state clean. Remove unsafe mappings/users under the reclaim protocol and release necessary references before reuse. A cached page can remain referenced while no application currently executes on it. Writeback cleanliness alone is not a full filesystem durability transaction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

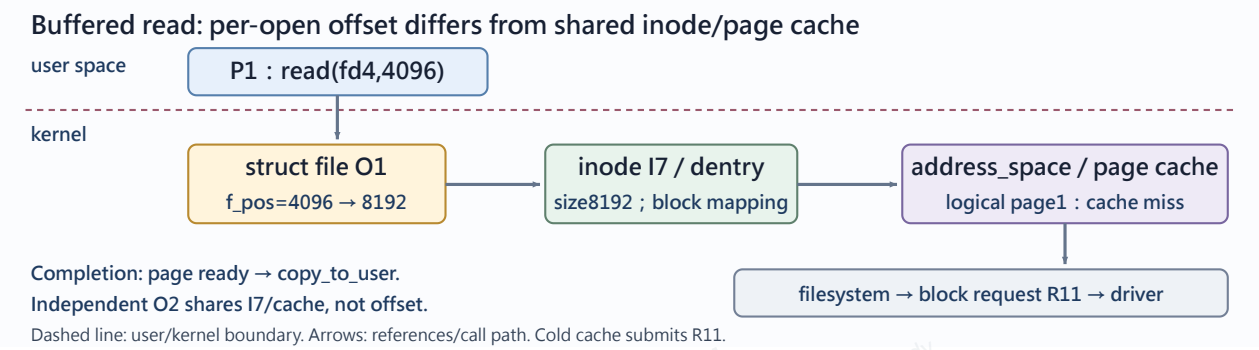
Four VFS objects in one buffered read

Fix v3.2, ordinary ext3 buffered read, size 8192, page 4096, opened P1 fd 4→struct file O1 with f_pos 4096, dentry/inode I7 references. I7.address_space manages shared file pages; superblock identifies the filesystem instance. This example assumes complete successful I/O and copy_to_user, available data, and no concurrent truncate or shared-offset change.

Object	State/references	Incorrect assumption
Superblock	Filesystem instance/operations	Contains per-open offset
Dentry	Parent/name→inode, possibly negative	One unique name per file
Inode	Identity, mode, size, mapping, address_space	One global current offset
Struct file O1	F_pos, flags, operations, path	Every fd necessarily has its own offset

Read 4096 retains O1, validates operation/user buffer, and dispatches file ops. A miss on logical page 1 submits mapped blocks, waits for page readiness, copies to user, and sets O1.f_pos to 8192. Independent O2 can use I7's page cache; dup shares O1. Cache hits save block I/O without eliminating lookup/copy costs.

Four VFS objects in one buffered read (continued)



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ext3 block groups and procfs boundaries

Ext3 organizes inodes, data, bitmaps, block groups for locality. Nearby allocation is a policy target, not an invariant that every file stays contiguous. Growth/fragmentation can move later blocks while mappings remain correct.

In the teaching data=ordered model, a 6000 B write dirties two page-cache pages and its metadata transaction publishes newly allocated block mappings. Relevant data must be ordered before the metadata commit that exposes those mappings, avoiding pointers to unready contents. Write success differs from fsync, and metadata atomicity does not make all in-place user writes atomic. Data=writeback and data=journal have different contracts.

Procfs /proc/123/status formats live kernel state rather than necessarily reading persistent blocks. Reused PID 123 can later denote another process;monitoring needs start time/other identity evidence. Separate proc reads do not form a simultaneous system snapshot. A fixed 16-bit PID or always-PID-in-upper-inode-bits rule is not assumed for v3.2.

Three driver paths and terminal states

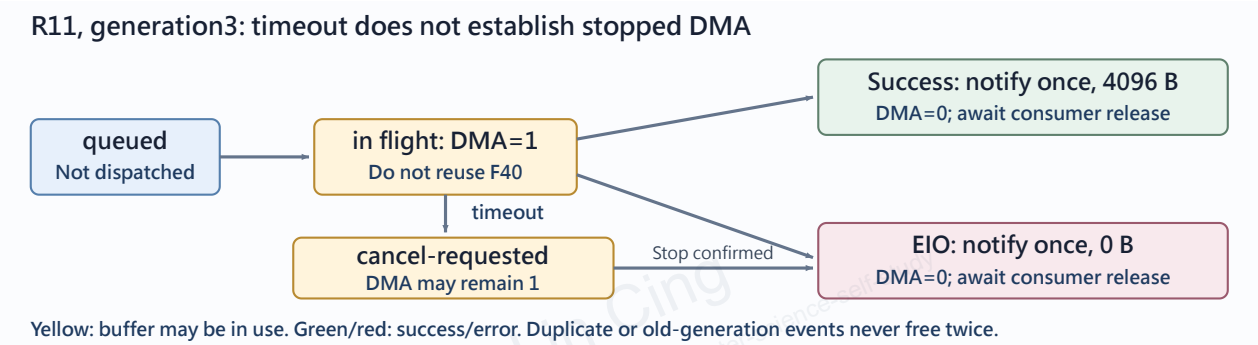
Block devices offer block/sector addressing through filesystem/request layers. Character drivers define file operations without universal seeking. Network devices use sockets/protocols/packets rather than universally seekable device files.

Fix one cold-page synchronous read, resident user buffer, no read-ahead, request R11 over kernel buffer F40. Queue→in-flight device ownership→terminal success/error. Publish results and release mapping according to completion/lifetime. Real layers can merge/split;one page/request is this input model.

Outcome	Hardware evidence	Terminal notifications	Application	Reuse F40
Success	I/O ends, ownership returns	One	4096 B	After copying/references end
EIO	Request stopped/error-completed	One	Error with no prior bytes	After DMA/references end
Timeout, hardware may continue	Stop unproved	Cannot claim success/safe release	Cancellation/reset protocol	Not yet

Three driver paths and terminal states (continued)

An error completion still completes;excluding it from completion counts strands waiters. Timeout is not hardware cessation. Late DMA after free corrupts the next owner's data.



Duplicate completion and reset

Submission identity is (id 11, generation 3). First valid terminal event atomically changes in-flight→terminal and notifies once. Duplicates of that generation are ignored/logged without another notify/free. Reusing id 11 with generation 4 rejects old generation 3 events.

Generation protects software identity, not DMA quiescence. Cancel/reset must establish hardware no longer uses old addresses before synchronization/unmap/reuse. Device-specific late-IRQ/error protocols matter beyond a timeout number.

Synchronous read finishes copying before return. Async user buffers remain valid through the completion contract. Queue state, ownership, and notification count belong in one consistent model.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Pipe EOF requires every writer to close

Parent fd 3→read description R, fd 4→write description W. Fork copies fd tables while retaining R/W objects. A parent consumer and child producer must close unused endpoints.

Event	Parent writer	Child writer	Buffered bytes	Read result
After fork	1	1	0	Block or EAGAIN, not EOF
Child writesABC/closes	1	0	3	ABC available
Consumer drains	1	0	0	Still no EOF
Parent closes last writer	0	0	0	Later read returns 0

Streams preserve byte order, but multiwriter atomicity depends on PIPE_BUF, size, and mode rather than every arbitrary write being indivisible. Shared memory supplies neither queueing nor exclusion;choose locks, semaphores, or an atomic protocol.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Signals, wait queues, and atomic semop

Blocked standard signals can coalesce repeated instances into one pending event, so they are not reliable counters. Real-time signals can queue and sigqueue can carry limited payload. Handlers remain constrained by async-signal-safety.

A waiter that tests done=false before enqueue/sleep can miss a producer's intervening done=true+wake. Establish a wakeable waiting state first, then recheck. If true, do not sleep. Recheck after waking because another waiter can consume the resource. Wake makes runnable rather than immediately executing or permanently satisfying the condition.

System V semop atomically batches operations within a semaphore set. With S0=1, S1=0, the batch S0-=1, S1-=1 must not permanently decrement S0 while waiting for S1. Publish no partial effect before all can complete. Permissions, limits, deletion, and errors remain part of the contract.

Application and wire byte denominators

Fix IPv4/TCP, no options, MTU 1500, no TSO/GSO. Send accepts 6000 B. The MTU permits at most 1460 B of TCP data, but peer MSS, path MTU, scheduling and window can change segmentation. In this specific trace, effective MSS is 1460 B, there is no retransmission, and the sender transmits four full segments followed by one 160 B segment; payloads sum to 6000 B. Five IP/TCP headers add 200 B, L3 bytes total 6200 B, excluding Ethernet, ACK, retransmission.

Sk_buff, TCP send queue, routing, and driver retain different states. Offload can split one large sk_buff into many wire packets, so counts differ. Send acceptance proves neither peer acknowledgment nor application processing nor disk durability.

For 10.1.2.3 with routes /0, 10.0.0.0/8, 10.1.0.0/16, 10.1.2.0/24, the longest match is /24. FIB structures, caches, hooks vary by version/CONFIG;longest-prefix semantics do not imply one universal hash layout.

Queueing, retransmission, and causal evidence

In a fixed trace relative to enqueue0, dispatch2 ms, transmit completion3, ACK13. Queue wait2, dispatch-to-transmit1, post-transmit ACK wait10. These claims follow linked timestamps.

Aggregate queue length20, retransmissions3, device errors1 cannot prove queueing caused a long tail. Loss, reordering, or unrelated device faults may explain counts. Link the same request/flow/window's enqueue, dispatch, completion, drop, ACK evidence, or use a controlled intervention before making a causal claim.

TCP sequence handling restores ordered delivered bytes despite IP loss/reordering. UDP preserves datagram boundaries without delivery guarantees. A TCP upper layer does not make every driver/IP queue lossless or ordered.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Authentication and authorization

PAM composes user-space authentication, account, session policies;kernel resource checks still use credentials. Salt separates stored values without adding password entropy. KDF cost/access protection differ. Old eight-character schemes are not current recommendations.

Credentials include real, effective, saved, filesystem IDs, groups, capabilities. Ordinary file DAC selects owner/group/other then requested bits;ACL, LSM, capability, mount conditions can change the decision. Successful open can create a continuing handle not revoked by later pathname-mode changes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Root and regular-file DAC

Fix v3.2, a non-immutable regular file using generic_permission, owner 1000, mode 0400, rw mount, no ACL/LSM denial, and successful path traversal. Caller current_user_ns equals inode_userns; each request independently asks for only read, write, or execute. Effective capabilities belong to the corresponding user namespace;exclude other policy.

Caller	Read	Write	Execute	Rule
Fsuid 1000, no DAC capability	Allow	Deny	Deny	Owner bits 100
Fsuid 2000, no capability	Deny	Deny	Deny	Other bits 000
Fsuid 0, CAP_DAC_OVERRIDE	Allow	Allow	Deny	Override read/write;regular execute needs some x bit
Fsuid 0, no relevant capability	Deny	Deny	Deny	Numeric UID alone adds no missing capability
Fsuid 2000, CAP_DAC_READ_SEARCH only	Allow	Deny	Deny	Supplies regular-file read only

Generic_permission first performs ordinary checking, then tests CAP_DAC_OVERRIDE on failure;execute also requires nonzero S_IXUGO. Mode 0401 supplies an x bit, allowing that DAC override, while noexec, format, and other policies still apply. Thus mode 0400 does not force capable-root write denial, and root does not bypass every security layer.

Temporary versus permanent privilege drop

A setuid-root program hasreal1000, effective0, saved0. Seteuid(1000) yields1000/1000/0 and can later restore effective0 under saved-ID rules. It is temporary.

Permanent dropping additionally handles saved IDs, groups, capability sets, and privileged handles. Changing euid alone does not recall opened descriptors. Securebits, user namespaces, and required resources affect implementation;this ID trace is not a complete sandbox recipe.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One successful read across mechanisms

Continue fd 4→O1→I7, offset 4096, read 4096, cold cache, resident user buffer, no concurrent truncate or shared-offset changes, and complete successful I/O and copy_to_user. Retain fd/validate→cache lookup→filesystem map→R11 block I/O→device completion→ready page→copy_to_user→O1.offset 8192→return 4096. Waiting can switch tasks, unlike an ordinary internal function call.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Errors, partial progress, and lifetimes

If R11 terminates EIO before any copy, read returns error without advancing offset. Terminal notification is still exactly one, and buffer reuse waits for stopped DMA/released references. With earlier copied bytes, an API may return that successful prefix and expose errors later;not every failed read has zero progress. This fixed example explicitly excludes prior bytes.

Fork sharing pages does not share tasks;independent opens sharing inode do not share offsets;send acceptance is not remote receipt;dirty-cache writes are not durable. Identity, authority, lifetime, and completion contracts preserve these distinctions.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz et al., Operating System Concepts, 9th ed., Ch. 18. Linux v3.2 fork, memory, module, DAC, ELF. fork(2), signal(7).

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Chapter route

How does layered Windows architecture organize objects, processes, I/O, and services across user/kernel boundaries?

Prerequisites are OS structure, protection domains, and file/I/O mechanisms.

- Identify user/kernel-mode components.
- Analyze object-manager handles and granted access.
- Separate Windows 7 history from contemporary implementations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One NT-family case

The case distinguishes user subsystems/services from kernel-mode executive, kernel, HAL, drivers. It does not substitute DOS-based Windows, Mach, or later Windows additions.

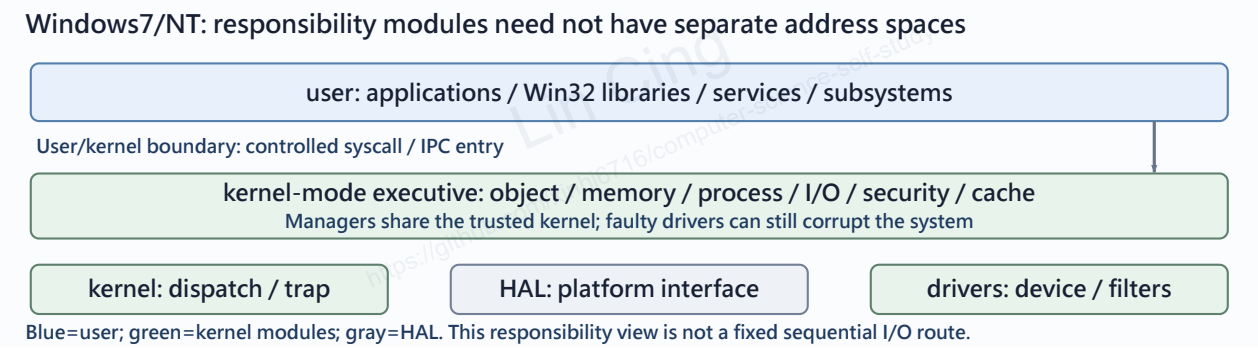
Term	Meaning	Does not imply
Win32 API	Application interfaces for processes, objects, I/O	Every API makes a syscall
NT kernel/executive	Trusted scheduling, memory, object, I/O	Every component has its own address space
Compatibility	Runs existing programs in specified environment	Every ISA, bitness, legacy subsystem supported
Portability	Concentrates hardware dependencies	Driver binaries run unchanged across CPUs

OS/2, Win16, POSIX labels in historical diagrams retain their period/version scope. They do not make every Windows 7 installation run every old binary. Subsequent cases specify execution, configuration, failure assumptions.

ReadFile across trusted components

Library preparation enters a controlled kernel interface that checks handle/buffer. I/O manager, filesystem, cache, memory manager select a hit or driver path. Device waits can block the thread before completion returns a result.

An API name alone specifies neither blocking, copy count, IRP presence, nor context-switch count. Cached/fast paths differ. Record open flags, object type, cache state, and completion mode first. Layering includes many kernel-mode components rather than a mandatory user-server hop or independent failure domain per component.



Goals and mechanisms

Goal	Mechanism	Not guaranteed by its name
Security	Tokens, descriptors, granted handle access	Bug-free kernel/drivers
Reliability	Boundaries, references, error recovery	Shared-kernel fault containment
Performance	Caches, async I/O, scheduling	Uniformly low latency
Extensibility	Object/driver interfaces	Every extension is safe
Portability	HAL, architecture-specific layers	Cost-free porting
Internationalization	Unicode, locale data	Characters equal bytes
Power/dynamic devices	Power states, PnP lifetime	Removal without outstanding I/O

HAL abstracts selected platform, interrupt, timer details without removing every memory-model/driver responsibility. SMP support does not make shared updates race-free.

Unicode length units

Win32 W interfaces use UTF-16 code units; A interfaces use code-page conversion. Neither count is necessarily a UTF-8 byte count or a Unicode scalar count.

A, U+1F600, B encodes as 0041 D83D DE00 0042: four 16-bit code units, three scalars, and 8 data bytes. If a terminating NUL is required, it adds 2 bytes, for 10 total; a 4-byte buffer cannot hold the complete data and terminator.

A capacity in wchar_t units is not a byte count, and a byte length is not a scalar count. Unicode APIs do not automatically normalize every filename into one canonical form.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

PnP/power depend on ownership

The lifecycle model is working→stop-pending→stopped→removed, not the complete PnP IRP list. Stop-pending blocks new submission, drains/cancels existing work with confirmed completion, then releases device context, DMA maps, buffers.

Freeing context before an outstanding IRQ/request ends causes use-after-free. Requested cancellation does not prove DMA stopped. Removal can finish requests with errors rather than guaranteed success. No resource can pass to an incompatible owner while execution/device references can still touch it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kernel, executive, HAL, and services

Component	Responsibility	State
Kernel dispatcher	Select/dispatch threads, synchronization/trap foundations	Ready queues, thread states
Executive managers	Objects, memory, process, I/O, security, cache	Handles, PTE/PFN, IRPs
HAL	Platform, interrupt, timer abstraction	Routing/control
Drivers	Devices, filters, filesystems	Device objects, queues, buffers
User services/subsystems	Login, background services, application environments	User address spaces, IPC

Kernel/executive are responsibility labels, both privileged. A thread entering kernel uses its kernel stack without creating a separately scheduled second thread.

Thread priority differs from IRQL

Dispatcher chooses the highest-priority eligible ready thread:variable 1–15, realtime 16–31, idle special 0. Variable boosts decay no lower than base and do not enter realtime range. Conditions affect boost/quantum;the later q2 is not universal.

IRQL governs current-processor interrupt/kernel activity and permitted operations, not another 0–31 thread priority.

Common x86/x64 IRQL	Execution	Constraint
PASSIVE_LEVEL0	Threads/workers	Waiting still follows API rules
APC_LEVEL1	Selected kernel APC work	Specific APC/call constraints
DISPATCH_LEVEL2	Ordinary DPC/spinlock regions	No nonzero blocking wait/pageable access fault
Device DIRQL	ISR/device synchronization	Shorter, more restricted work

Raising one CPU's IRQL does not exclude other CPUs. Realtime thread priority does not guarantee hard deadlines. Driver priority rules define these constraints.

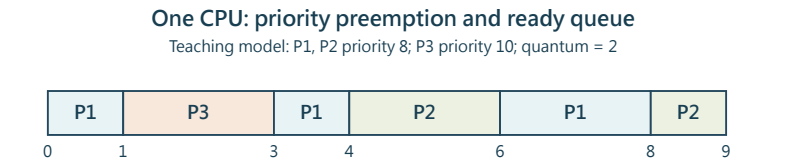
A fixed priority trace

One CPU, no boosts/decay, equal-priority q2, higher priority immediately preempts. A preempted task keeps residual slice and returns to its priority queue's front. P1/P2 priority 8, ready at 0, bursts 4/3. P3 priority 10, ready at 1, burst 2.

Interval	Reason	Result
0–1 P1	First equal-priority arrival	Preempted with burst 3/slice 1
1–3 P3	Higher priority 10	Complete
3–4 P1	Queue front, residual slice	Burst2, quantum expires, tail
4–6 P2	Priority8 queue front	Burst1, tail
6–8 P1	Queue front	Complete
8–9 P2	Only ready task	Complete

Completions P1/P2/P3=8/9/3, total CPU 9. This isolates queue/quantum rules; actual boosts, quanta, processors require their own calculation.

Priority preemption and queue trace



Time and event	Priority 8 ready queue before dispatch	
t=0	P1 → P2	P1 enters first
t=1	P1 → P2	P3 preempts; P1 retains 1 slice unit
t=3	P1 → P2	P3 finishes; P1 uses remaining slice
t=4	P2 → P1	P1 slice expires; P1 moves to tail
t=6	P1 → P2	P2 has 1 left; moves to tail

Completion: P1 = 8, P2 = 9, P3 = 3

Fixed teaching rules, not Windows 7 actual quantum or complete scheduling.

P3 preempts P1 at t=1; P1 retains one unit of its slice. P1 resumes at t=3 and moves to the priority 8 queue tail when its slice expires at t=4. Completion times are P1=8, P2=9, and P3=3. These are teaching rules, not a Windows 7 scheduling guarantee.

Dispatcher objects are not universal queues

Manual-reset events remain signaled until ResetEvent and can release many waiters. Auto-reset satisfies one wait then resets. Repeated SetEvent on an already signaled event does not accumulate independent tokens. Counted resources need semaphores or explicit queues.

A mutex has an owner. WAIT_ABANDONED signals abnormal prior-owner termination and requires checking protected-data consistency;acquisition does not repair it. Mutexes alone do not prevent multi-lock ordering deadlocks.

This model exposes semaphore count 0…limit. At 0, waiters sleep; release can hand off a permit. Do not confuse it with Chapter 5's alternative internal signed counter.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ISR, DPC, APC, and worker contracts

ISR checks/acknowledges source and performs minimum work. Ordinary DPC defers work but remains DISPATCH_LEVEL, so it cannot perform arbitrary blocking filesystem I/O. Sleepable/pageable work goes to an appropriate PASSIVE worker with retained context references.

An APC targets a thread. Ordinary user APCs generally run during its alertable wait, not arbitrary user instructions. Sleep differs from SleepEx(TRUE). Special kernel APC/startup paths have additional rules outside this summary.

ISR→DPC→worker retains context before queueing and releases after worker completion. Freeing before the queued worker uses it is invalid. IRQL and ownership jointly determine legal work.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Handles are process-table indexes

A's 0x40→F, granted R while B's 0x40 can name event E. Integers alone do not transfer authority. Inheritance, DuplicateHandle, or named reopening provide explicit authorized sharing.

Step	A table	B table	F lifetime
Initial	0x40→F, R	Empty	A retains
SAME_ACCESS duplicate	Unchanged	0x88→F, R	Both retain
A closes	Empty	Still 0x88	Retained
B closes, internal I/O remains	Empty	Empty	Still retained
Last internal reference ends	Empty	Empty	Cleanup permitted by type rules

SAME_ACCESS preserves rights. Other desired-access behavior depends on object/API authority, not a universal subset rule. Source/target process handles need rights such as PROCESS_DUP_HANDLE. DuplicateHandle.

Tokens and ordered DACLs

Choose the effective token, including impersonation, with SIDs, groups, privileges. Descriptor contains owner, DACL, SACL;DACL grants/denies while SACL principally audits.

This simplified model uses enabled SIDs, ordinary ACEs, specific rights, excluding privilege, integrity, owner/object-ACE exceptions. Abstract R=1/W=2 are not actual Win32 masks. Request R/W starts remaining 3.

ACE	Match	Remaining/result
ALLOW Alice R	Yes	RemoveR, leaveW
DENY Staff W	Alice inStaff	IntersectsW, deny/stop
ALLOW Staff RW	Not reached	Cannot repair denial

Moving the final allow before deny can satisfy all remaining bits first. Order matters;canonical ACE ordering is preferred. NULL DACL imposes no DACL restriction;empty DACL grants nothing in this ordinary model. Ordered access checks.

Reservation, commitment, and residency

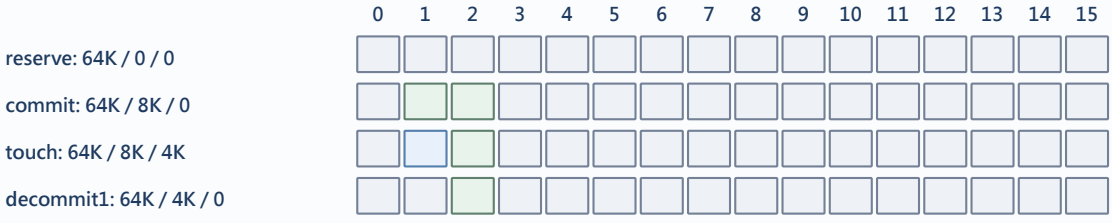
Fix 4 KiB pages, 64 KiB allocation granularity, private base 0x100000. Reserve 64 KiB of process VA, then commit pages 0x101000/0x102000 with 8 KiB charge, not necessarily physical frames.

Step	Reserved VA	Private committed	Resident here
Reserve	64 KiB	0	0
Commit two pages	64 KiB	8 KiB	0
Touch 0x101123	64 KiB	8 KiB	4 KiB zero-fill
Decommit first, no other reference	64 KiB	4 KiB	0
Release region	0	0	0

Reservation, commitment, and residency (continued)

Sixteen 4 KiB pages in 64 KiB: distinguish reserve, commit, and resident

Gray=reserved, uncommitted; green=committed, nonresident; blue=resident and committed.



Rows show reserved VA / committed private / resident. Releasing the entire region makes all three zero.

Reserve/commit can fail;commit follows system backing commitments rather than immediately zero-writing pagefile pages. Equal VA numbers in different processes do not compete for one shared virtual address. VirtualAlloc.

PTE, PFN, and working-set trimming

PTEs describe virtual mappings/software states;PFN database describes frames, references, lists. A legal nonresident page can be demand-zero, section/file/pagefile-backed, or recoverable from RAM. Not every fault reads storage.

VA 0x401123 has VPN 0x401, offset 0x123. A valid PTE→PFN 0x2A gives PA 0x2A123. Writing RO without legal COW is an access violation, not permission to allocate around protection.

Trimmed clean backed pages can enter standby rather than vanish. Modified pages need writeback before reuse;age alone cannot free pinned/I/O-held frames. Standby recovery can be a soft fault;missing RAM backing may require storage. Windows 7 reference-history/working-set heuristics are neither future-aware OPT nor exact LRU. Track frames, PTEs, pending writes together.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Processes, sections, and ALPC authority

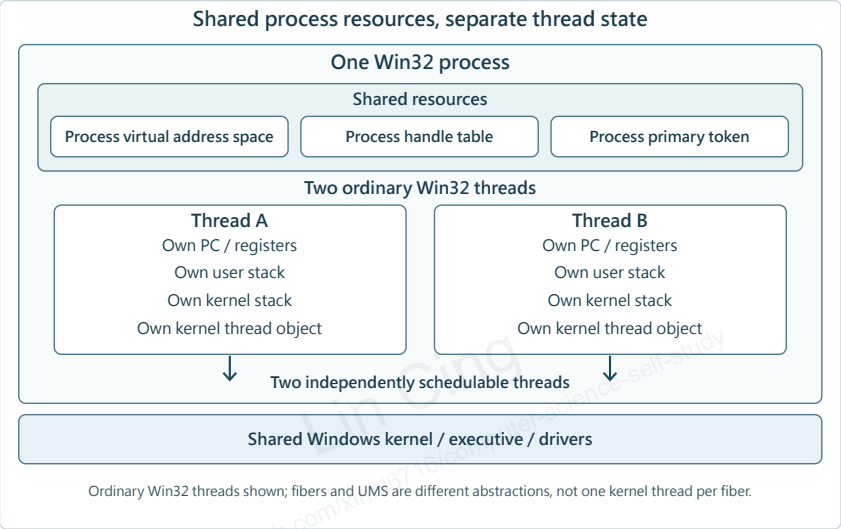
CreateProcess includes an initial thread;another CreateThread makes two. Process owns address space, handles, token;threads own registers, user/kernel stacks, execution state. Closing a process handle does not terminate it. Exited-thread objects can remain observable/waitable through retained handles.

Sections map file/pagefile backing at different process VAs with independent rights and shared-data synchronization. ALPC/ports package client/server requests and authority. Shared sections save copies without removing lengths/ownership checks.

A maps offset 0 at 0x100000, B at 0x500000. A+16 and B+16 can share bytes, but literal pointer 0x100010 is not B's pointer. Transfer section-relative offset 16 and length.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two Win32 threads in one process



Each ordinary Win32 thread has a kernel-schedulable thread object and its own execution state. Threads share the process address space and handle table, but have separate registers, user stacks, and kernel stacks. The token shown is the process primary token; thread impersonation changes which identity is effective.

Fibers and UMS are other user-level scheduling abstractions; a fiber is not automatically a kernel thread. Shared resources still require synchronization.

IRP dispatch down, completion up

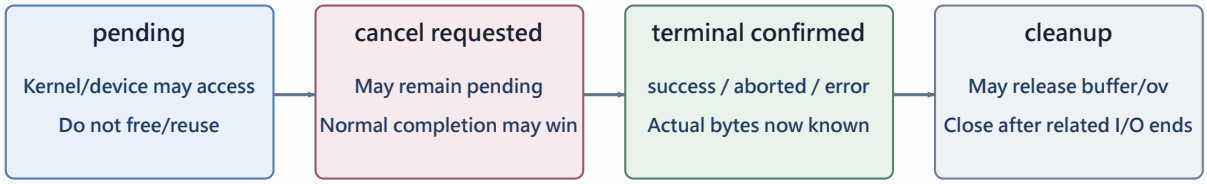
I/O manager can send IRPs through filesystem/filter/device stacks with per-stack state. Lower completion returns status/actual bytes through completion paths. Cached/fast operations need not repeat the same full chain.

Time	Ownership/state	Allowed action
Submit	Valid buffer, unique operation state	Do not reuse while pending
Pending	Driver/device can access	Cancel requests work;does not free
Hardware/cancel completion	Status/bytes established	Notify under completion rules
Caller confirms terminal	Success/aborted/error known	Release buffer/OVERLAPPED

CancelIoEx races normal completion. Success means cancellation requested;ERROR_NOT_FOUND can mean no matching pending operation. Collect the original result regardless. Cancellation contract.

IRP dispatch down, completion up (continued)

Cancel requested ≠ I/O complete: retain buffer/OVERLAPPED until terminal



This path requests cancellation. Pending may complete normally; cancel success is not the operation result.
Use unique ov/buffer per outstanding operation. Win7 needs a count argument; final bytes come from completion.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Cache/memory managers and reads

Cached reads find file ranges in filesystem/cache views. Resident data copies to caller. A view fault invokes paging read through storage to fill a frame, then satisfies the original read. Paging/cached requests are related but not two application deliveries.

For 4096 B, a hit copies 4096 to caller. A miss adds 4096 B device→cache DMA before that copy. CPU copy cost, DMA bytes, storage reads differ; this is not two CPU copies. Lazy writing batches dirty data while delaying durability. Filesystem/storage log/data flush ordering establishes persistence rather than hit/write success alone.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

PnP, power, registry, and boot dependencies

PnP coordinates discovery, driver stacks, removal;power manager coordinates power states. Drain hardware-touching requests before releasing resources. Driver objects, device objects, and physical presence/usability have different lifetimes.

Registry hives, keys, values store configuration, possibly volatile state, rather than ordinary file paths. Changes take effect under service reload/restart/notification rules, not instantaneously in every process.

Boot progresses through firmware/boot manager/OS loader, kernel/HAL/boot drivers, kernel initialization, sessions/services, logon. BIOS/UEFI differ;no universal MBR file sequence is assumed. Page tables/storage drivers must exist before dependent memory/file services.

Session, window station, desktop, and process

Terminal services/fast user switching preserve interactive sessions, desktop/input views, and processes. Windows 7 service/session arrangements must not be confused with older versions.

Resource	Independence across sessions	Reason
User address space	Per-process mappings	Session numbers do not replace MMU
Window stations/desktops, some names	Session-related scope	Object rights/namespace still matter
Kernel, drivers, RAM	Shared	Common faults/resource limits
NTFS/network services	Can be shared	Credentials/object policy

The abstraction does not prove unrestricted simultaneous interactive connections;product, licensing, deployment limits are separate.

Local/Global scope is not authorization

S1/S2 Local\Ready events can be distinct session objects. Authorized Global\Ready access can reach one global object while retaining DACL/create/open rights. Global never means bypass checks.

Handle 0x80 in different processes can denote unrelated objects. Equal names require matching namespace and authority. Session ID is not trusted user identity. Switching the foreground user need not terminate background-session processes or release their locks, CPU, memory.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Volumes, MFT, attributes, extents

Windows 7 supports multiple filesystems;not every volume is NTFS. Clusters allocate storage;LCNs are volume-relative. Disk offsets additionally include partition/volume start.

MFT records contain attributes. Small data can be resident in its record;nonresident data uses VCN→LCN extents. Nonresident here does not mean absent from RAM. In a 1024 B record with 700 already used,324 remains. Data 300 fits this budget;512 does not. Names/attributes/format change the threshold;300 is not universal.

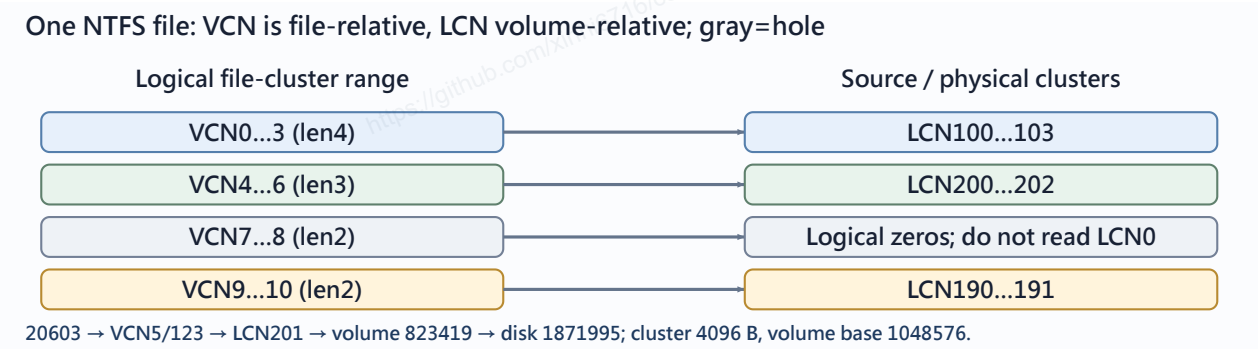
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One complete VCN-to-LCN example

Cluster 4096 B, volume starts at disk byte 1048576. Half-open extents are:

Start VCN	Length	Start LCN	Mapping
0	4	100	VCN0–3→LCN100–103
4	3	200	VCN4–6→LCN200–202
7	2	None	VCN7/8 are holes, logical zeros
9	2	190	VCN9/10→LCN190/191

File byte 20603=5・4096+123 selects extent2. LCN=200+(5−4)=201. Volume byte201・4096+123=823419;disk byte=1048576+823419=1871995. A hole returns zeros without treating None as LCN 0.



File references, indexes, and metadata files

The book's NTFS reference combines 48-bit MFT record and 16-bit sequence. Reuse changes sequence to reject mismatched stale references. Finite 16-bit sequences do not prove everlasting uniqueness against wrap.

Directory indexes sort keys/references. Balanced B+ paths have logarithmic height, but insertion/splitting can reorganize nodes. Small indexes fit root/record;larger ones allocate index blocks. Cached directory sizes/times need not reread current contents on every enumeration.

MFT, bitmap, log, volume, and other metadata use special files and common cache/I/O services. Cluster bitmap, extent, size updates still need coordinated consistency rather than independent successful writes.

Metadata WAL before home updates

This is a single-transaction undo/redo teaching model with exclusive reservation of the new block, not a complete NTFS log encoding or concurrent recovery protocol. All before/after records and a completeness marker must be durable **before any home field is changed**. Complete records and commit are distinguishable and flush is trusted. Steal allows home writes before commit; no-force allows home writes to lag after commit.

Old metadata is size 4096, extents [100], bitmap[200]=0; new metadata is size 8192, extents [100,200], bitmap[200]=1. Persist the complete log before changing home. Announce success only after commit is durable.

Crash prefix	Possible home state	Recovery
Complete log not durable	All old	This model prohibits any home update
Complete log durable, no durable commit	Old or mixed	Undo before-images to complete old
Complete log and commit durable	Old, mixed, or new	Redo after-images to complete new

Recovery chooses the entire metadata state. Actual NTFS also handles LSNs, concurrency and checkpoints.

Recovery invariant and checkpoint scope

In this model, all before/after records and their completeness marker are durable before any home update. Without a durable commit, undo changed fields using before-images; with one, redo all after-images. Assigning old/new values is idempotent, so repeated recovery chooses the same state. Logs must remain available and the flush and failure assumptions must hold.

A checkpoint does not mean every older log can be discarded. Active-transaction undo and dirty-metadata redo remain until safe truncation conditions hold.

Metadata consistency does not make user data an atomic application transaction. Cached write success or NTFS logging does not atomically update two business files. Applications need their own flush, publication, or transaction protocol.

ACLs, encryption, and repair

NTFS uses Windows descriptors/DACLs, not POSIX owner/group/other masks. Traverse-bypass privilege and explicit FILE_TRAVERSE checks are different contexts;Windows does not universally ignore path protection.

EFS encrypts file contents;BitLocker protects volume/storage under its keys/unlock conditions. ACLs alone do not protect unencrypted media read elsewhere. Encryption supplies neither backups nor application transactions.

Bad-sector remapping supplies new locations, not unreadable lost contents without a valid mirror/parity/source. RAID/spares, metadata recovery, and data repair guarantee different things.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sparse allocation, compression, USN, and snapshots

The example spans 11 logical clusters, including two holes. Nine data clusters allocate 36864 B without compression while logical size can be 45056 B. Hole writes later require allocation. This is not a process VA reservation.

Compression units and content/configuration determine compression results, not a fixed ratio. USN tracks changes for consumers/indexes;it differs from recovery logging and can wrap. A history gap requires reconstruction rather than treating it as an eternal backup.

VSS provides a point-in-time storage view. Application updates across related fields can require writer coordination/quiescence for application consistency.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

COW snapshots retain the old version

Live X=41, Y80 creates snapshotS. Before first update X to 42, retain old X=41 for S, then publish new live. After live X=43, S still sees 41 rather than 42. Unchanged Y can share old 80.

Time	Live X	S X	Y
After snapshot	41	41	Shared 80
First write	42	41	80
Second write	43	41	80

Reclaim old backing only after every retaining snapshot/reader releases it. Shared-storage failure can destroy live and snapshot together, so it is not independent backup media.

Interfaces and protocol responsibilities

User Winsock offers sockets;NDIS connects adapter/protocol driver models. TDI is historical;Microsoft directs new Vista-and-later kernel network modules toward WSK. It is not Windows 7's only new-driver route. These responsibilities do not determine a fixed process-switch count per packet.

Mechanism	Role	Does not supply
TCP/IP	Addressing, packets, reliable bytes	Application framing/exactly-once effects
SMB redirector/server	Remote file operations	Identical local/remote failures
Named pipe	Local/remote pipe semantics	Interchangeable byte/message modes
RPC	Request/reply call abstraction	Timeout proves nonexecution
COM	Object interfaces/marshalling	Every call crosses machines

Specify versions, authentication, encryption, errors separately. HTTP/WebDAV/VPN names do not replace settings or imply contemporary security guidance.

UNC paths and partial failures

Opening `\\server\share\report` selects a redirector encoding network requests. Server applies its credentials/filesystem policy and replies status/bytes. Client, network, server, storage can fail independently.

Server writes but reply is lost. Client timeout does not prove no write;retrying append can duplicate. Protocol request identity, deduplication/retry, and durability rules must handle it. TCP reliability alone is not business exactly-once.

Similar APIs can have different share/offline-cache visibility and error timing. Open success does not promise an immortal connection.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Named-pipe messages and directory identities

Fix a message-type named pipe whose receiving handle uses `PIPE_READMODE_MESSAGE`. If the next message has 500 B and the receive buffer holds 200 B, this example can return 200 B with `ERROR_MORE_DATA`; read the remaining 300 B of that same message. FALSE need not mean zero-byte fatal failure. A message-type pipe handle may use byte-read mode, so this example does not describe that mode or TCP framing.

Domains integrate shared identity and management. Active Directory combines directory and authentication-related services rather than merely an encrypted database. Equal display names can have different SIDs; servers check effective credentials, token and object policy.

Offline client caching creates reconnection versions and conflicts. Cached success is not remote durable commit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Scheduling units and pools

CreateProcess includes an initial thread and separately observable/waitable process/thread handles. Additional threads share address space/resources but have separate PC, stack, TLS. Handle names do not replace references.

Fibers switch in user code within a thread. A blocking syscall on that thread does not let ordinary fiber switching make the same kernel thread run another fiber simultaneously. Windows 7 UMS coordinates user scheduling with managed kernel state under platform/API constraints;it does not make all blocking free or every API fiber-compatible.

Pools reuse workers/queues while retaining work-data lifetimes. One-worker jobA waiting for queued jobB in the same pool can starve/deadlock it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sharing handles and file positions

Inheritance requires handle inheritability and CreateProcess conditions/handle list. DuplicateHandle requires source/target process authority. Named opening checks the selected namespace/policy again.

DUPLICATE_SAME_ACCESS retains granted mask;copying read-only to a child creates no write right. Different requested access follows object/API rules, not arbitrary amplification or a universal subset theorem.

Duplicated handles to one file object can share position;independent CreateFile can have separate positions. Seek/read races remain. Overlapped operations carry their own offsets/state.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three ReadFile starting outcomes

Use regular files opened FILE_FLAG_OVERLAPPED, independent OVERLAPPED and valid buffer per outstanding operation, and a manual-reset event. Windows 7 requires non-NULL lpNumberOfBytesRead, so provide a DWORD;pending final bytes come from completion.

Initial result	Meaning	Lifetime
TRUE	Operation completed;inspect actual bytes	Process final result
FALSE, ERROR_IO_PENDING	Submitted, not ordinary failure	Retain buffer, ov, event
FALSE, other error	Follow file/pipe/device contract	Establish whether work remains

Confirm pending work with an appropriate mechanism such as GetOverlappedResult. Do not assume later GetOverlappedResultEx exists in Windows 7.

Six bytes, four bytes, then EOF

Fix a 10-byte regular file ABCDEFGHIJ, with no concurrent write or truncation. Each 6-byte request uses a `FILE_FLAG_OVERLAPPED` handle, its own valid OVERLAPPED and buffer, and an explicit offset. R0 and R1 succeed; initiation may complete immediately or first return pending.

Operation	Offset	Request	Final result	Next step
R0	0	6	Success: 6 B, ABCDEF	Reuse state only after terminal
R1	6	6	Success: 4 B, GHIJ	Completed prefix totals 10 B
R2	10	6	EOF: `ERROR_HANDLE_EOF`	Stop

For this positive-length overlapped regular-file read, R2 may return `ERROR_HANDLE_EOF` at `ReadFile` initiation or after pending through `GetOverlappedResult`; an immediate failure has no pending operation to wait for. A separate non-overlapped synchronous read can instead succeed with zero bytes at EOF. Pipes and devices differ; cancellation can lose to normal completion, so collect the actual result.

Virtual memory, heaps, mappings, and TLS

VirtualAlloc manages reserve/commit/protection. Heap managers subdivide regions;HeapFree need not release the whole region. Sections/views map file/pagefile backing at process-local VAs with compatible offset/size/file bounds.

TLS provides per-thread slots/logical ownership rather than MMU isolation. Another thread with a pointer can access the same process memory. Objects outliving threads need ownership/synchronization.

Commit, working set, and heap counts differ. Reserve 64 KiB, commit 8 KiB, resident 4 KiB can hold simultaneously;one memory number cannot describe all three.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete cancellation and cleanup

While kernel/device may access an operation, retain buffer, OVERLAPPED, event, context. For pendingR, request CancelIoEx(h,&ov), collect terminal through the original completion path, handle normal/aborted/other error and actual bytes, then release data/ov. Close shared handle/event only after all related operations terminate.

Normal completion may win;successful abort can yield ERROR_OPERATION_ABORTED. ERROR_NOT_FOUND from cancellation does not replace original outcome. Never free merely because cancel returned or reuse one ov for simultaneous R1/R2. IOCP/other notification modes have their own delivery rules;follow the selected contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

States of one read

Layer	Track	Earlier success can still leave
Authority	Local handle, granted mask, effective token	NoW or later network failure
Virtual memory	Buffer VA, commit, residency, rights	Reserved but uncommitted memory
I/O	IRP/operation, pending, actual bytes	Submission without completion
Cache/filesystem	Offset, VCN/LCN, metadata	Cache write without user durability
Device/network	DMA, connection, storage contract	Requested cancel without stopped hardware

IRQL constrains present kernel activity;thread priority selects ready execution. Confusing them invalidates ISR/DPC waiting and lifetime reasoning.

Reconstructing the examples

The fixed q2 trace gives completions8/9/3 from residual slices/queues. One region can reserve 64 KiB, commit 8, and reside 4 KiB. File byte 20603→VCN5/123→LCN201→volume823419→disk 1871995, with holes separate. Reads at 0/6/10 finish 6/4/EOF; pending, cancel request, terminal differ.

These instantiate general invariants:operations need granted rights;live I/O forbids premature free;extent, size, bitmap agree;one CPU's IRQL does not exclude other CPUs/shared-memory users.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz et al., Operating System Concepts, 9th ed., Ch. 19. Microsoft: IRQL, VirtualAlloc, AccessCheck. ReadFile, EOF, CancelIoEx, named pipes.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Chapter route

Which computing needs and constraints drove OS evolution, and how do historical designs relate to today's abstractions?

Prerequisites are OS fundamentals, process/memory/file abstractions, and architecture history.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Learning objectives

- Compare historical systems by requirements/constraints.
- Distinguish inherited concepts from version-specific mechanisms.
- Avoid equating historical products with current systems.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Feature migration

Compare constraints, mechanisms, and new contracts. Similar names do not establish identical address units, execution agents, or protection boundaries.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Concepts migrate without identical implementations

The chapter follows constraint→mechanism→contract, not a strict chronology. Batch sequencing reduces operator waits;spooling decouples device producers/consumers;time sharing adds interaction/CPU reclamation;virtual memory separates information identity from residence.

Constraint	Mechanism	New obligations
Slow manual job changes	Resident monitor	Detect termination/protect monitor
Slow cards/printing	Offline I/O, spool	Queues, capacity, errors/retries
Long exclusive occupancy	Time sharing	Timers, isolation, response, fairness
Expensive fast memory	Paging/backing	Restartable faults, mapping, replacement

Multics, UNIX, and smaller successors share multiuser, hierarchical-file, and sharing concepts. Direct code descent requires source/design evidence beyond conceptual similarity.

Four comparison dimensions

A process can mean static worker, dynamically created process, or a Mach task's thread. Paging can mean relocation without demand loading. Identify:

1. Whether execution agents are dynamic and dedicated per job or reused.
2. Byte/word addressing, page size, and table purpose.
3. Software-layer, privilege, address-space, or machine boundaries.
4. Ownership of faults, timeouts, buffers, and retry/release conditions.

Different answers prohibit transferring modern API/performance promises solely by name. Numerical demonstrations specify rules rather than claim historical measurements.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Early systems

Batch/monitors reduce manual waits;spooling overlaps device work. One printer remains bounded by its service demand.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From manual operation to resident monitors

Compilation, assembly, loading, running required media changes/termination observation. Batching groups similar needs;monitor control records choose next steps. Controllers/drivers still perform device operations.

Three jobsF, C, F each need2 CPU minutes. Each compiler setup costs 3, including initial setup. Original order has9 setup+6 CPU=15. ReorderingF, F, C has6+6=12, assuming reorderability and excluding response/dependencies.

Protect the monitor against user overwrite or halt. Control-record/data separation needs syntax, length, and escaping rules rather than assuming data rarely contains control characters.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Offline I/O, buffers, spools, and multiprogramming

Offline equipment converts cards to tape before mainframe processing, saving host waiting but adding media-batch/movement latency. Buffers absorb production/consumption mismatch until finite capacity backpressures. Disk spools retain independent jobs/data/queue metadata for overlap, unlike imagining random simultaneous access to both tape ends. Multiprogramming separately runs another ready job while one waits, and requires scheduling/isolation.

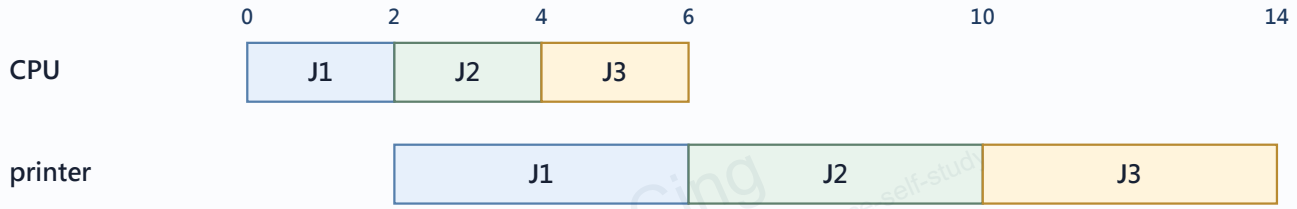
For three jobs each CPU 2 then print4, with one CPU/printer, CPU runsJ1 0–2, J2 2–4, J3 4–6. Printer runsJ1 2–6, J2 6–10, J3 10–14. Total14, CPU utilization6/14, printer 12/14. Serial execution takes 18. Spooling does not halve the printer's12 units of work.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Offline I/O, buffers, spools, and multiprogramming (continued)

Three jobs: CPU 2 each, one printer 4 each; total 14

One color per job. CPU and printer each serialize; different devices overlap.



J2/J3 output can queue. Printer work remains 12. CPU busy 6/14, printer 12/14; serial total 18.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Atlas

One-level storage separates information addresses from core/drum residence. Learning replacement includes the all-t=0 fallback.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Information identity and automatic backing

Atlas combines fast core/drum behind a one-level store with 512-word pages and associative mappings. The book's16 Kiwords core has32 frames, not16 KiB. A four-frame teaching model retains512 words/page.

Virtual word address3597=7・512+13 maps page 7→frame 2 to core word address1037. If absent, preserve restartable execution, load into an available frame, update mapping, and retry. One-level store does not make drum as fast/directly accessible as core. Transfers, replacement, and live-frame ownership remain necessary.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three learning-replacement branches

Kilburn1962 defines t as time since last use and T as the preceding inactive interval, sampled by executed instructions rather than wall time. Choose first any t>T+1;otherwise among t≠0 choose maximalT−t;otherwise all t=0, choose maximalT. This example breaks ties by table order.

Case	(t, T) entries	Branch/victim
A	(6,3),(2,5),(0,9)	First:6>4, first page
B	(2,4),(1,6),(0,8)	Second:nonzero scores2/5, second
C	(0,2),(0,5),(0,3)	Third:largestT5, second

Past intervals predict future use rather than supply OPT knowledge. A predicted-distant page used next can add a fault. Branch3 makes valid all-recently-used states selectable. Original paper.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

XDS-940

Paging can relocate without demand paging. Reentrant code can share while writable state remains private.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Relocation is not demand loading

Historical virtual space16 Kiwords, physical64 Kiwords, page 2 Kiwords give eight virtual pages/process and32 frames. Register-held tables scatter resident pages;whole-image swapping changes resident processes.

P virtual page 3→frame 11, offset 17 gives VA6161 and PA11・2048+17=22545 words. No byte conversion or unspecified word width is assumed. A legal missing-page demand fault is additional behavior absent from this relocation-only case. Atlas automatic demand loading and XDS relocation differ despite both having tables.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Shared code and controlled execution

Two processes each use eight pages. Sharing two safely read-only reentrant code pages needs2+6・2=14 frames instead of 16. Shared instruction pages do not privatize a secretly shared writable global buffer;separate code and mutable state.

Converting XDS-930 to940 added user/monitor distinction, privileged I/O/halt, and controlled system calls. Dynamic subprocesses/shared memory additionally require lifecycle, synchronization, termination cleanup. These protections work together.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

THE

Layers and explicit synchronization decompose reasoning. A software layer is not automatically a hardware protection domain.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Lower-layer contracts before upper-layer reasoning

Dijkstra's THE is batch multiprogramming by cooperating sequential processes, not primarily terminal interaction.

Level	Responsibility/hidden details
0	Processor allocation/timer;sequential-process abstraction
1	Segment controller/backing;information-segment identities
2	Console-message interpreter;physical console allocation
3	I/O buffering/logical communication units
4	User programs
5	Operator, not an implemented software level

Layers describe dependency rather than separate address spaces or mandatory traps. Segments identify information;core/drum pages hold it. EWD196.

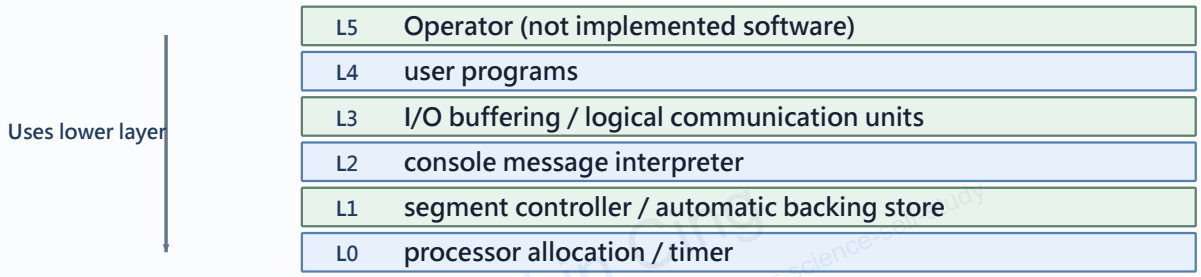
Induction over layer contracts

Assume layer i guaranteesCi fromC0…C(i−1), with stated interference constraints, and does not corrupt lower hidden state. Hardware/processor control establishesC0;induction establishes each successive contract. Layering alone proves nothing without those obligations.

Console messages can reside in segments and use level 1 retrieval. If level 1 instead needs level 2 to fetch its own pages, the dependency cycle invalidates bottom-up induction. Synchronization cannot assume relative speed:consume only produced segments, and give a printer only one incompatible owner. Waiting changes progress, not correctness based on "usually slower."

Induction over layer contracts (continued)

THE: higher levels use lower abstractions, not separate address spaces



Induction: Ci uses/preserves C0…C(i−1). Cycles or uncontrolled shared state invalidate the argument.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

RC4000

A nucleus supplies primitives while upper processes choose policy. Finite buffers require unique ownership and explicit return conditions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Policy above the nucleus

RC4000 supports constructing different systems rather than hardwiring one batch/time-sharing policy. Nucleus supplies process control, protection, messages. The book's fixed eight-word messages use a shared pool and per-process queues. Send-message/wait-message and send-answer/wait-answer form request/reply. Primitive atomicity protects internal transitions, not an entire server transaction.

Drivers translate registers/interrupts into process-like message interfaces. Devices do not thereby become ordinary processors executing arbitrary user code.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Conservation with two buffers

Pool={b0, b1};retain one buffer through request/reply completion. This is an ownership model, not instruction-level original-machine emulation.

Step	Free	Server queued/processing	Awaiting client reply
Initial	b0, b1	Empty	Empty
C1 sends	b1	b0 queued	Empty
C2 sends	Empty	b0, b1 queued	Empty
S receives/answers b0	Empty	b1 queued	b0
C1 consumes/returns	b0	b1 queued	Empty

Each buffer has exactly one state;free+queued+processing+reply=2. Third submission must wait/fail as specified rather than allocateb0 again. S waiting for C1 while C1 waits for S still deadlocks. Small kernels/atomic send do not eliminate circular wait.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CTSS

Interaction and swapping shape feedback. Specify initial level, quantum growth, and promotion separately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Short interaction versus long CPU bursts

MIT CTSS demonstrates interactive time sharing compatible with existing batch programs. Moving images between scarce memory/backing makes excessively short slices costly. Compatibility concerns preserving that environment, not every future API. The7090/7094 memory counts36-bit words, not equal KiB.

Exponential feedback demotes work exhausting a full slice and gives longer lower-priority slices, while short interactions gain earlier service. I/O promotion, aging, and starting level depend on the chosen version/model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete exponential-feedback trace

Fix q_i=2^i, start0, one burst11, no competitors/promotions;demote only when a full slice expires before completion.

Dispatch	Level	Allowed	Used	Total CPU	Remaining
1	0	1	1	1	10
2	1	2	2	3	8
3	2	4	4	7	4
4	3	8	4	11	0

Full slices throughk sum2^(k+1)−1 by the geometric series. With swap overhead.5/dispatch, four dispatches cost2, total 13. RR q1 needs11 dispatches, overhead5.5, total 16.5. This compares the fixed long-burst/cost model rather than every interactive workload.

Multics

Segments, files, paging, and selective sharing combine with ring enforcement. Rings and software layers provide different guarantees.

A computer utility with controlled sharing

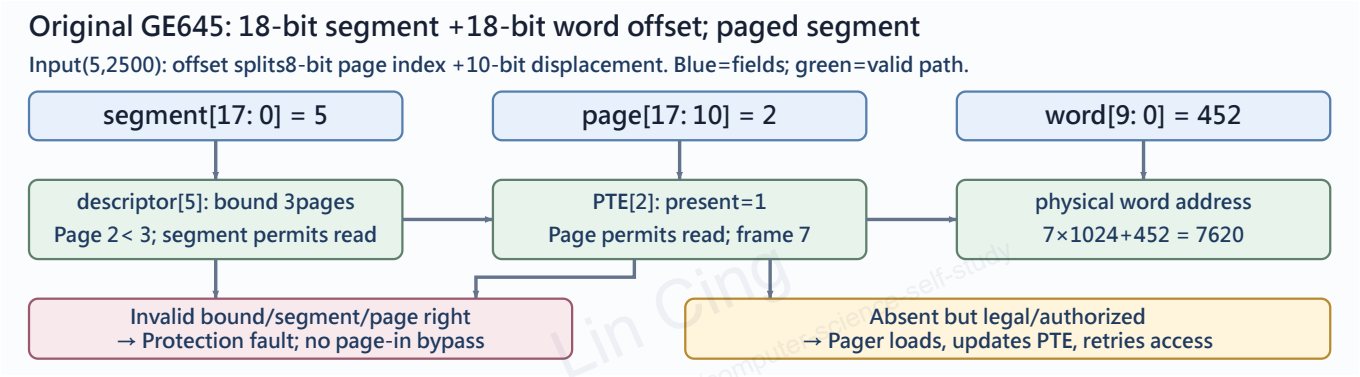
Multics targets remote terminals, continuous service, growing capacity, and selective sharing. Hierarchical files, segments, paging, rings jointly support those goals rather than one vague early-large-OS label.

Fix original GE645 hardware:18-bit segment number and18-bit word offset, each0…2^{18}−1. A 1024-word page uses10 offset bits, leaving8 page-index bits. Descriptor bounds can be smaller than field capacity. Original design.

Segment5, offset 2500=2 • 1024+452:first inspect descriptor, bound, rights, then segment page 2. Frame7 gives core word address7620. Segment identity is independent of physical frame.

Here segment 5 is paged, bound 3 pages, and segment/page permit read. Page 2<3 and presentPTE2→frame 7 allow access. Invalid bound/permission is a protection fault that page-in cannot bypass. Only an authorized absent page invokes pager, mapping update, retry.

A computer utility with controlled sharing (continued)



Rings versus layers

Rings enforce privilege/entry rules;THE layers organize abstraction/dependency. Modules need not each own a ring, and drawing layers proves no protection boundary.

Resolving a segment name still requires object access lists, execution context, entry rules. Shared code/private writable state need mapping/protection contracts. The developers'1972 retrospective describes integration exposing unexpected path costs and iterative redesign. Useful concepts do not prove the first interface partition efficient;smaller successors need not retain every mechanism. Retrospective.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

IBM OS/360

Family compatibility does not erase MFT/MVT differences. Later virtual memory and CP/CMS add distinct abstractions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Compatibility and allocation variants

System/360 sought common software/tools across capacities. OS/360 MFT uses fixed regions;MVT variable regions. One variant does not specify the entire family.

A 256 KiB example has four64 KiB regions and jobs50,50,70. Two50 jobs waste28 KiB internally. Remaining128 total cannot fit70 into one64 region. This is the chosen configuration, not all possible OS/360 layouts.

Variable holes50,30,40 total 120 but lack a contiguous70, an external-fragmentation example. Moving/compacting requires relocation support/cost;load-time-bound addresses cannot be arbitrarily changed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Later virtual memory and CP/CMS

System/370-era OS/VS/MVS uses new VM hardware rather than original360 paging semantics. CP/67 provides VMs;CMS is a single-user system within them. CP supplies multi-CMS isolation rather than CMS itself becoming a shared multiuser kernel.

Two guests can both see address1000 while VMM maps their memory views to different host resources. Mediate guest privilege;guest supervisor status does not authorize arbitrary host writes. Family compatibility, relocation, VM isolation, and interaction are distinct. JCL describes batch resources/steps;option count proves neither correctness nor a fixed performance ratio.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

TOPS-20

TENEX/TOPS-20 combines command assistance and memory support. Syntax usability does not replace object authorization.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Interaction and virtual memory

In the book's history, BBN added paging to PDP-10-related hardware and developed TENEX. DEC later acquired rights and formed DECSYSTEM-20/TOPS-20. Command completion/help reduces syntax-memory burden;virtual memory manages addresses/residence. Shell suggestions do not authorize page-table changes, and paging does not infer desired commands.

A machine-like private execution view with many simultaneous users still needs CPU scheduling, terminal I/O, file rights, reclamation. Faster CPUs alone do not provide interaction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Usability and security complement one another

In an explicit toy COPY source destination parser, valid syntax creates a request. Execution still checks source read and destination-parent/file write contracts. Autocomplete success does not grant access.

Checking destination then later writing its pathname can race name replacement, as in Chapter 11 TOCTOU. Protected references, clear errors, and understandable interaction must work together. This toy illustrates responsibility, not literal TOPS-20 syntax.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CP/M and MS-DOS

Small-platform compatibility differs from modern protected kernels. A shared-address-space convention does not confine untrusted code.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Resource limits and isolation

CP/M supplies early microcomputer commands/program/device services. MS-DOS follows an IBM-PC single-user/compatibility path. Their protection differs from later multiuser systems;they are not merely smaller NT/Linux.

Counts such as64 KiB/640 KiB must distinguish CPU addressability, conventional layout, OS reservations, and extensions. A common program region is not every version's absolute physical limit.

Without enforced memory isolation, programs can overwrite system data despite APIs instructing them to use services. Hostile programs need not follow conventions. Hardware privilege plus trusted mediation is required.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A shared-space counterexample

Toy RAM[0,1024) has monitor[0,256), job[256,1024), without hardware bounds. Store100 corrupts monitor even if correct libraries never do so. This is not an actual DOS layout.

Protected base256, limit768 validates0≤a<768 then translates256+a. Addresses−1/768 fail;0/767 reach256/1023. Users cannot modify base/limit. This proves range confinement, not progress against infinite loops, which needs a timer, or shared-file policy. One hardware feature supplies one boundary rather than a full modern OS.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Macintosh and Windows

GUI design does not determine preemption, memory protection, or driver boundaries. Cooperative/preemptive progress rests on different premises.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Desktop interaction versus kernel guarantees

Early Macintosh/Windows made desktop interaction central;later generations added fuller memory, multitasking, networking. Specify generation. Windows 7 here is NT, not early DOS-dependent Windows.

Windows, mouse, menus are interaction models;preemption, address protection, compatibility, drivers are separate. Two windows prove neither parallel processes nor isolated address spaces. A blocked UI thread can freeze a GUI while kernel scheduling continues. Trace event, queue, handler, and awaited service.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Cooperative and preemptive progress

On one CPU, A loops forever without yield while B is ready. If switching requires yield/block, B may never run. With serviceable timer and a scheduler giving both slices, preemption runsB at a slice boundary.

If A instead loops unbounded in a nonpreemptible kernel lock region, user slices alone cannot guarantee progress. Include kernel/IRQ bounds. This toy compares protocols without classifying every desktop generation identically. Even a scheduledB may wait for A's global resource;preemption does not eliminate deadlock/priority inversion.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mach

Mach2.x and3 place BSD differently. Separate task/thread/port authority from copy/mapping ownership.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Versioned abstractions

Mach builds on Accent-related research with tasks/address spaces, threads, IPC, VM. A task owns address space/port rights;threads execute and share their task's memory.

Model	BSD placement	Boundary
Mach2.x/2.5	Much remains in kernel	File service need not cross a user server
Mach3 microkernel	UNIX services in user servers	Controlled IPC across address spaces

Kernel-checkable port rights authorize messages;equal integers do not transfer rights across tasks. Shared memory also exists, so not all exchange requires byte-copy messages.

A fixed IPC cost and isolation model

ClientC sends32 B to user serverS, which replies4096 B. Each direction copies user→kernel→other user twice, without shared-page/out-of-line optimization.

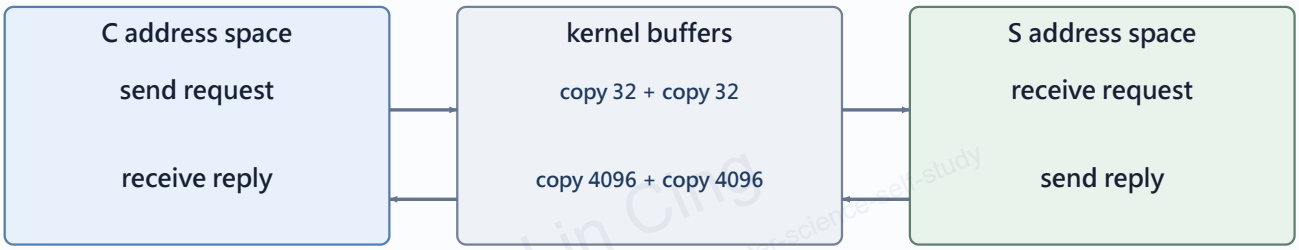
Path	Mode transitions	Address-space changes	Copy bytes
C user→K→S user	2	1	64
S user→K→C user	2	1	8192
Total	4	2	8256

Address protection confines direct corruption from server crash, but client still needs errors, timeouts, reconstruction. Smaller kernel does not prove universal availability. IPC/VM integration can avoid copies while adding mapping, rights, page lifetime, COW costs rather than becoming free.

A fixed IPC cost and isolation model (continued)

Mach3 copy model: 32 B request +4096 B reply; 8256 B copied

Blue=client C; green=server S; gray=kernel. Two copies/direction, no shared-page optimization.



Controlled kernel transfer: 4 mode transitions,2 address-space changes. Check port rights and buffer lifetime.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Other systems and unequal processor roles

The book also discusses Burroughs MCP's systems language, segmentation, multiprocessing, and CDC6600 coordination among processing activities. Multiple processors need not be symmetric peers. Identify roles, memory authority, I/O responsibility before applying an SMP scheduler.

High-level languages improve expression, type/module checking, maintenance, while synchronization, ABI, hardware control still need correctness. Assembly does not logically imply failure. Language choice changes inspectable conditions rather than replacing contracts.

Compatibility ecosystems, hardware costs, deployment organizations also affect survival. No single feature here explains commercial success or supplies current-market evidence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mechanisms across cases

Problem	Case	Required conditions
CPU/slow-I/O overlap	Monitors/spools	Queues, buffers, ownership, overlappable work
Nonresident information	Atlas	Mapping, restartable fault, total victim selection
Scattered/shared code	XDS-940	Relocation, RO code, private mutable state
Decomposed correctness	THE	Acyclic dependencies, layer contracts
Policy above nucleus	RC4000	Message ownership, pool limits, wait relationships
Interaction/long-burst tradeoff	CTSS	Feedback, promotion, swap-cost rules
Selective sharing/naming	Multics	Separate segment, page, protection
Service isolation	Mach3	IPC rights, address spaces, failure handling

General proofs beyond finite demonstrations

A trace exposes missing t=0 branches, word/byte confusion, or duplicate buffer allocation. General claims require rules covering all legal states.

Atlas selects a victim from every nonempty finite frame set:if branch 1 applies, use it;otherwise if any nonzero t exists, a finite maximum score exists;otherwise branch 3 chooses maximalT. This proves total selection, not minimum faults.

Buffer conservation holds initially and under each atomic ownership transition. Appearing simultaneously free and awaiting reply violates that invariant. Layer induction requiresCi to depend only on lower contracts while preserving them;cycles/uncontrolled hidden-state mutation invalidate it.

These prove total selection, resource safety, and compositional correctness, respectively. None alone establishes an entirely reliable OS.

References

- Silberschatz, Galvin, and Gagne,*Operating System Concepts*,9th edition, Chapter 20, printed pages 887–904. Historical cases retain their configurations. Numerical models, questions, SVGs are original;word counts are not byte counts.
- Kilburn1962 one-level store, pp231–232 supplies the three learning branches, including all-t=0. Historical intervals predict rather than implementOPT.
- Dijkstra EWD196 defines THE levels/information segments versus storage pages without implying independent address spaces.
- GE645 design specifies18-bit segment and18-bit word offset, split8+10 for 1024-word pages, not later6180 pointer formats.
- Multics first seven years,1972 describes utility goals/iteration rather than current markets.
- Tom Van Vleck on CTSS supplies7094/36-bit/compatibility context. Fixed q_i=2^i cases are not historical per-tick reconstructions.

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study